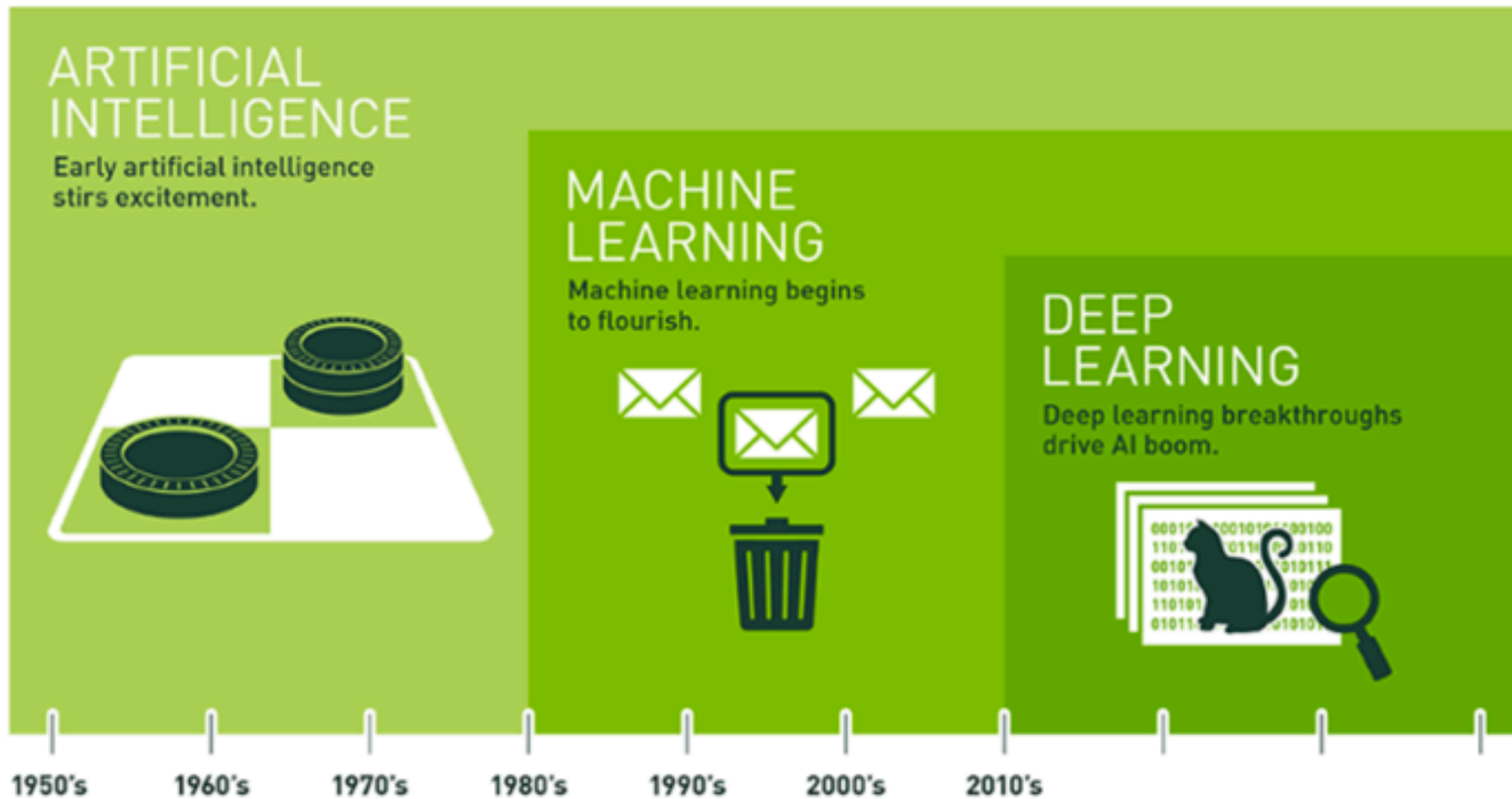




인공지능 개요

인공지능의 분류



Since an early flush of optimism in the 1950s, smaller subsets of artificial intelligence – first machine learning, then deep learning, a subset of machine learning – have created ever larger disruptions.

인공지능

Artificial Intelligence

- 인공지능(Artificial Intelligence)은 학습, 문제 해결, 패턴 인식 등과 같이 주로 인간 지능과 연결된 인지 문제를 해결하는 데 주력하는 컴퓨터 공학 분야 혹은 실현하는 기술, 개념
- 인공지능이라는 개념은 1956년 미국 다트머스 대학에 있던 존 매카시 교수에 의해 ‘인공지능’이란 용어가 처음 등장
- 최초의 인공지능의 개념은 인간과 유사한 특성을 가진 복잡한 컴퓨터를 제작
- 1970년대 당시 컴퓨터의 성능 때문에 인공지능 연구는 거의 없어짐

기계학습

Machine Learning

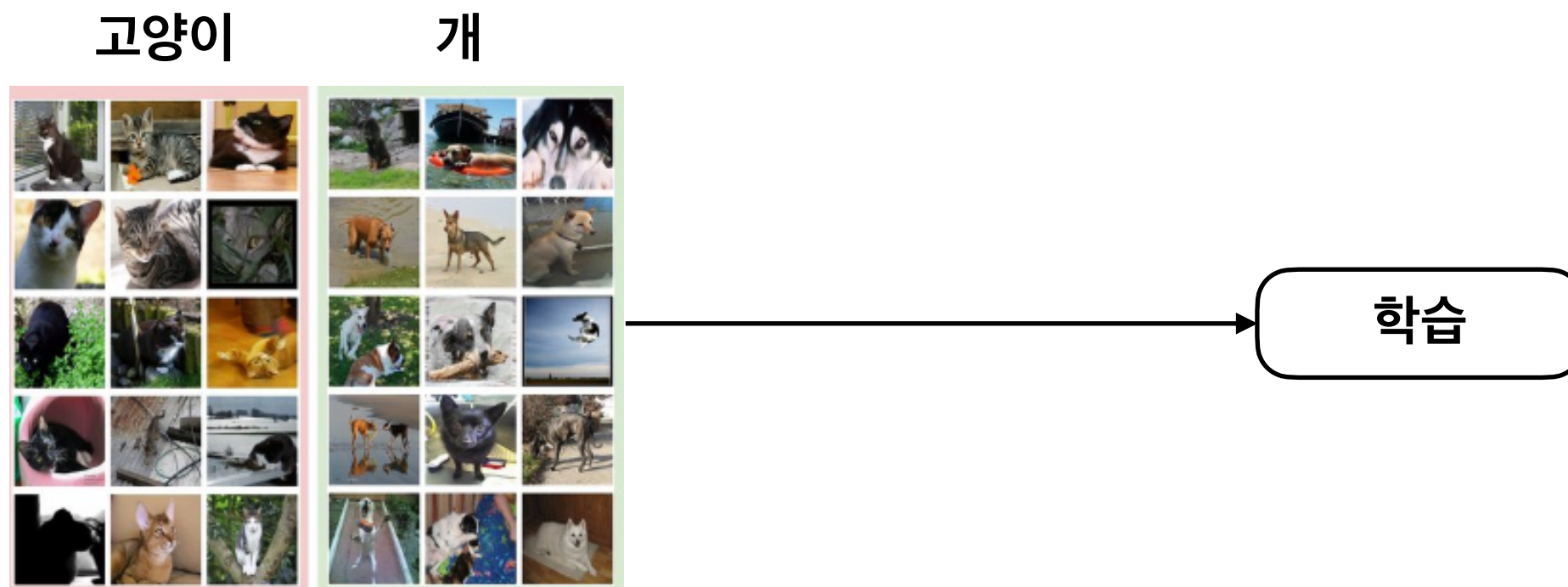
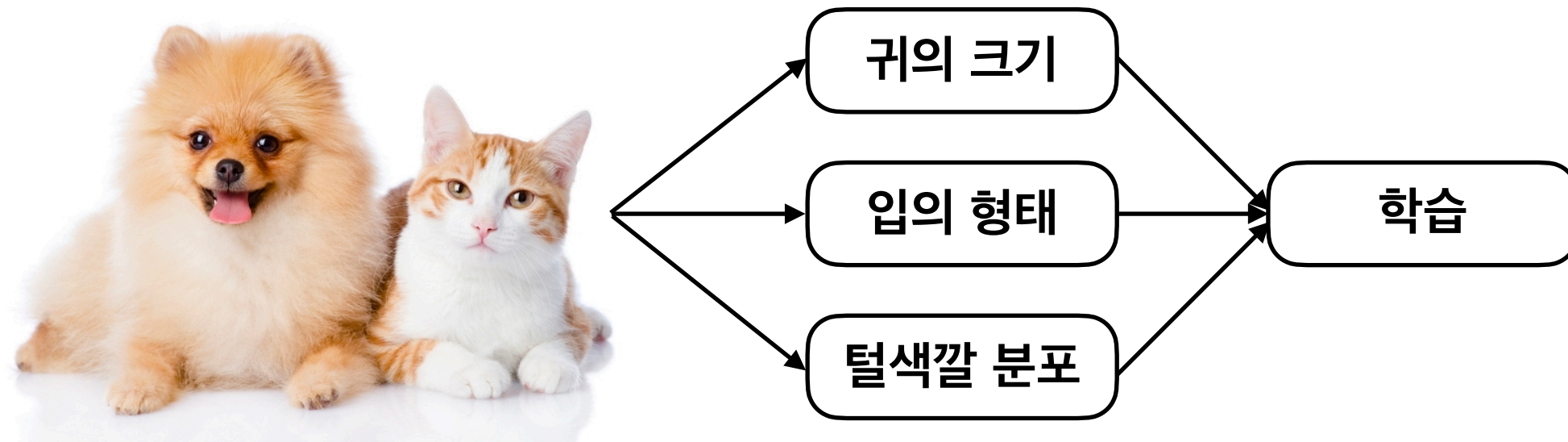
- 분류를 위한 데이터의 패턴이나 특징을 사람이 지정
- 10년간의 강수량 변화추이를 관찰해서 수확량을 예측하는 시스템
- 직업에 따라 수익 예측하는 시스템
- 결정트리를 이용한 전문가 시스템

딥러닝

Deep Learning

- 데이터의 특성을 사람이 지정하지 않음
- 컴퓨터가 스스로 학습해서 패턴을 찾아내고 자동으로 분류
- 많은 데이터가 필요함(빅데이터)
- 주로 인공 신경망을 사용해서 학습
- 컴퓨터 성능의 급격한 개선으로 인해서 활발한 연구가 진행중

머신러닝 vs 딥러닝



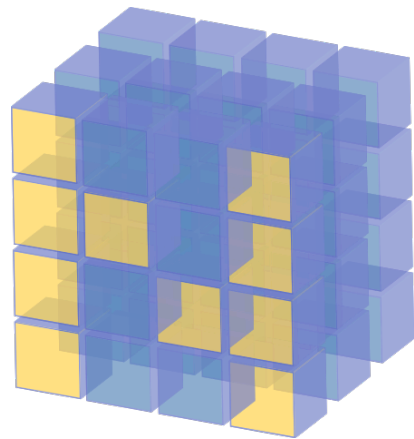
Python



- 1989년 Guido van Rossum에 의해 개발
- 인터프리터식 객체지향언어
- 상대적으로 배우기 쉬움
- 간결한 프로그래밍이 가능
- 풍부한 머신러닝 라이브러리



관련라이브러리



NumPy

행렬이나 다차원배열을 쉽게 처리

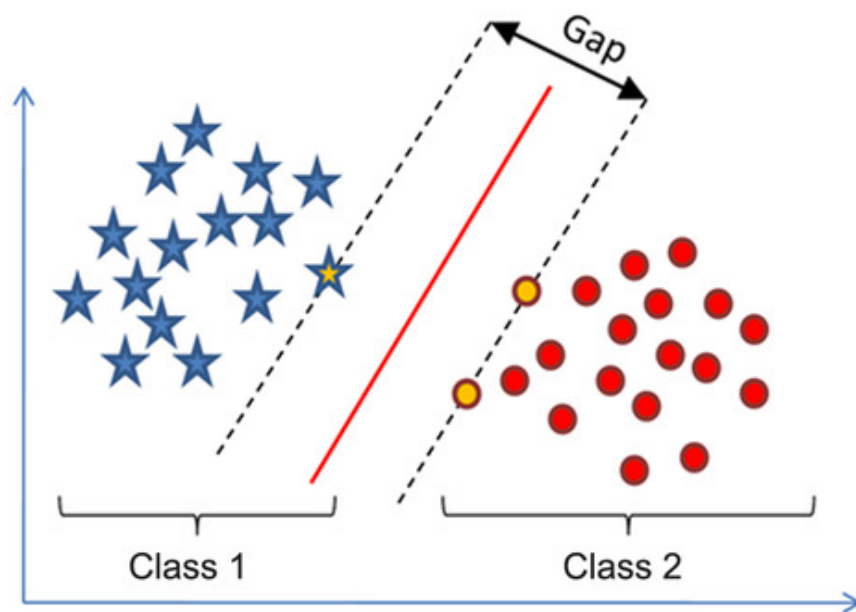


데이터를 쉽게 핸들링

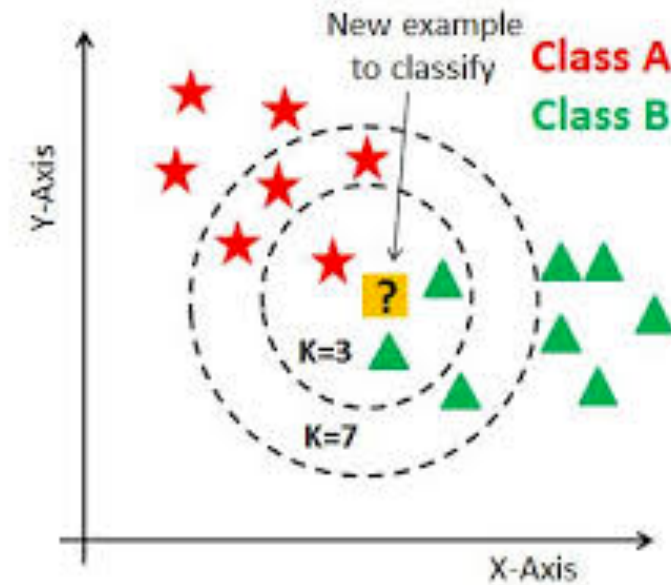
matplotlib

데이터를 차트나 플롯으로 시각화

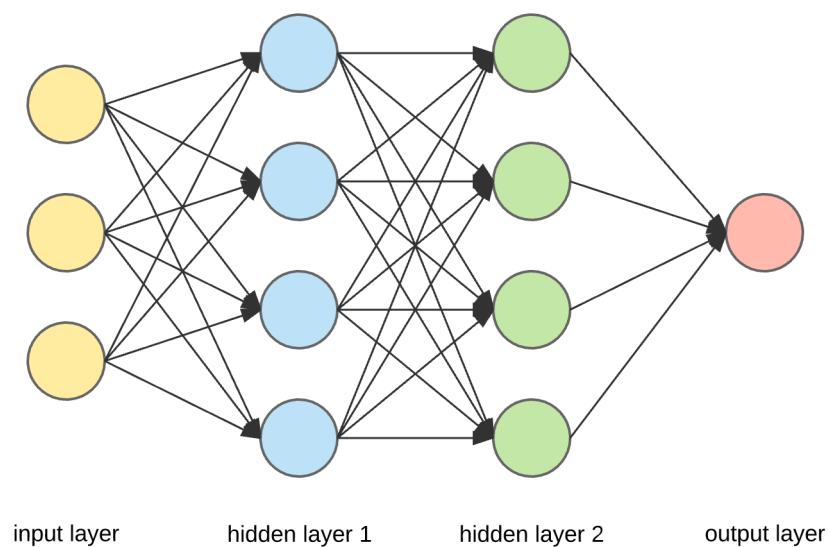
학습알고리즘



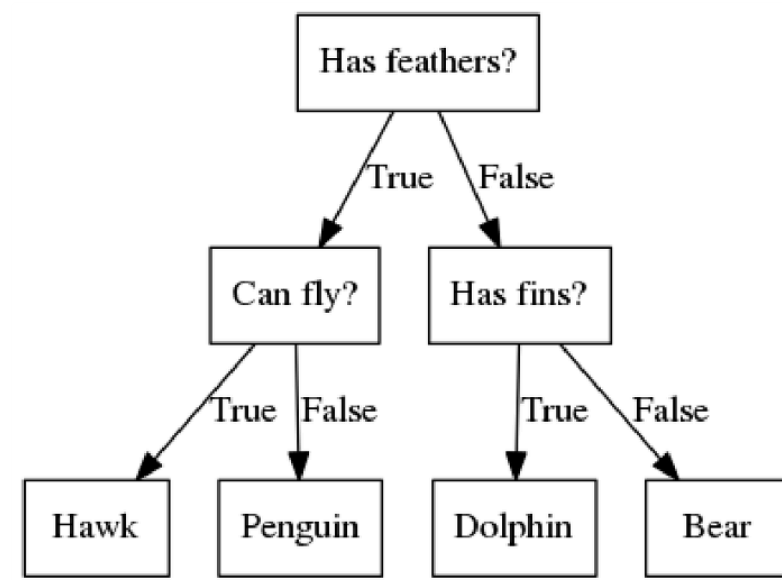
Support Vector Machine



k-nearest neighbors



Neural Network



Decision Tree

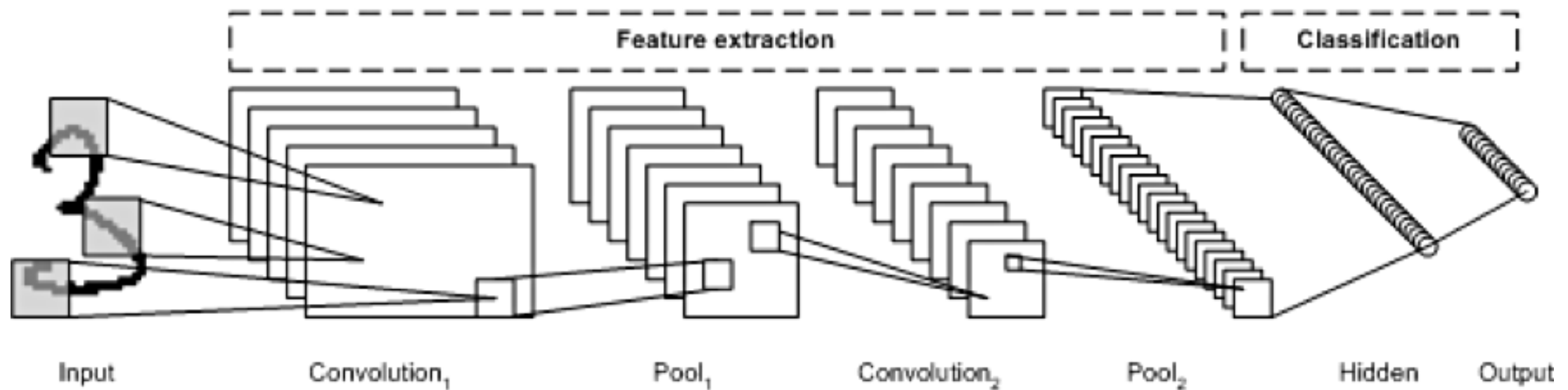
머신러닝 라이브러리



PYTORCH

합성곱 신경망

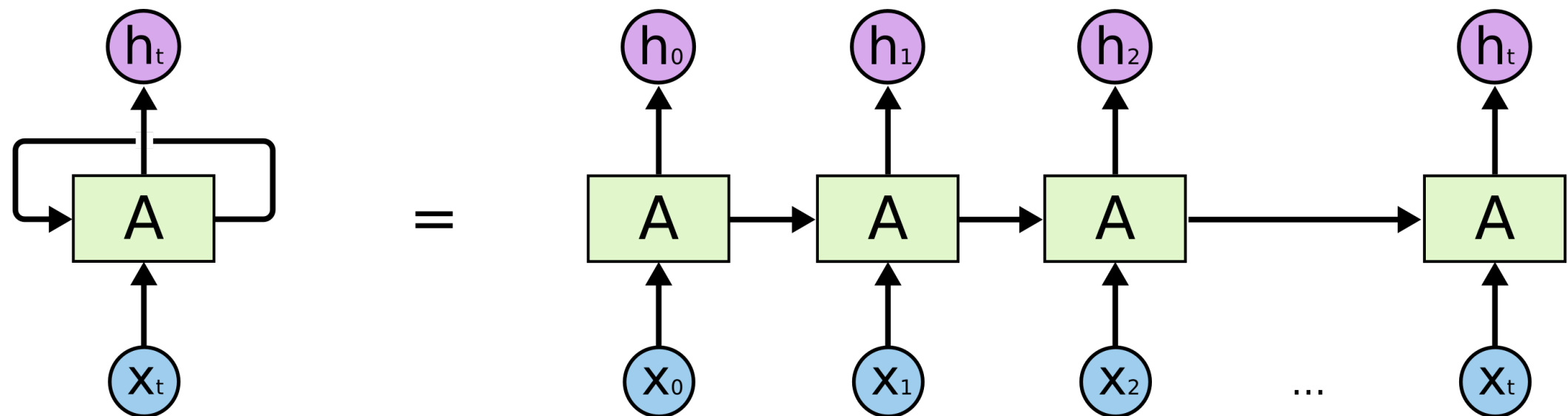
Convolution Neural Network



- 인간의 시신경 구조를 모방
- 이미지 인식에서 탁월한 성능을 보임

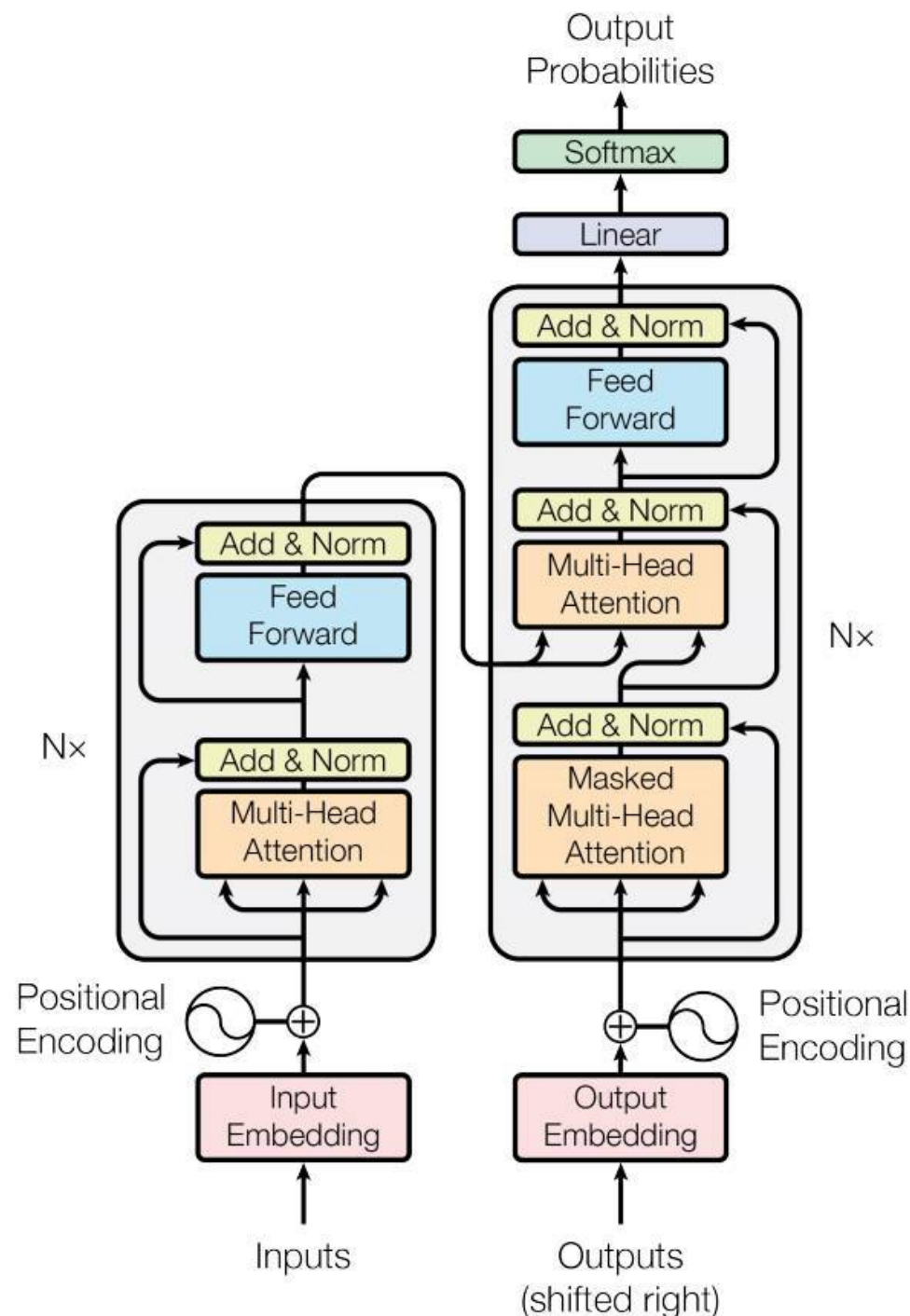
순환 신경망

Recurrent Neural Network

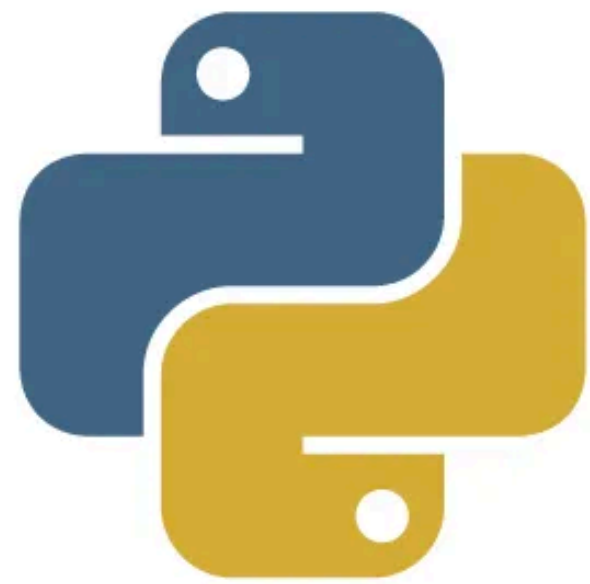


- RNN 알고리즘은 반복적이고 순차적인 데이터(Sequential data) 학습에 특화된 인공신경망의 한 종류
- 이전과 현재의 데이터를 이용해 이후를 예측

트랜스포머(Transformer) 모델



- 2017년 구글이 발표한 트랜스포머는 기존 RNN과 CNN의 구조적 한계를 극복하기 위해 등장한 혁신적인 인공지능망
- 어텐션 메커니즘만을 사용해 데이터의 장기 의존성 문제를 해결하고 효율적인 병렬 처리를 가능하게 설계 GPT, BERT 등 현대 주요 언어 모델의 근간
- 현재는 자연어를 넘어 시각·음성 등 AI 전 분야의 핵심 기술

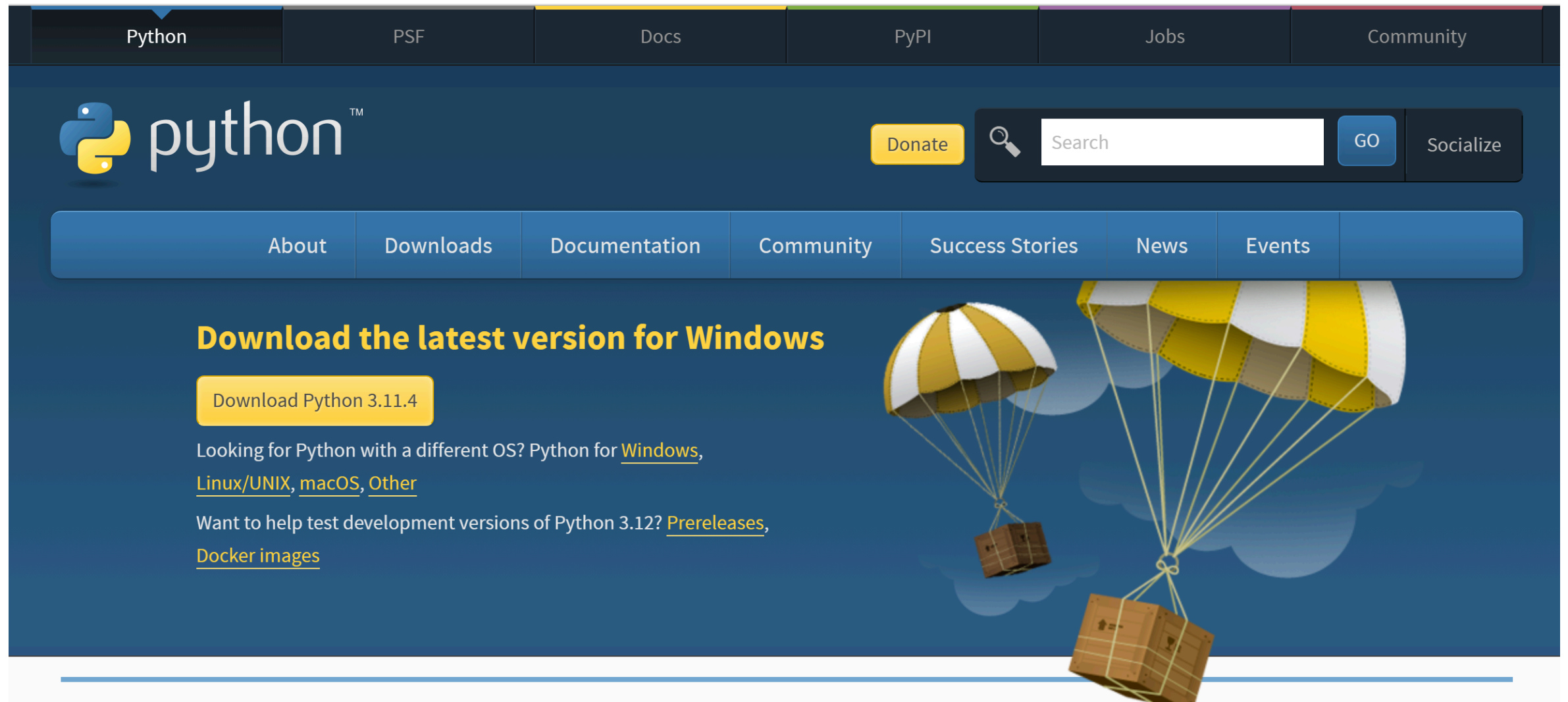


python

Python언어의 특징

- 인터프리터 언어
- 플랫폼 독립적
- 직관적이고 단순한 문법
- 동적 타이핑
- 방대한 라이브러리 보유
- 데이터 처리, 머신러닝, 웹 개발 등 사용

Python 설치하기



<https://www.python.org/downloads/>

개발도구-PyCharm



PyCharm Professional

전문 개발자용 Python IDE

[다운로드](#) [.dmg](#)

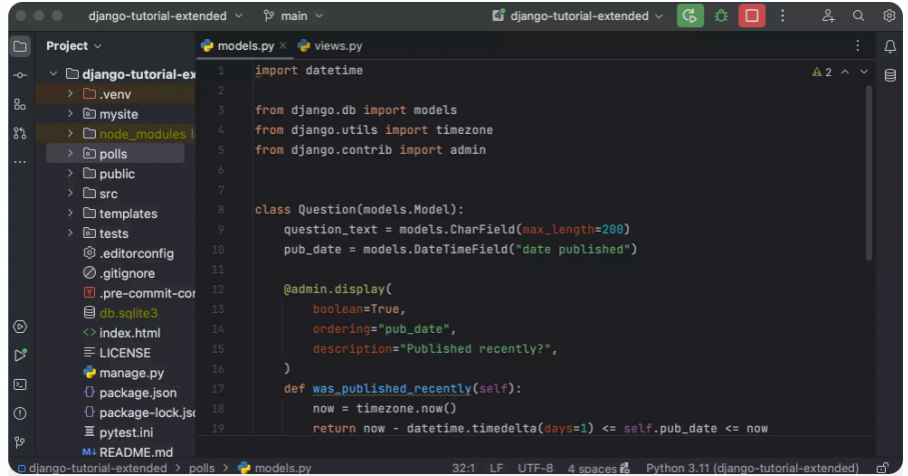
30일 무료 평가판

Intel 또는 Apple Silicon용 설치 프로그램 선택

개발자 도구 팀 도구 교육 솔루션 지원 스토어

새 버전이 제공됩니다 새로운 기능 기능 자세히 알아보기 [가격 책정](#) [다운로드](#)

Windows macOS Linux



버전: 2023.1.3
빌드: 231.9161.41
2023년 6월 21일

[시스템 요구 사항](#)
[설치 안내](#)

[기타 버전](#)
[타사 소프트웨어](#)

개발도구-Visual Studio Code

 Visual Studio Code

[Docs](#) [Updates](#) [Blog](#) [API](#) [Extensions](#) [FAQ](#) [Learn](#)



[Download](#)

[Version 1.79](#) is now available! Read about the new features and fixes from May.

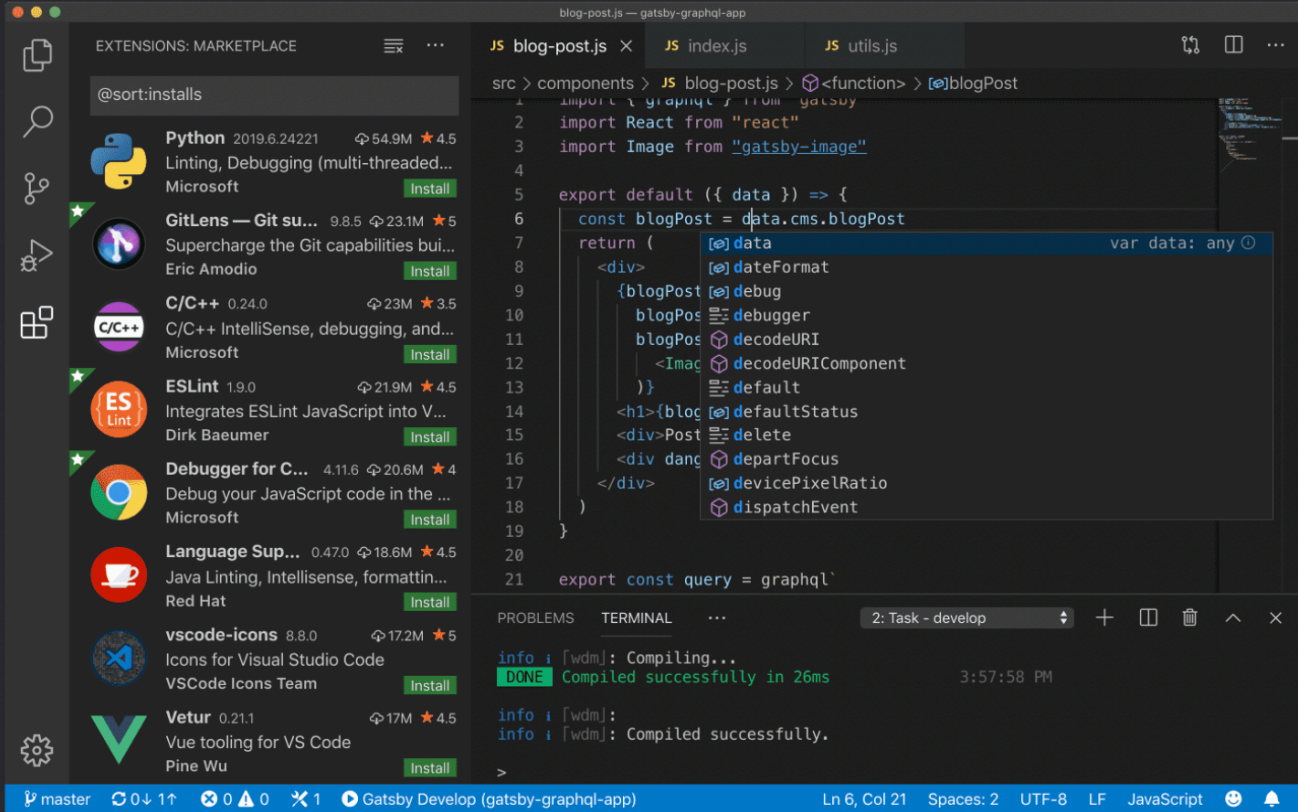
Code editing. Redefined.

Free. Built on open source. Runs everywhere.

[Download Mac Universal](#)
Stable Build

[Web, Insiders edition, or other platforms](#)

By using VS Code, you agree to its [license and privacy statement](#).



IntelliSense



Run and Debug

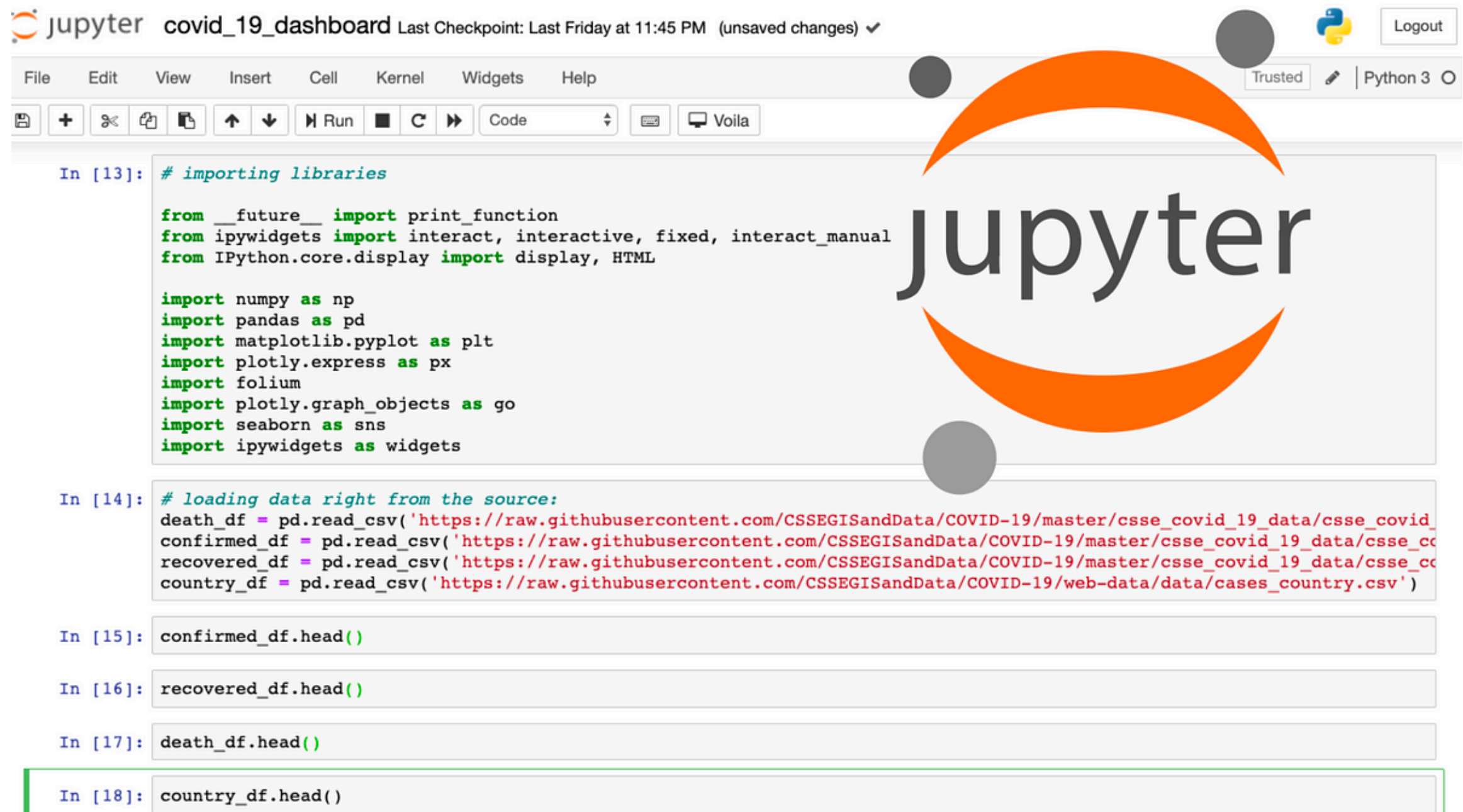


Built-in Git



Extensions

개발도구-Jupyter notebook



```
jupyter covid_19_dashboard Last Checkpoint: Last Friday at 11:45 PM (unsaved changes) ✓

File Edit View Insert Cell Kernel Widgets Help

In [13]: # importing libraries

from __future__ import print_function
from ipywidgets import interact, interactive, fixed, interact_manual
from IPython.core.display import display, HTML

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import plotly.express as px
import folium
import plotly.graph_objects as go
import seaborn as sns
import ipywidgets as widgets

In [14]: # loading data right from the source:
death_df = pd.read_csv('https://raw.githubusercontent.com/CSSEGISandData/COVID-19/master/csse_covid_19_data/csse_covid_19_data/confirmed_df = pd.read_csv('https://raw.githubusercontent.com/CSSEGISandData/COVID-19/master/csse_covid_19_data/csse_covid_19_data/recovered_df = pd.read_csv('https://raw.githubusercontent.com/CSSEGISandData/COVID-19/master/csse_covid_19_data/csse_covid_19_data/country_df = pd.read_csv('https://raw.githubusercontent.com/CSSEGISandData/COVID-19/web-data/data/cases_country.csv')

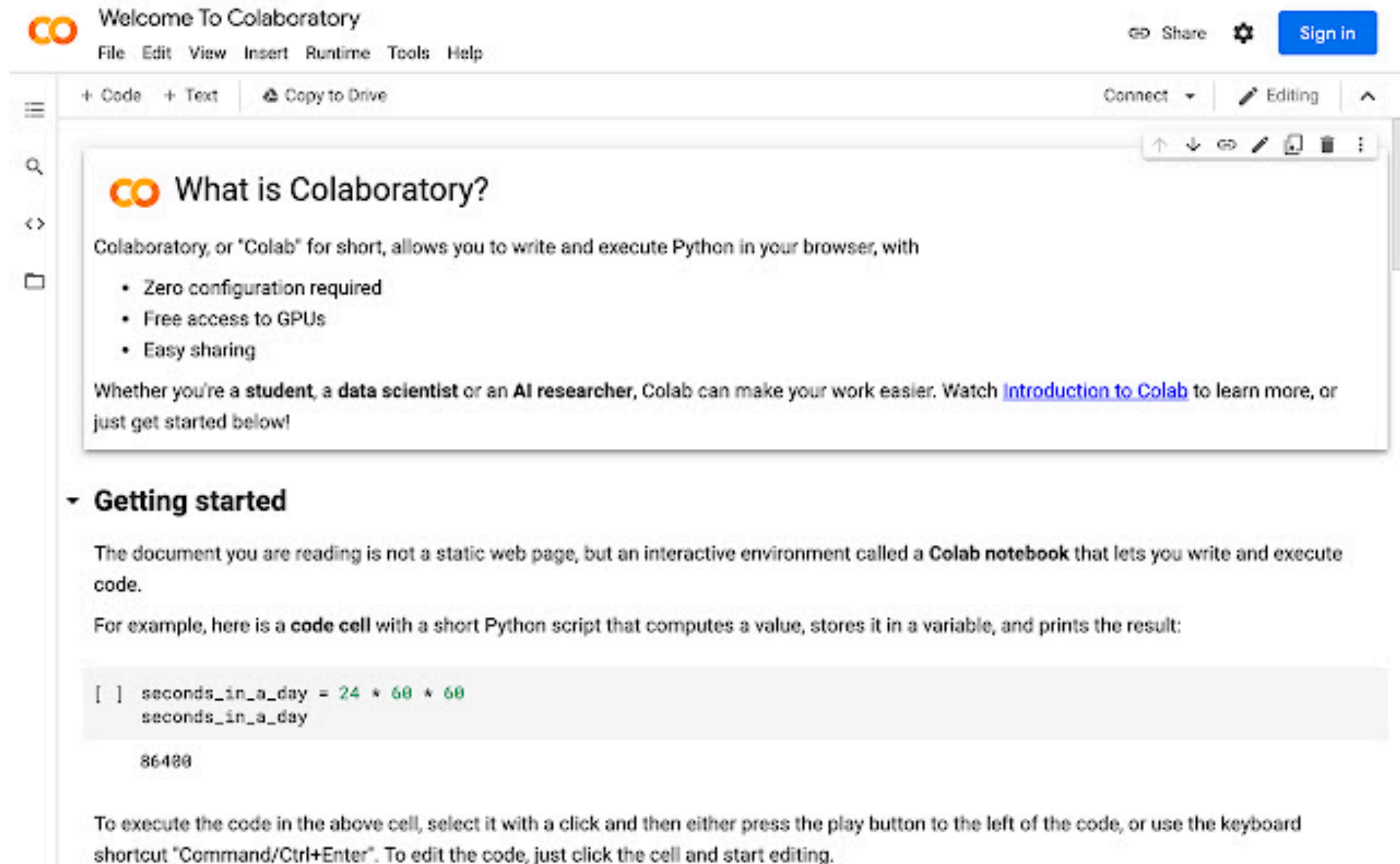
In [15]: confirmed_df.head()

In [16]: recovered_df.head()

In [17]: death_df.head()

In [18]: country_df.head()
```

개발도구-colab



The screenshot displays the Google Colaboratory (Colab) web interface. At the top, there's a header with the Colab logo, the text 'Welcome To Colaboratory', and a menu bar with 'File', 'Edit', 'View', 'Insert', 'Runtime', 'Tools', and 'Help'. On the right side of the header, there are links for 'Share', 'Settings', and a 'Sign in' button. Below the header, a toolbar shows '+ Code', '+ Text', and 'Copy to Drive'. The main content area features a 'What is Colaboratory?' section with a list of bullet points: 'Zero configuration required', 'Free access to GPUs', and 'Easy sharing'. Below this, a paragraph explains that Colab is an interactive environment for writing and executing Python code in a browser. A section titled 'Getting started' follows, explaining that the document is a Colab notebook and providing an example of a code cell. The code cell contains a Python script that calculates the number of seconds in a day (24 * 60 * 60) and prints the result, which is 86400. At the bottom, a paragraph explains how to execute the code in the cell by clicking the play button or using the keyboard shortcut 'Command/Ctrl+Enter'.

Welcome To Colaboratory

File Edit View Insert Runtime Tools Help

Share Settings Sign in

+ Code + Text Copy to Drive

Connect Editing

What is Colaboratory?

Colaboratory, or "Colab" for short, allows you to write and execute Python in your browser, with

- Zero configuration required
- Free access to GPUs
- Easy sharing

Whether you're a **student**, a **data scientist** or an **AI researcher**, Colab can make your work easier. Watch [Introduction to Colab](#) to learn more, or just get started below!

Getting started

The document you are reading is not a static web page, but an interactive environment called a **Colab notebook** that lets you write and execute code.

For example, here is a **code cell** with a short Python script that computes a value, stores it in a variable, and prints the result:

```
[ ] seconds_in_a_day = 24 * 60 * 60
    seconds_in_a_day

86400
```

To execute the code in the above cell, select it with a click and then either press the play button to the left of the code, or use the keyboard shortcut "Command/Ctrl+Enter". To edit the code, just click the cell and start editing.

변수(variable)

- 변수는 값을 저장하기 위해 사용
- 프로그램에서 필요한 데이터를 임시적으로 저장할 수 있는 공간에 이름(식별자) 붙인것
- 변수명을 통해 자유롭게 데이터에 대해 읽기, 쓰기, 덮어쓰기 가능
- 변수의 선언없이 바로 사용가능
- age = 27 라고 입력하면 age란 변수가 만들어지고 27을 저장
- name = '홍길동'

변수 작명규칙

- 영문자(alphabet), 숫자, 언더스코어("_") 만 사용가능
- 중간에 공백이 들어갈 수 없음
- python 키워드는 사용할 수 없음
- 숫자는 제일 앞에 올 수 없음
- 영문자 혹은 언더스코어("_") 만이 첫글자로 올 수 있음
- 대소문자 구분, 변수의 시작은 소문자를 사용
- 가능한 변수명에 역할에 맞는 의미를 부여할것

기본 데이터형

- 정수형(int) - 1, 2, 500
- 실수형(float) - 1.5, 3.14
- 복소수(complex) - $v = 2 + 3j$
- 이진형(bool) - True(참), False(거짓)
- 문자열형(str) - "I love you", '사랑합니다.'

기본데이터형

- 파이썬에서 모든 데이터 형은 class
- 데이터형의 type을 알고 싶으면 type(데이터)

```
[30] 1 | type(.10.)
```

```
↳ int
```

```
[31] 1 | type(.1.1.)
```

```
↳ float
```

```
[33] 1 | s = 'Hello'
      2 | type(s.)
```

```
↳ str
```

연산자

산술연산자

연산자	의미	예	결과
+	덧셈	$4 + 10$	14
-	뺄셈	$4.0 - 0.1$	3.9
*	곱셈	$20 * 20$	400
/	실수 나눗셈	$1 / 2$	0.5
//	정수 나눗셈(몫)	$11 // 2$	5
**	누승	$4 ** 2$	16
%	나머지	$20 \% 3$	2

산술연산자 실습

```
1 print(2+3)
2 print(5-2)
3 print(4*5)
4 print(1/2)
5 print(5//3)
6 print(5**2)
7 print(5**3)
8 print(20%3)
```

```
5
3
20
0.5
1
25
125
2
```

```
1 print('Hello'+'World!')
2 # print('Hello'-'World!')
3 # print('Hello'*'World!')
4 print('Hello'*2)
```

```
HelloWorld!
HelloHello
```

```
[36] 1 num = 3
      2 print(num+5)
      3
      4 num1 = 5
      5 num2 = 6
      6 print(num1+num2)
      7
```

```
8
11
```

대입연산자

연산자	사용예	의미
=	a = 4	변수 a에 4을 대입
+=	a += 1	a = a + 1
-=	a -= 3	a = a - 3
*=	a *= 3	a = a * 3
**=	a **= 3	a = a ** 3
/=	a /= 3	a = a / 3
//=	a //= 3	a = a // 3
%=	a %= 3	a = a % 3

대입연산자 실습

```
1 num=4
2 print(num)
3 num+=4
4 print(num)
5 num-=3
6 print(num)
7 num*=3
8 print(num)
9 num/=3
10 print(num)
11 num**=2
12 print(num)
```

4
8
5
15
5.0
25.0

```
14 hello= 'Hello '
15 hello+='World!'
16 print(hello)
17 hello*=3
18 print(hello)
```

Hello World!
Hello World!Hello World!Hello World!

비교연산자

연산자	사용예	의미
<	$a < b$	a가 b보다 작으면 True, 그렇지 않으면 False
>	$a > b$	a가 b보다 크면 True, 그렇지 않으면 False
<=	$a \leq b$	a가 b보다 작거나 같으면 True, 그렇지 않으면 False
>=	$a \geq b$	a가 b보다 크거나 같으면 True, 그렇지 않으면 False
==	$a == b$	a가 b와 같으면 True, 그렇지 않으면 False
!=	$a != b$	a가 b와 같지 않으면 True, 그렇지 않으면 False

비교연산자 실습

1	<code>print(5 < 6)</code>	True
2	<code>print(5 > 6)</code>	False
3	<code>print(11 <= 11)</code>	True
4	<code>print(15.1 >= 12)</code>	True
5	<code>print(13 == 13)</code>	True
6	<code>print(11 != 5)</code>	True
7	<code>print('Hello' > 'World!')</code>	False
8	<code>print('홍길동' != '허균')</code>	True
9	<code>result = 13==4</code>	False
10	<code>print(result)</code>	

논리연산자

연산자	사용예	의미
not	not a	a가 True이면 False, False 이면 True 반환
and	a and b	a와 b 모두 True일때 True, 하나라도 False 이면 False
or	a or b	a와 b 중 하나라도 True일때 True, 둘다 False 이면 False

비교, 논리연산자 실습

1	<code>print(True and True)</code>	True
2	<code>print(True and False)</code>	False
3	<code>print(True or False)</code>	True
4	<code>print(False or False)</code>	False
5	<code>print(not False)</code>	True
6	<code>print(5<6 and True)</code>	True
7	<code>num1=3</code>	
8	<code>num2=5</code>	
9	<code>print(not (num1>num2))</code>	True
10	<code>print(num1==num2.)</code>	False

문장의 구분

- 많은 언어에서 문장의 끝을 표시하기 위해서 ‘;’을 사용
- 파이썬에서는 문장끝에 ‘;’을 표시하지 않고 줄바꿈을 인식
- 줄바꿈을 하지 않고 두개이상의 문장을 사용할때 구분을 위해서 사용

```
print('Hello, world!')
```

```
print('Hello'); print('world!')
```

들여쓰기

- 들여쓰기는 코드를 읽기 쉽도록 일정한 간격을 띄워 작성하는 방법
- 파이썬은 들여쓰기 자체가 문법, 들여쓰기를 하지 않으면 에러 발생
- tab키나 space를 사용하는데 space를 사용하는것을 권장
- tab키와 space를 혼용해서는 안됨

코드블럭

- 코드블럭은 특정한 동작을 위해서 코드가 모여있는 상태
- 다른 언어에서는 주로 { }, begin end, 같이 명시적으로 블럭을 표시
- 파이썬에서는 : 와 들여쓰기로 코드블럭을 구성

```
if age == 10:  
    print('10살')  
    print('입니다.')
```

```
print('여긴 다른 블럭입니다.')
```

파이썬

```
if age == 10 {  
    print('10살')  
    print('입니다.')
```

```
}  
print('여긴 다른 블럭입니다.')
```

swift

주석

- 코드가 아니지만 사람이 참조할 목적으로 작성 하는 부분
- 코드가 아니기 때문에 인터프리터가 처리하지 않음
- 프로그램 실행에 영향을 주지않음

```
print('Hello, world!')
print('Hello'); print('world!')

# Hello, world! 출력
print('Hello, world!')

#print('Hello, world!')

a = 1 + 2 # 더하기
print('Hello, world!') #print('1234567890')
```

print 함수

- 변수의 내용이나 특정 내용을 출력하기 위해서 사용
- `print('Hello world!')`
- `print(age)`
- `name = '홍길동'`
- `print('나의 이름은' + name)`

```
[4]      1 | age = 20  
      2 | print(age.)
```

```
↳ 20
```

```
[5]      1 | print("Hello Python!")
```

```
↳ Hello Python!
```

```
[7]      1 | name = '홍길동'  
      2 | print('나의 이름은 ' + name)
```

```
↳ 나의 이름은 홍길동
```

print 함수

```
age = 20
```

```
name = '홍길동'
```

```
print(name, age)
```

```
print('이름은', name, '나이는', age)
```

```
print('이름은 ' + name + '나이는 ' + str(age))
```

string. format()

```
str1 = "{}".format(10)
```

```
print(str1)
```

```
str2 = "{}과 {}".format(10, 20)
```

```
print(str2)
```

```
num1 = 10
```

```
num2 = 20
```

```
str3 = "{}x{}={} ".format(num1, num2, num1*num2)
```

```
print(str3)
```

string. format()

```
age = 20
```

```
name = '홍길동'
```

```
str4 = '이름은 {} 나이는 {}'.format(name, age)
```

```
print(str4)
```

```
str5 = '이름은 {0} 나이는 {1}'.format(name, age)
```

```
print(str5)
```

```
str6 = '이름은 {1} 나이는 {0}'.format(age, name)
```

```
print(str6)
```

```
print('이름은 {1} 나이는 {0}'.format(age, name))
```

string. format()

```
age = 20
```

```
name = '홍길동'
```

```
str7 = '이름은 {:s} 나이는 {:d}'.format(name, age)
```

```
print(str7)
```

```
print('pi = {:f}'.format(3.141592))
```

```
print('pi = {:10f}'.format(3.141592))
```

```
print('pi = {:5.2f}'.format(3.141592))
```

```
print('pi = {:.2f}'.format(3.141592))
```

string 서식지정자

```
print('이름은 %s 나이는 %d'%(name, age))
```

```
print('이름은 %s 나이는 %5d'%(name, age))
```

```
print('이름은 %s 나이는 %05d'%(name, age))
```

```
print('pi = %f'%3.141592)
```

```
print('pi = %5.2f'%3.141592)
```

```
print('pi = %.2f'%3.141592)
```

시퀀스 자료형

list

- list는 일련번호로 구분되는 순서에 따라 데이터가 정렬된 목록 형태의 자료형
- 리스트는 문자열, 정수, 실수 등 모든 자료형을 섞어서 저장할 수 있음
- 리스트의 값은 변경가능(mutable)

score01=30

score02=50

score03=90

.

.

score30=89

```
scores = [30, 50, 90, 89, 56, 87, 45]
```

```
print(scores)
```

```
items = [1, 3, 'Hello', 5.6, False, 89, 'World!']
```

```
print(items)
```

list 활용

- 개별데이터를 조회할 때는 index를 사용
- index의 범위는 0 부터 list의 길이-1
- 범위를 벗어나면 오류
- -index는 역순

```
scores = [30, 50, 90, 89, 56, 87]
score0 = scores[0]
print(score0)
print(scores[5])
print(scores[0])
print(scores[-1])
print(scores[-3])
```

list 활용

- 개별데이터의 값을 바꿀때는 index를 사용
- -index는 역순
- 범위를 벗어나면 오류

```
scores[1] = 100
```

```
print(scores)
```

```
scores[3] = 200
```

```
print(scores)
```

```
scores[-1] = 300
```

```
print(scores)
```

```
scores[6] = 400
```

```
IndexError: list assignment index  
out of range
```

list 활용

추가

추가할 데이터

`scores.append(99)`

`scores.append('Hello')`

삽입

삽입할 위치

삽입할 데이터

`scores.insert(1, 33)`

`scores.insert(2, 'World')`

list 활용

- list는 +, * 연산자 사용가능

```
list1 = [1,2,3,4,5]
list2 = [5,6,7,8,9,10]
print(list1 + list2)
print(list2 + list1)
print(list1 * 3)
print(list2 * 3)
```

list활용

- list의 길이는 len함수를 사용해서 조회

```
scores = [30, 50, 90, 89, 56, 87, 45]
```

```
length = len(scores)
```

```
print('scores의 길이는 {}입니다.'.format(length))
```

```
bts = ["진", "슈가", "제이홉", "RM", "지민", "뷔", "정국"]
```

```
print('bts의 멤버는 {}명입니다.'.format(len(bts)))
```

list분할

```
numbers = [0, 10, 20, 30, 40, 50, 60, 70, 80, 90, 100]

numbers1 = numbers[0:4]

print(numbers1)

print(numbers[:4])

print(numbers[7:11])

print(numbers[7:])

print(numbers[:])
```

range()

`range([start], stop, [step])`

start, step 생략가능

start의 기본값 0

step의 기본값 1

범위를 생성할때 stop값은 포함되지 않음

`range(10)` # 0 ~ 9까지의 범위를 생성

`range(1, 10)` # 1 ~ 9까지의 범위를 생성

range()

```
r1 = range(1, 10, 1) # 1~9까지 1씩 증가하는 범위  
print(r1)
```

```
r2 = range(10, 20, 2) # 10~20까지 2씩 증가하는 범위  
print(r2)
```

```
r3 = range(10, 1) # start=10, stop=1, step=1로 인식  
되어 범위가 만들어지지 않는다.  
print(r3)
```

range()

10 ~ 1까지 1씩 감소하는 범위

```
r4 = range(10, 0, -1)
```

```
print(r4)
```

10 ~ 1까지 2씩 감소하는 범위

```
r5 = range(10, 0, -2)
```

```
print(r5)
```

range를 이용한 list 생성

```
list1 = list(range(10))  
print(list1)  
list2 = list(range(1, 10))  
print(list2)  
list3 = list(range(1, 10, 2))  
print(list3)  
list4 = list(range(10, 0, -1))  
print(list4)
```

Tuple

- tuple는 일련번호로 구분되는 순서에 따라 데이터가 정렬된 목록 형태의 자료형
- tuple은 문자열, 정수, 실수 등 모든 자료형을 섞어서 저장할 수 있음
- list와 달리 내용을 변경 할수 없음(immutable)
- ()를 사용함

Tuple

```
student = ('전정국', '인공지능학과', 3, 175.3, 3.5, True)
```

```
print(student)
```

```
print(student[0])
```

```
student[0] = '정국' #에러 immutable
```

```
car = ('Tesla', 'model3', 'red', 500)
```

```
print(car)
```

range를 이용한 tuple 생성

```
range1 = range(10)
```

```
tuple1 = tuple(range1)
```

```
print(tuple1)
```

```
range2 = range(-5, 15, 2)
```

```
print(tuple(range2))
```

input()

#사용자의 입력을 받을때 사용

#입력값은 문자열로 반환

```
input("프롬프트 문자열")
```

```
age = input('나이를 입력하시오')
```

```
print(age)
```

```
num = 3
```

```
diff = input('변화량을 입력하시오')
```

```
print(num + diff) # 에러
```

```
#TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

```
print(num + int(diff))
```

제어문

if 조건문

if 조건식 :

코드	조건식의 결과가
·	참일때만
·	바로 아래의
코드	코드블럭을 실행

```
adult = 18
```

```
age = 17
```

```
if age < adult :
```

```
    print('당신은 미성년자입니다.')
```

```
    print('당신은 입장할 수 없습니다.')
```

if~else 조건문

```
adult = 18
age = 17

if age < adult :
    print('당신은 미성년자입니다.')
else :
    print('당신은 성인입니다.')
```

```
gender = 'male'

if gender == 'male' :
    print('당신은 남자입니다.')
else :
    print('당신은 남자가 아닙니다.')
```

if~elif

```
if 조건식1 :
```

```
    코드블럭1
```

```
elif 조건식2 :
```

```
    코드블럭2
```

```
elif 조건식3 :
```

```
    코드블럭3
```

```
elif 조건식4 :
```

```
    코드블럭4
```

```
else :
```

```
    코드블럭5
```

if~elif

```
score = 90
if score >= 90:
    result = 'A'
elif score >= 80:
    result = 'B'
elif score >= 70:
    result = 'C'
elif score >= 60:
    result = 'D'
else :
    result = 'F'
print(result)
```

```
score = 90
if score >= 90:
    result = 'A'
if score >= 80:
    result = 'B'
if score >= 70:
    result = 'C'
if score >= 60:
    result = 'D'
else :
    result = 'F'
print(result)
```

for ~ in 구문

```
# Hello World! 1000번출력
print('Hello World!')
print('Hello World!')
print('Hello World!')

.

.

.

print('Hello World!')
```

```
for 변수 in range(반복횟수) :
    반복할 코드
```

```
for i in range(10):
    print('Hello World!')
```

```
for i in range(1, 10):
    print('Hello World!', i)
```

```
for i in range(10, 0, -1):
    print(i)
```

for ~ in 구문

```
list1 = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]  
for num in list1:  
    print(num)
```

```
fruits = ('apple', 'orange', 'grape')  
for fruit in fruits:  
    print(fruit)
```

```
tuple1 = (1, 2, 3, 4, 5, 6, 7, 8, 9, 10)  
for num in tuple1:  
    print(num)
```

for ~ in 구문

input 함수를 이용해서 입력받은 횟수만큼 반복

```
str_count = input('반복할 횟수를 입력하세요: ')\ncount = int(str_count)\n\nfor i in range(count):\n    print('Hello, world!', i)
```

for루프를 중첩해서 사용

```
for i in range(10):\n    for fruit in fruits:\n        print(i, fruit)
```

구구단 출력

원하는 단수를 입력받아 for루프를 이용해서 출력하시요.

구구단 출력

```
dan = int(input('출력할 단을 입력하시오.'))
```

```
for i in range(1, 10):
```

```
    print('{} X {} = {}'.format(dan, i, dan * i))
```

구구단 출력

전체구구단을 이중 for루프를 이용해서 출력하시오.

구구단 출력

```
for dan in range(2,10):  
    print('{}단'.format(dan))  
    for i in range(1,10):  
        print('{} x {} = {}'.format(dan, i, dan*i))
```

while 구문

```
age = 0
```

```
while age < 5:  
    print('{}살입니다.'.format(age))  
    age += 1
```

```
age = 0
```

```
num = 0
```

```
while age < 5 :  
    while num < 10 :  
        print('{}살입니다.'.format(age))  
        num += 1  
    num = 0  
    age += 1
```

break, continue

```
for row in range(10) :  
    if row == 2 :  
        break  
    print('{}번째 row'.format(row))
```

```
for row in range(10) :  
    if row == 2 :  
        continue  
    print('{}번째 row'.format(row))
```

홀수 출력

for루프, continue, %(나머지 연산자)를 이용해서
0~10 사이의 홀수를 출력하라.

Dictionary

딕셔너리

- 키(key)-값(value)로 연관된 데이터들이 순서 없이 모인 자료형
- 키를 통해서 데이터에 접근 가능
- { } 를 사용하고 키:값 형식으로 저장하고 ','(쉼표)로 구분한다.
- 키는 중복될 수 없다.

```
student = {'name': '홍길동', 'major': '인공지능학과', 'score': 3.5}  
print(student)  
scores = {1: 3.5, 2: 4.0, 3: 4.3, 4: 4.2}  
print(scores)
```

생성

```
dic1 = {} # 빈 딕셔너리 생성
print(dic1)
dic2 = dict() # 생성자를 이용해서 딕셔너리 생성
print(dic2)
dic3 = {'name': 'item', 1: 3.5, False: [1, 2, 3]}
print(dic3)
dic4 = dict(name='홍길동', height=180, age=20)
print(dic4)
```

```
{}
```

```
{}
```

```
{'name': 'item', 1: 3.5, False: [1, 2, 3]}
```

```
{'name': '홍길동', 'height': 180, 'age': 20}
```

아이템 값 가져오기

```
student = {'name': '홍길동', 'major': '정통과', 'score': 3.5}
print(student['name'])
print(student['score'])
scores = {1: 3.5, 2: 4.0, 3: 4.3, 4: 4.2}
print(scores[1])
print(scores[2])
```

홍길동

3.5

3.5

4.0

아이템 수정, 추가

```
student['major'] = '정보통신과'
print(student)
student['grade'] = 4
print(student)
scores[2] = '4.5'
print(scores)
scores[5] = '5.0'
print(scores)
```

{'name': '홍길동', 'major': '정보통신과', 'score': 3.5}

{'name': '홍길동', 'major': '정보통신과', 'score': 3.5, 'grade': 4}

{1: 3.5, 2: '4.5', 3: 4.3, 4: 4.2}

{1: 3.5, 2: '4.5', 3: 4.3, 4: 4.2, 5: '5.0'}

아이템 삭제

```
del student['name']  
print(student)  
  
del scores[1]  
print(scores)
```

```
{'major': '정보통신과', 'score': 3.5}
```

```
{2: 4.0, 3: 4.3, 4: 4.2}
```

Key, Value 가져오기

```
student = {'name': '홍길동', 'major': '정통과', 'score': 3.5}
```

```
print(student.items())
```

```
print(student.keys())
```

```
print(student.values())
```

```
dict_items([('name', '홍길동'), ('major', '정통과'), ('score', 3.5)])
```

```
dict_keys(['name', 'major', 'score'])
```

```
dict_values(['홍길동', '정통과', 3.5])
```

Key, Value 가져오기

```
student = {'name': '홍길동', 'major': '정통과', 'score': 3.5}
```

```
print(list(student.items()))
```

```
print(list(student.keys()))
```

```
print(list(student.values()))
```

```
[('name', '홍길동'), ('major', '정통과'), ('score', 3.5)]
```

```
['name', 'major', 'score']
```

```
['홍길동', '정통과', 3.5]
```

Set

Set

- 중복되지 않은 데이터들이 모인 집합 형태의 자료형
- 순서가 없음
- { } 를 사용하고 아이템들은 ','(쉼표)로 구분

```
fruits = {'banana', 'apple', 'cherry', 'kiwi'}  
print(fruits)  
numbers = {1, 2, 3, 4, 5}  
print(numbers)
```

아이템 수정, 삭제

```
set1 = {1, 2, 3}
set1.add(4)
print(set1)
set1.update([4, 5, 6])
print(set1)
set1.remove(5)
print(set1)
```

{1, 2, 3, 4}

{1, 2, 3, 4, 5, 6}

{1, 2, 3, 4, 6}

아이템 수정, 삭제

```
set1 = set([0, 1, 2, 3, 4, 5, 6])
set2 = set([4, 5, 6, 7, 8, 9, 10])
print(set1 & set2) # 교집합
print(set1.intersection(set2))
print(set1 | set2) # 합집합
print(set1.union(set2))
print(set1 - set2) # 차집합
print(set1.difference(set2))
```

{4, 5, 6}

{4, 5, 6}

{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10}

{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10}

{0, 1, 2, 3}

{0, 1, 2, 3}

packing, unpacking

packing, unpacking

```
nums = 1,2,3 # packing
print(nums)
print(type(nums))
num1, num2, num3 = nums # unpacking
print(num1, num2, num3)
```

```
(1, 2, 3)
<class 'tuple'>
1 2 3
```

unpacking

```
nums = [1, 2, 3, 4, 5]
num1, num2, num3, num4, num5 = nums
print(num1, num2, num3, num4, num5)
num1, num2, *num3 = nums
print(num1, num2, num3)
*num1, num2, num3 = nums
print(num1, num2, num3)
```

1 2 3 4 5

1 2 [3, 4, 5]

[1, 2, 3] 4 5

unpacking

```
nums = (1, 2, 3)
num1, num2, num3 = nums
print(num1, num2, num3)
num1, num2, _ = nums
print(num1, num2)
_, _, num3 = nums
print(num3)
```

```
1 2 3
1 2
3
```

unpacking

```
student = {'name': '홍길동', 'major': '정보통신', 'grade': 3}
a, b, c = student
print(a, b, c)

set1 = {1, 2, 3, 4}
a, *b, c = set1
print(a, b, c)
```

name major grade

1 [2, 3] 4

함수

함수 Function

- 프로그램의 기본적인 구성요소
- 어떤 작업을 수행하는 문장의 그룹
- 하나의 프로그램은 여러 개의 함수들이 모여서 이루어진다
- 프로그램에서 레고 블록과 같은 요소
- 가독성의 증대, 유지관리가 쉽다.

함수 정의

```
# 매개변수가 없는 함수  
def 함수이름():  
    코드
```

```
def hello():  
    print('Hello World!')
```

hello() # 함수호출

함수 정의

```
# 매개변수가 있는 함수  
def 함수이름(매개변수):  
    코드
```

```
def hello1(name):  
    print('Hello World!', name)  
  
hello('홍길동')  
hello('트와이스')# 함수호출
```

함수 정의

```
# 매개변수가 복수인 함수  
def 함수이름(매개변수1, 매개변수2):  
    코드
```

```
def add(num1, num2):  
    print(num1 + num2)  
  
add(2, 3)  
add(3.5, 3.14)  
add('Hello ', 'World!')
```

함수 정의

```
# 반환값이 있는 함수  
def 함수이름():  
    코드  
    return 반환값
```

```
def hello2():  
    return 'Hello World!'  
  
print(hello2())
```

함수 정의

```
# 반환값과 매개변수가 있는 함수
def 함수이름(매개변수):
    코드
    return 반환값
```

```
def hello3(name):
    return 'Hello ' + name

print(hello3('홍길동'))
```

함수 정의

```
# 가변 매개변수가 있는 함수, *를 사용
def 함수이름(매개변수1, *매개변수2):
    코드
    return 반환값
```

```
def hello4(greeting, *names):
    for name in names:
        print(greeting, name)
```

```
hello4('안녕', "진", "슈가", "제이홉", "RM", "지민")
```

```
# 가변 매개변수는 하나만 사용 가능
# 가변 매개변수는 매개변수중 마지막에만 위치 가능
```

함수 정의

```
# 함수 호출시 매개변수명을 사용
def 함수이름(매개변수):
    코드
    return 반환값
```

```
def function(first, second, third):
    return first + second + third

print(function(3,5,7))
print(function(first=3, second=5, third=7))
print(function(second=5, third=7, first=3))
```

실습

두 수와 연산자(문자열형)를 입력받아서
사칙연산을 수행하는 함수를 만드시오.

실습

```
def calc(num1, op, num2):  
    if op == '+':  
        result = num1 + num2  
    elif op == '-':  
        result = num1 - num2  
    elif op == '*':  
        result = num1 * num2  
    elif op == '/':  
        result = num1 / num2  
    else : result = 0  
    return result  
  
print(calc(10, '+', 5))  
print(calc(10, '-', 5))  
print(calc(3, '*', 5))  
print(calc(2.5, '/', 3.5))
```

함수 정의

```
# 매개변수명의 기본값  
def 함수이름(매개변수=기본값):  
    return 반환값
```

```
def function1(first=1, second=3, third=5):  
    return first + second + third  
  
print(function1())  
print(function1(1, 2, 3))  
print(function1(1, 2))  
print(function1(1))  
print(function1(first=1))  
print(function1(second=5))  
print(function1(first=1, third=5))  
print(function1(first=1, second=5))
```

함수 정의

```
# 매개변수명의 기본값
```

```
def 함수이름(매개변수=기본값):  
    return 반환값
```

```
def function1(first, second, third=5 ):  
    return first + second + third
```

```
print(function1(1, 2, 3))
```

```
print(function1(1, 2))
```

```
print(function1(first=1, second=2, third=10))
```

```
print(function1(1, second=5))
```

```
print(function1(second=5, first=2))
```

실습

점수를 입력받아 평점을 구하는 함수를 만드시오.

calcGrade(95)를 실행하면 'A'를 반환하는 식으로 만드시오.

실습

```
def calcGrade(score):  
    if score >= 90:  
        result = 'A'  
    elif score >= 80:  
        result = 'B'  
    elif score >= 70:  
        result = 'C'  
    elif score >= 60:  
        result = 'D'  
    else:  
        result = 'F'  
    return result  
  
print(calcGrade(89))  
print(calcGrade(95))  
print(calcGrade(77))  
print(calcGrade(50))
```

반환형이 콜렉션일때 unpacking

```
# packing을 사용한 다중 반환값
def 함수이름():
    코드
    return 반환값1, 반환값2, 반환값3
```

```
def function():
    return 1, "Hello", True

a, b, c = function()
print(a, b, c)
a = function()
print(a)
```

반환형이 콜렉션일때 unpacking

```
def ret_list():  
    return [1, 2]  
l = ret_list()  
print(l)  
n1, n2 = ret_list()  
print(n1, n2)  
n1, _ = ret_list()  
print(n1)
```

반환형이 콜렉션일때 unpacking

```
def ret_tuple():  
    return (1,2)  
t = ret_tuple()  
print(t)  
n1, n2 = ret_tuple()  
print(n1, n2)  
n1,_ = ret_tuple()  
print(n1)
```

실습

젯수와 피젯수(정수형)을 매개변수로 전달받아 몫과 나머지를 튜플로 반환하는 함수를 만드시오.

`divide(15, 4)` 실행하면 `(3, 3)`을 반환하는 식으로 만드시오.

실습

```
def divide(num1, num2):  
    q = num1 // num2  
    r = num1 % num2  
    return (q, r)
```

```
t = divide(25, 6)  
print(t)  
q, r = divide(25, 6)  
print(q, r)  
q, _ = divide(25, 6)  
print(q)
```

객체지향 프로그래밍

객체지향의 용어

- 객체 : 데이터, 행위, 아이덴티티를 가지고 있는 것.
- 클래스 : 클래스란 객체를 생성하는 틀이다.
- 캡슐화 : 행위와 상태를 포장하고 외부에 노출할 것과 감출 것을 결정하는 것
- 상속 : 코드의 재사용성과 계층구조의 표현
- 다형성 : 캡슐화, 상속과 함께 동작 함으로써 객체-지향 프로그램의 흐름 제어를 단순화

인스턴스(instance)

인스턴스(instance)

- 클래스로부터 만들어지는 각각의 객체를 그 클래스의 인스턴스라고 한다.
- 서로 다른 인스턴스는 서로 다른 속성 값을 가진다.

클래스의 정의

```
class Person:  
    def hello(self):  
        print('Hello')
```

```
person = Person()  
person.hello()  
person1 = Person()  
person1.hello()
```

속성 정의

```
class Person:
    def __init__(self):
        self.hi = 'Hello'

    def hello(self):
        print(self.hi)

person = Person()
person.hello()
print(person.hi)
```

속성 정의와 초기화

```
class Person:
    def __init__(self, n, a):
        self.name = n
        self.age = a

    def hello(self):
        print('Hello {}'.format(self.name))
        print('당신은 {}살입니다.'.format(self.age))

person = Person('홍길동', 20)
person.hello()
```

클래스 정의 실습

- 학생 클래스를 정의하라.
- 학년, 반, 이름의 세 가지 속성을 가진다.
- 속성은 생성자를 통해서 설정된다.
- '몇학년 몇반 누구입니다.' 라고 출력하는 introduce 라는 메소드를 정의하라.

비공개 속성 정의

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.__age = age

    def hello(self):
        print('Hello {}'.format(self.name))
        print('당신은 {}살입니다.'.format(self.__age))

person = Person('홍길동', 20)
person.hello()
print(person.__age) # 에러
person.__age = 100 # 에러
```

비공개 속성 사용

```
class Person:
    def __init__(self, name, age):
        self.name = name
        if 0 <= age <= 20: self.__age = age
        else: self.__age = 0

    def inc_age(self):
        self.__age += 1

    def info(self):
        print(self.__age)

person = Person('홍길동', 20)
person.inc_age()
person.info()
```

클래스 속성 사용

```
class Person:
    count = 0

    def __init__(self, name):
        self.name = name
        Person.count += 1

person1 = Person('홍길동')
print(person1.count)
person2 = Person('허균')
print(person2.count)
```

클래스(class) 메서드 정의

- 클래스메서드는 클래스에서 바로 호출할 수 있는 메서드
- 클래스명.메서드명()으로 호출
- 메서드 위에 @classmethod를 붙임
- 매개변수에 self를 사용하지 않음

```
class 클래스이름:
```

```
    @classmethod
```

```
    def 메서드명(cls, 매개변수1, 매개변수2):
```

```
        코드
```

정적(static) 메서드 정의

- 정적메서드는 클래스에서 바로 호출할 수 있는 메서드
- 클래스명.메서드명()으로 호출
- 메서드 위에 @staticmethod를 붙임
- 매개변수에 self를 사용하지 않음

```
class 클래스이름:  
    @staticmethod  
    def 메서드명(매개변수1, 매개변수2):  
        코드
```

정적(static) 메서드사용

```
class Math:
    @staticmethod
    def add(a, b):
        return a + b

    @staticmethod
    def sub(a, b):
        return a - b

print(Math.add(4,5))
print(Math.sub(9,5))
```

클래스 정의 실습

- 사각형을 관리하는 클래스를 정의하라.
- 가로, 세로를 저장하는 속성을 정의하라.
- 가로, 세로로 면적을 반환하는 메소드를 정의하라
- 가로, 세로를 각각 두배로 만드는 메서드를 정의하라

클래스 정의 실습

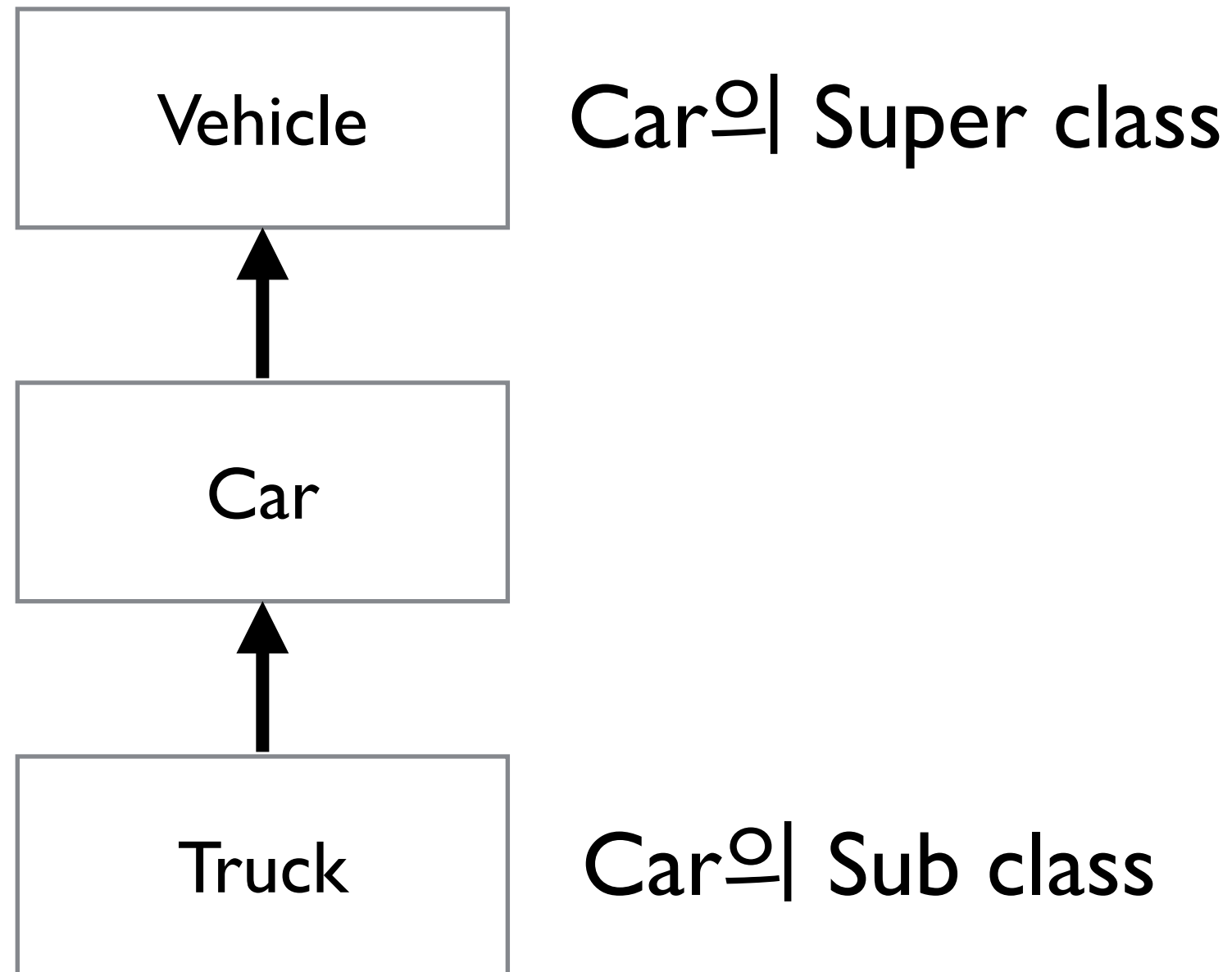
```
class Rectangle:
    def __init__(self, width, height):
        self.width = width
        self.height = height

    def area(self):
        return self.width * self.height

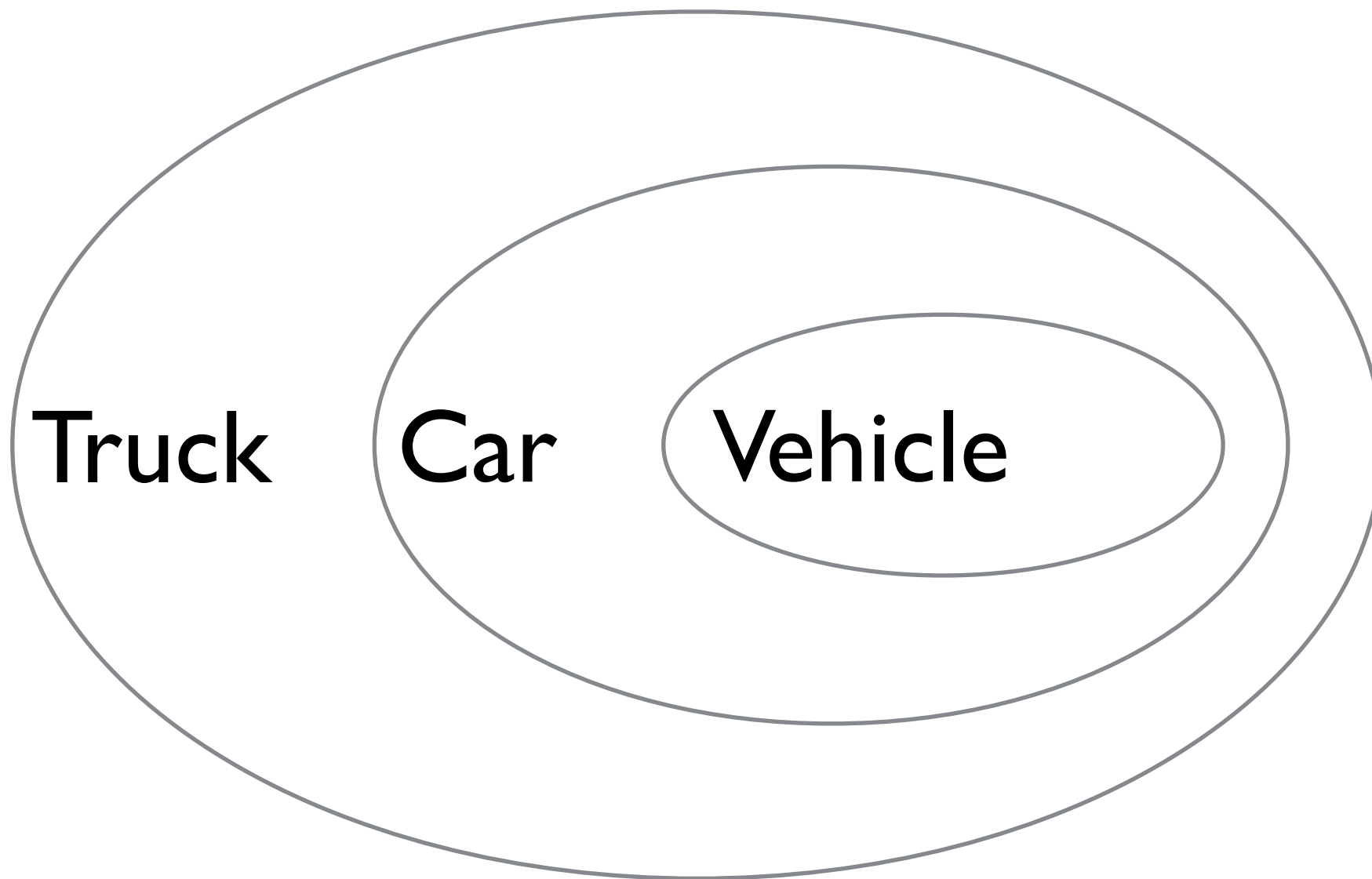
    def double(self):
        self.width *= 2
        self.height *= 2

rect = Rectangle(10, 20)
print(rect.area())
rect.double()
print(rect.width, rect.height)
```

클래스의 상속



클래스의 상속



클래스의 상속

```
class Vehicle:
    def __init__(self, speed):
        self.speed = speed

    def speed_up(self):
        self.speed += 10

    def speed_dn(self):
        self.speed -= 10
```

클래스의 상속

```
class Car(Vehicle):  
    def __init__(self, speed, wheels, seats):  
        Vehicle.__init__(self, speed)  
        self.wheels = wheels  
        self.seats = seats  
  
    def info(self):  
        print(self.speed, self.wheels, self.seats)  
  
car = Car(100, 4, 4)  
car.speed_up()  
car.info()
```

클래스 상속 실습

- Car class를 상속받아서 Truck class를 정의하라.
- 적재량을 관리하는 loadage 변수를 정의하라.
- ‘load’를 화면에 인쇄하는 load 메소드를 정의하라.
- ‘unload’를 화면에 인쇄하는 unload 메소드를 정의하라.

클래스 상속 실습

```
class Truck(Car):
    def __init__(self, speed, wheels, seats, loadage):
        Car.__init__(self, speed, wheels, seats)
        self.loadage = loadage

    def load(self):
        print('load')

    def unload(self):
        print('unload')

truck = Truck(100, 6, 2, 30)
truck.load()
truck.unload()
truck.info()
```

메서드 재정의

```
class Truck(Car):
    def __init__(self, speed, wheels, seats, loadage):
        Car.__init__(self, speed, wheels, seats)
        self.loadage = loadage

    def load(self):
        print('load')

    def unload(self):
        print('unload')

    def info(self):
        print(self.speed, self.wheels, self.seats, self.loadage)

truck = Truck(100, 6, 2, 30)
truck.load()
truck.unload()
truck.info()
```

객체지향의 동작방식

메모리란?

플립플롭(Flip-Flop)

- 출력값을 유지(저장)하는 회로
- 기본적으로 1비트의 정보를 저장하는 회로
- 0V의 경우 Low라고 (논리적으로 FALSE, 이진수로 0으로 표기)
- 5V(3.3V, 1.8V)의 경우 High (논리적으로 TRUE, 이진수로 1로 표기)

bit ☐

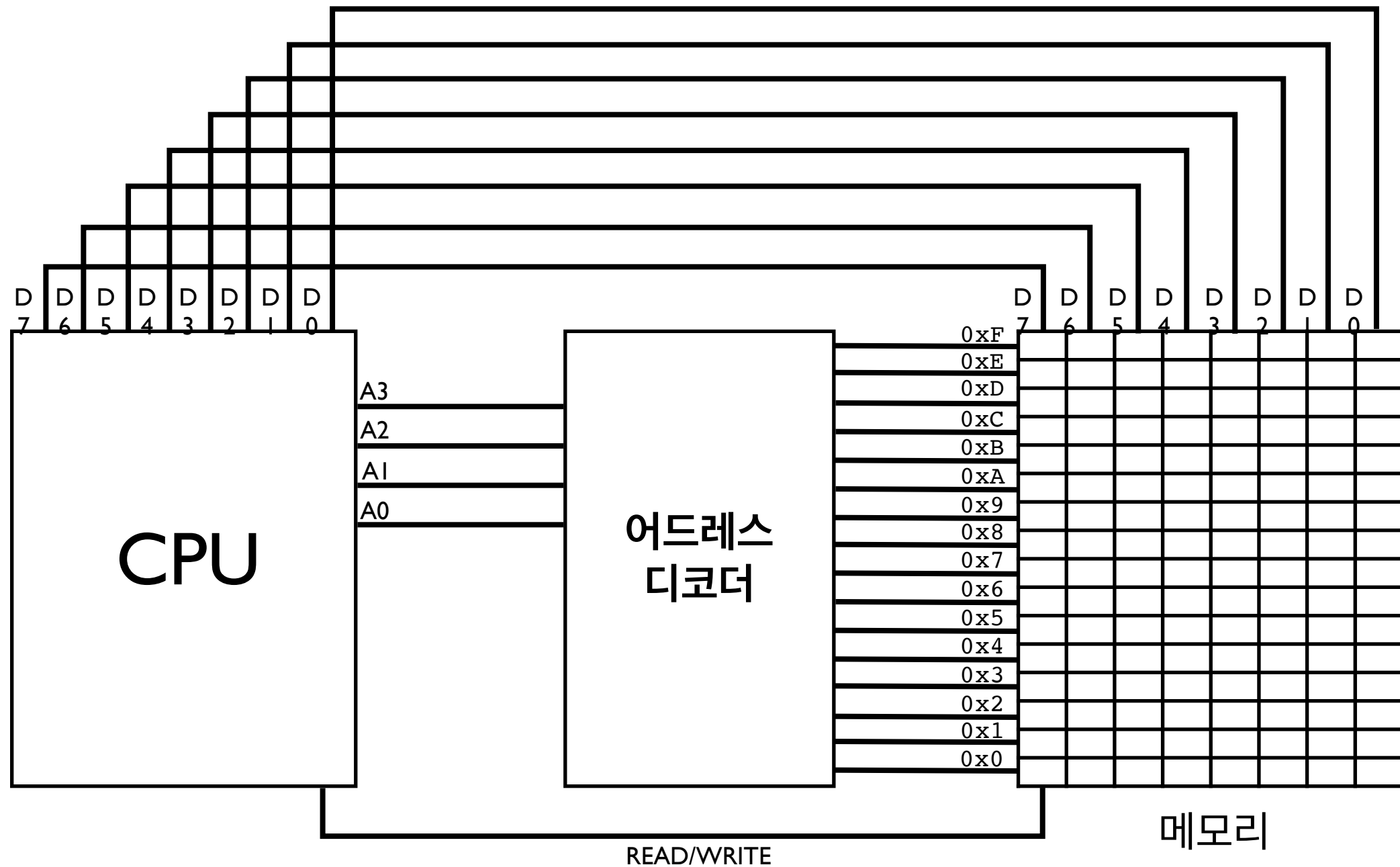
nibble

--	--	--	--

byte

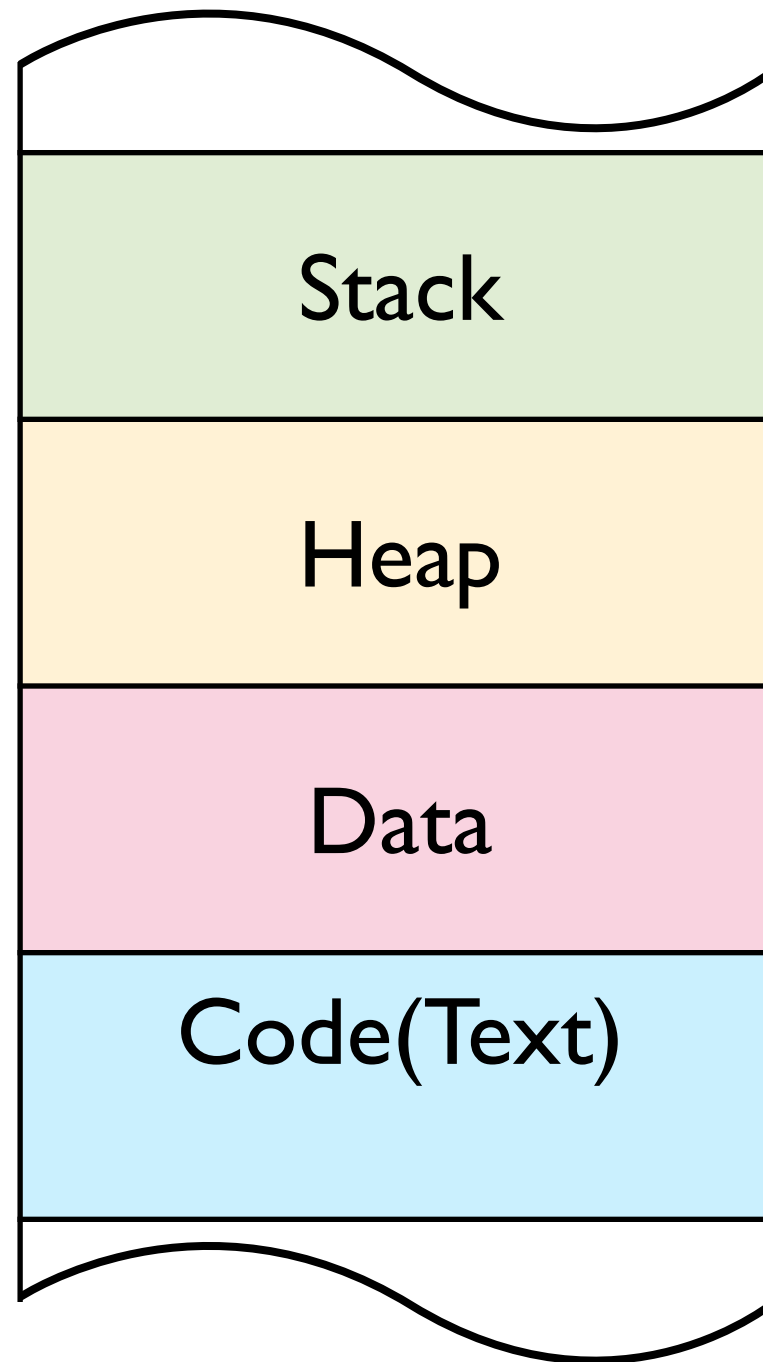
--	--	--	--	--	--	--	--

메모리주소란?

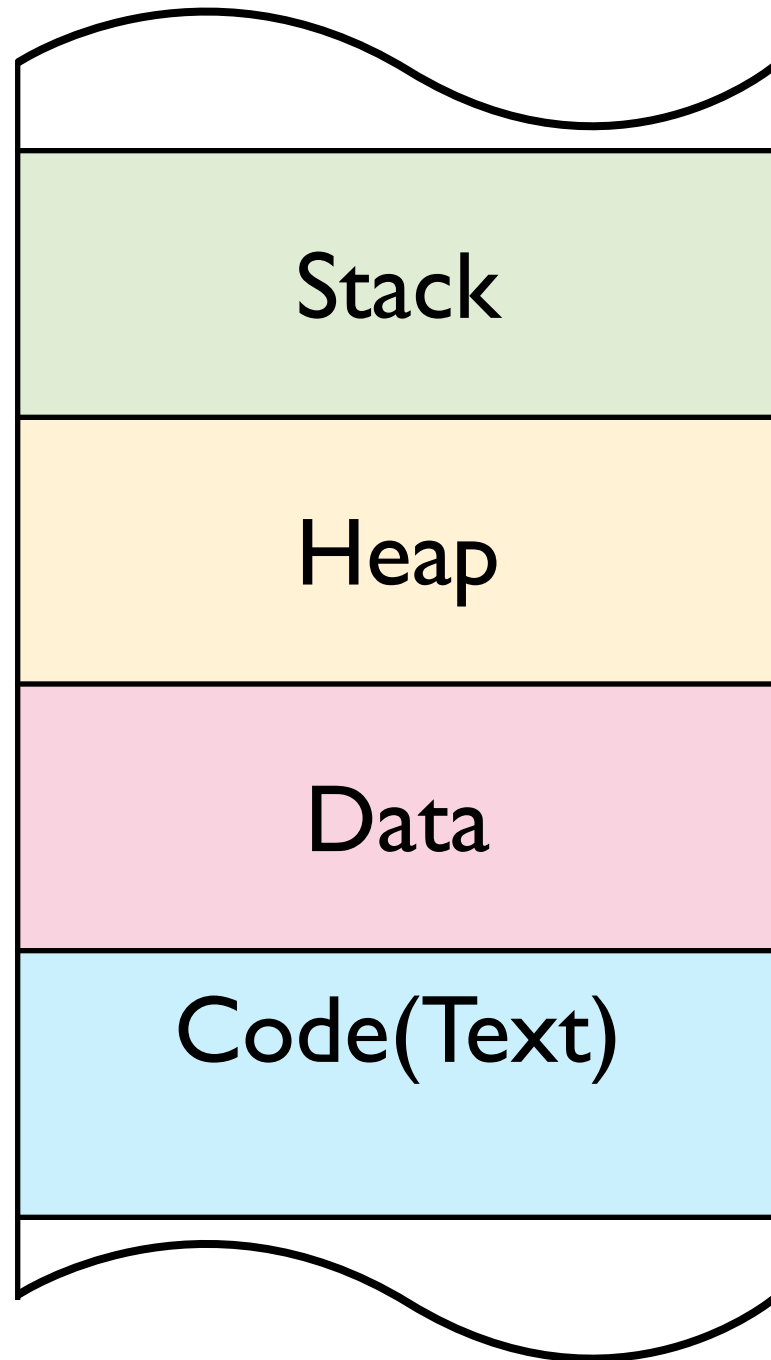


OS에 의한 메모리 구분

OS에 의한 메모리 구분

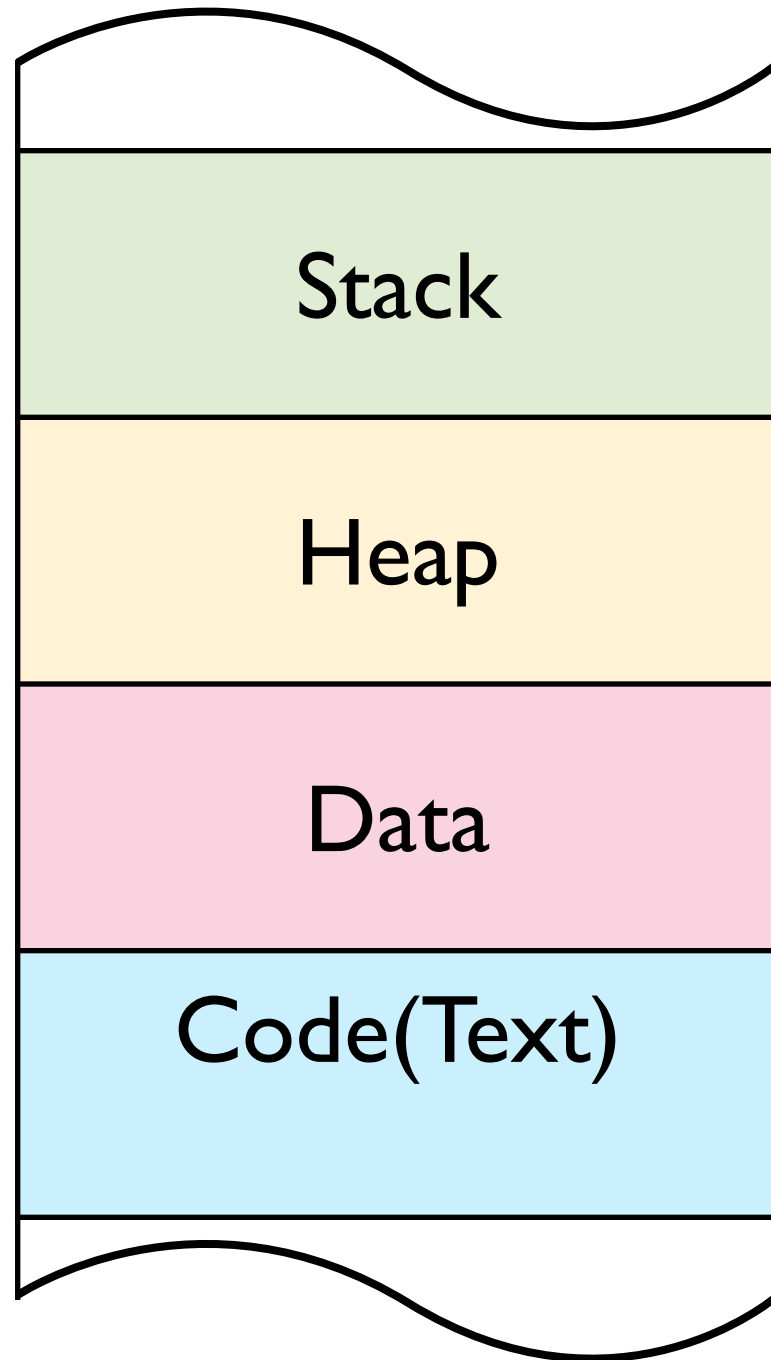


OS에 의한 메모리 구분



프로그램코드가 저장

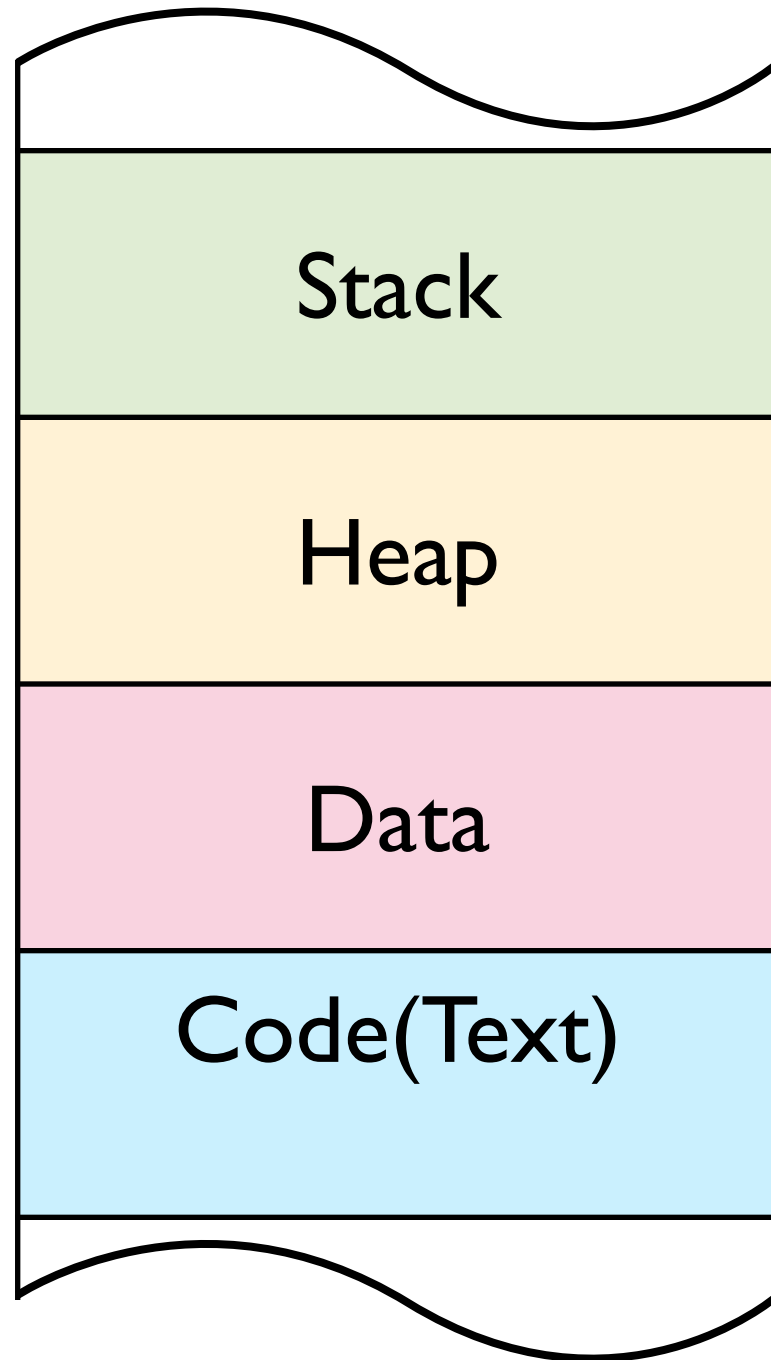
OS에 의한 메모리 구분



전역변수와 정적변수가 저장

프로그램코드가 저장

OS에 의한 메모리 구분

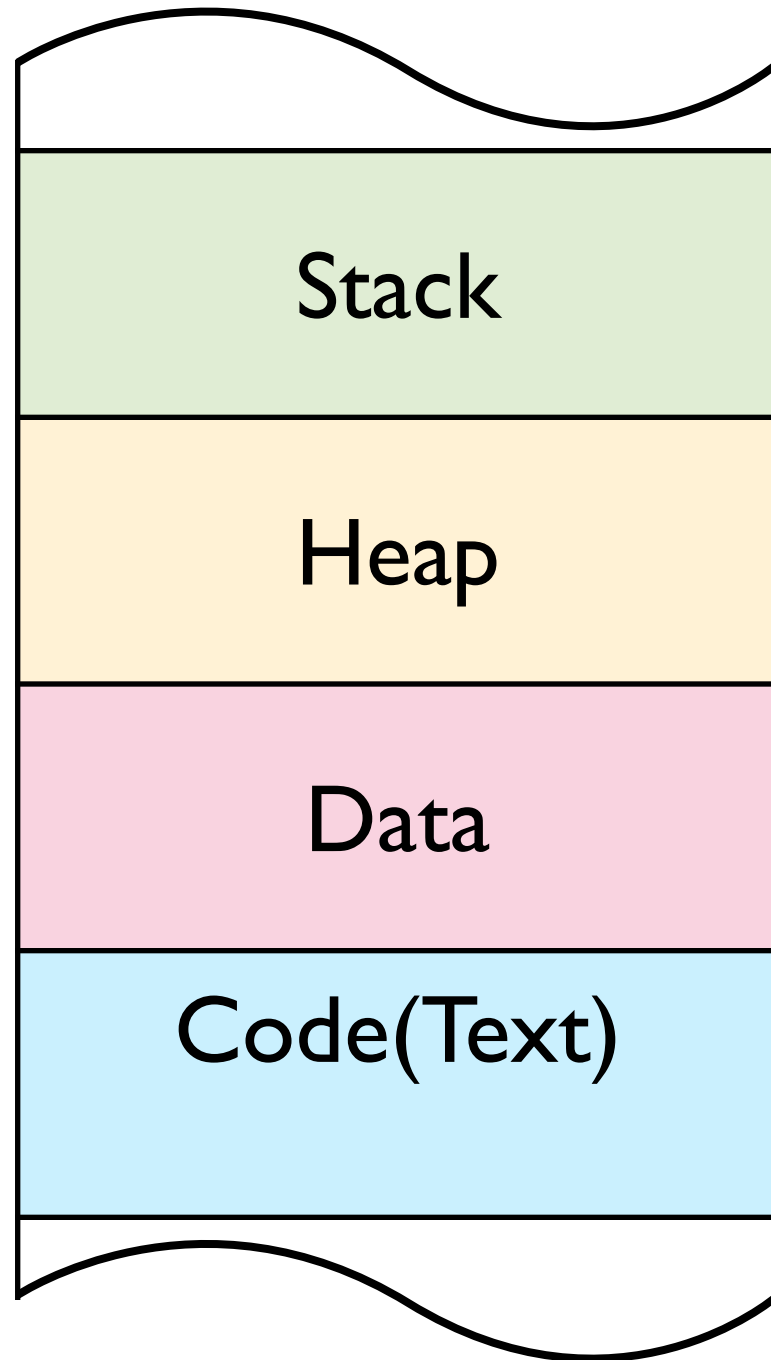


동적할당을 위한 공간

전역변수와 정적변수가 저장

프로그램코드가 저장

OS에 의한 메모리 구분



지역변수와 매개변수가 저장

동적할당을 위한 공간

전역변수와 정적변수가 저장

프로그램코드가 저장

클래스의 선언

```
class Vehicle:
```

```
    def __init__(self, wheels, seats, engine):
```

```
        self.wheels = wheels
```

```
        self.seats = seats
```

```
        self.engine = engine
```

```
    def drive(self):
```

```
        print('Run')
```

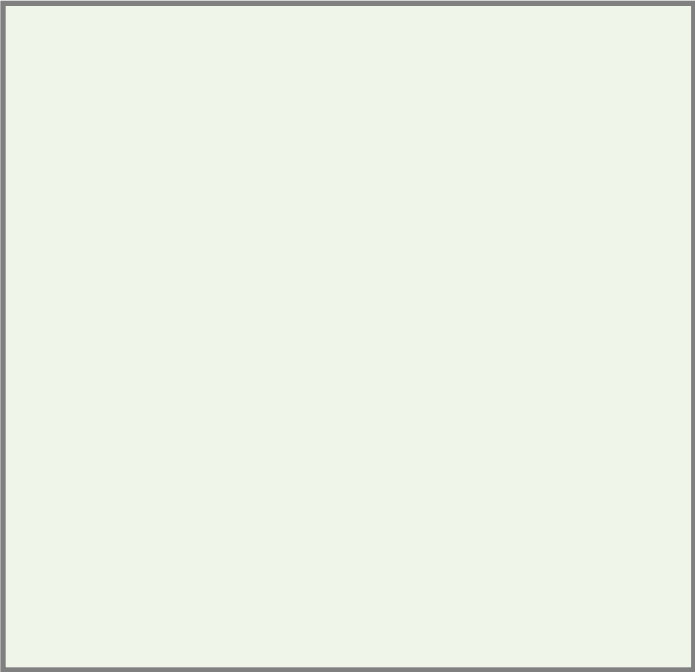
인스턴스 생성 및 사용

```
bike = Vehicle(2, 1, 2)
print('bike.wheels = ', bike.wheels)
print('bike.seats = ', bike.seats)
bike.wheels = 3
print('bike.wheels = ', bike.wheels)
bike.drive()
```

```
car = Vehicle(4, 4, 4)
print('car.wheels = ', car.wheels)
print('car.seats = ', bike.seats)
car.drive()
```

```
copy_bike = bike
copy_bike.wheels = 4
print('bike.wheels = ', bike.wheels)
```

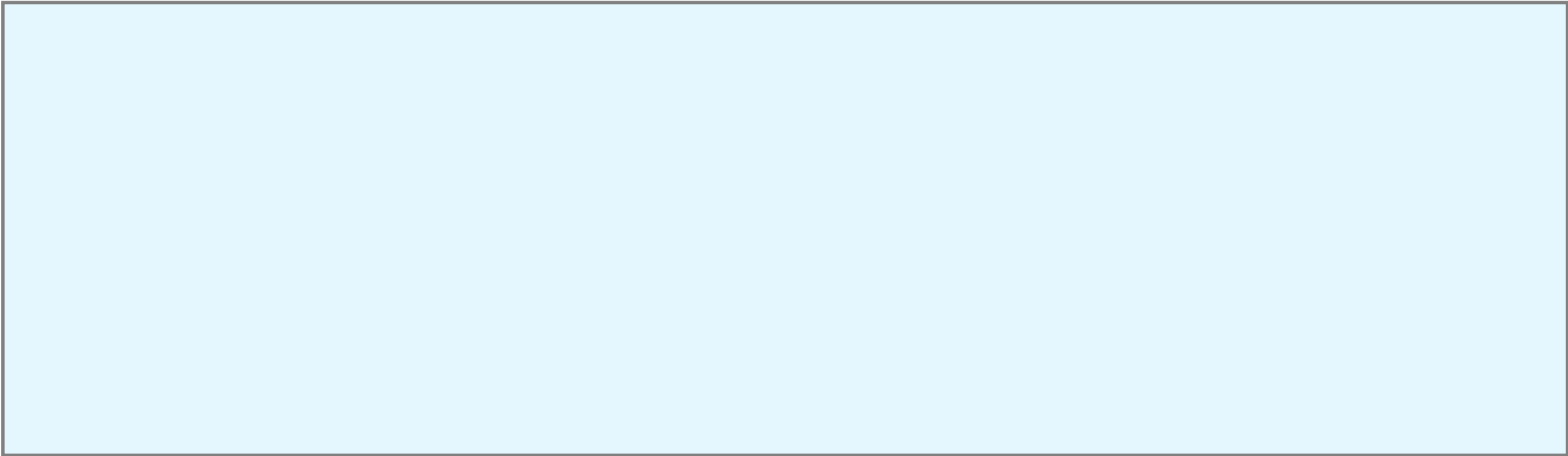
Stack



Heap

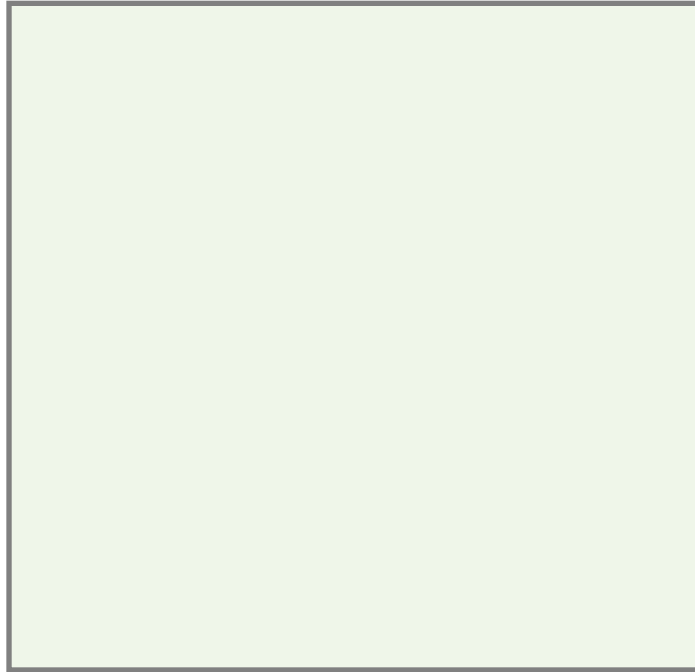


Code



```
bike = Vehicle(2, 1, 2)
```

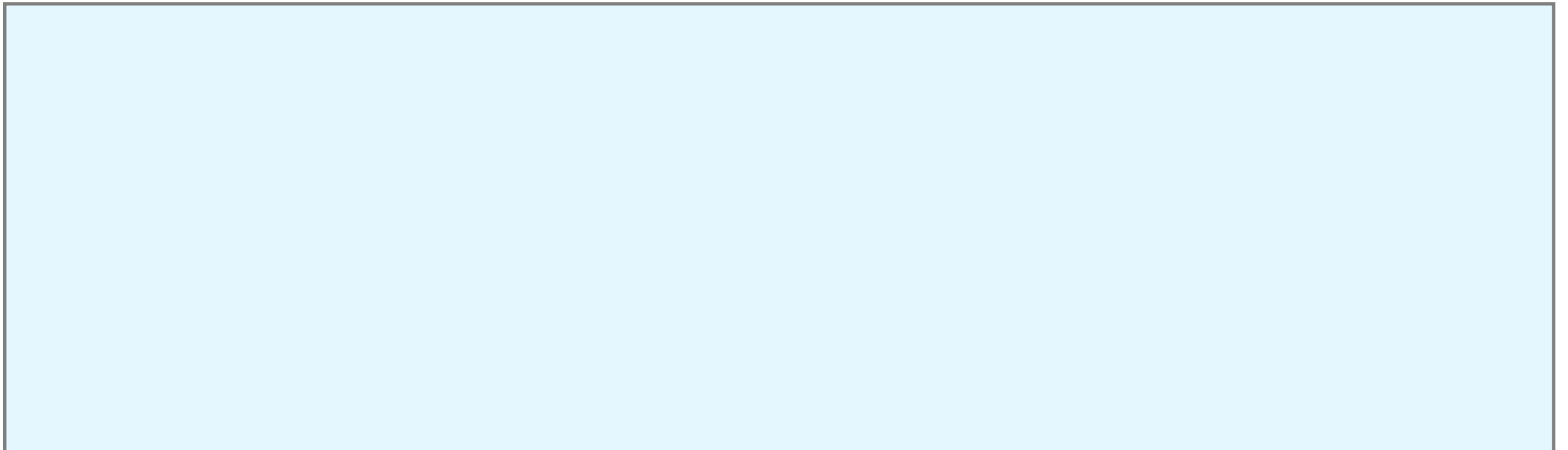
Stack



Heap

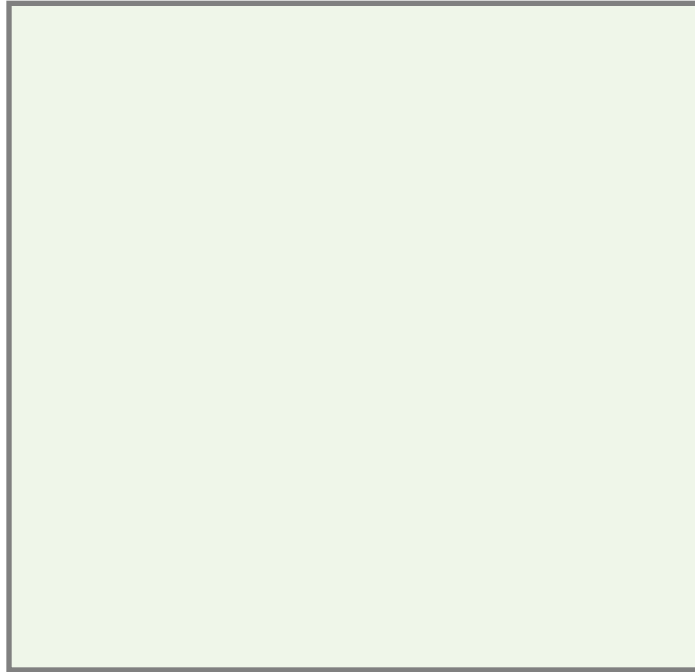


Code



```
bike = Vehicle(2, 1, 2)
```

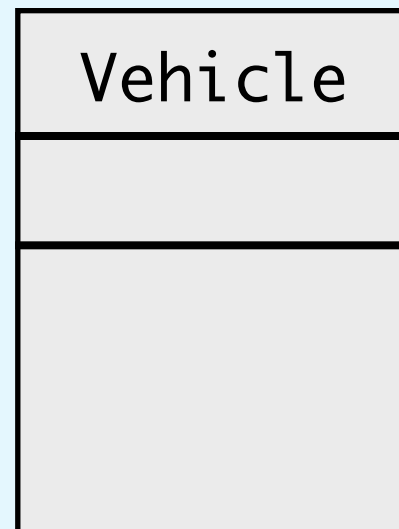
Stack



Heap

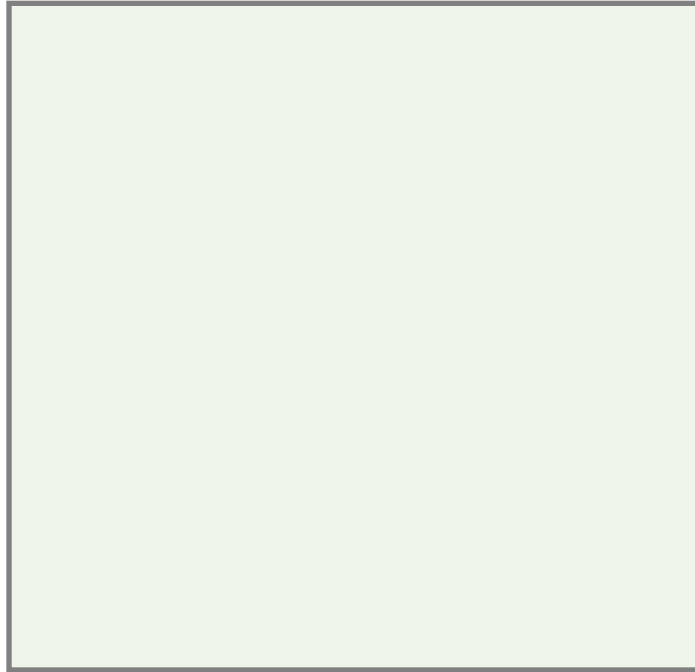


Code

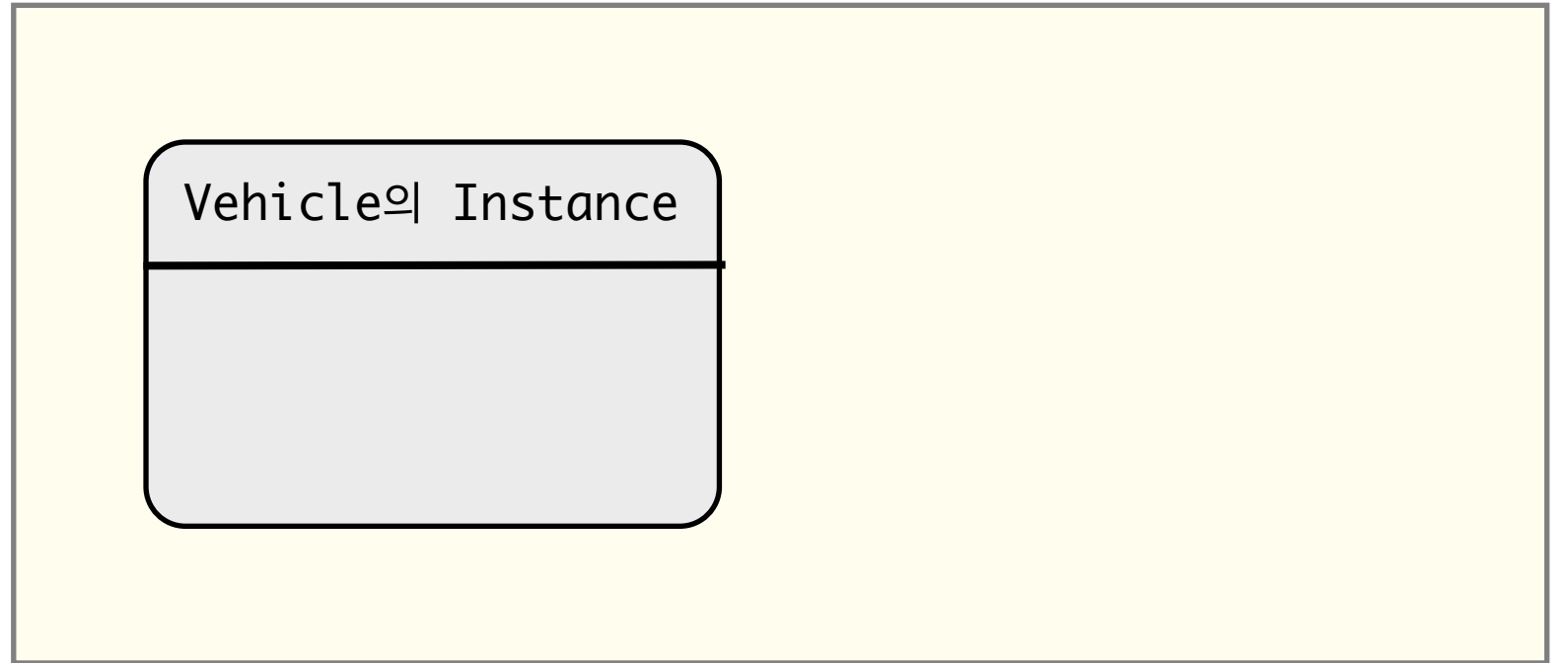


```
bike = Vehicle(2, 1, 2)
```

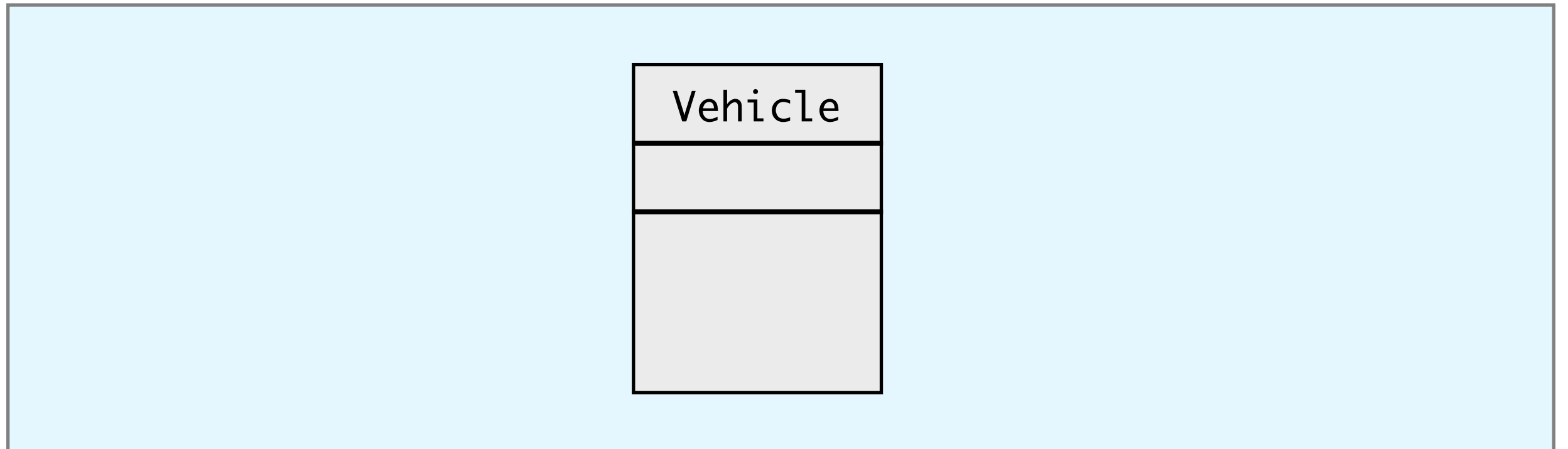
Stack



Heap

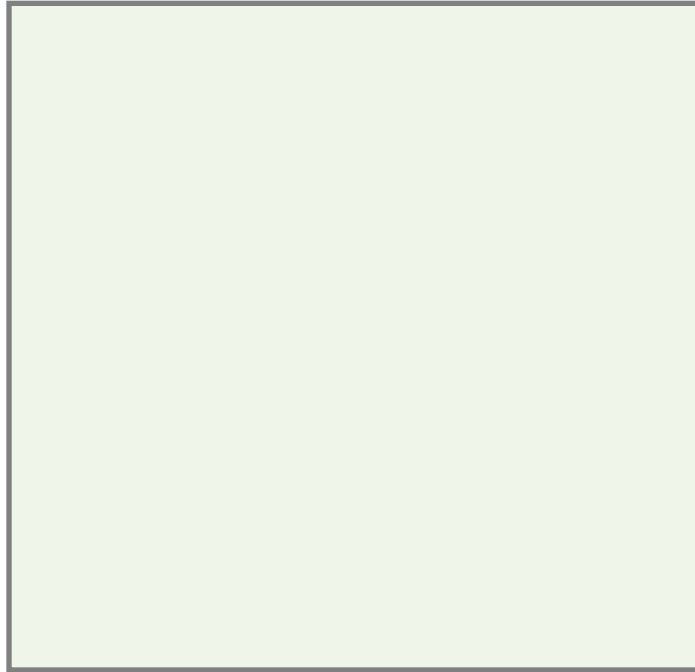


Code

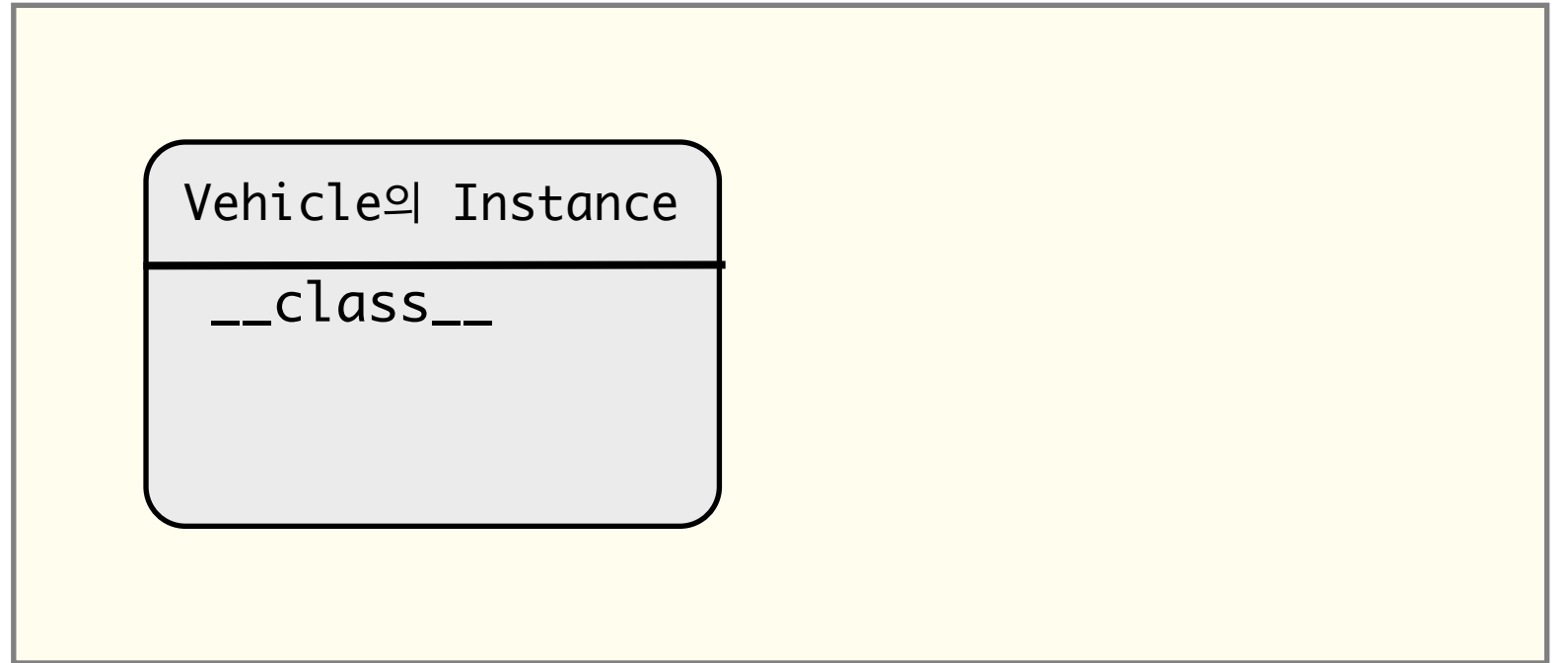


```
bike = Vehicle(2, 1, 2)
```

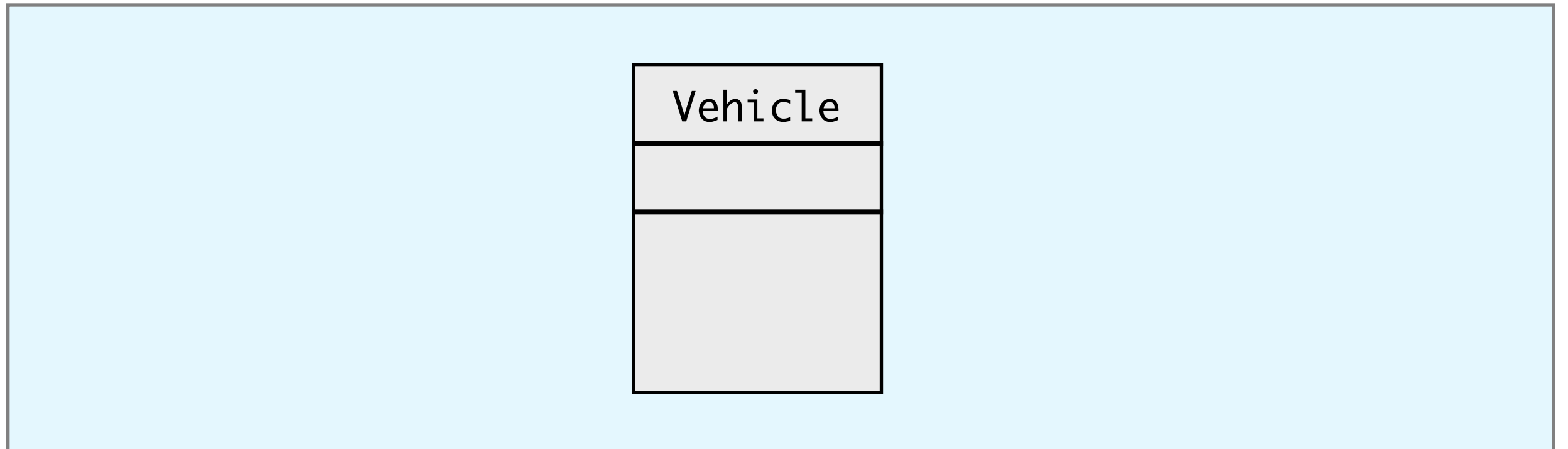
Stack



Heap

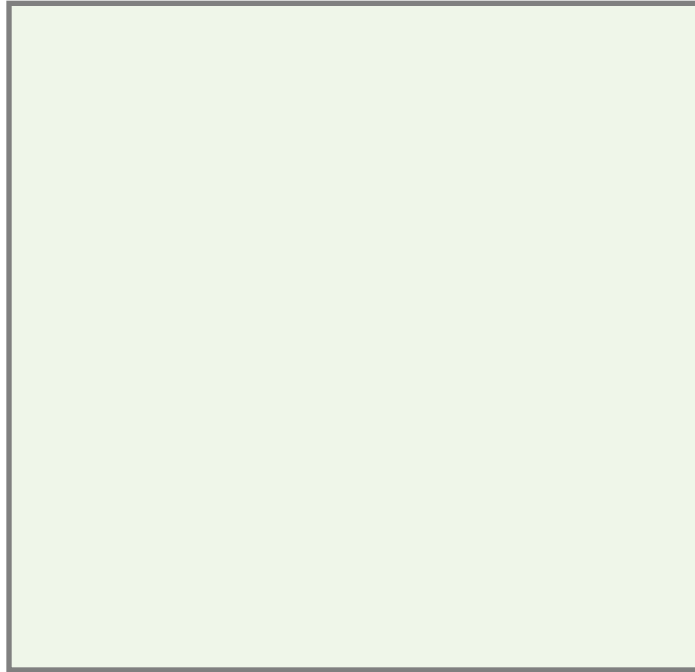


Code

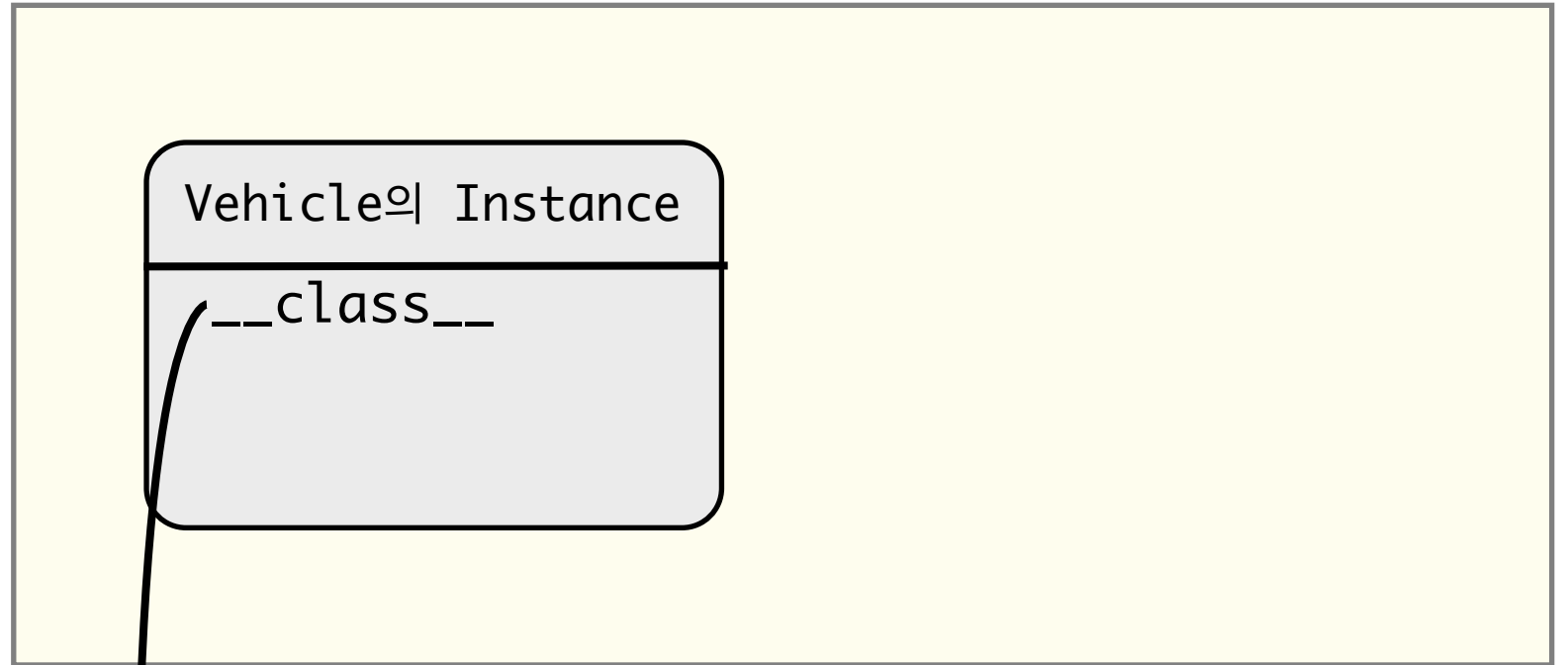


```
bike = Vehicle(2, 1, 2)
```

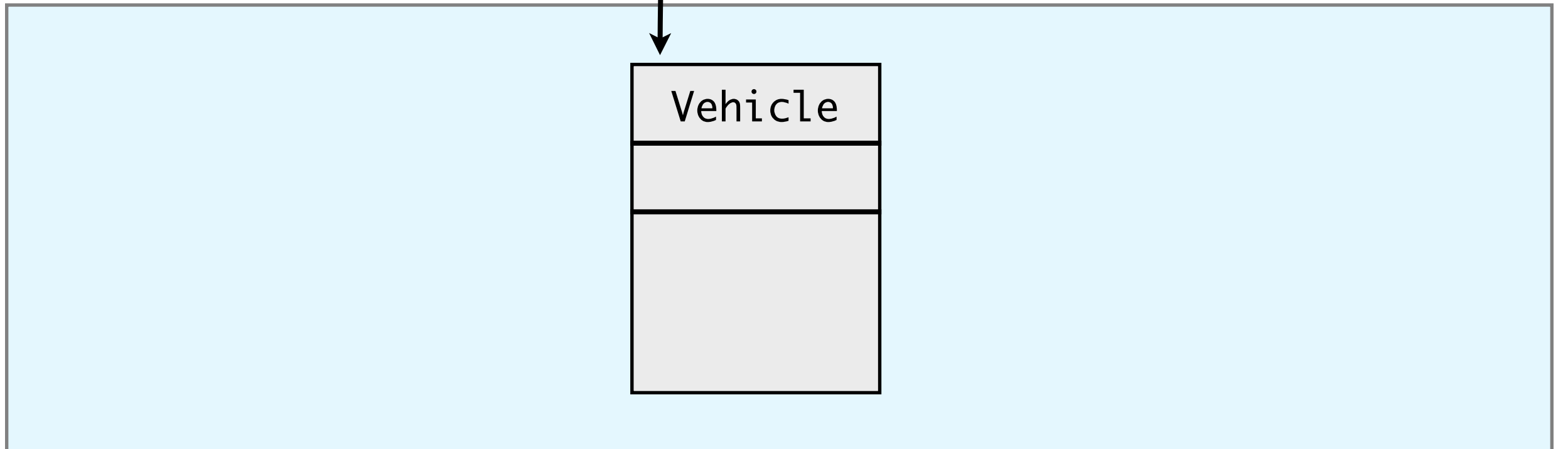
Stack



Heap

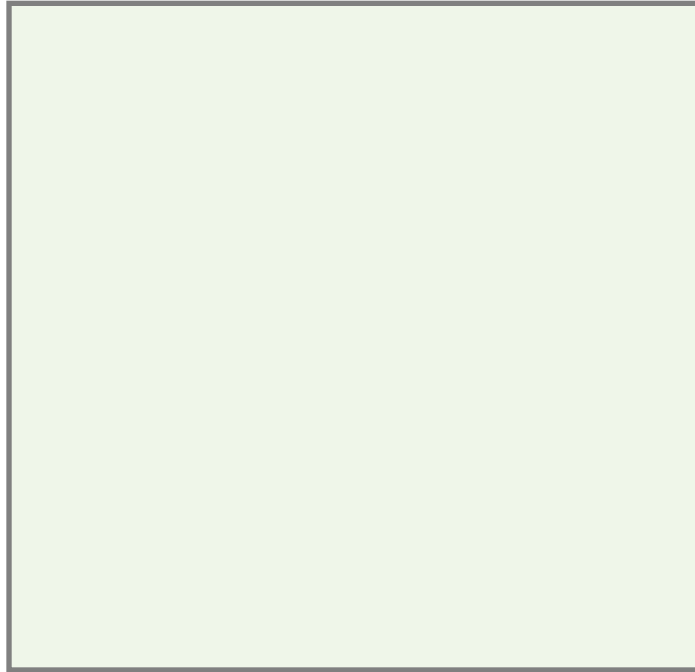


Code

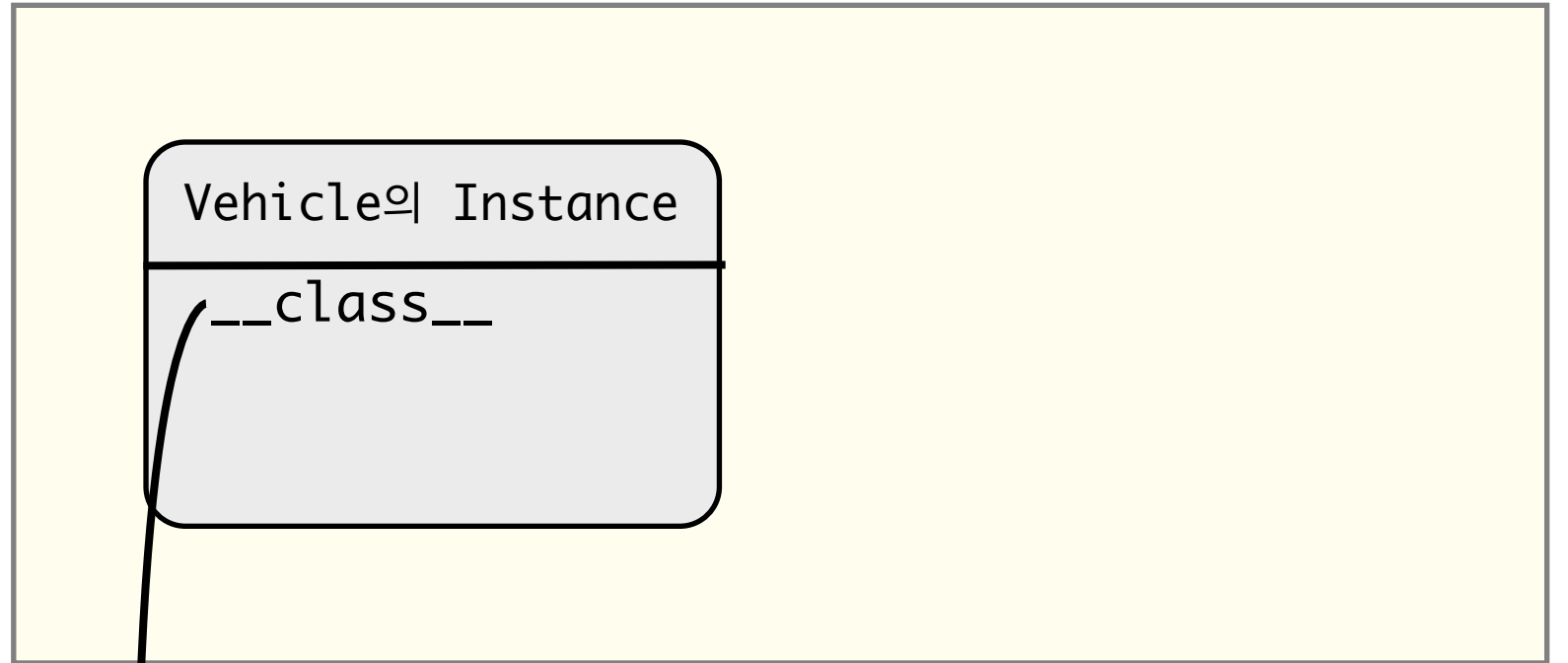


```
bike = Vehicle(2, 1, 2)
```

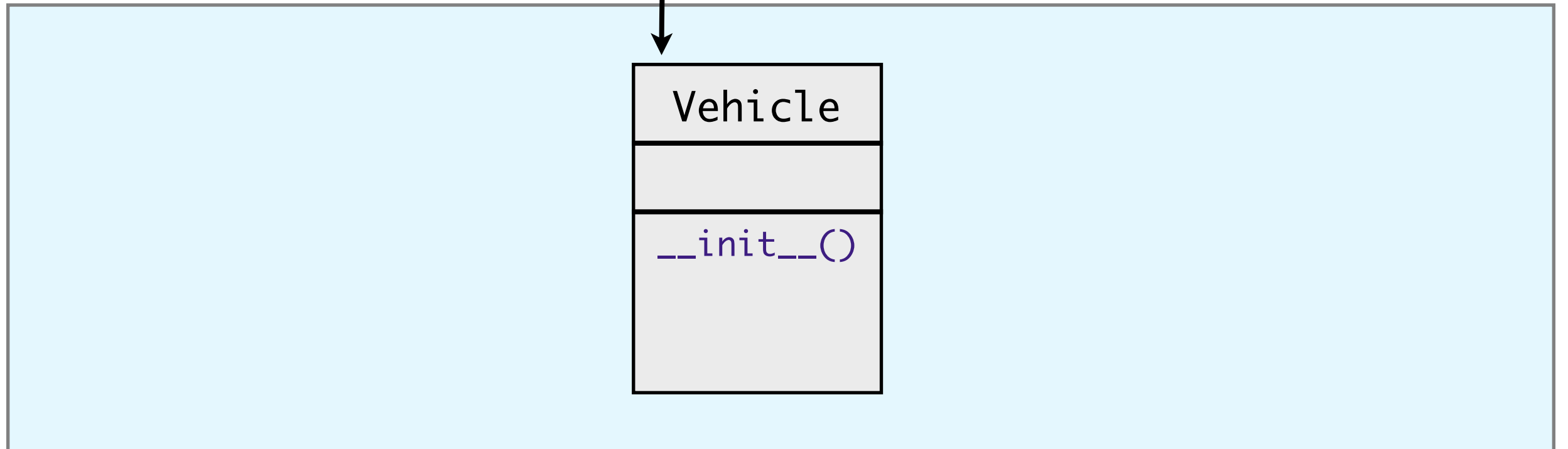
Stack



Heap

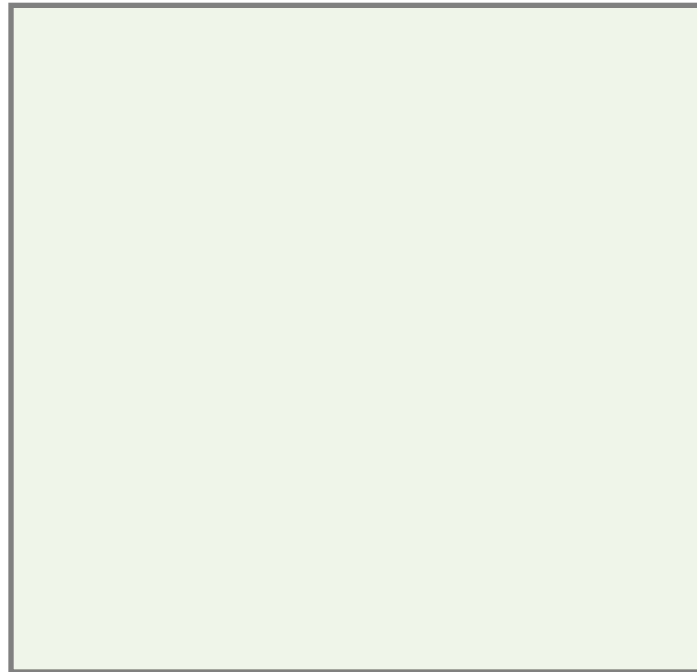


Code

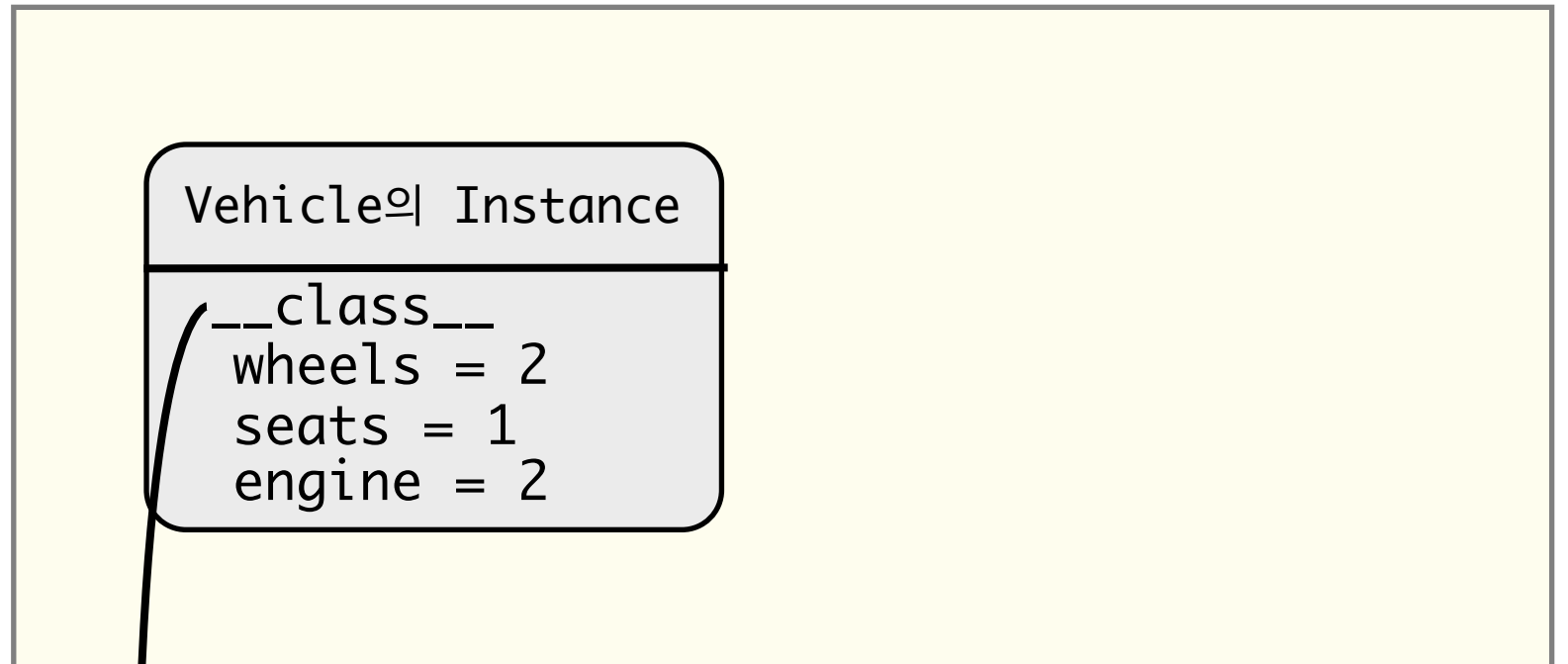


```
bike = Vehicle(2, 1, 2)
```

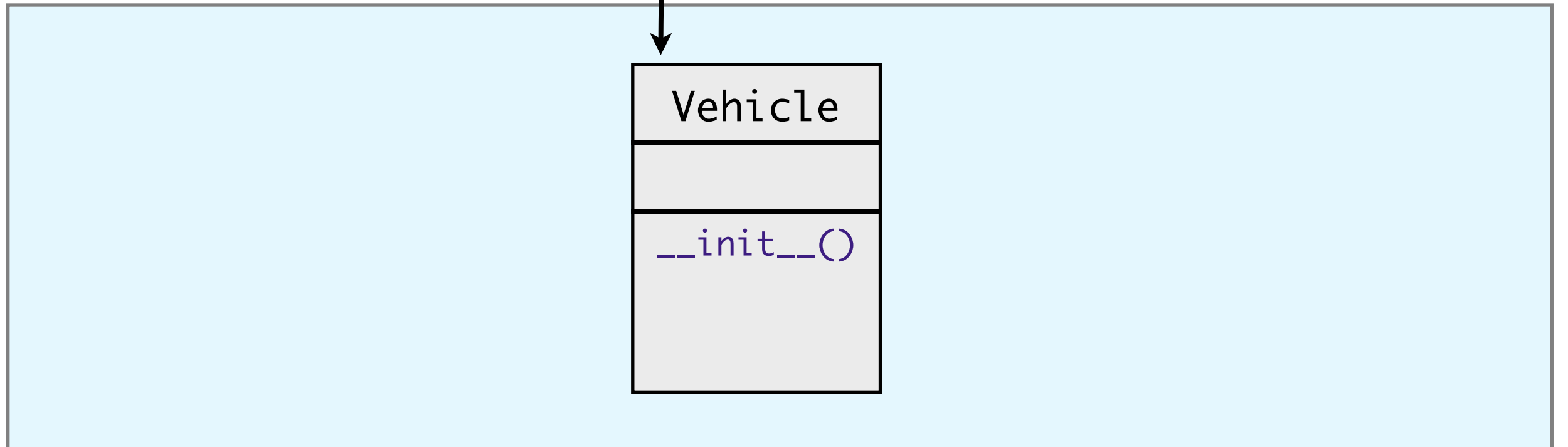
Stack



Heap

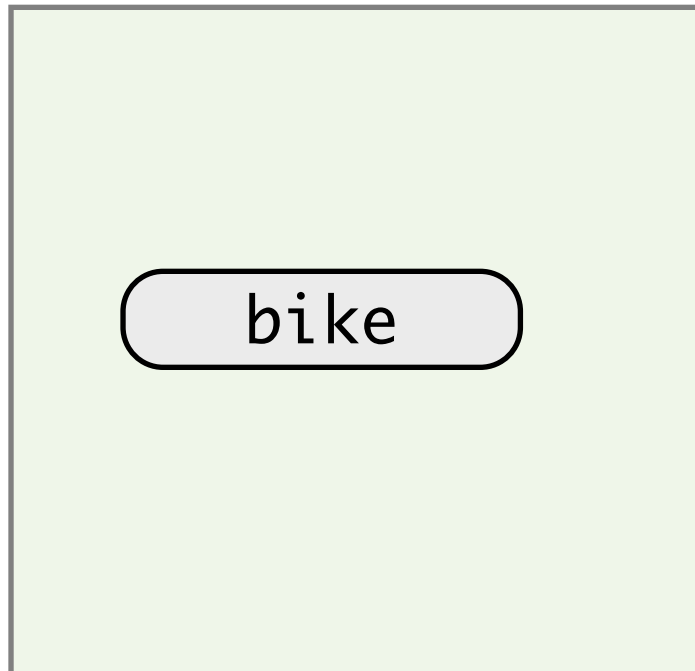


Code

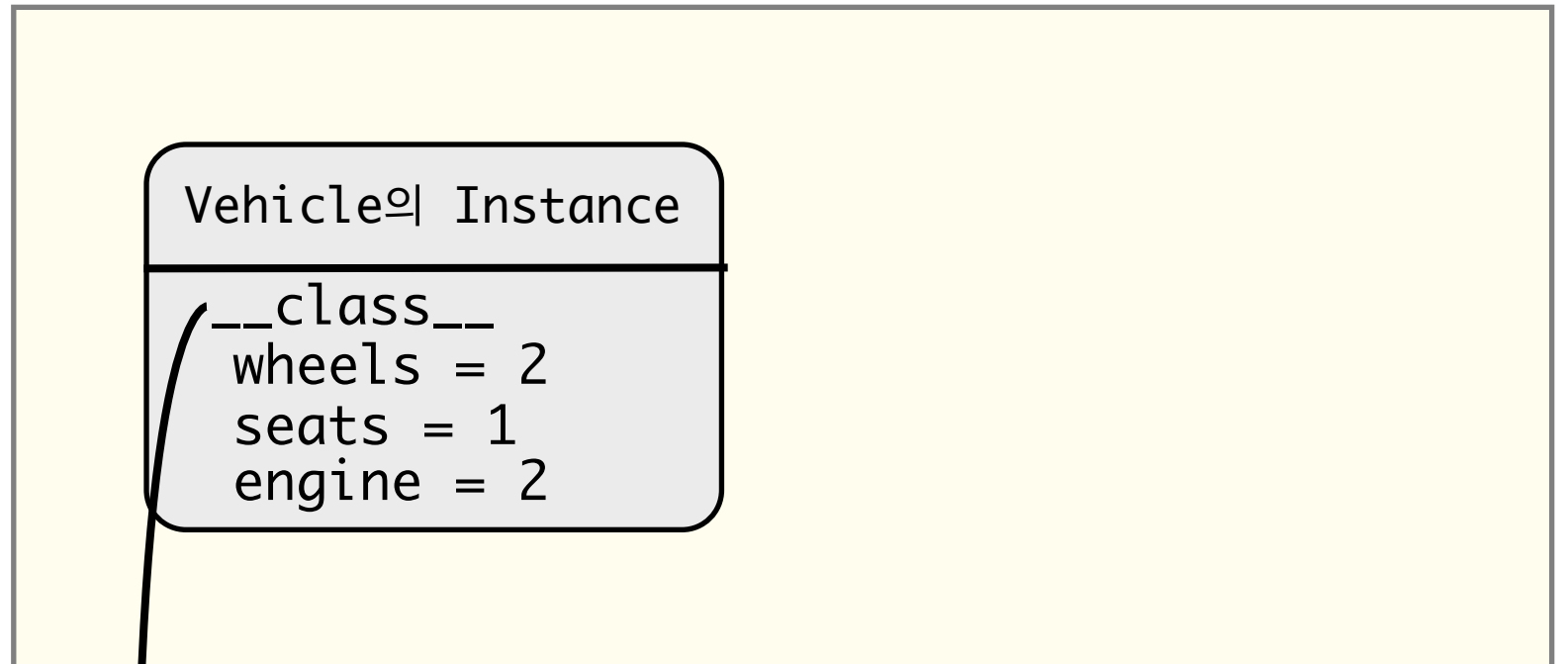


```
bike = Vehicle(2, 1, 2)
```

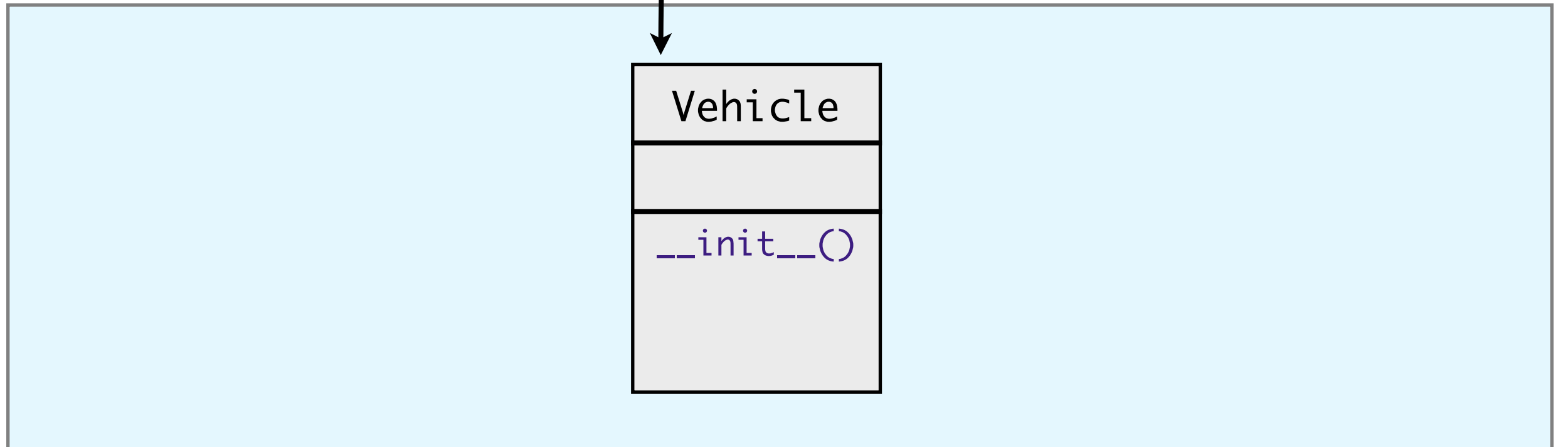
Stack



Heap

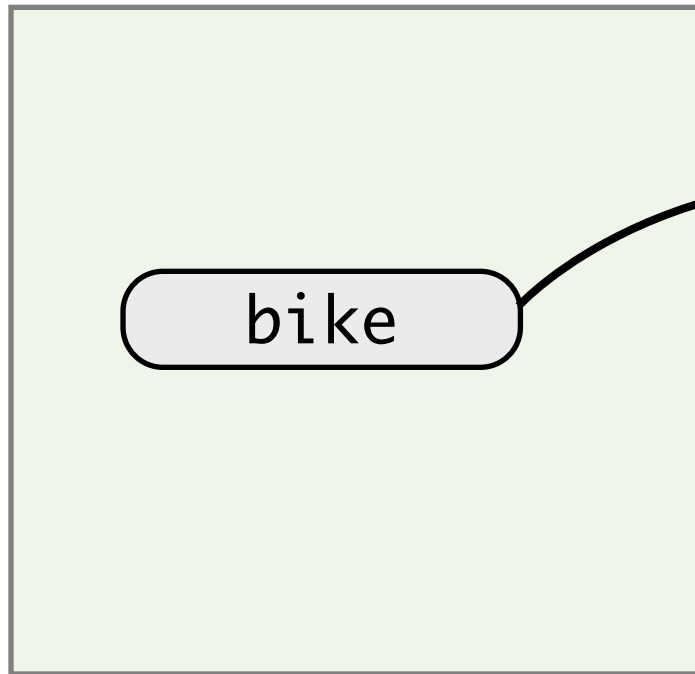


Code

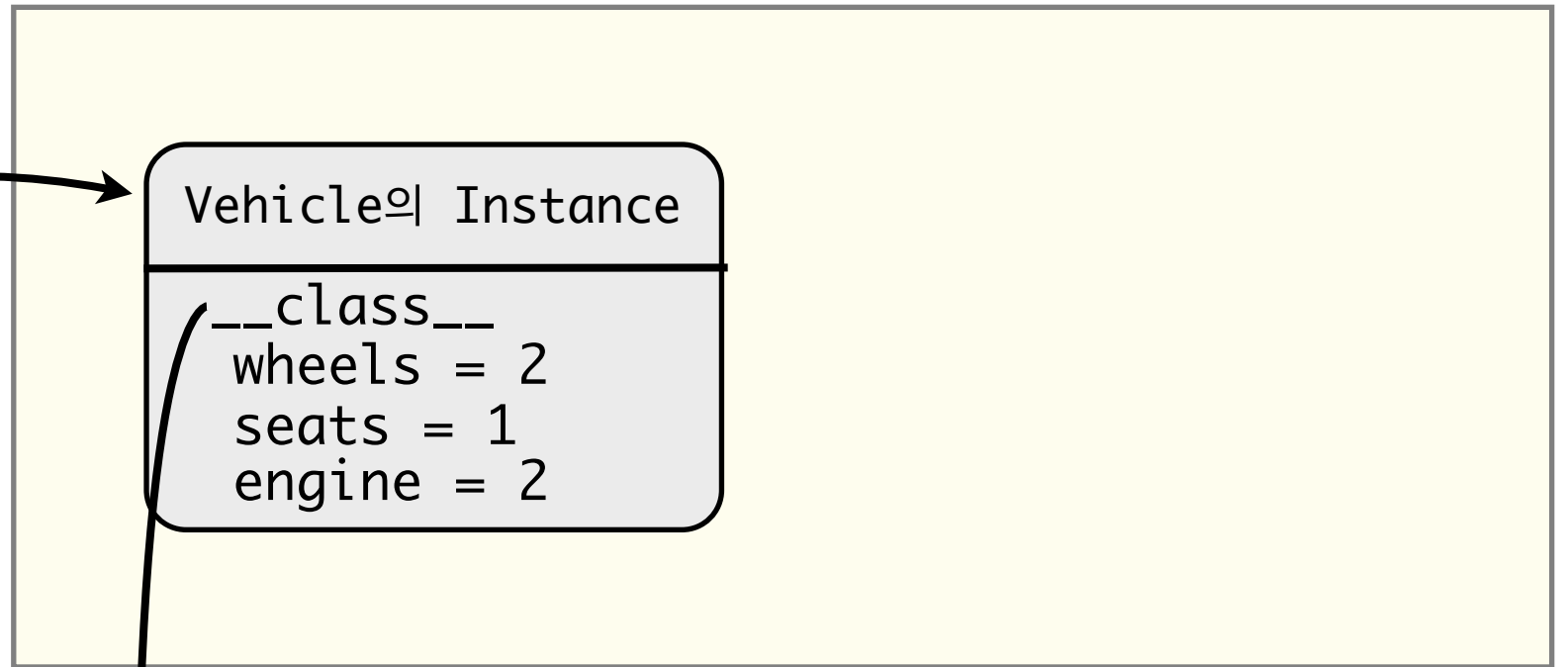


```
bike = Vehicle(2, 1, 2)
```

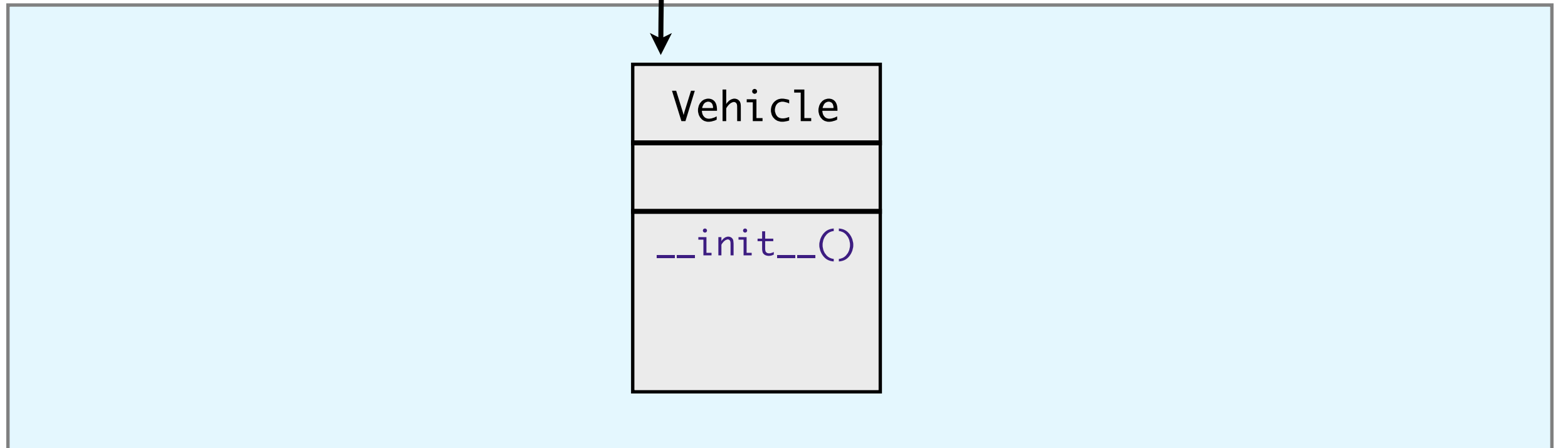
Stack



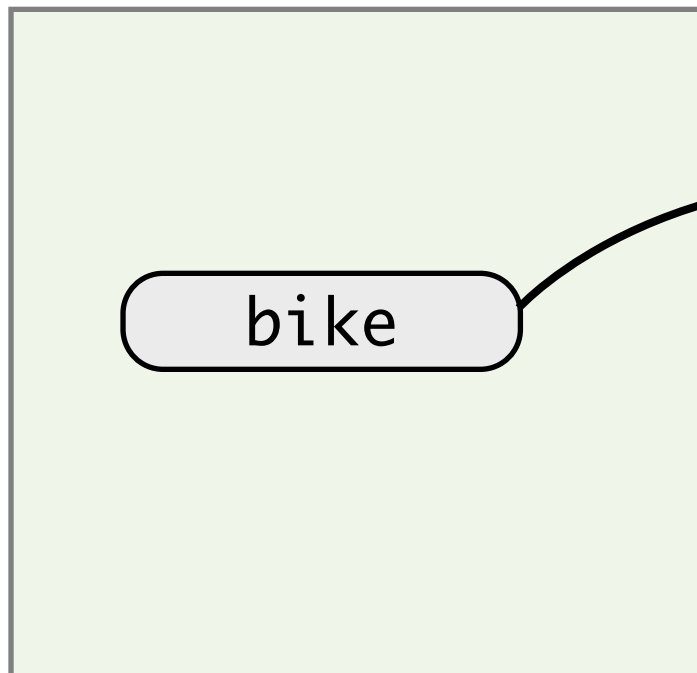
Heap



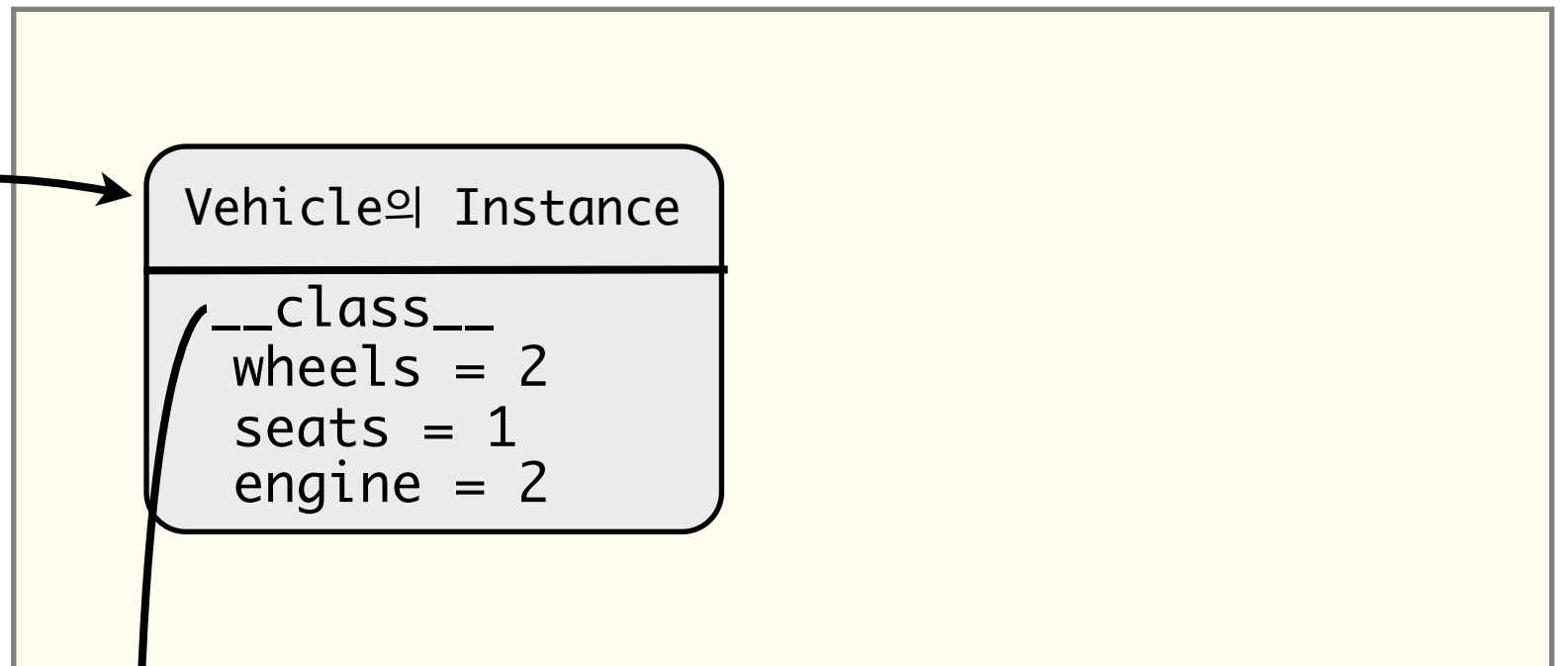
Code



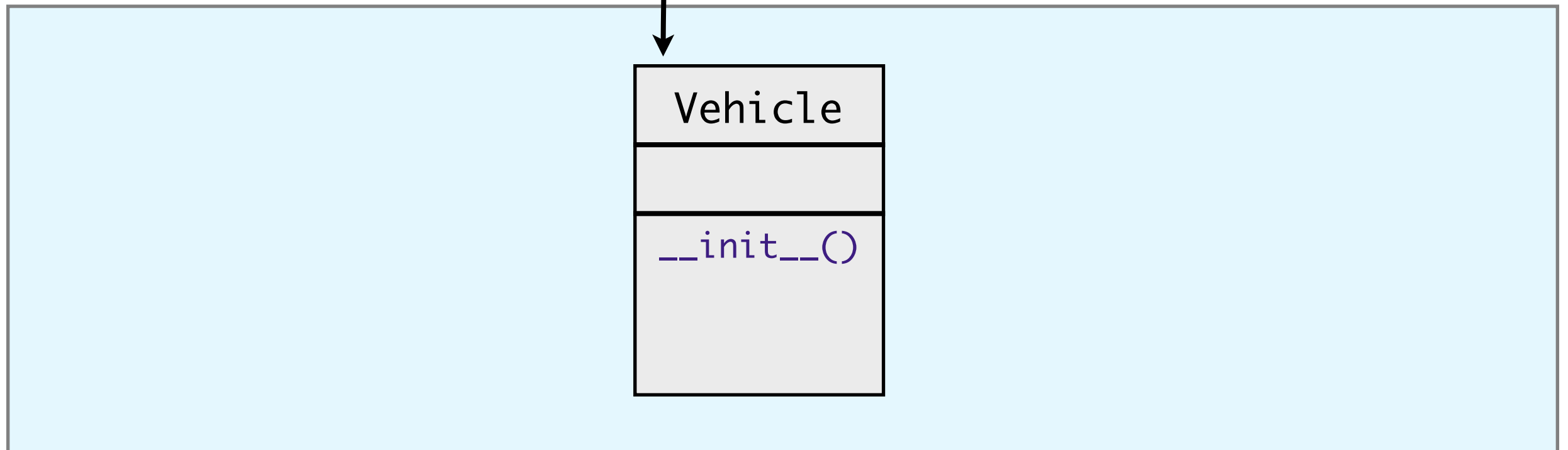
Stack



Heap

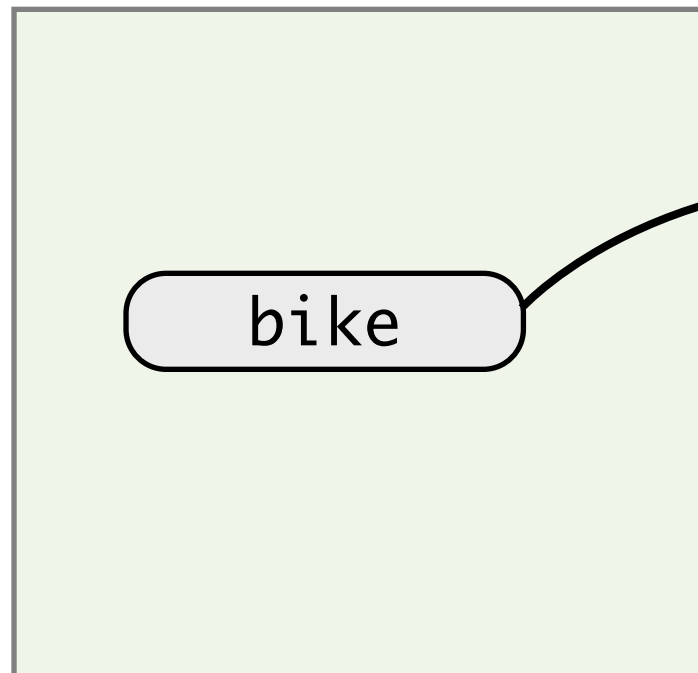


Code

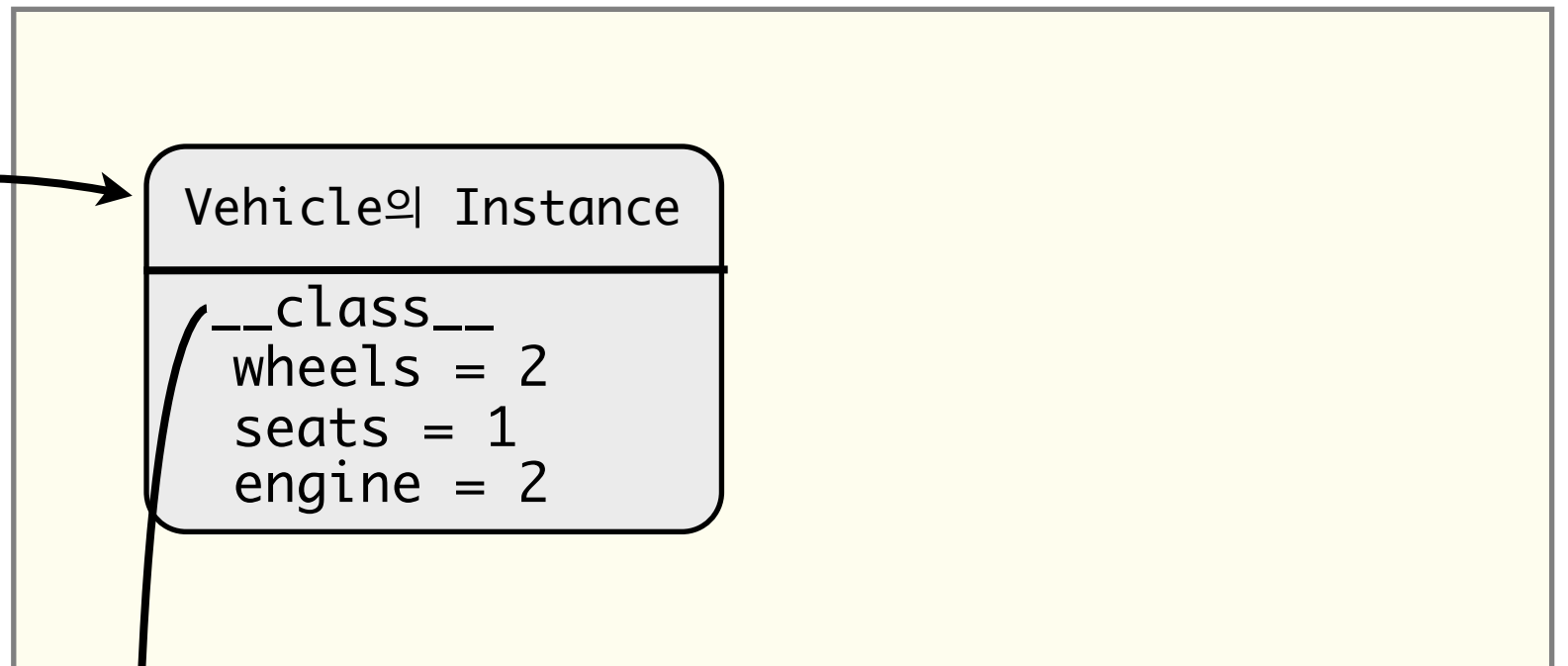


```
print('bike.wheels = ', bike.wheels)
```

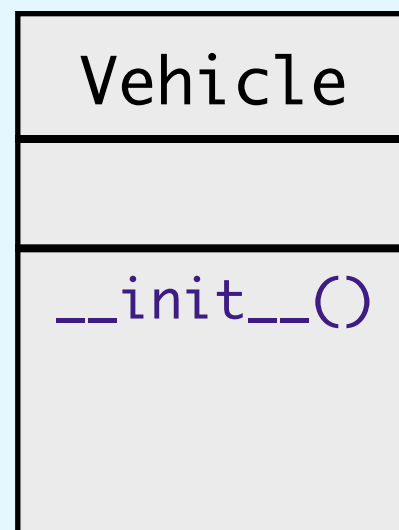
Stack



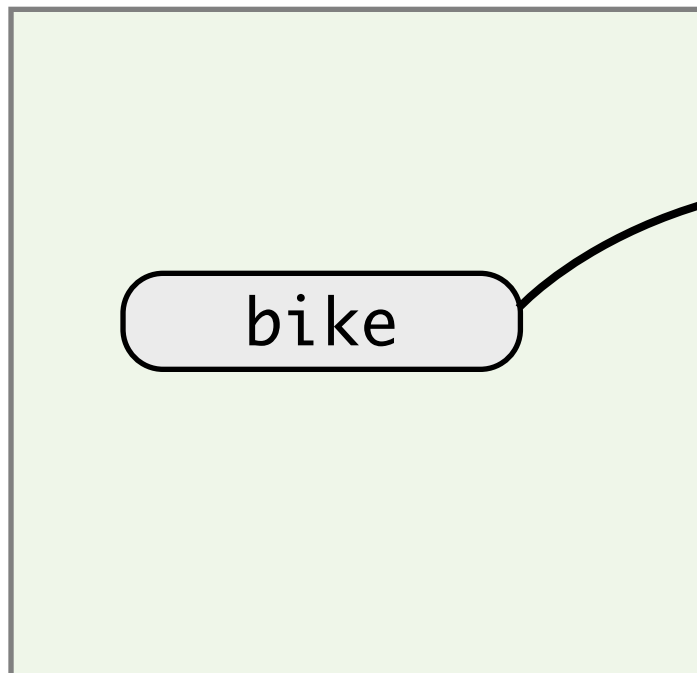
Heap



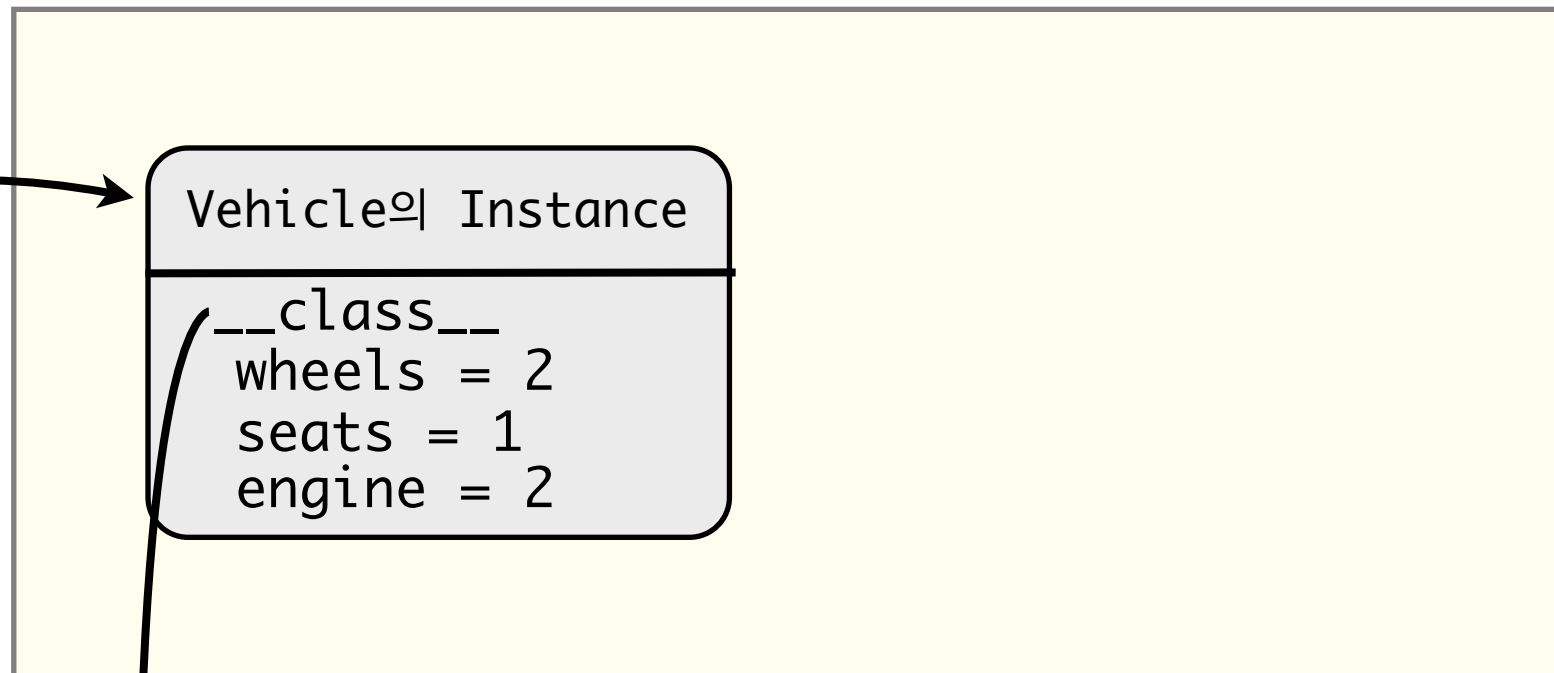
Code



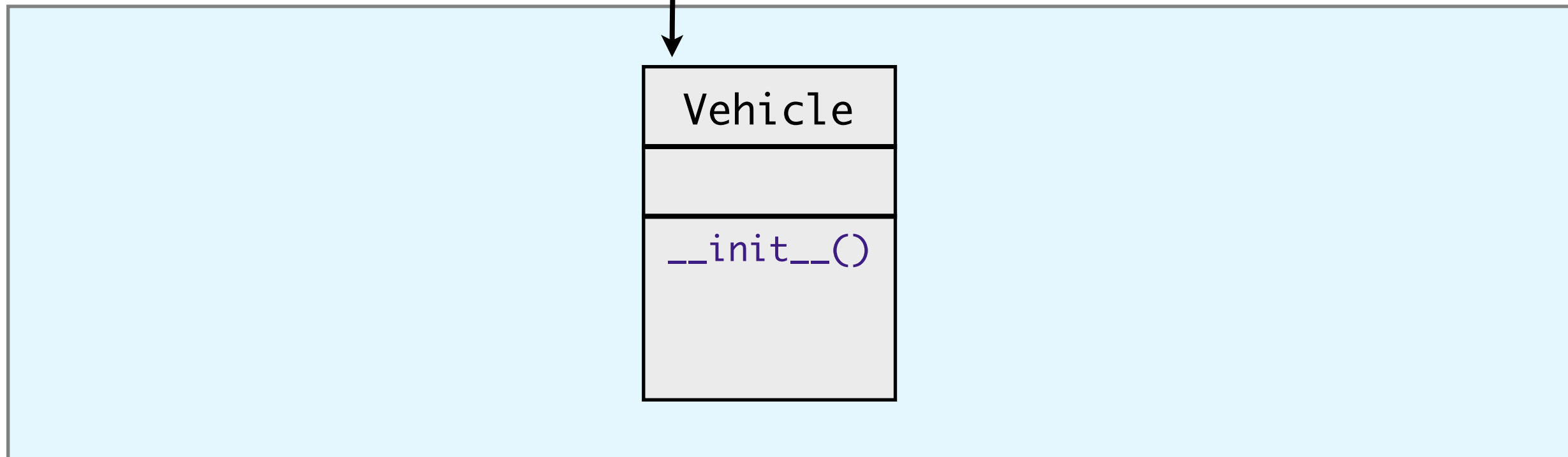
Stack



Heap

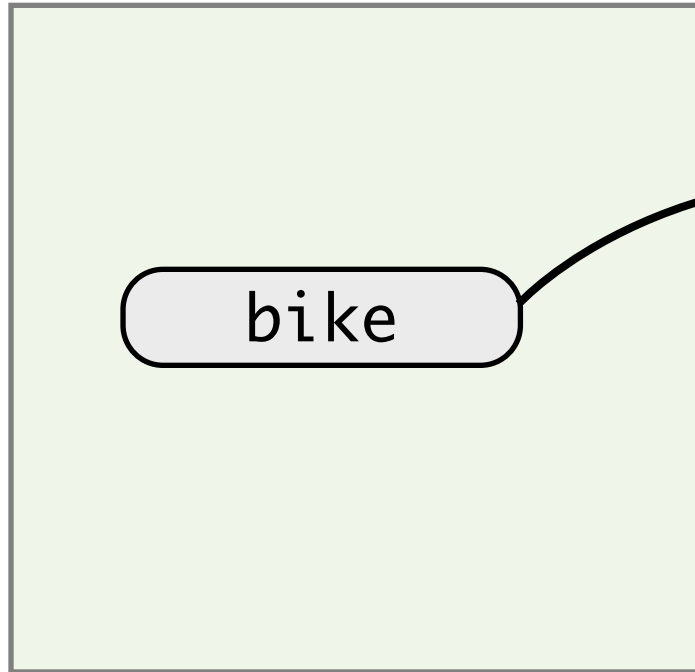


Code

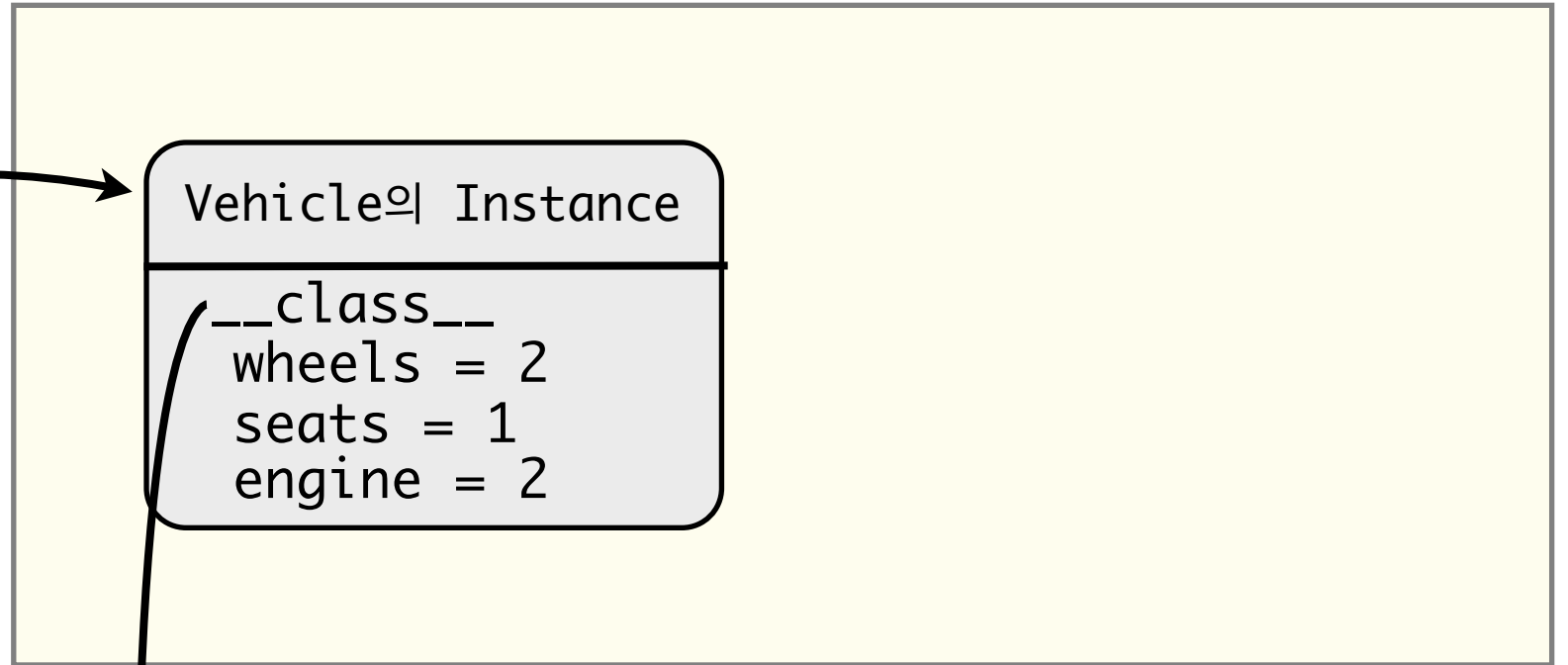


bike.drive()

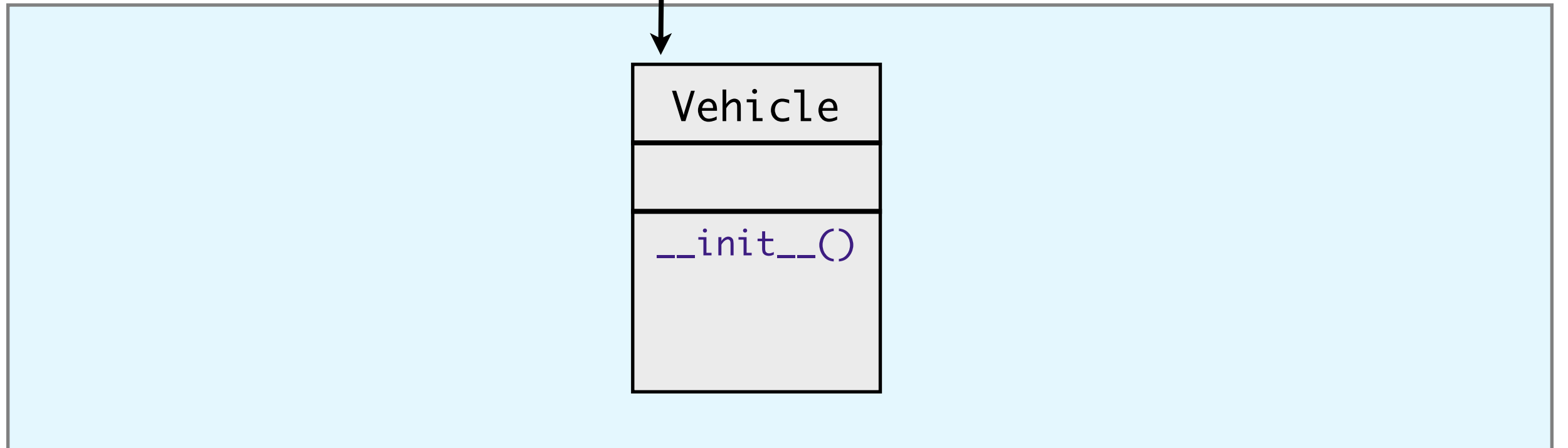
Stack



Heap

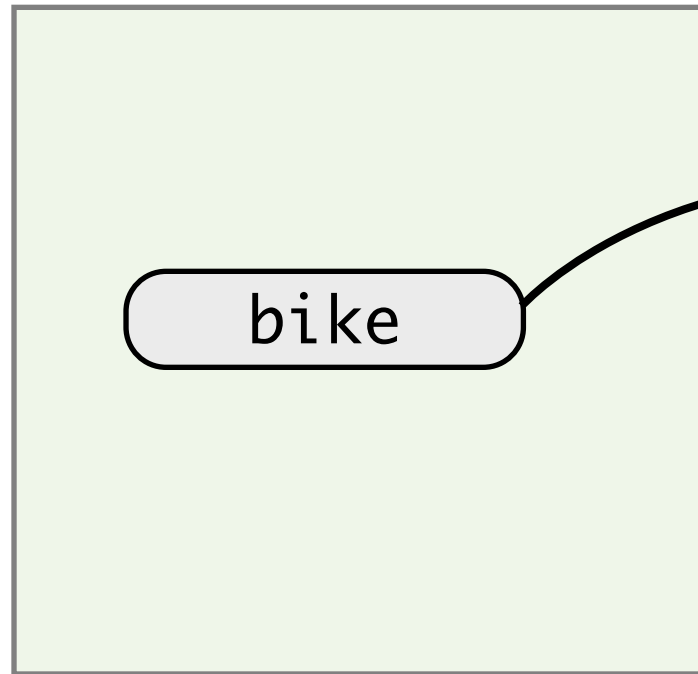


Code

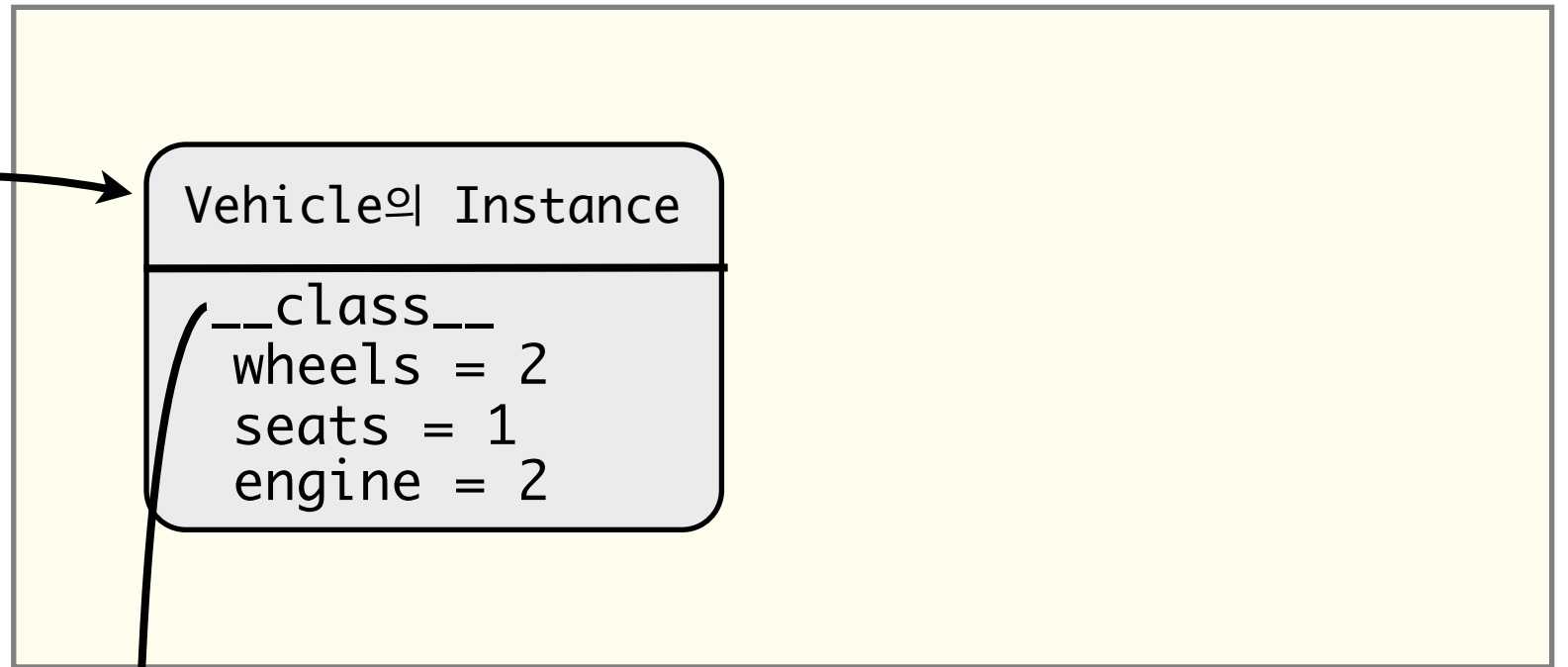


bike.drive()

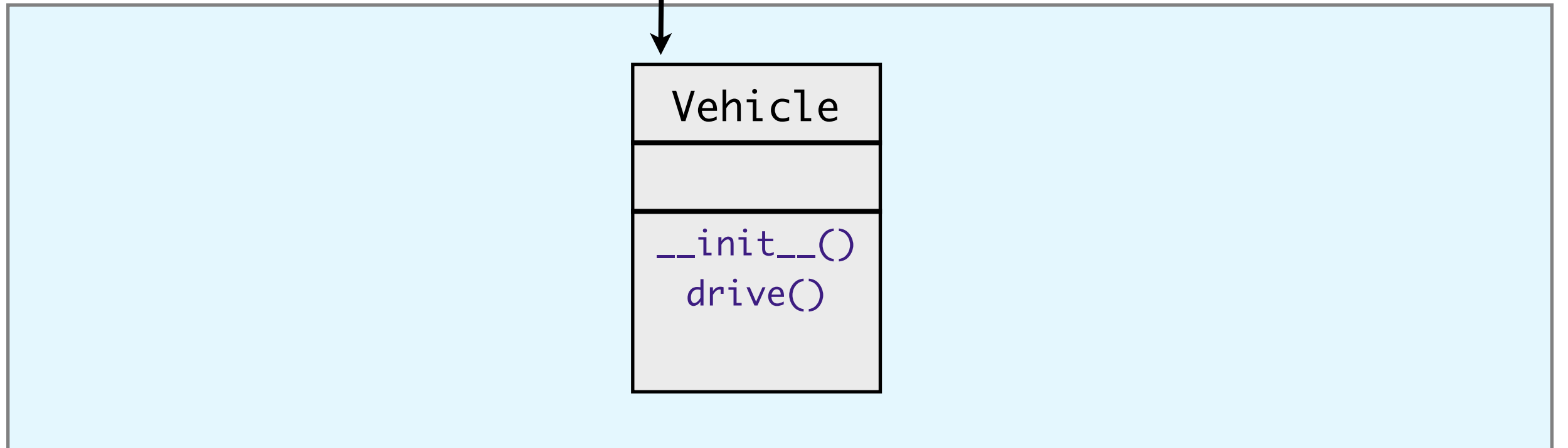
Stack



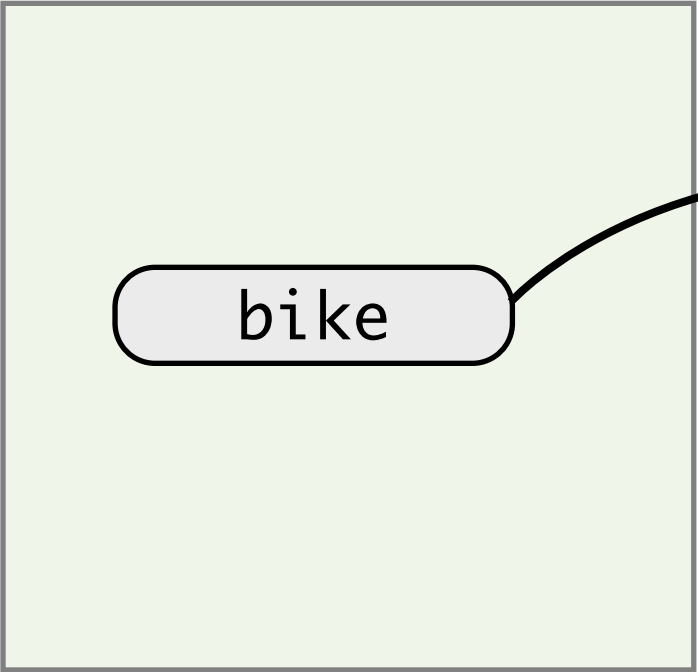
Heap



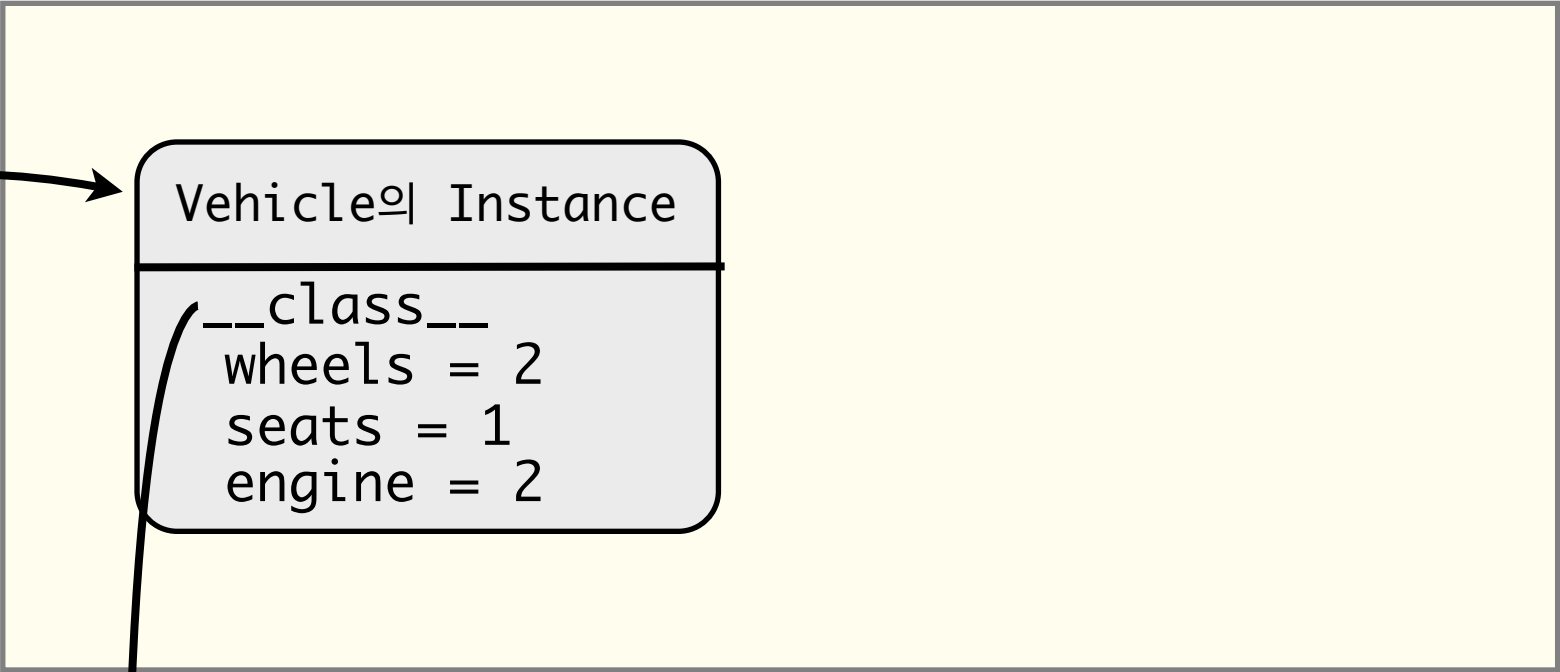
Code



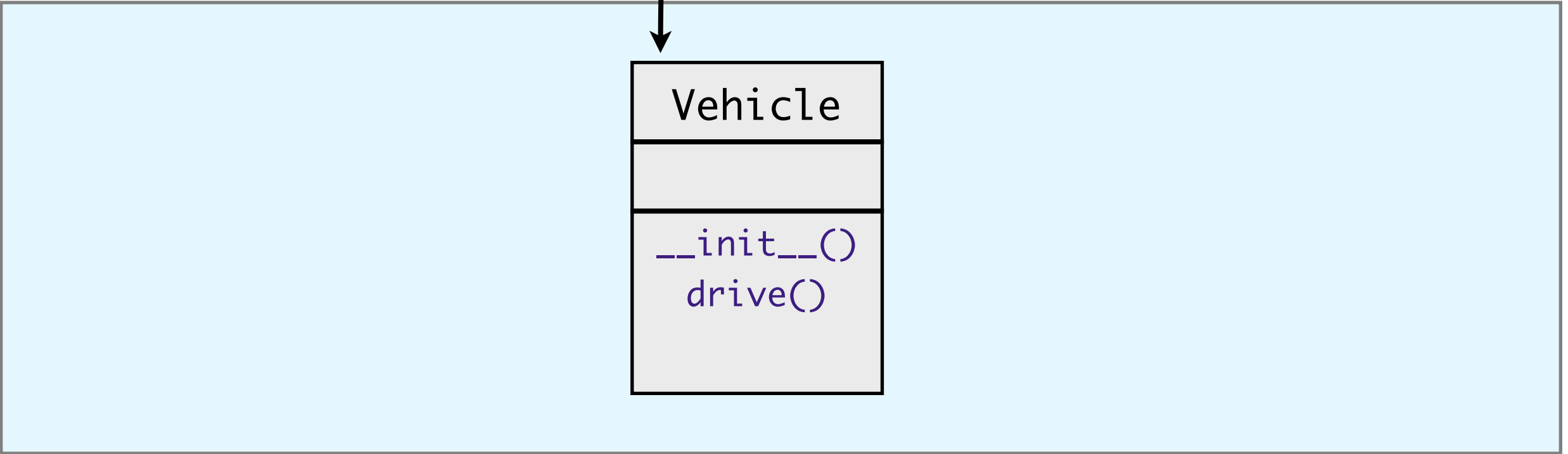
Stack



Heap

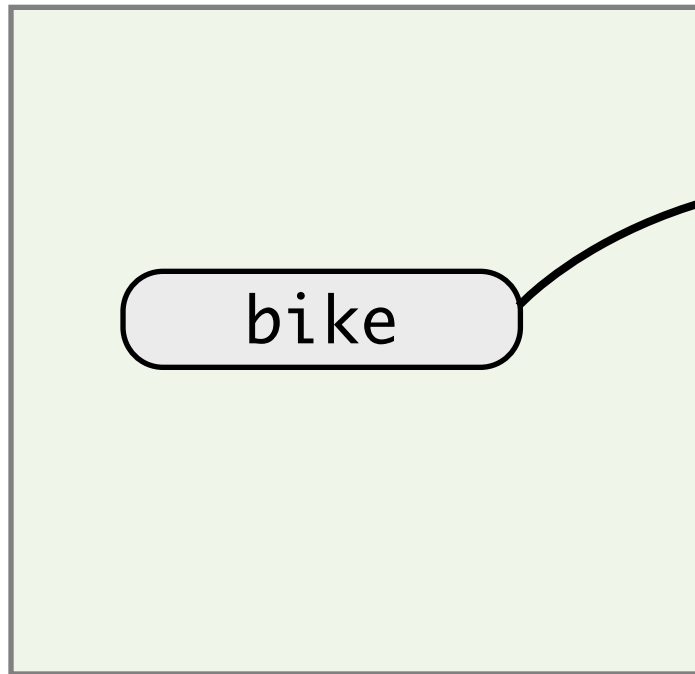


Code

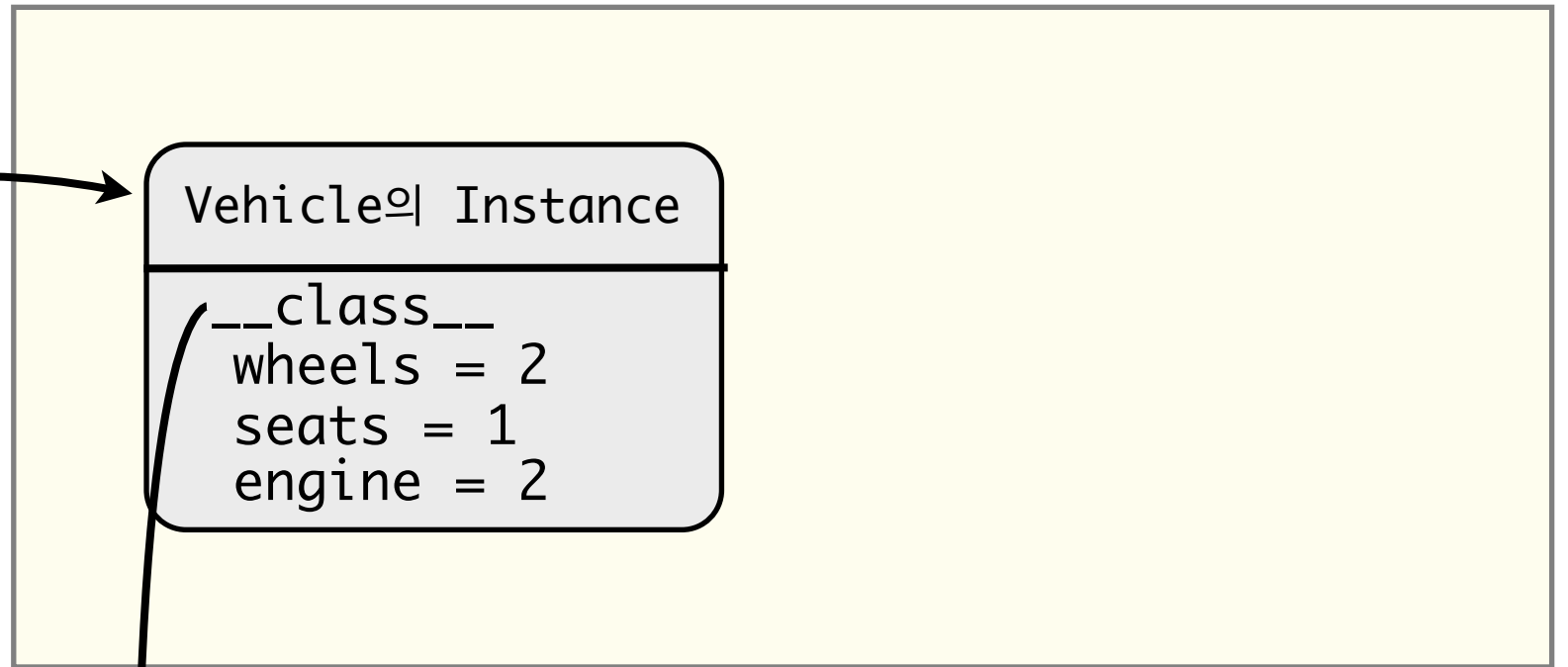


```
car = Vehicle(4, 4, 4)
```

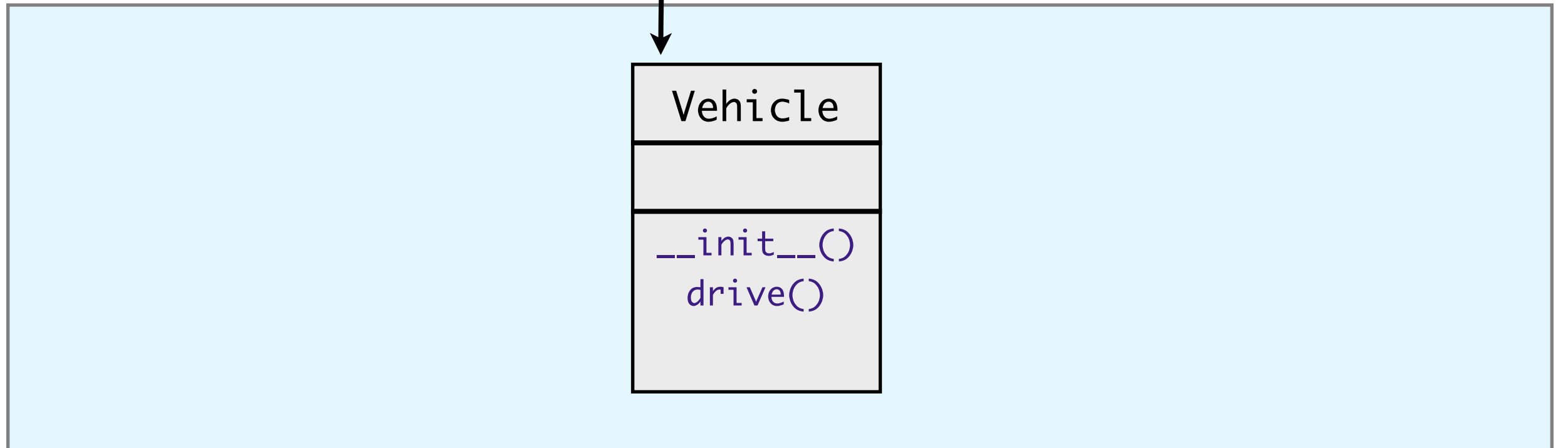
Stack



Heap

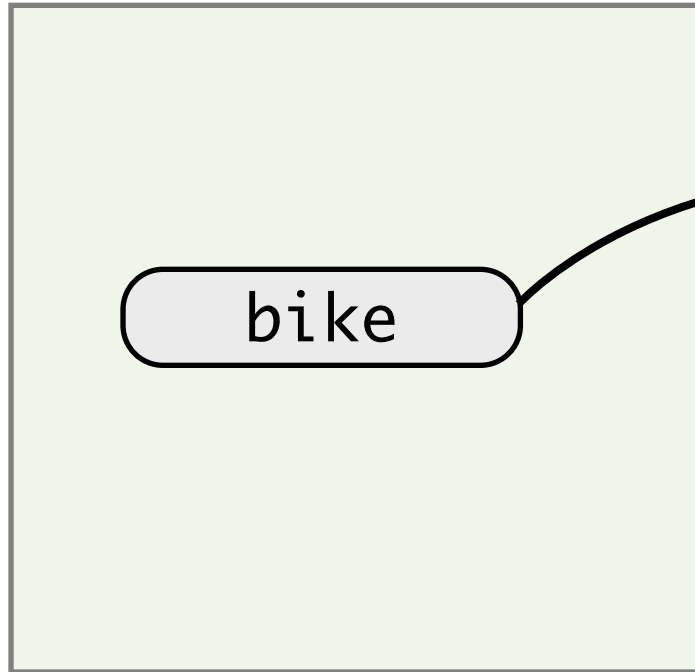


Code

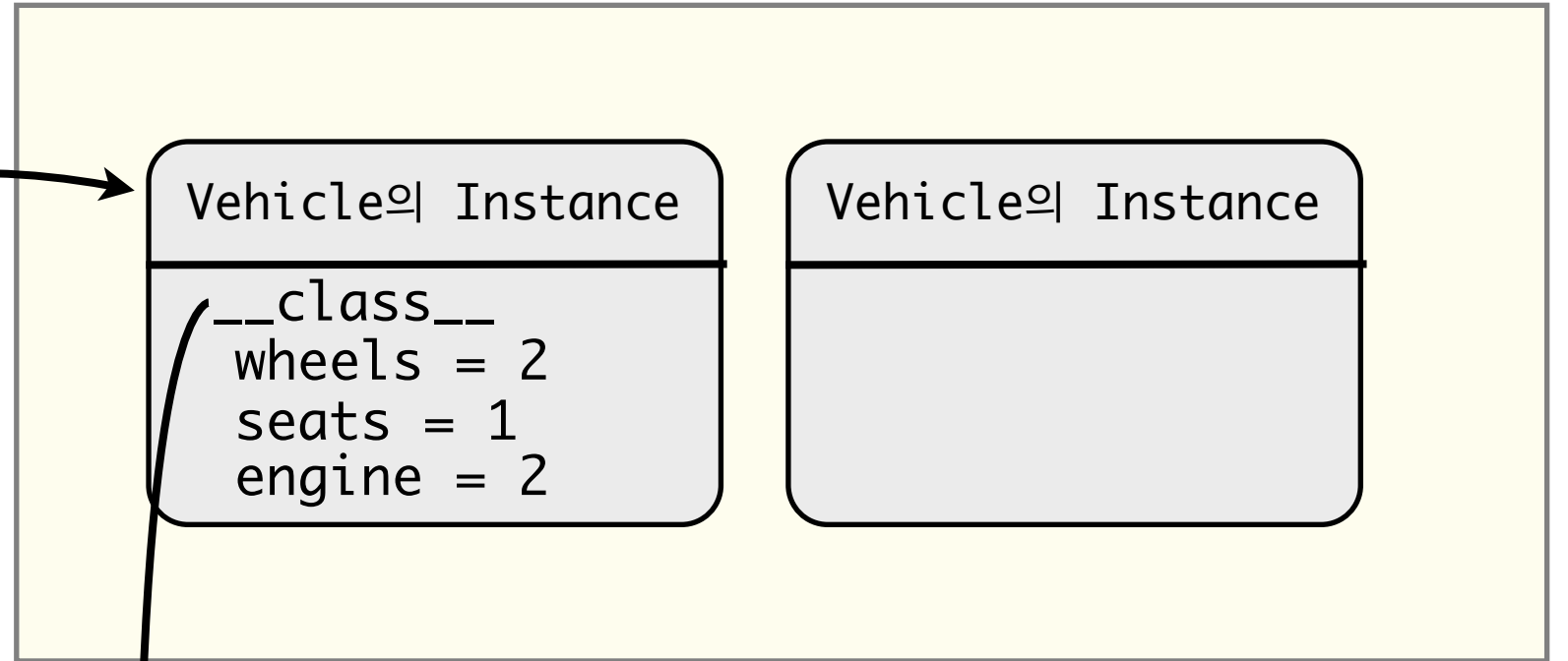


```
car = Vehicle(4, 4, 4)
```

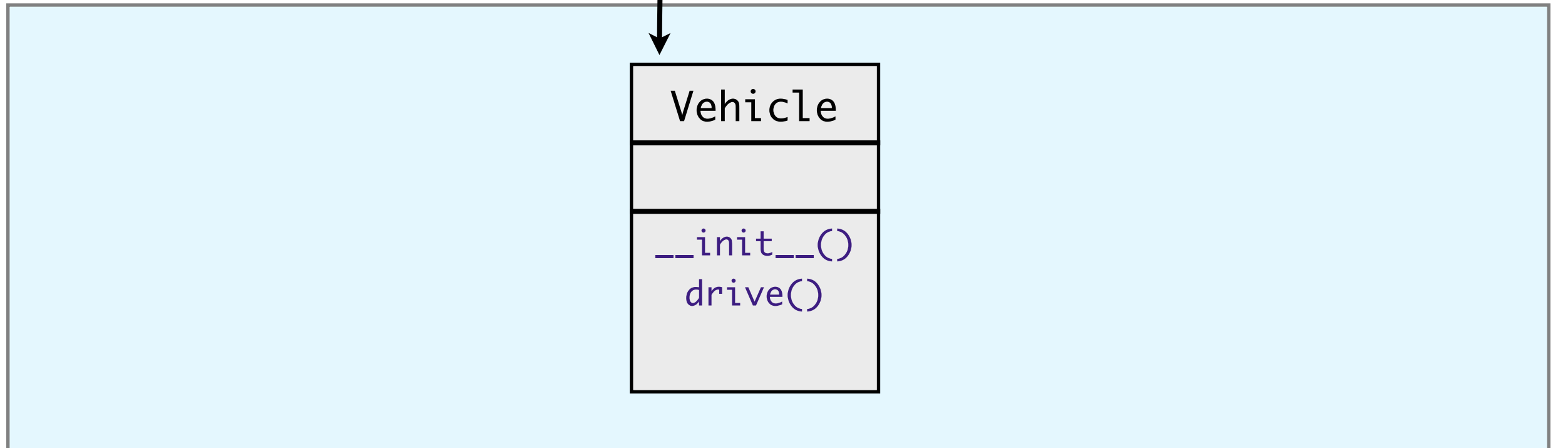
Stack



Heap

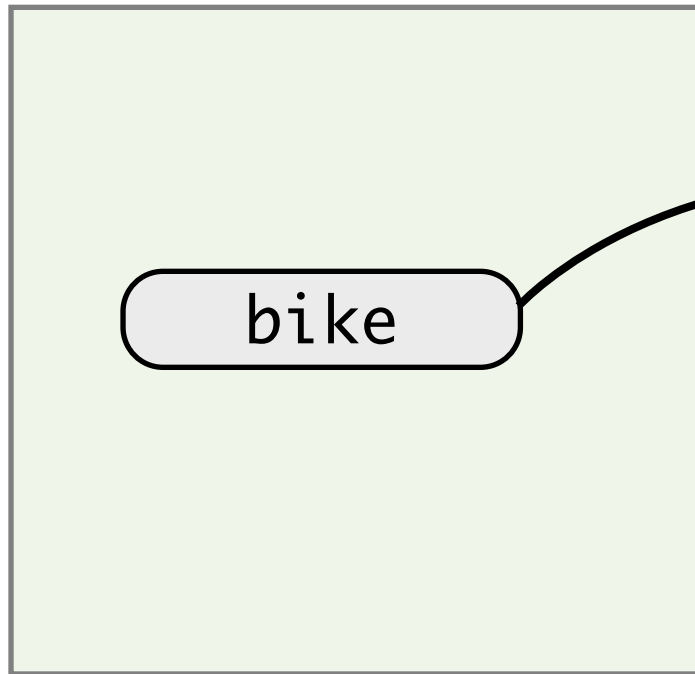


Code

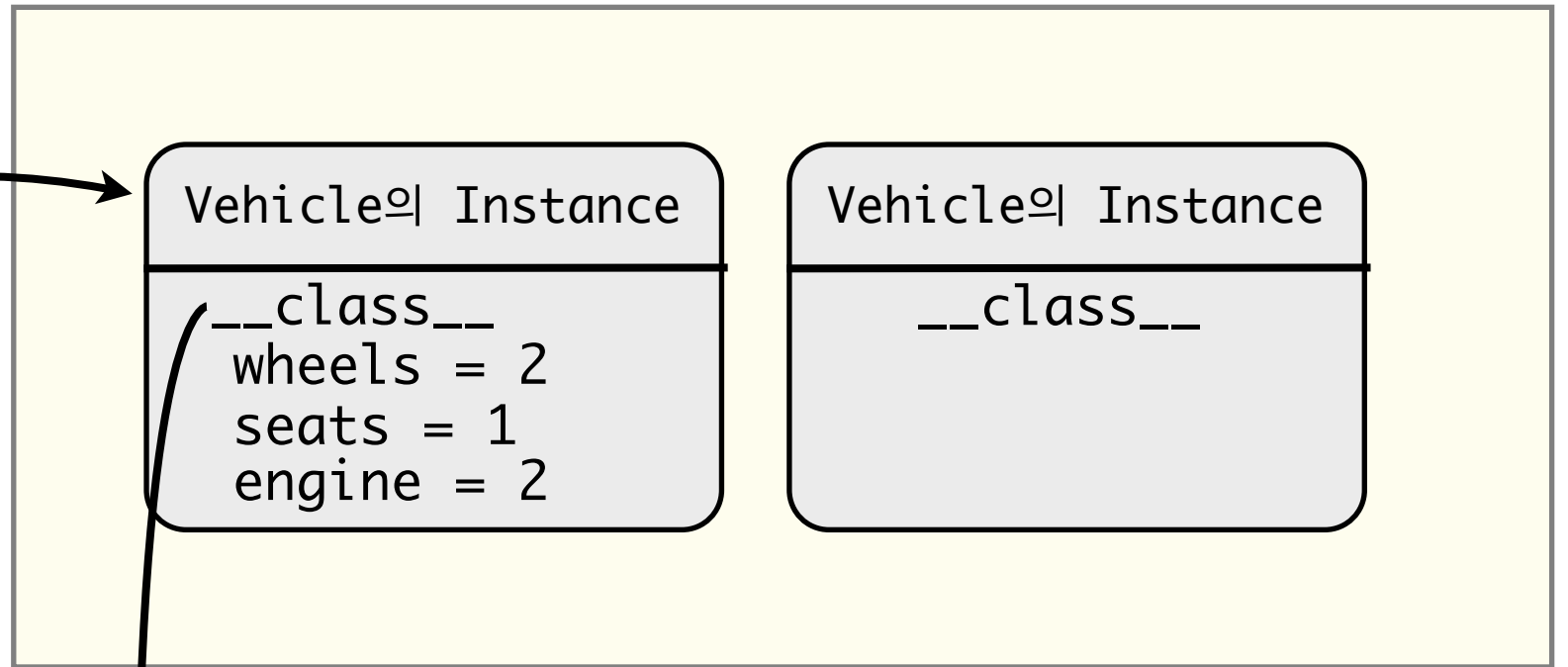


```
car = Vehicle(4, 4, 4)
```

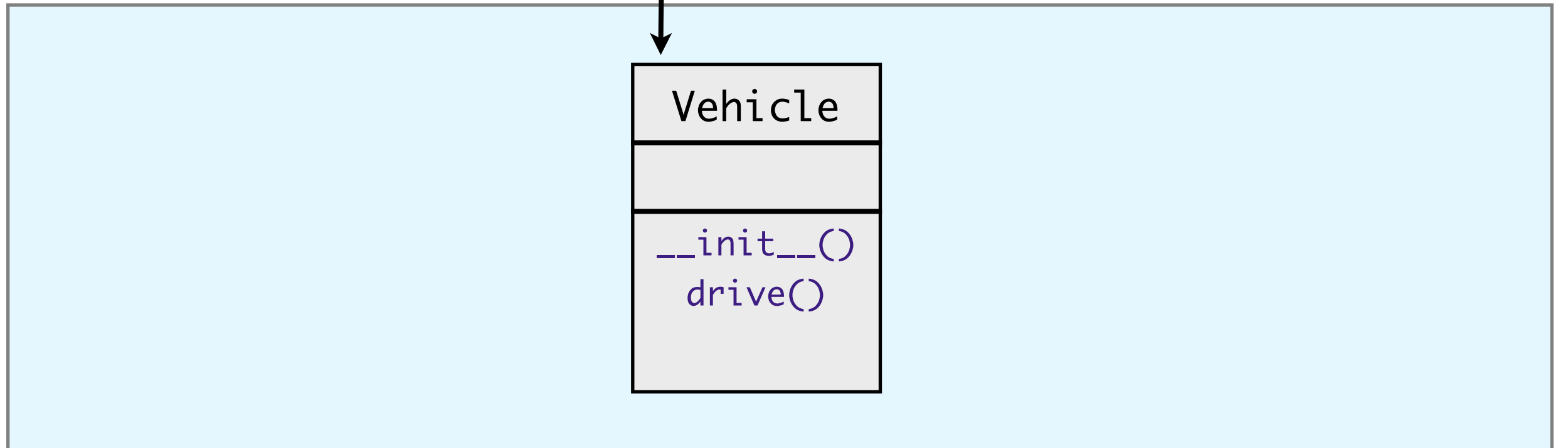
Stack



Heap

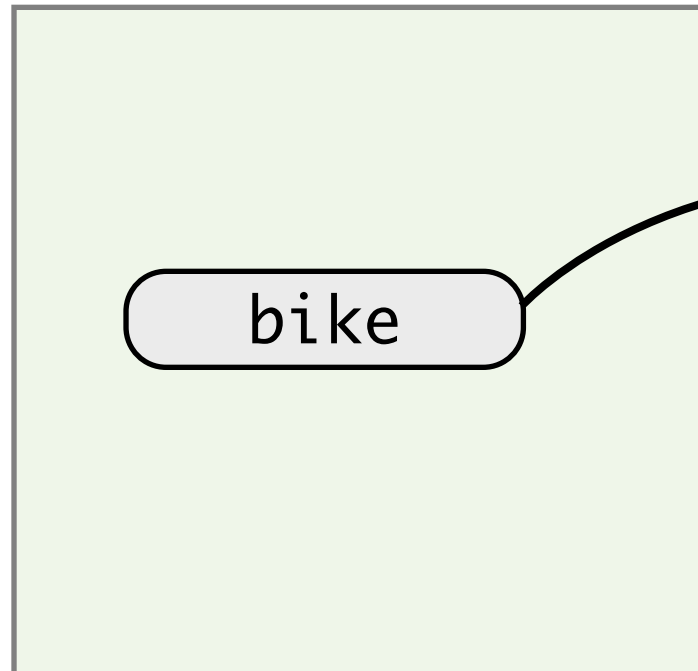


Code

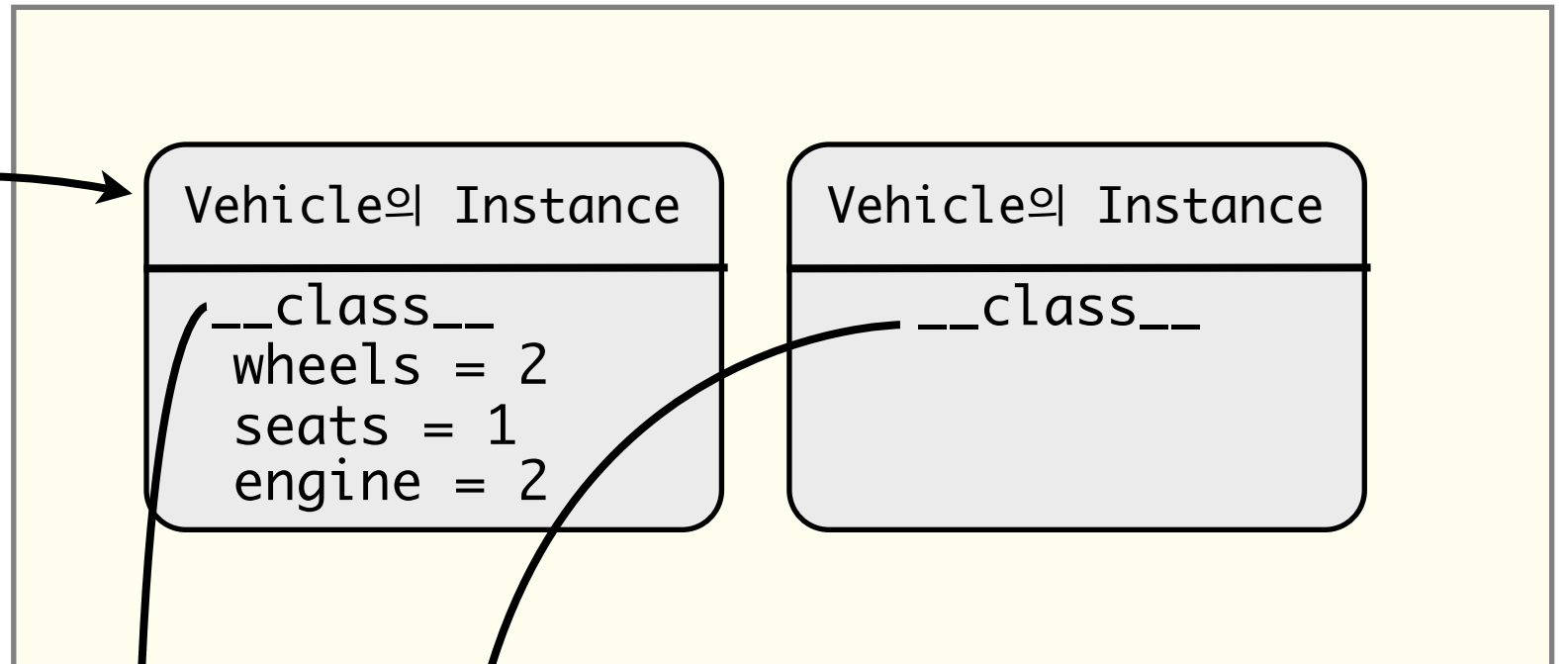


```
car = Vehicle(4, 4, 4)
```

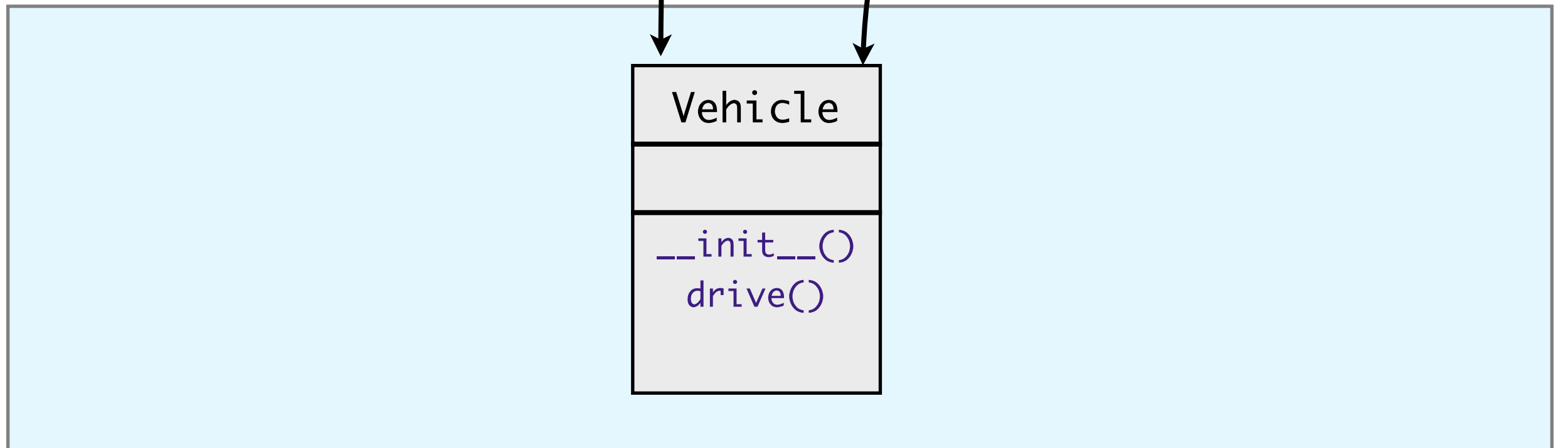
Stack



Heap

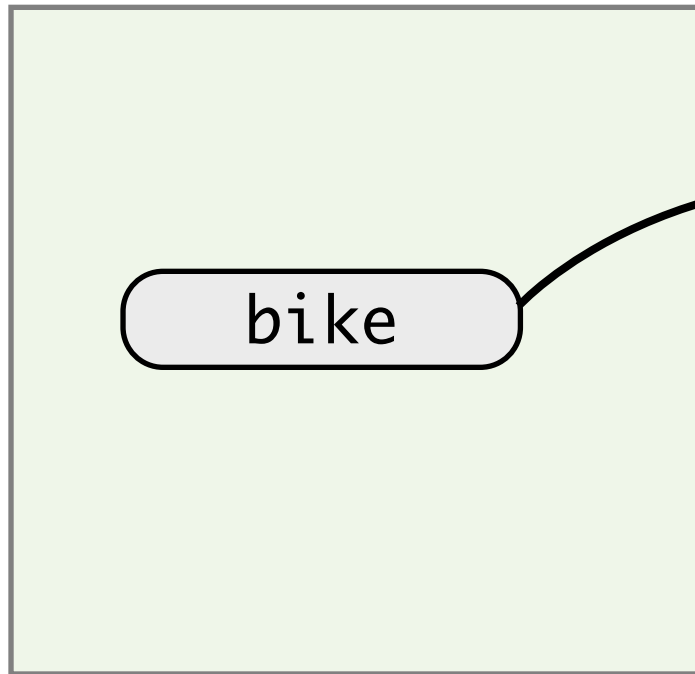


Code

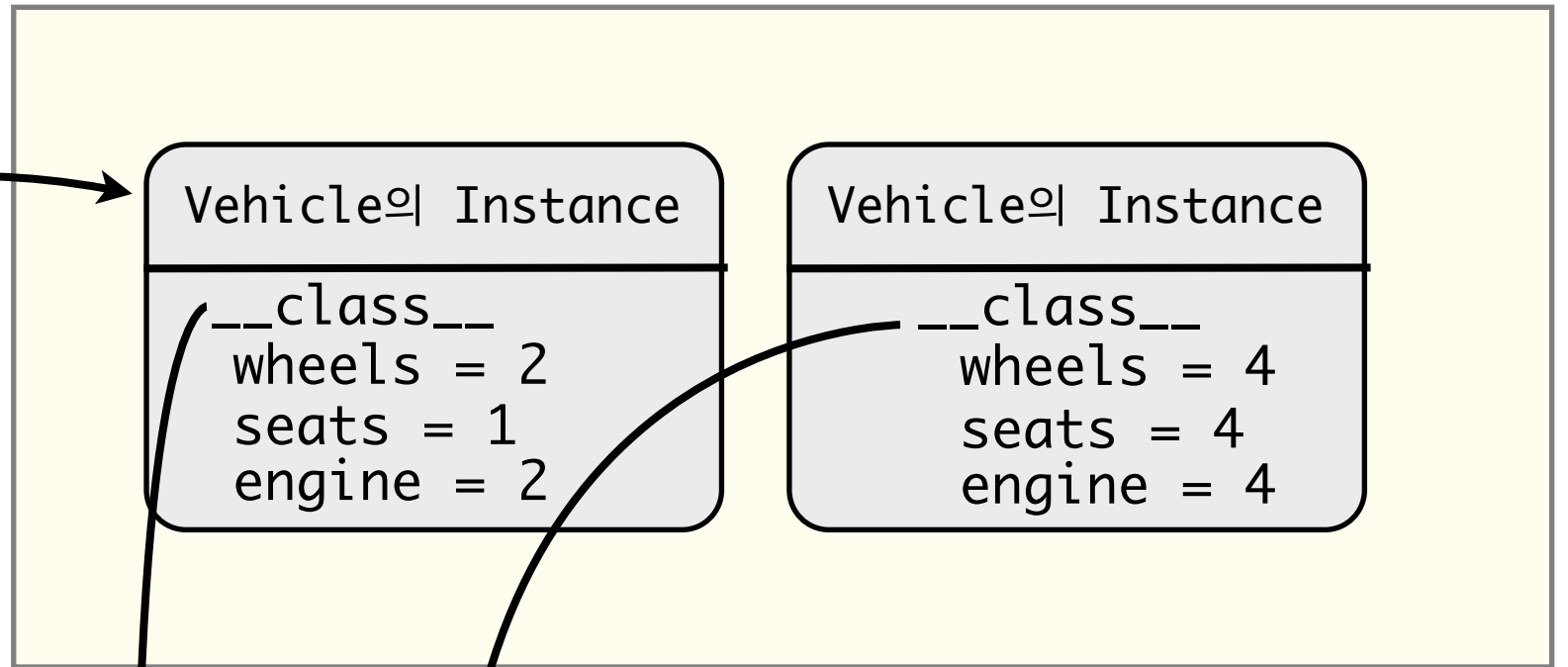


```
car = Vehicle(4, 4, 4)
```

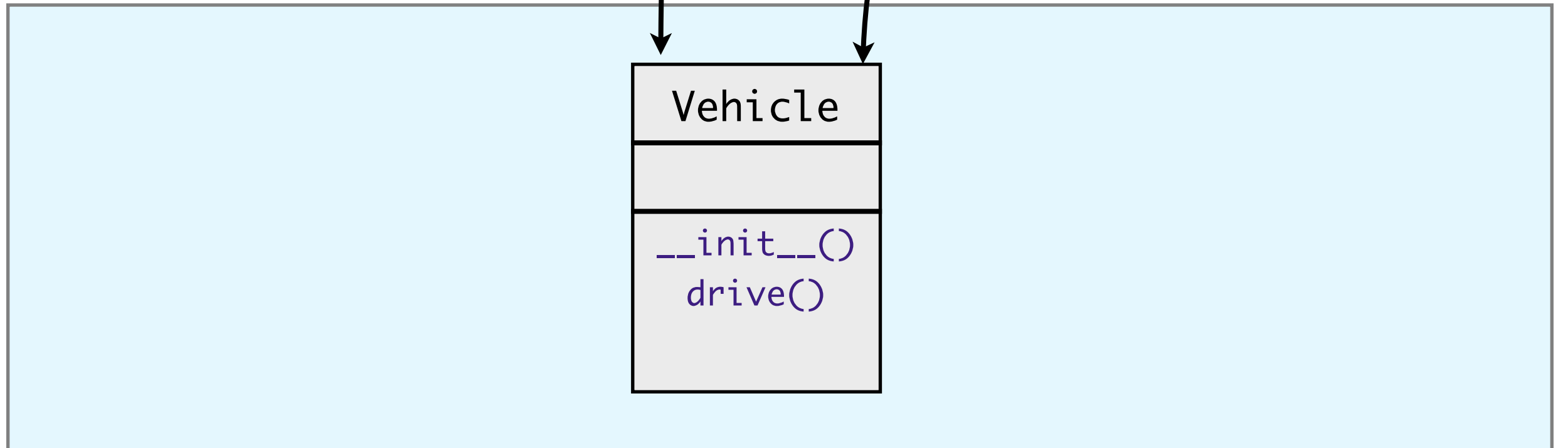
Stack



Heap

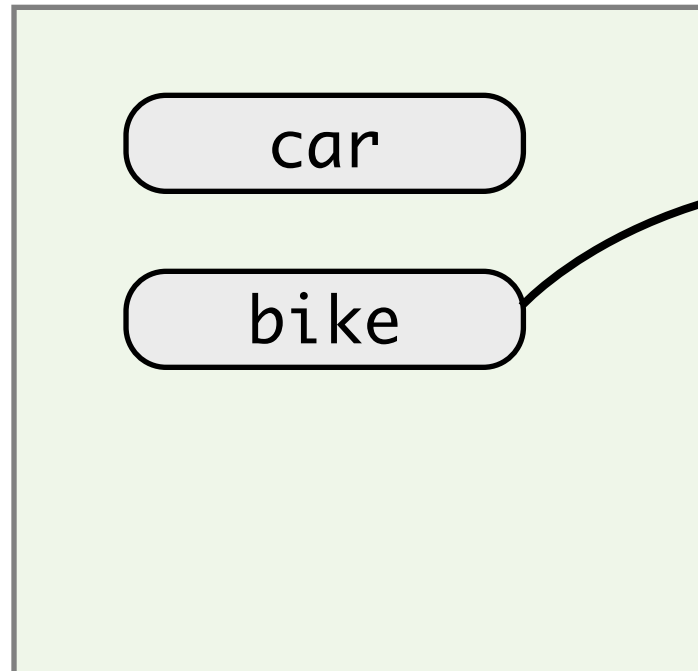


Code

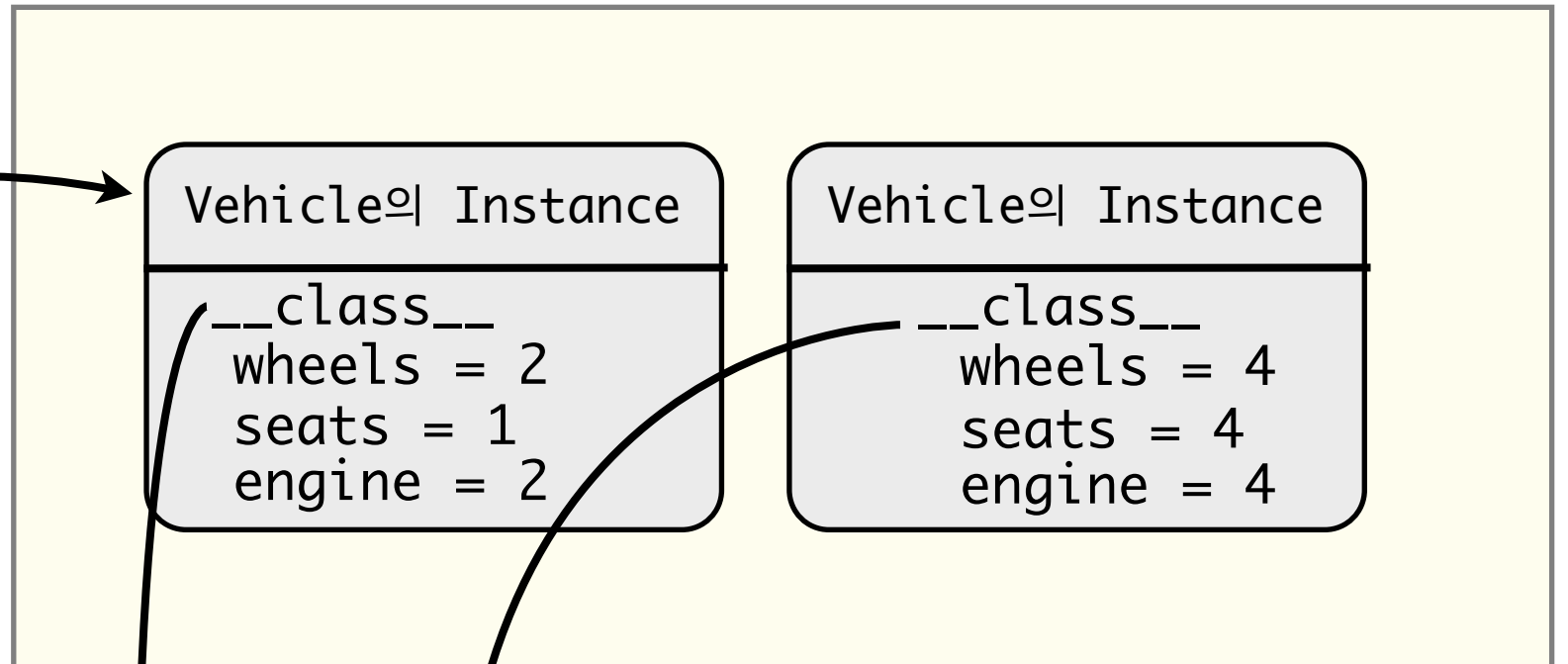


```
car = Vehicle(4, 4, 4)
```

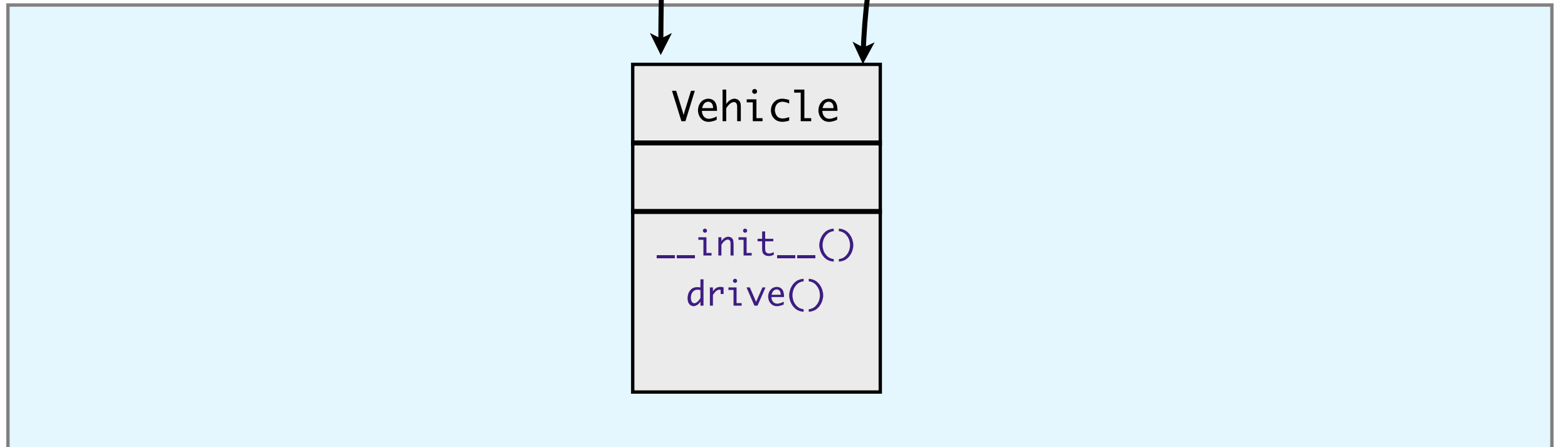
Stack



Heap

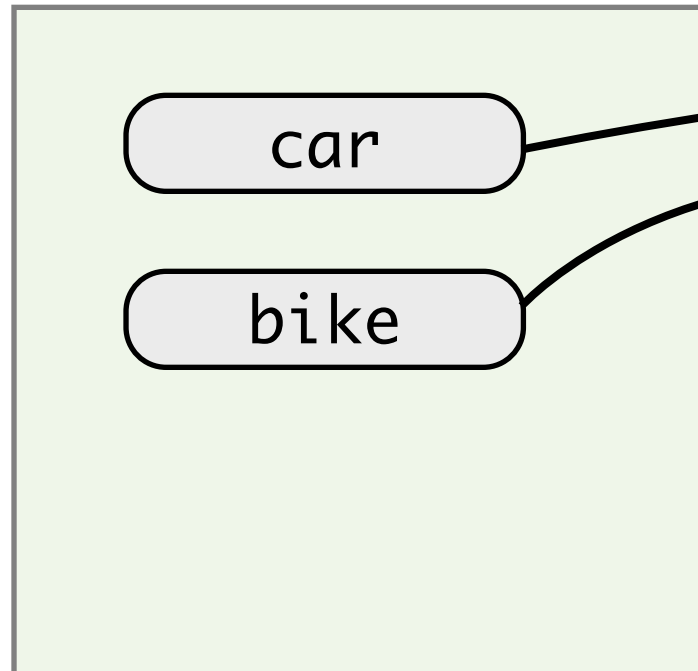


Code

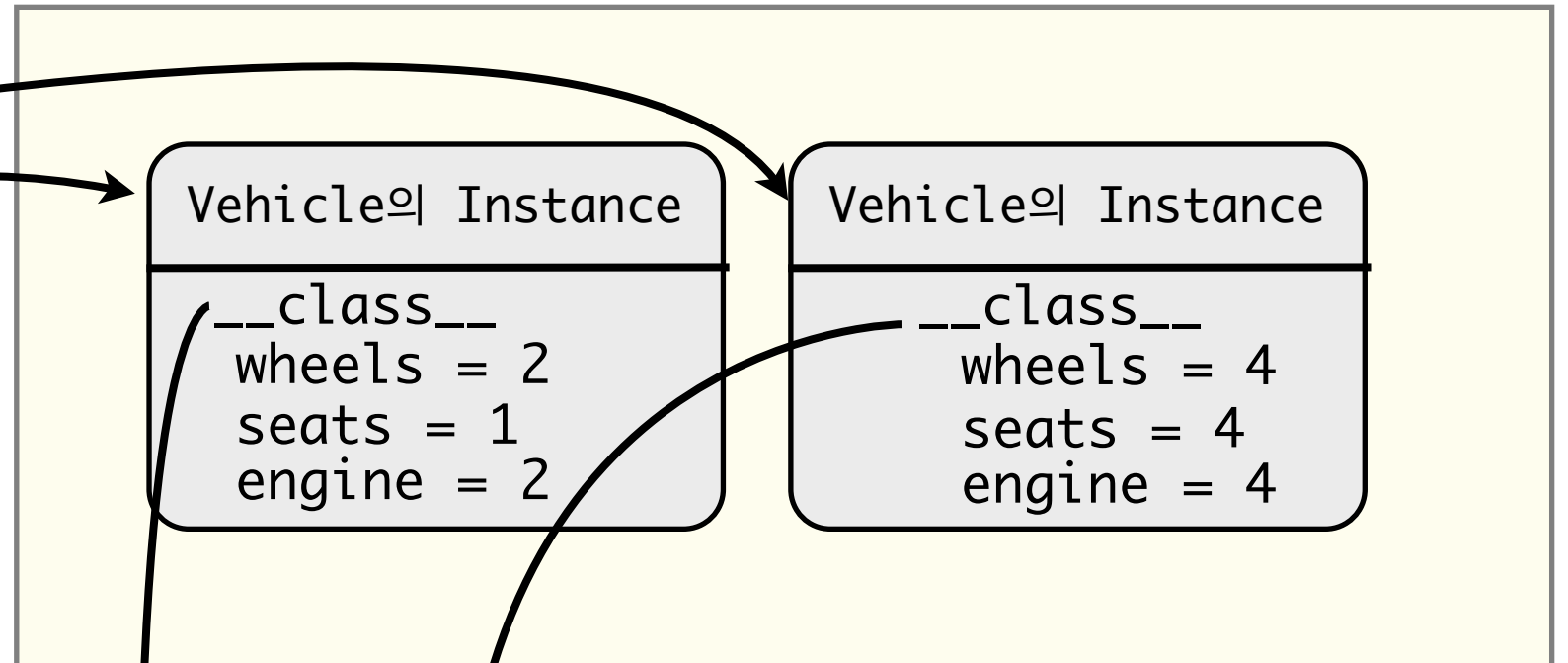


```
car = Vehicle(4, 4, 4)
```

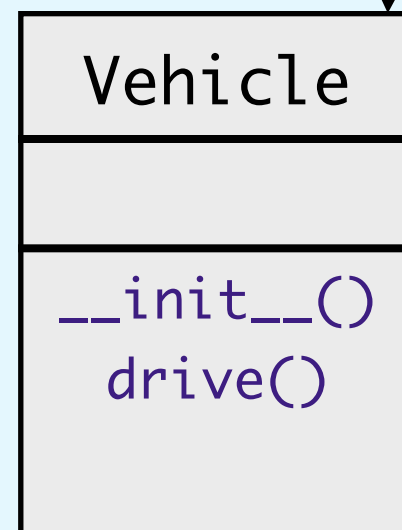
Stack



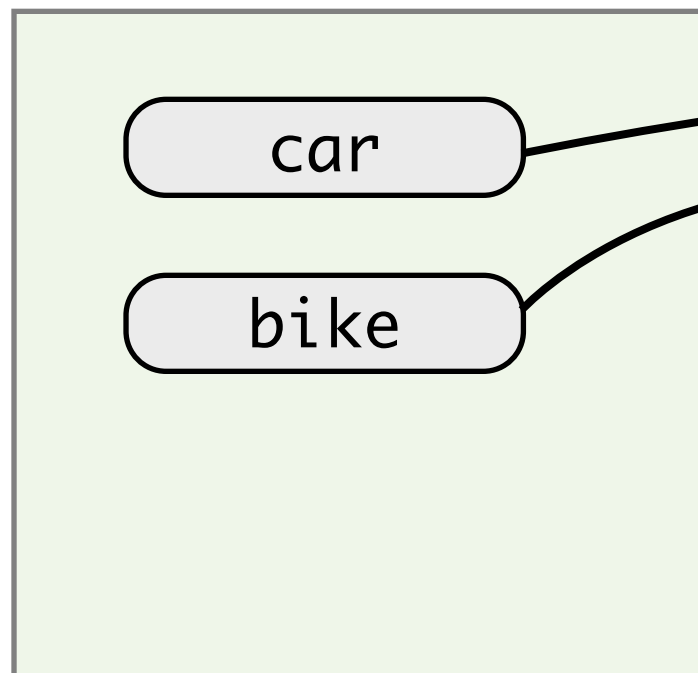
Heap



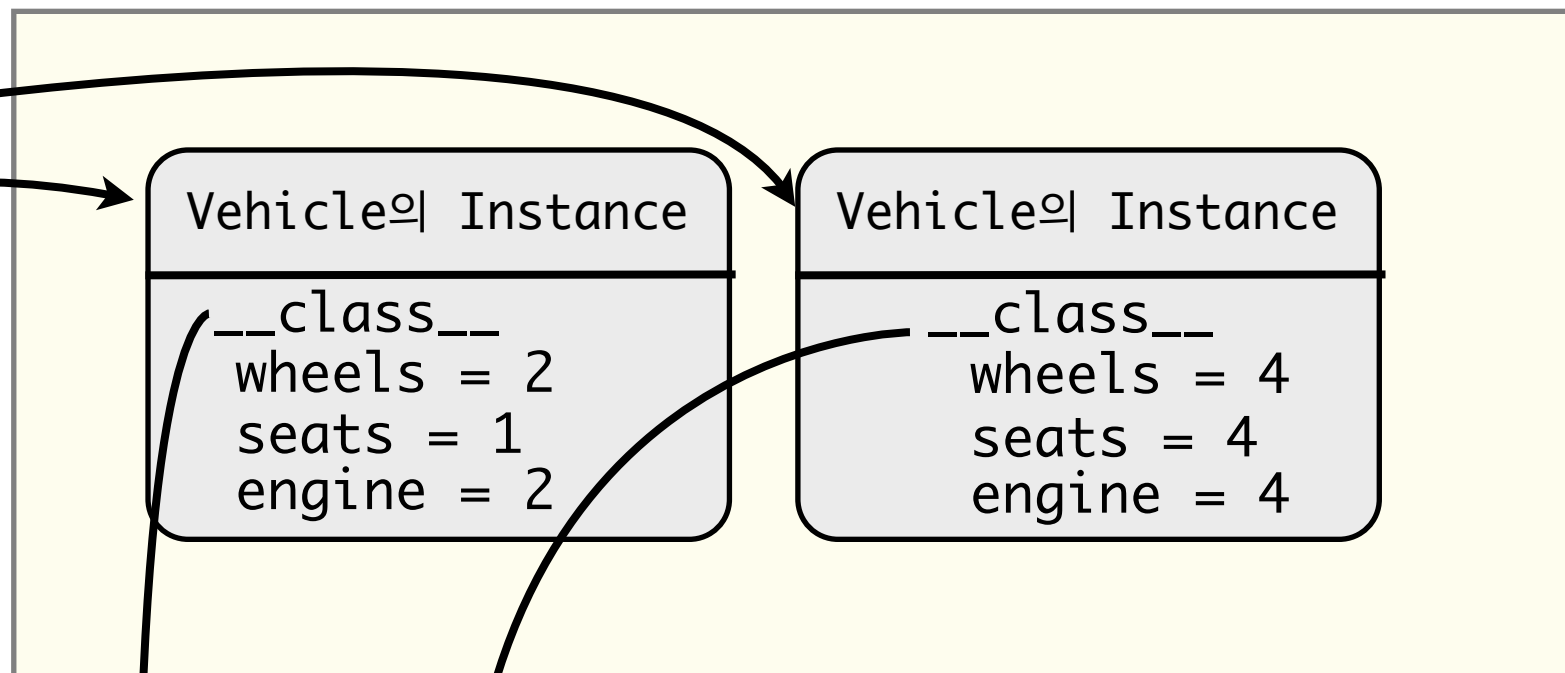
Code



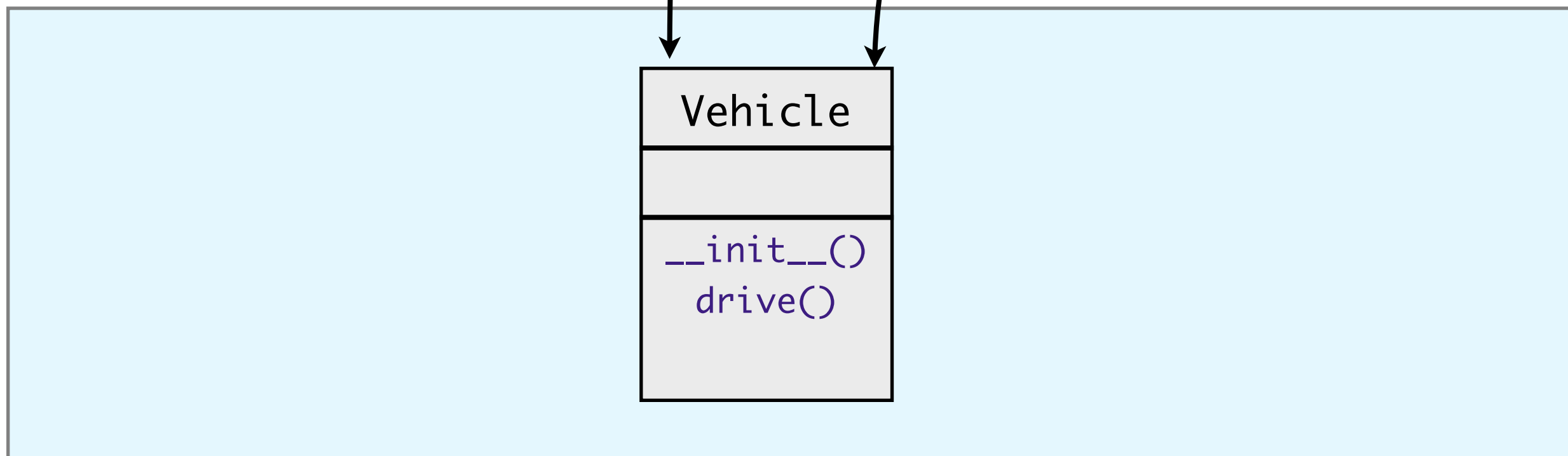
Stack



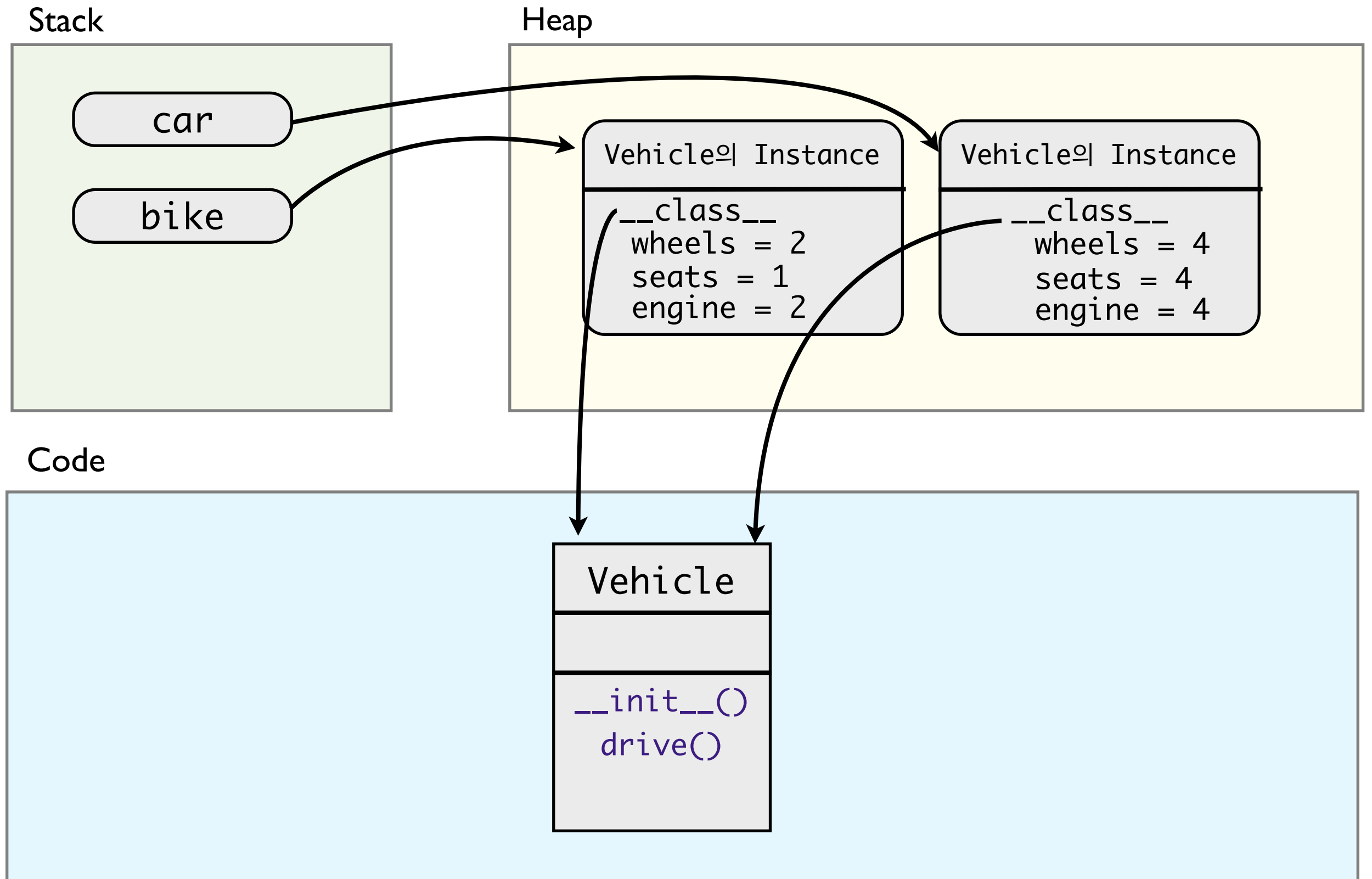
Heap



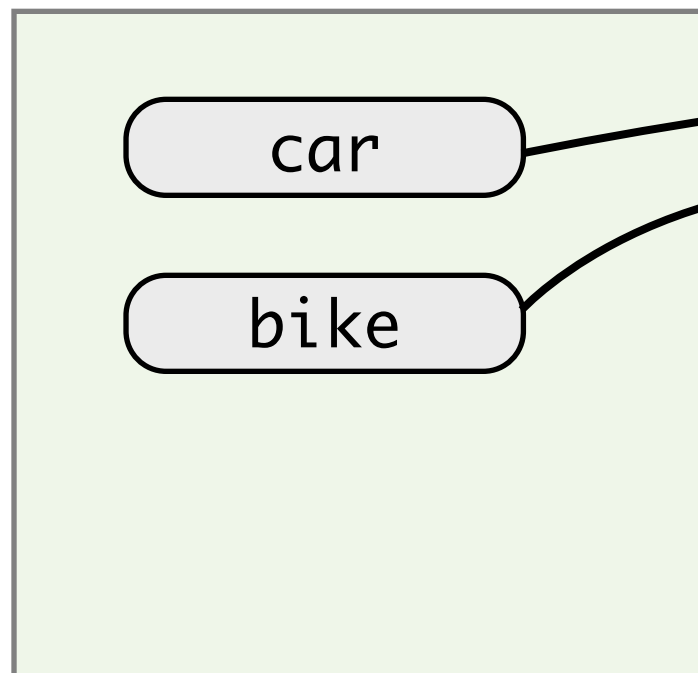
Code



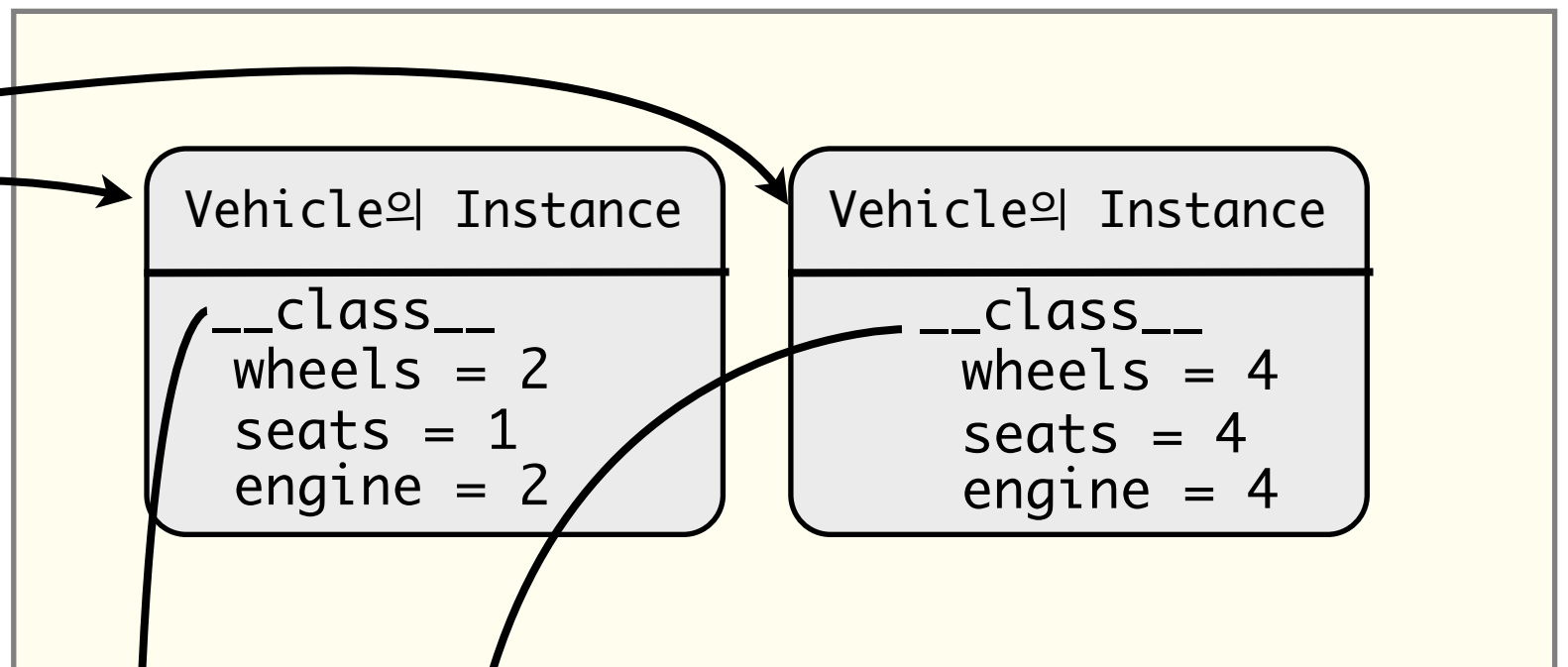
```
print('car.wheels = ', car.wheels)
```



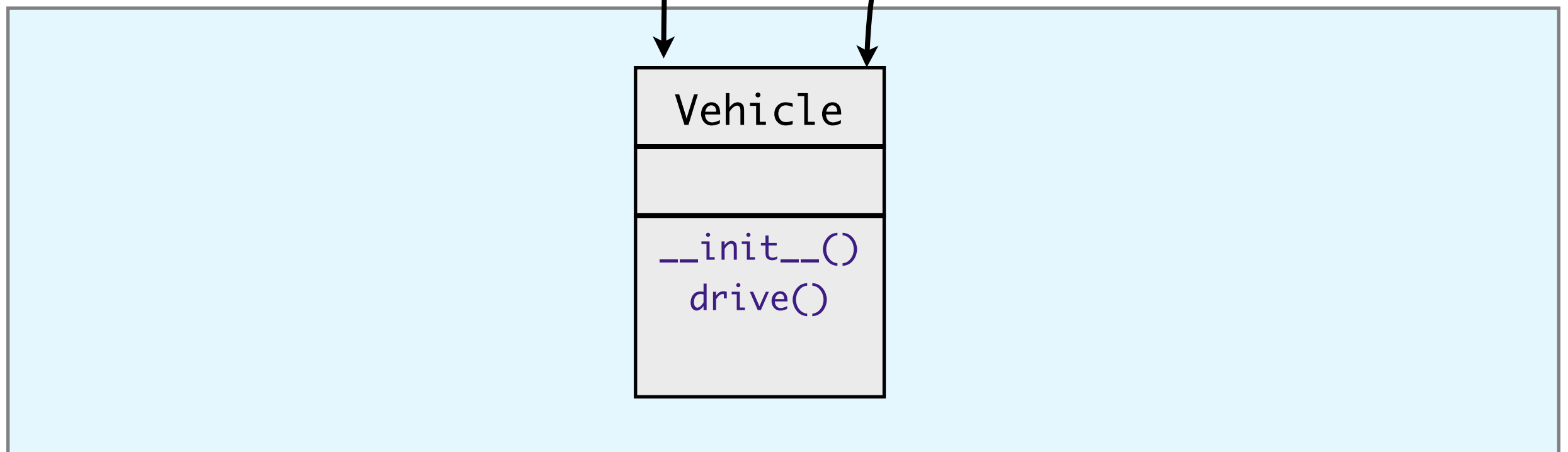
Stack



Heap

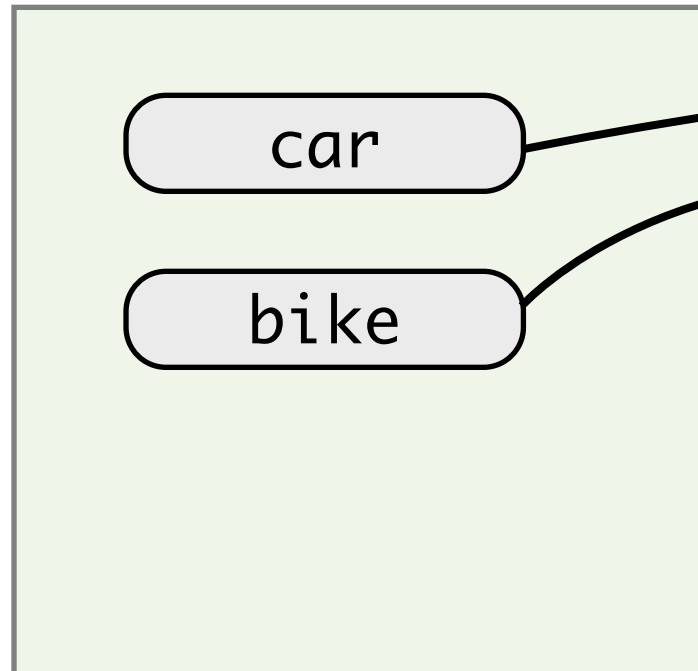


Code

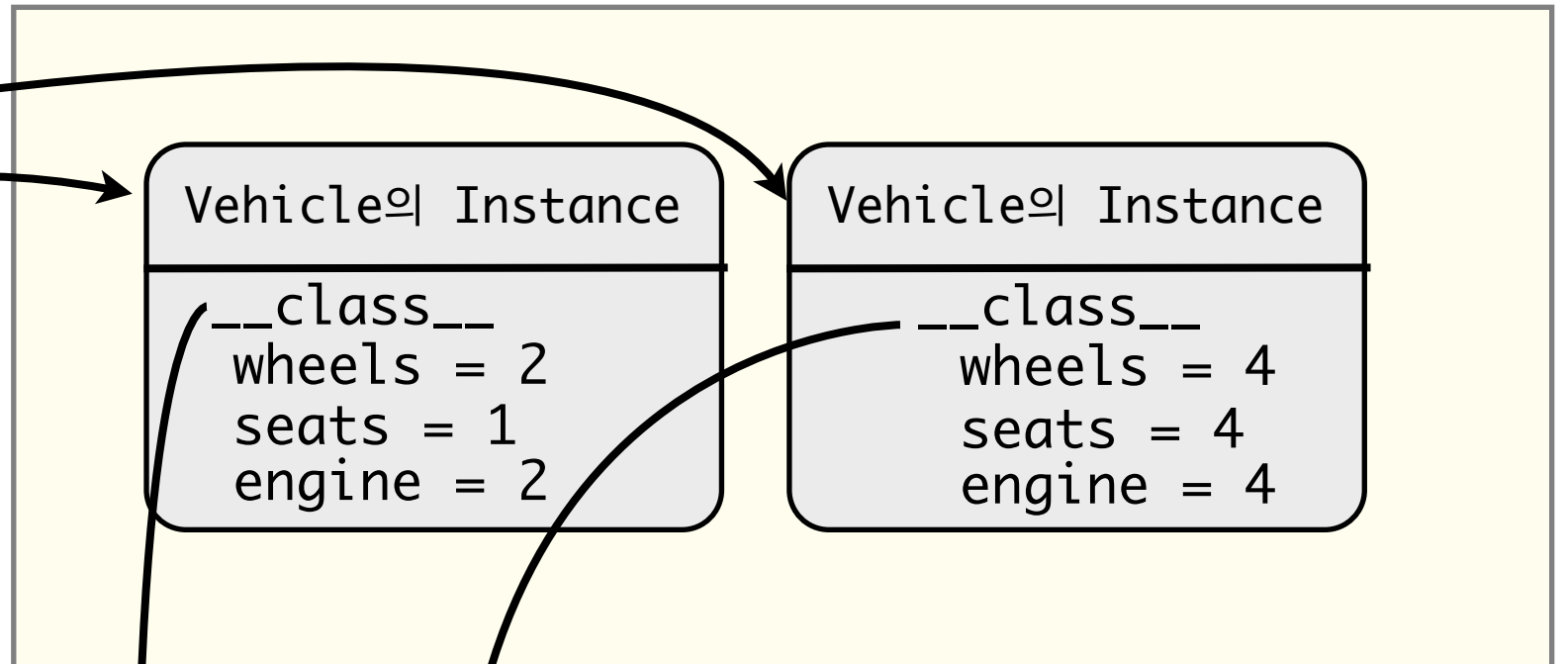


`car.drive()`

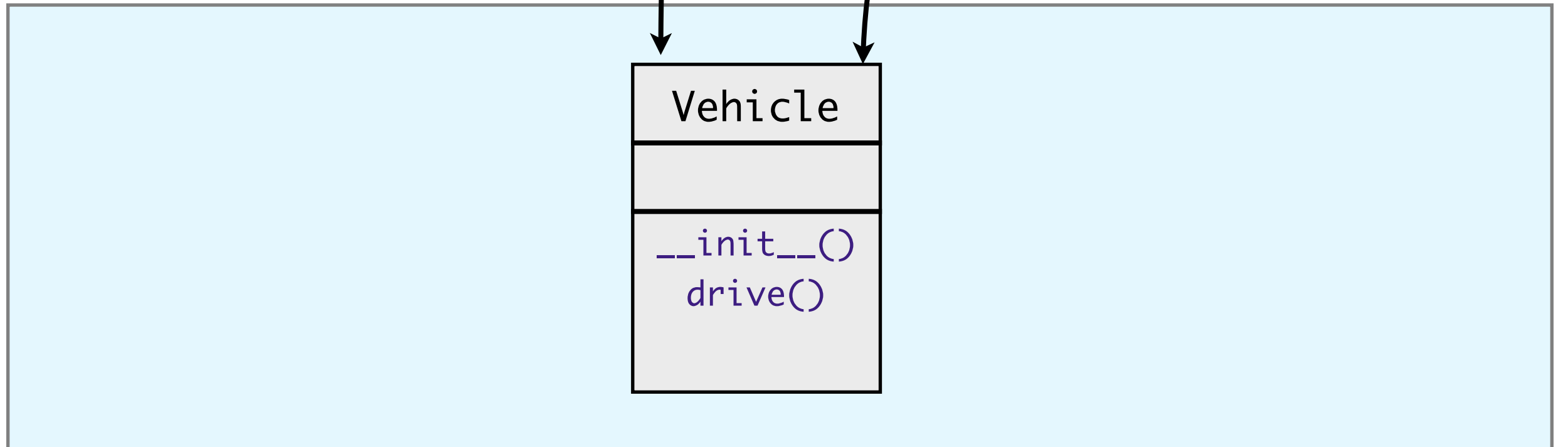
Stack



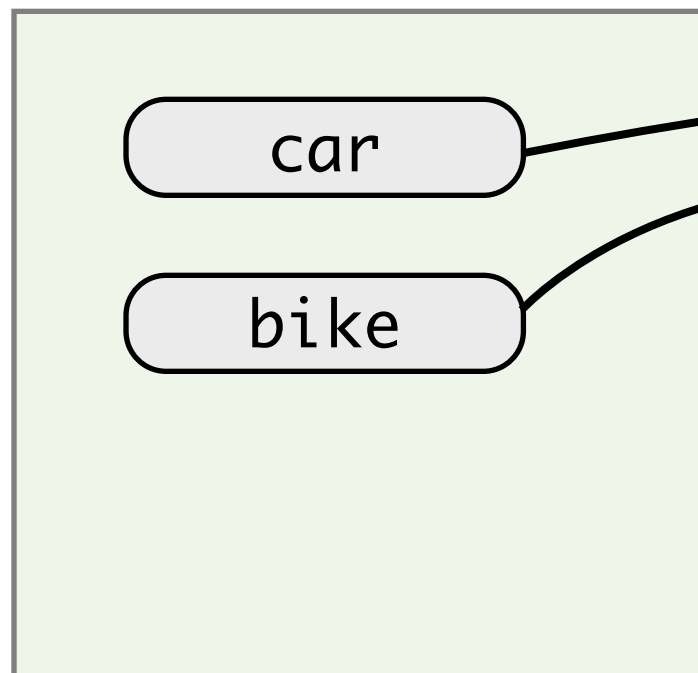
Heap



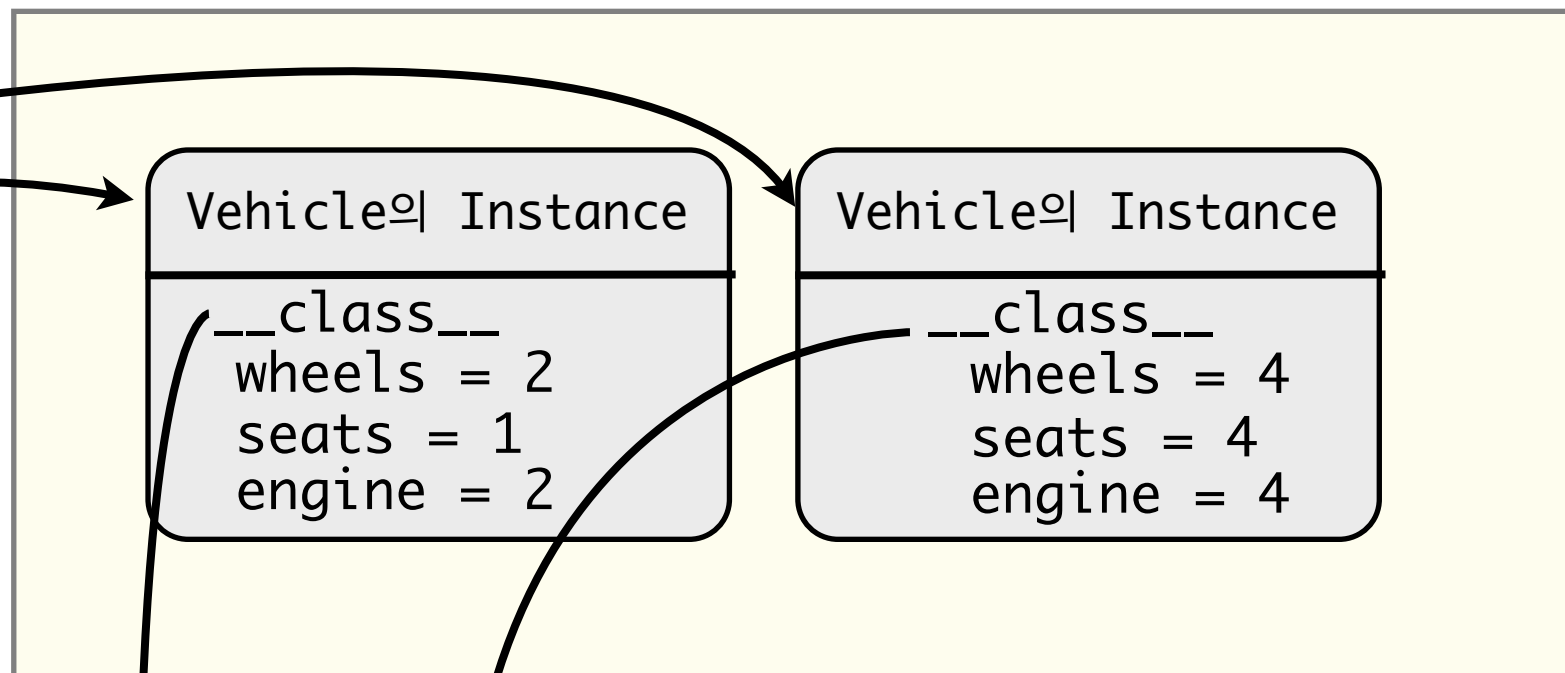
Code



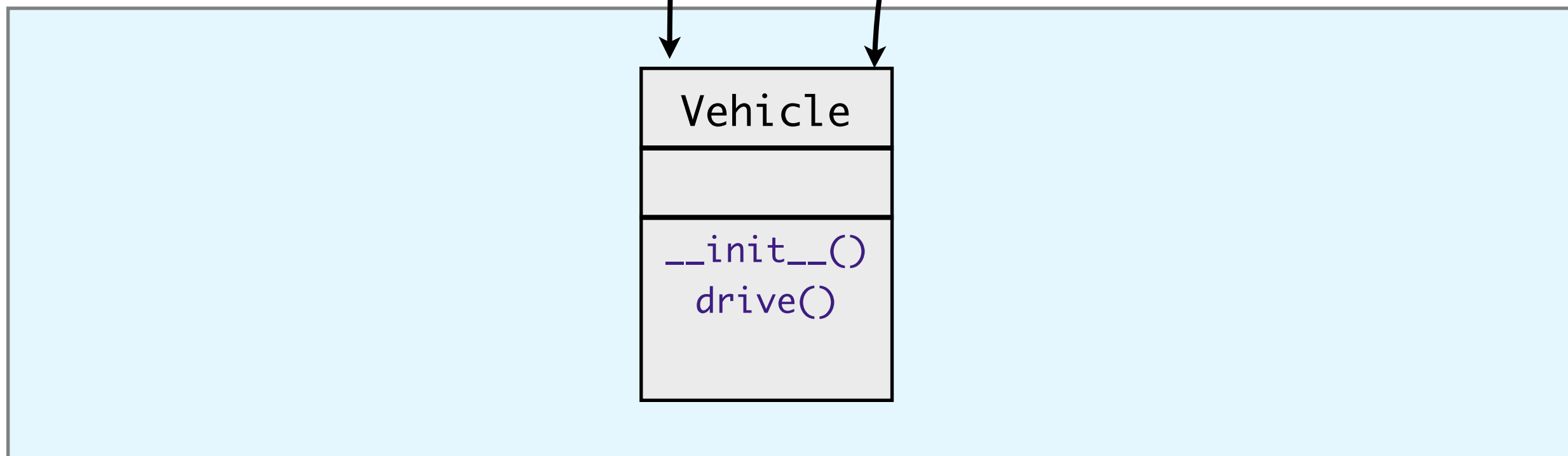
Stack



Heap

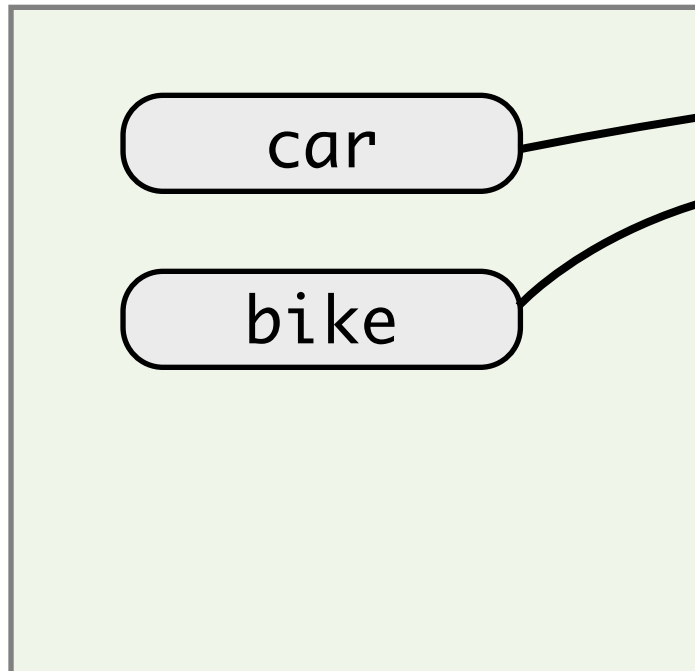


Code

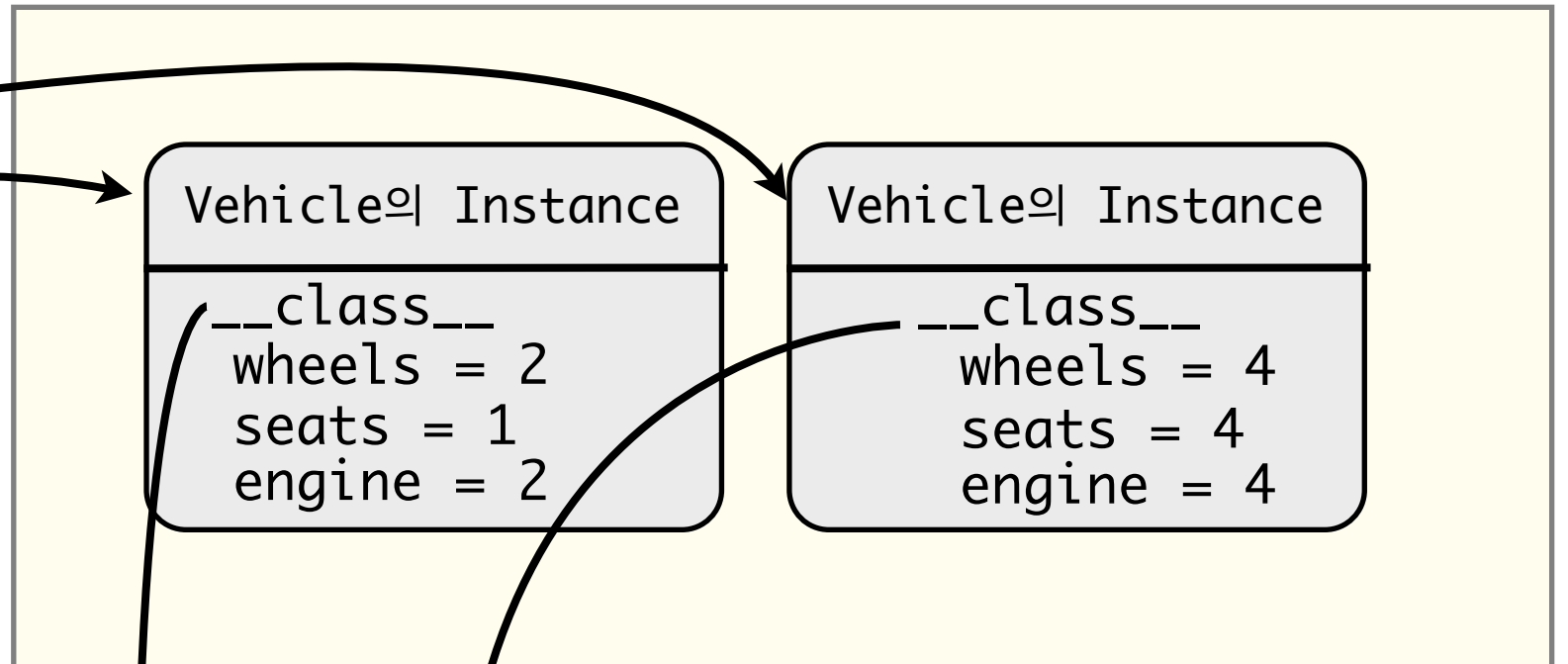


copy_bike = bike

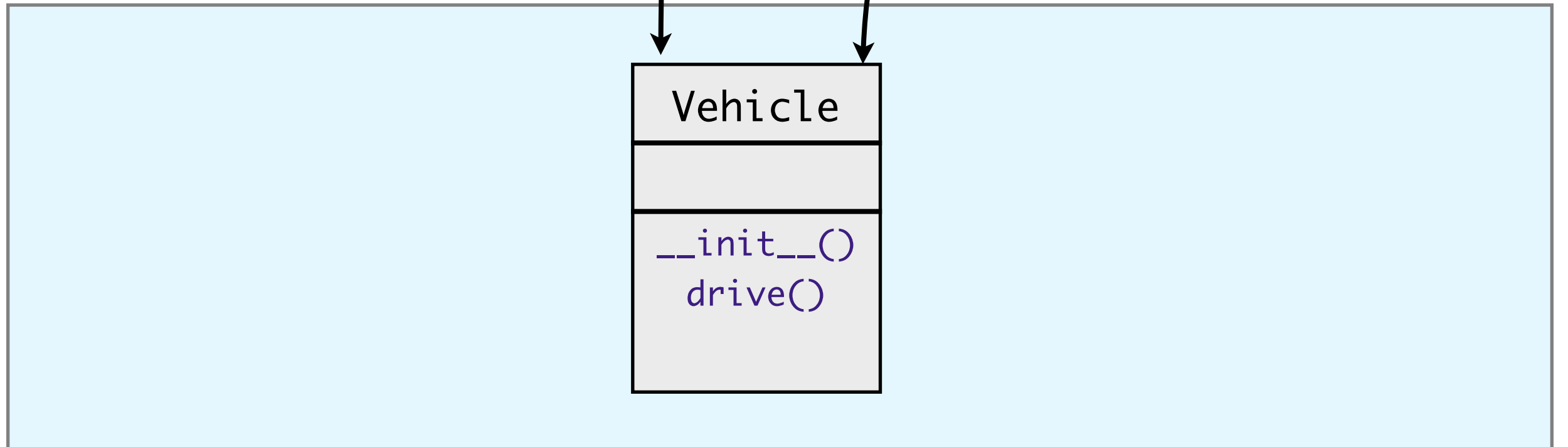
Stack



Heap

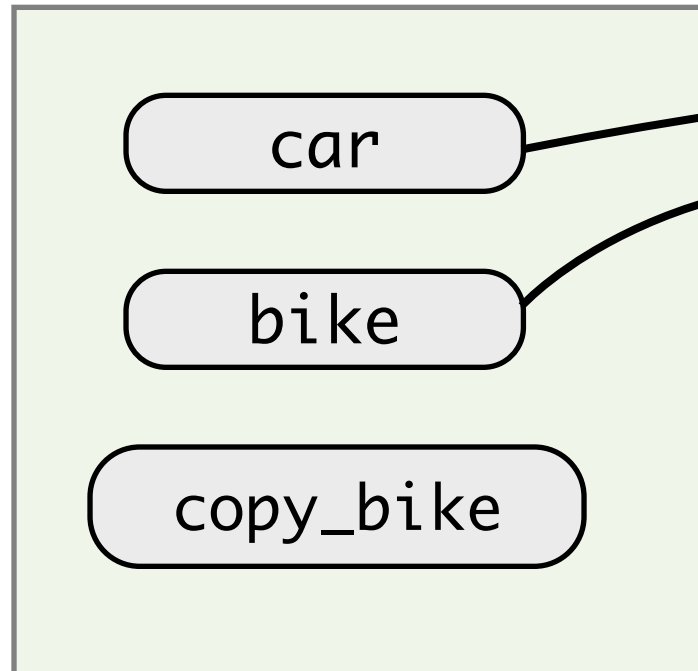


Code

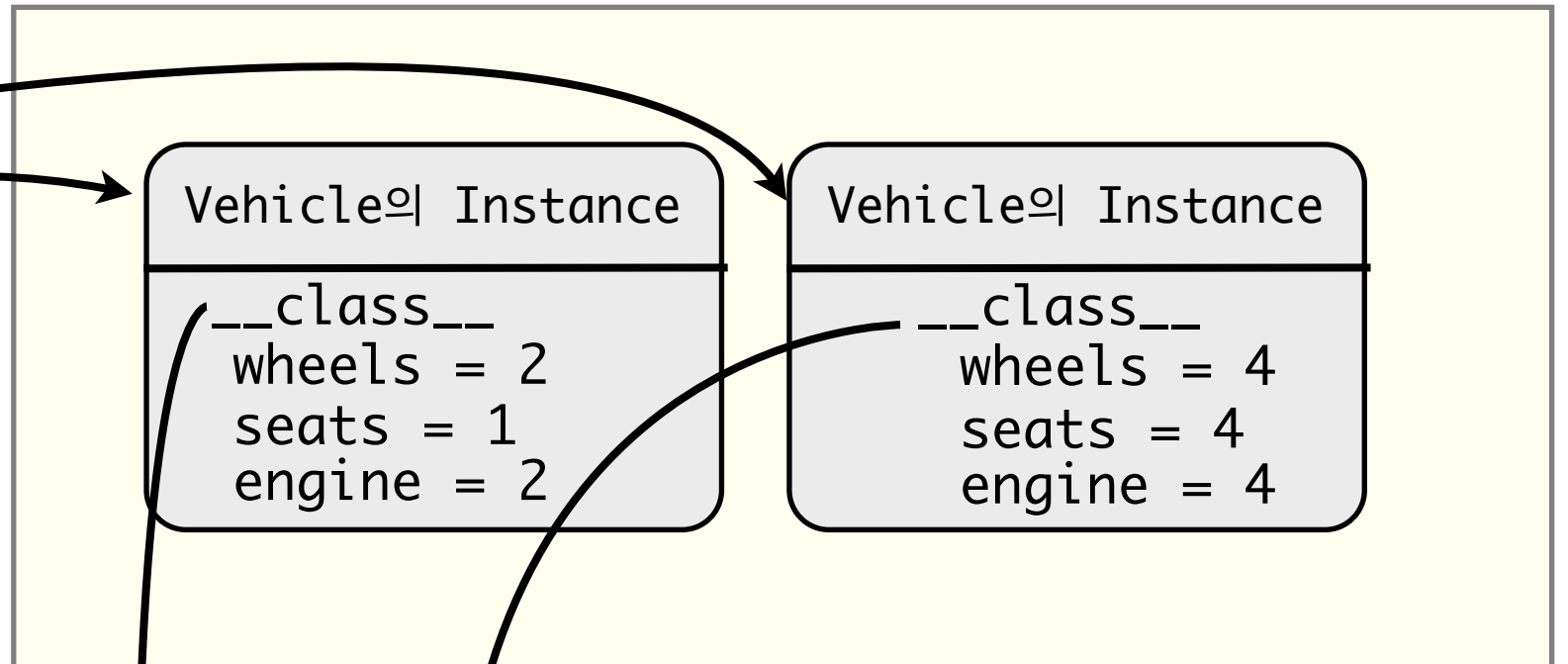


copy_bike = bike

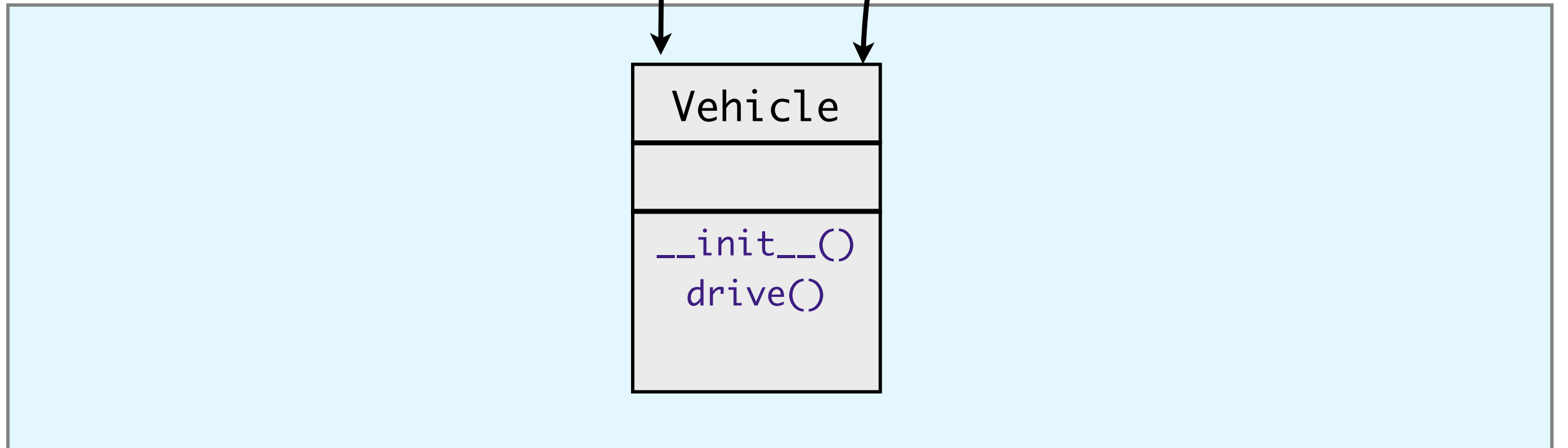
Stack



Heap

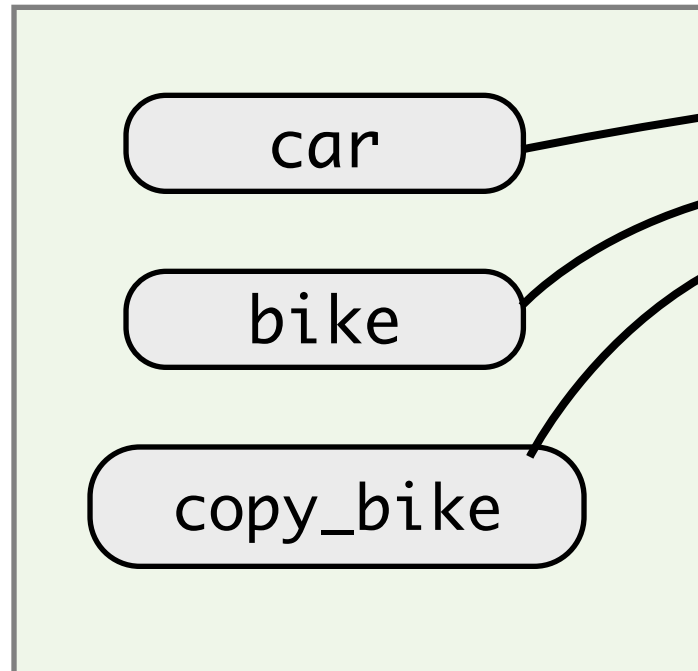


Code

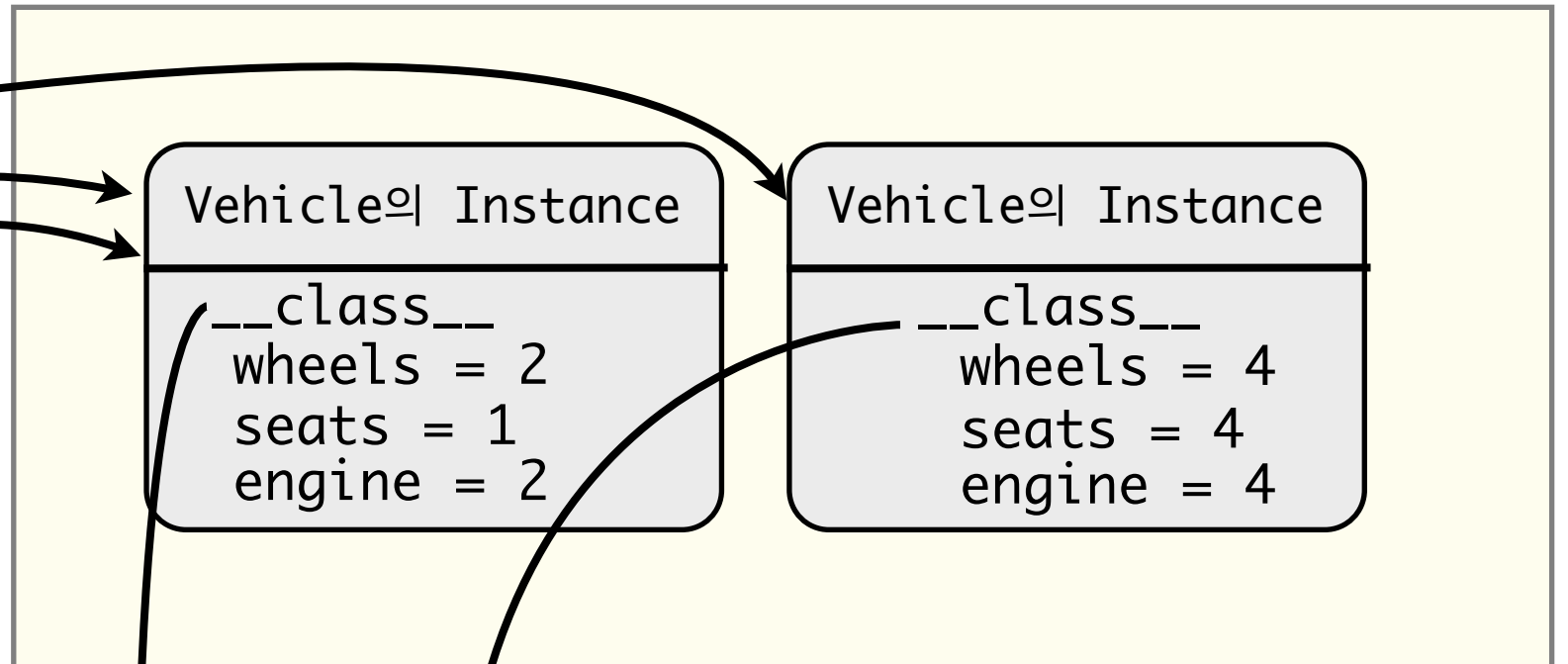


copy_bike = bike

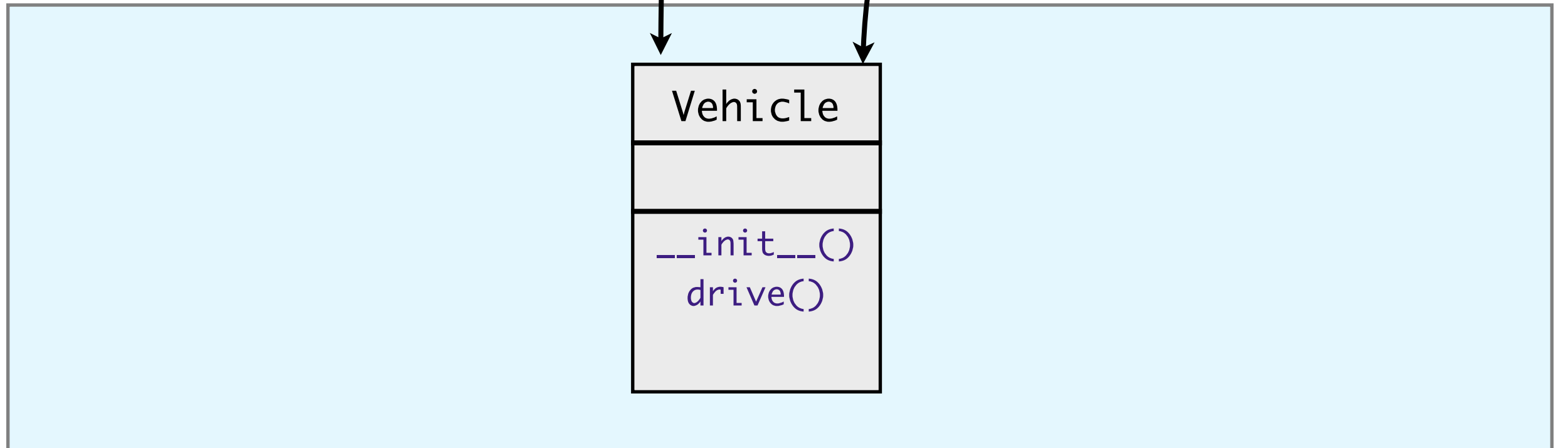
Stack



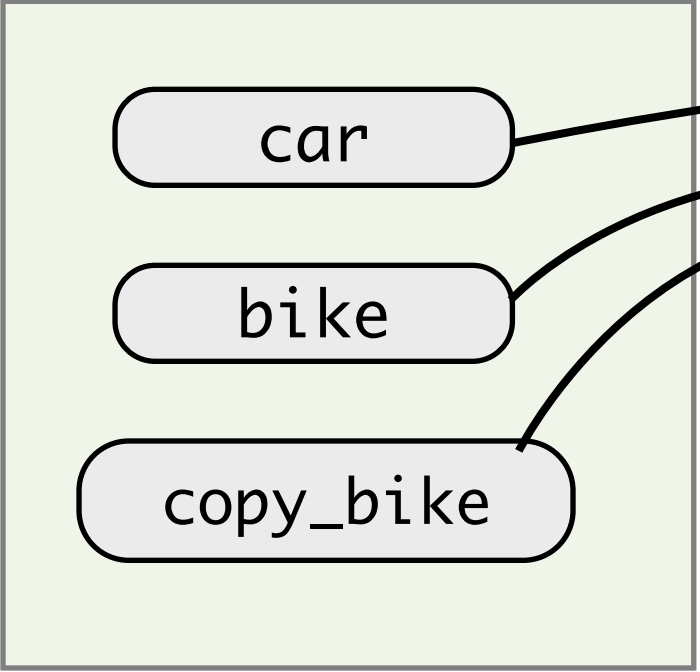
Heap



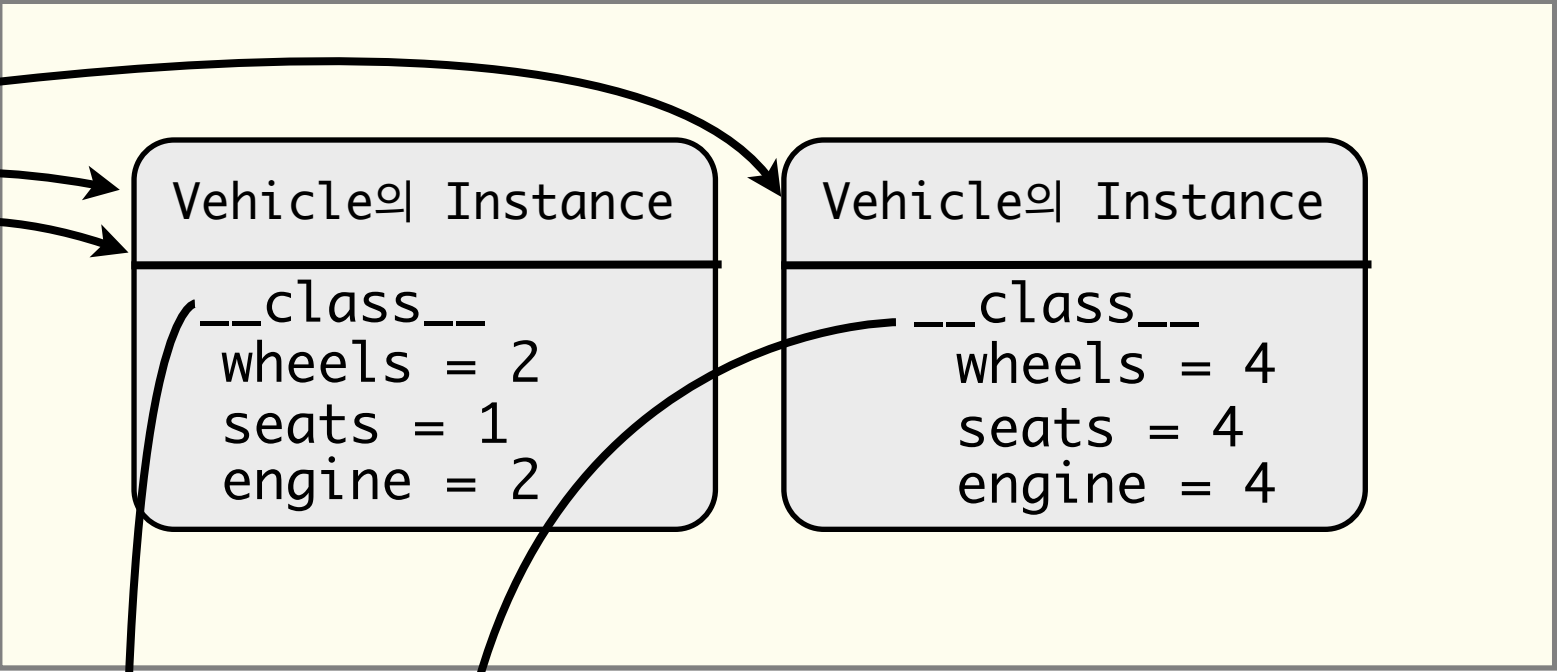
Code



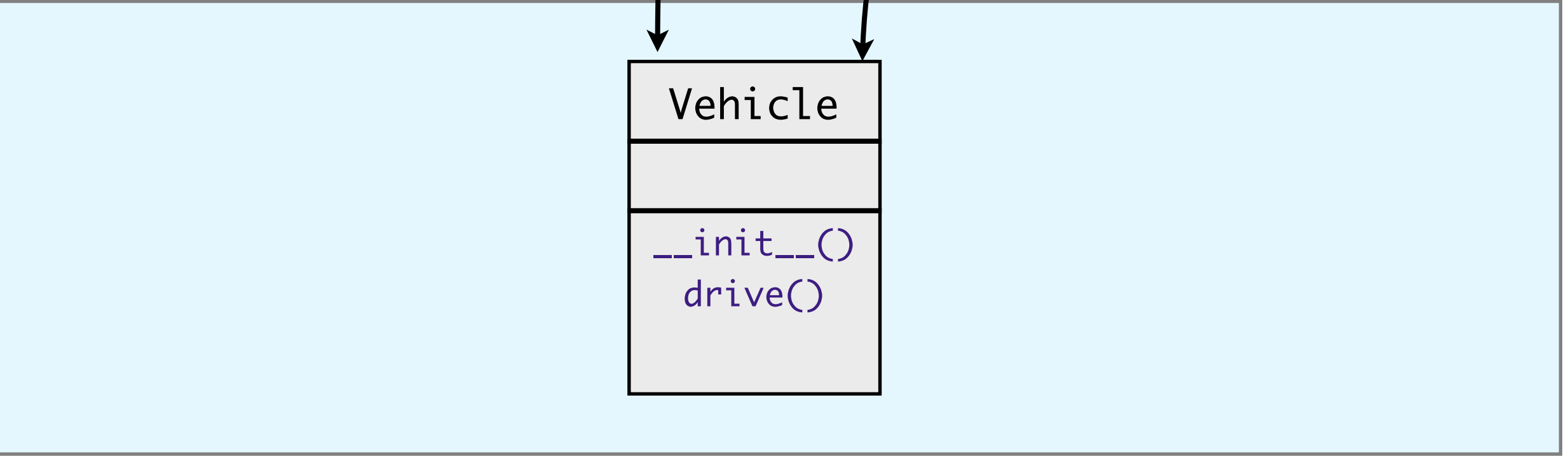
Stack



Heap

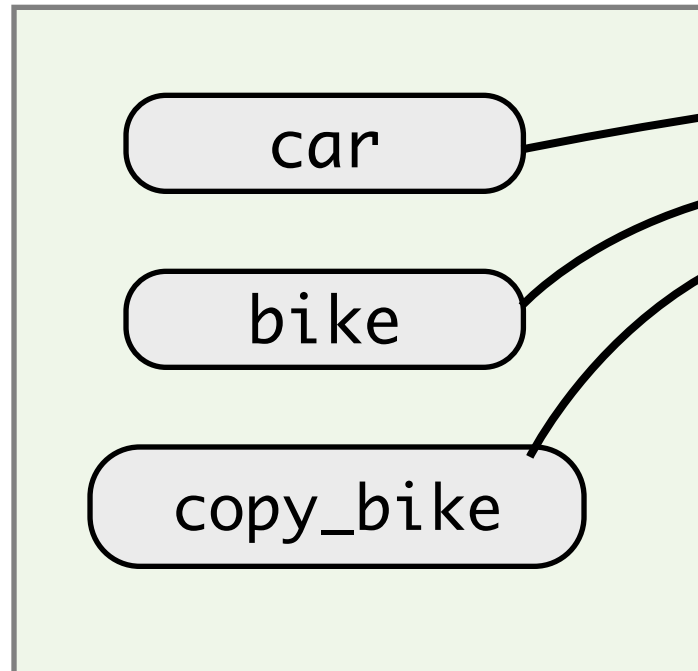


Code

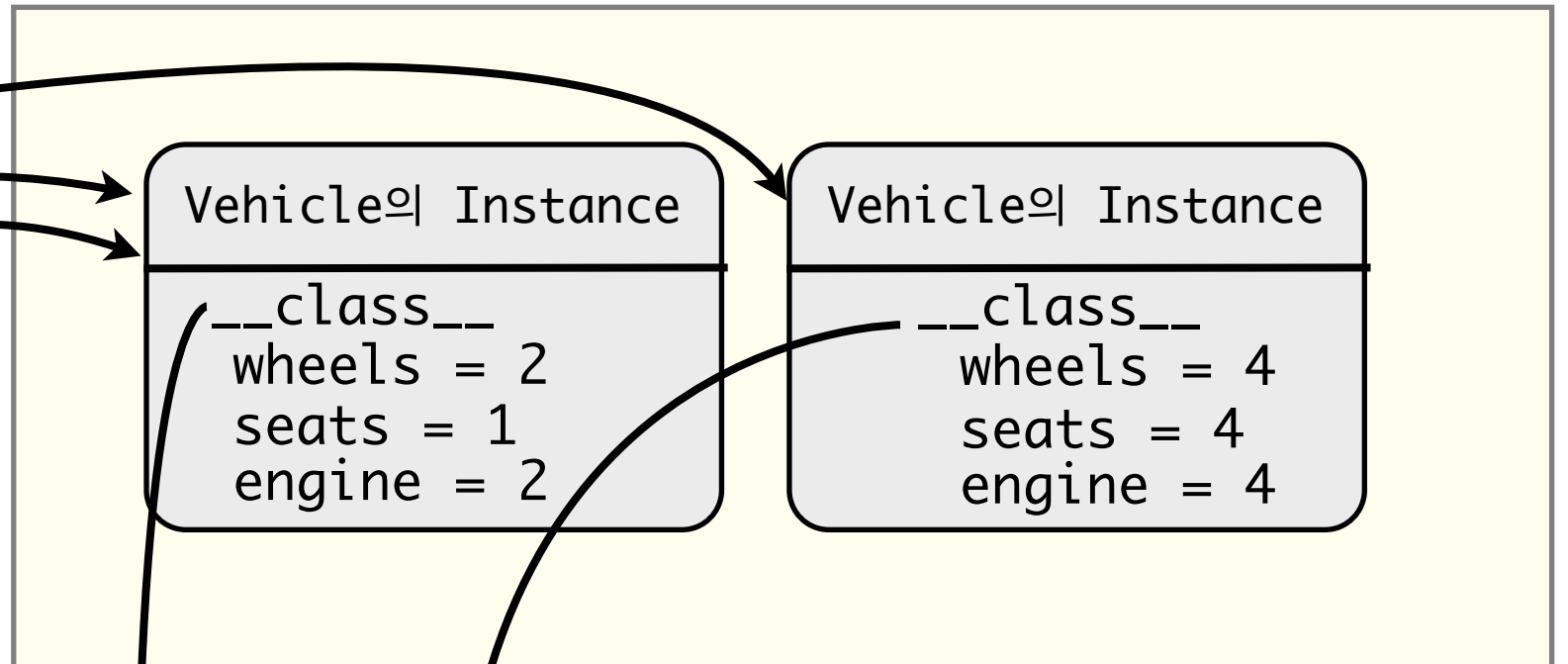


copy_bike.wheels = 4

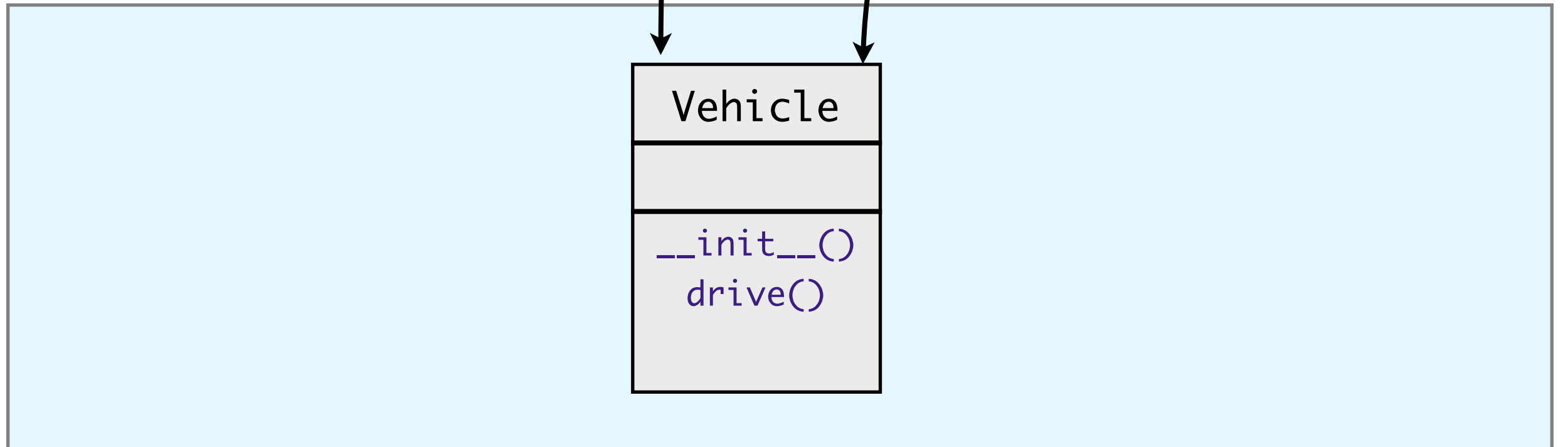
Stack



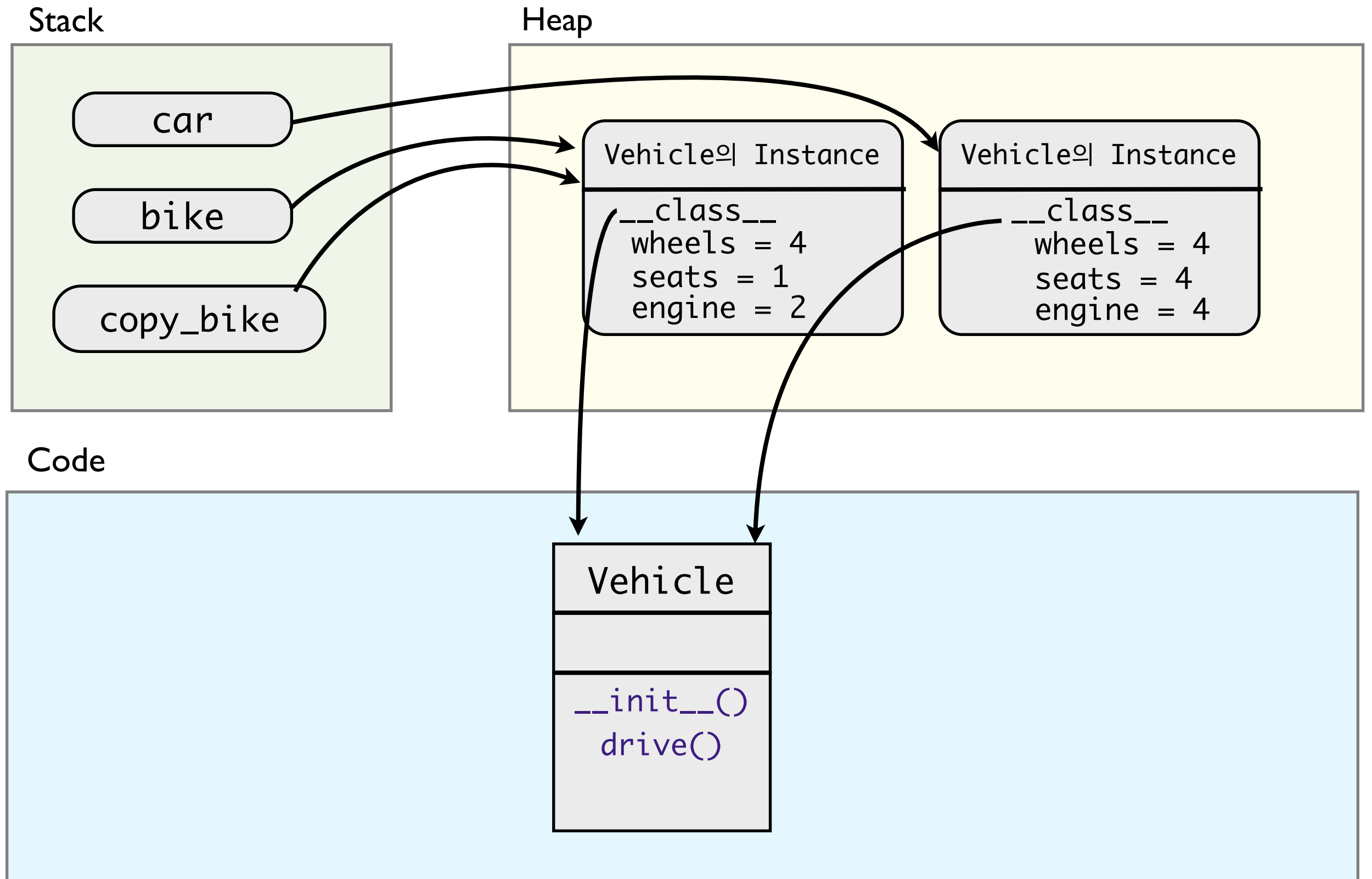
Heap



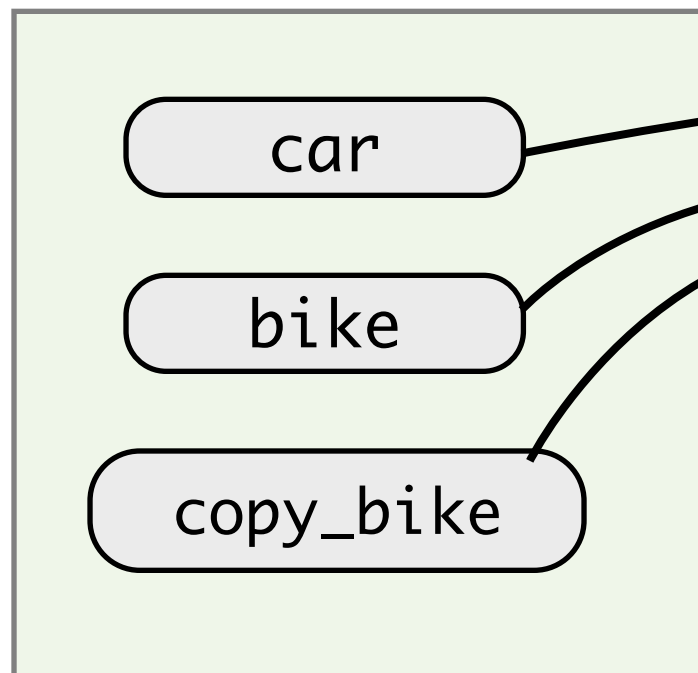
Code



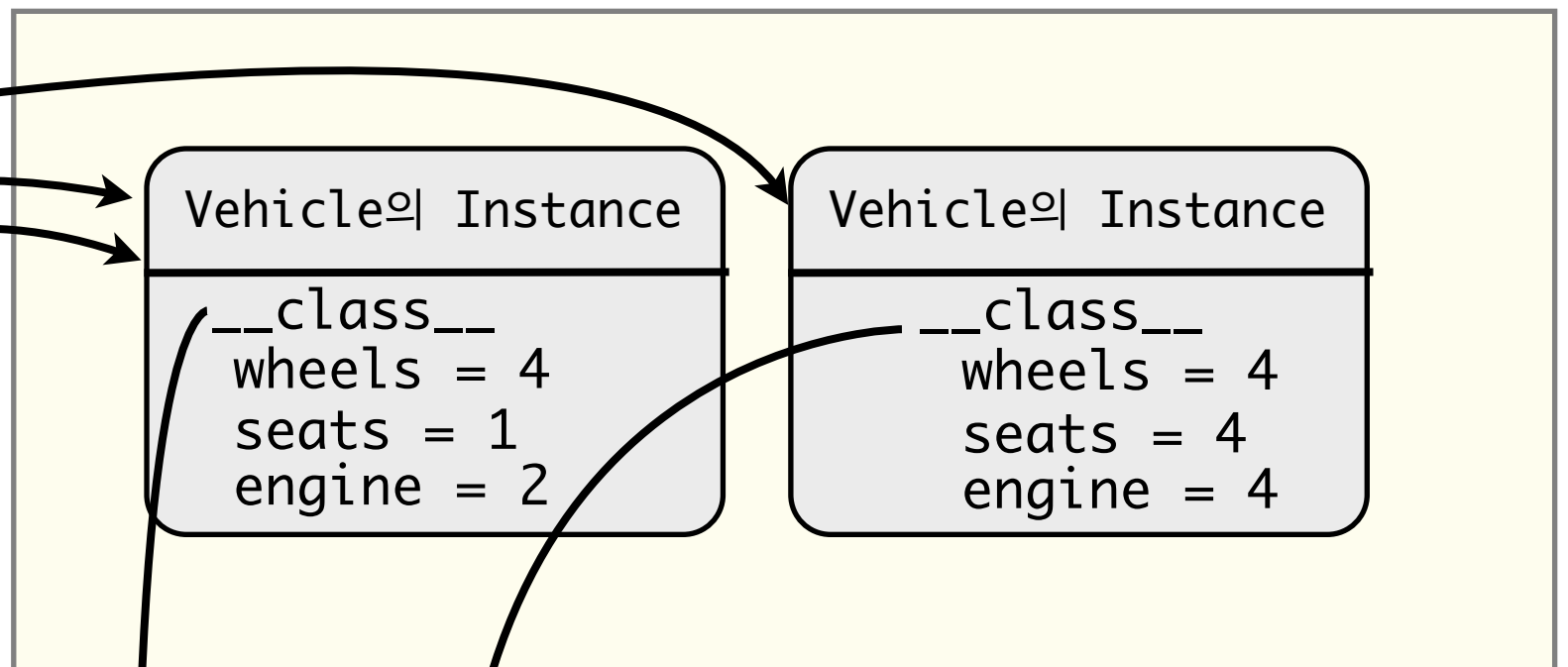
copy_bike.wheels = 4



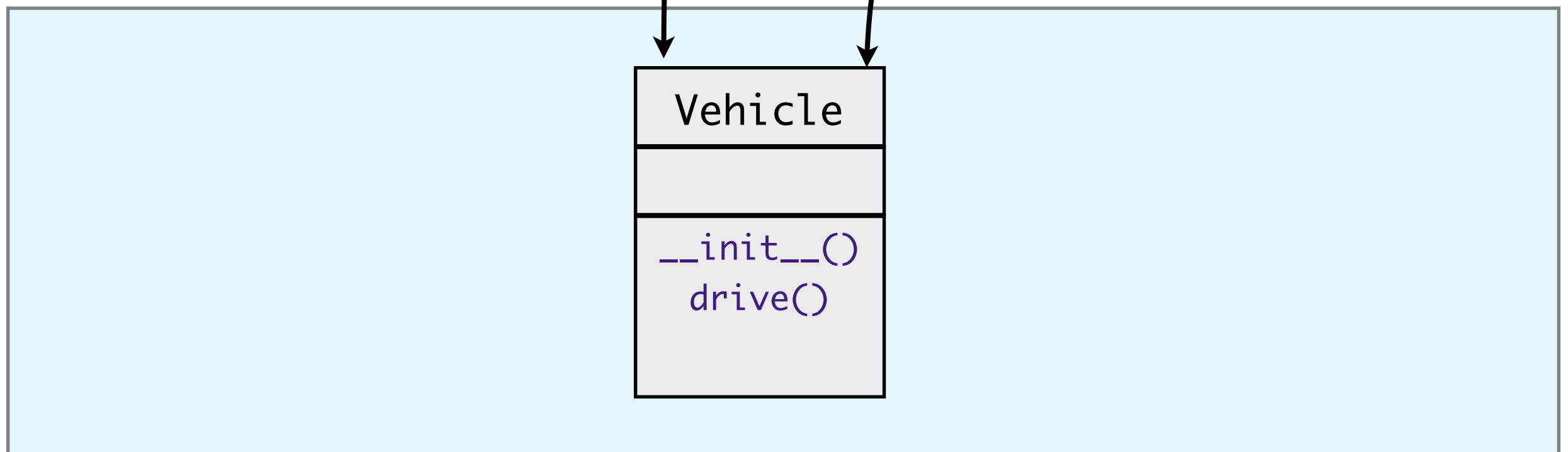
Stack



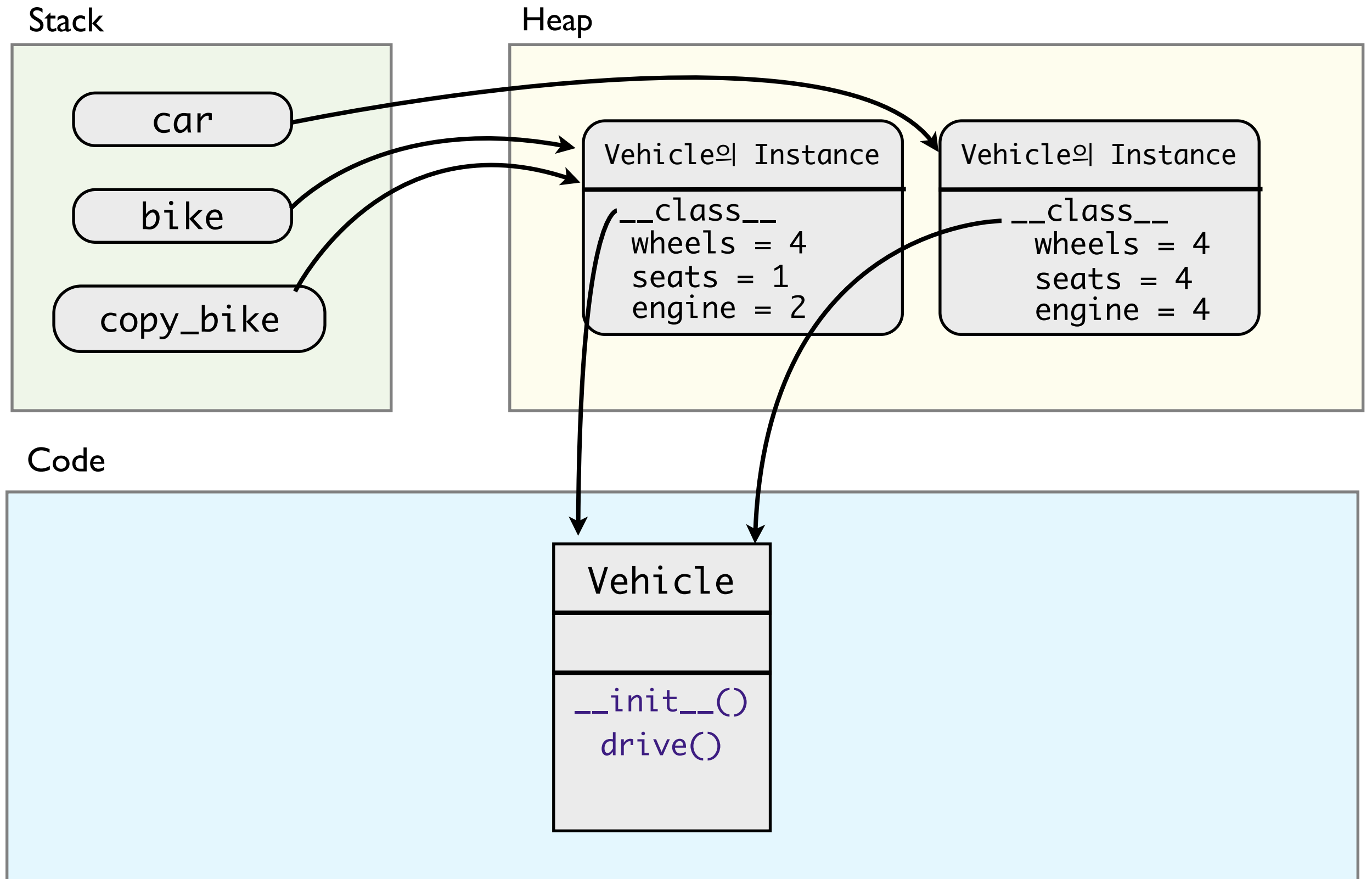
Heap



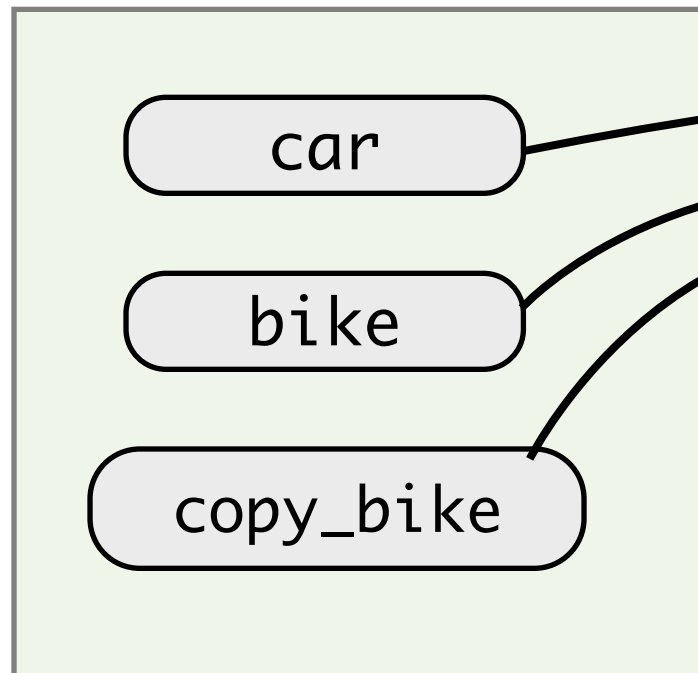
Code



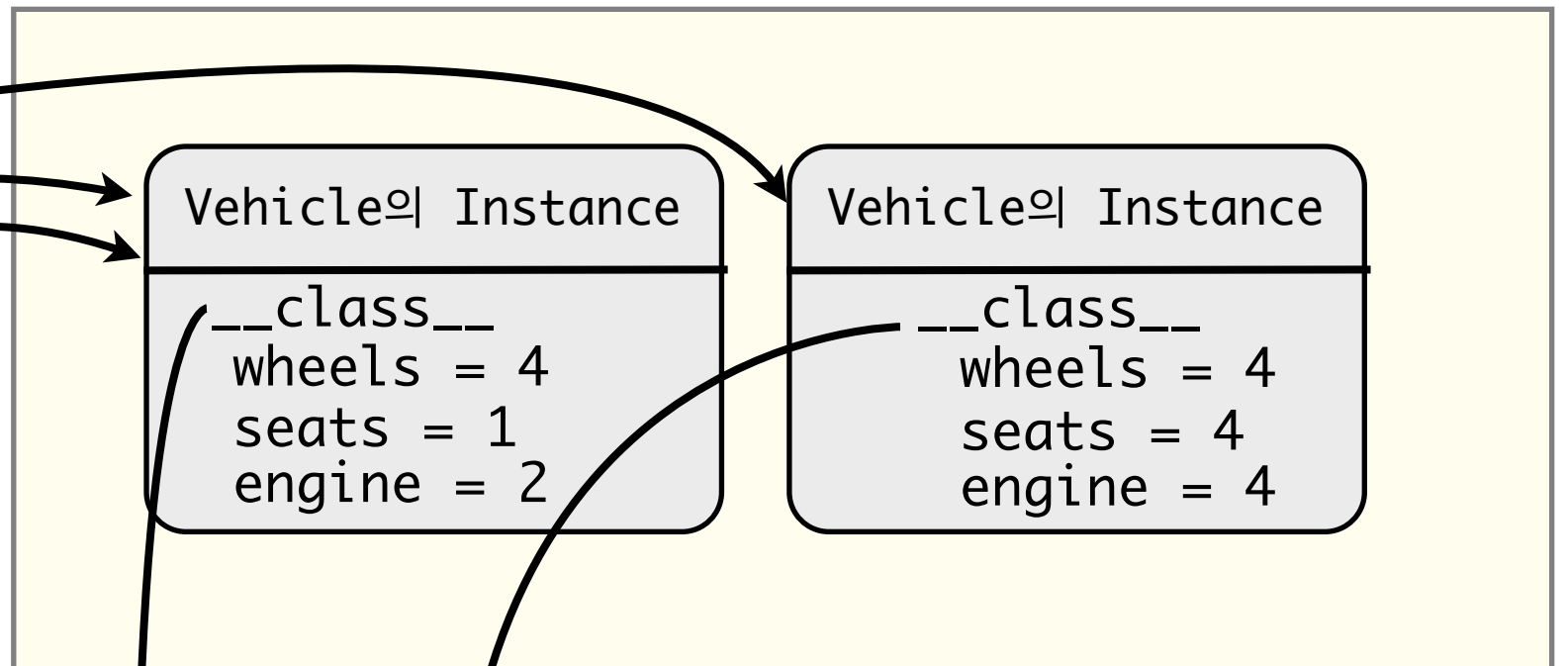
```
print('bike.wheels = ', bike.wheels)
```



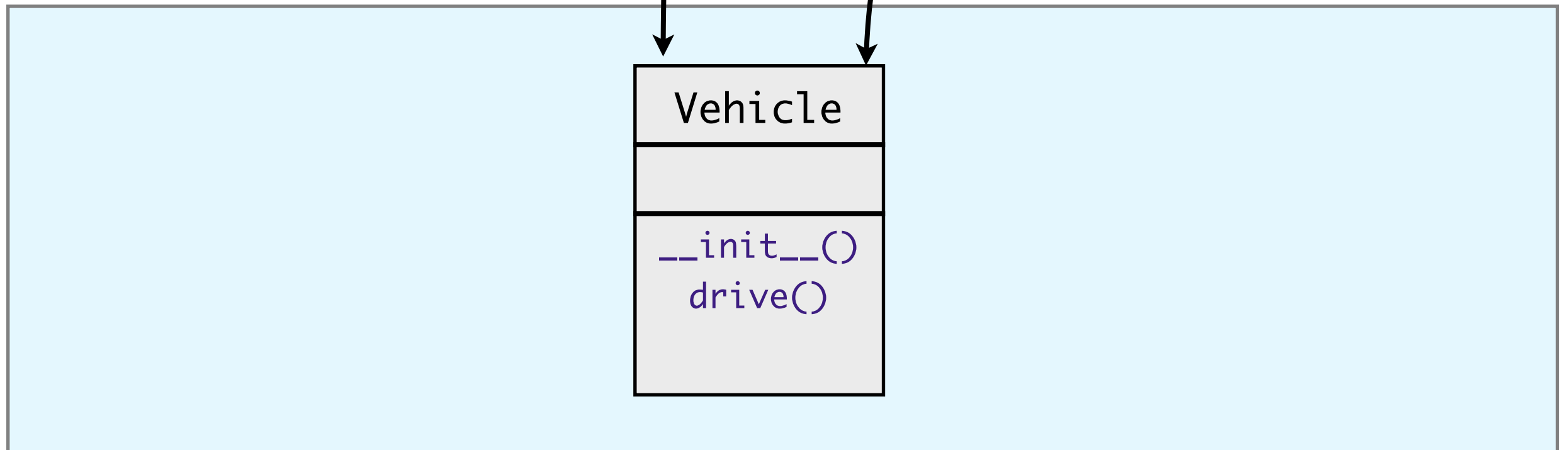
Stack



Heap



Code



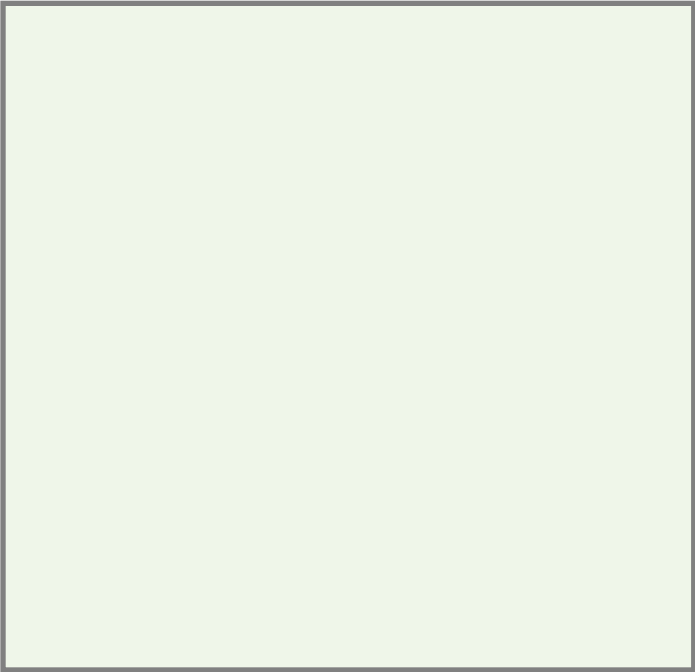
상속

```
class Car(Vehicle):
    def __init__(self):
        super().__init__(4, 5, 4)

    def shift_gear(self):
        print('변속')

car1 = Car()
car1.wheels = 6
print('car1.wheels = ', car1.wheels)
print('car1.seats = ', car1.seats)
car1.drive()
```

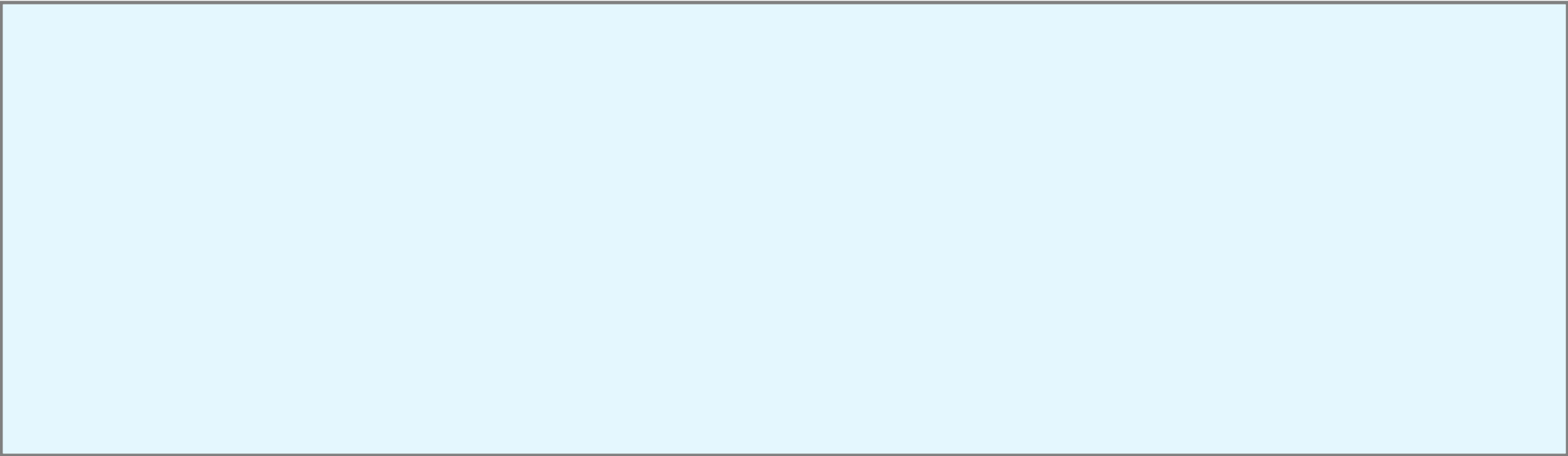
Stack



Heap

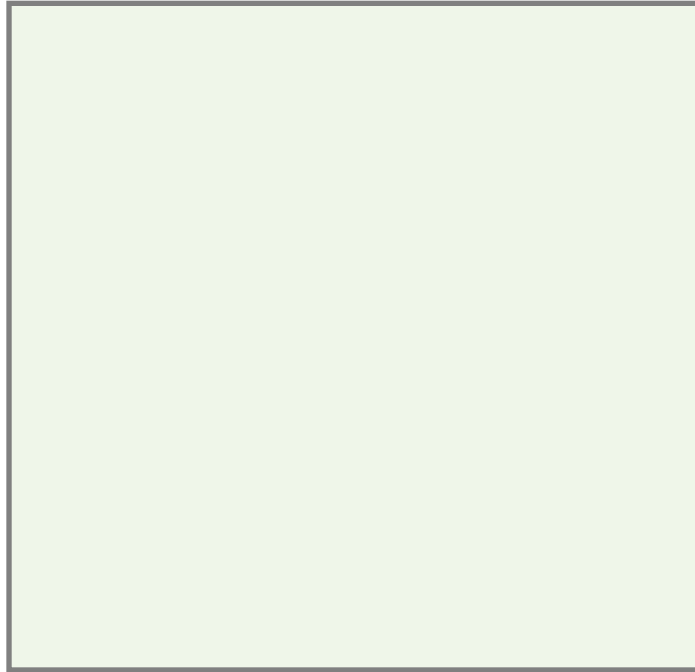


Code



`car1 = Car()`

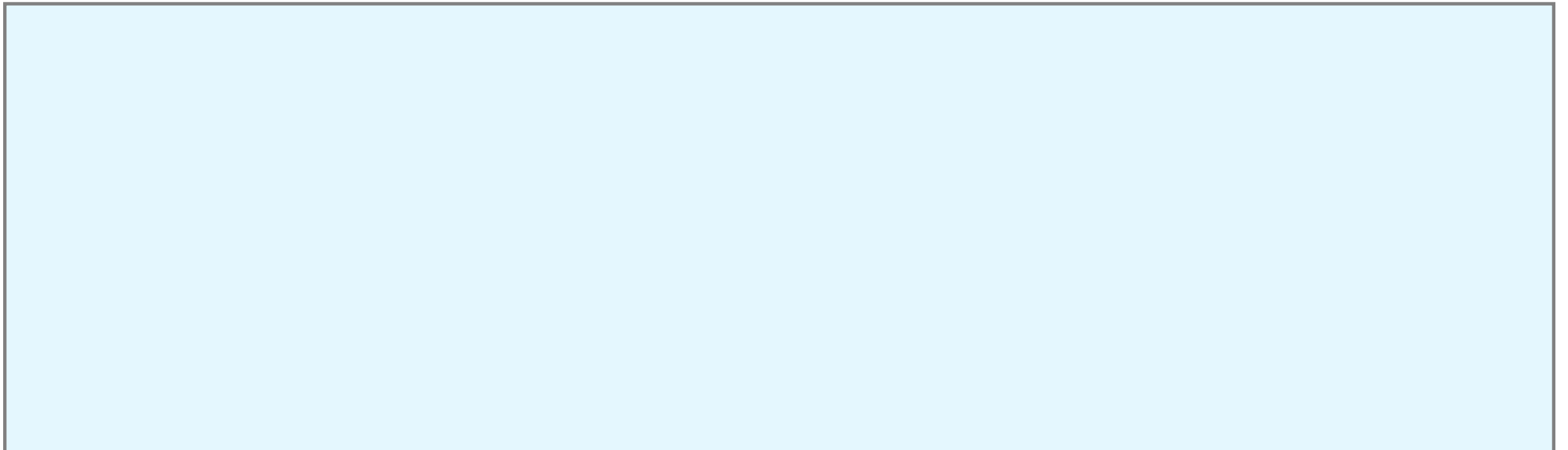
Stack



Heap

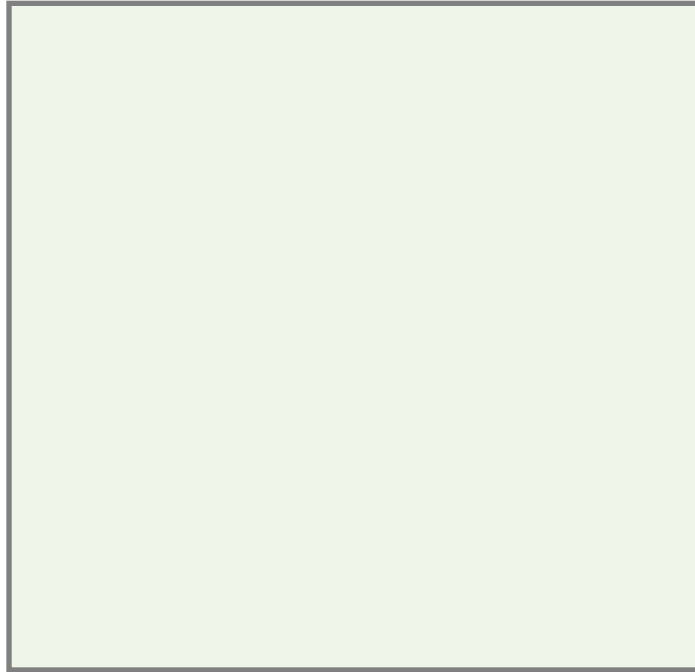


Code



car1 = Car()

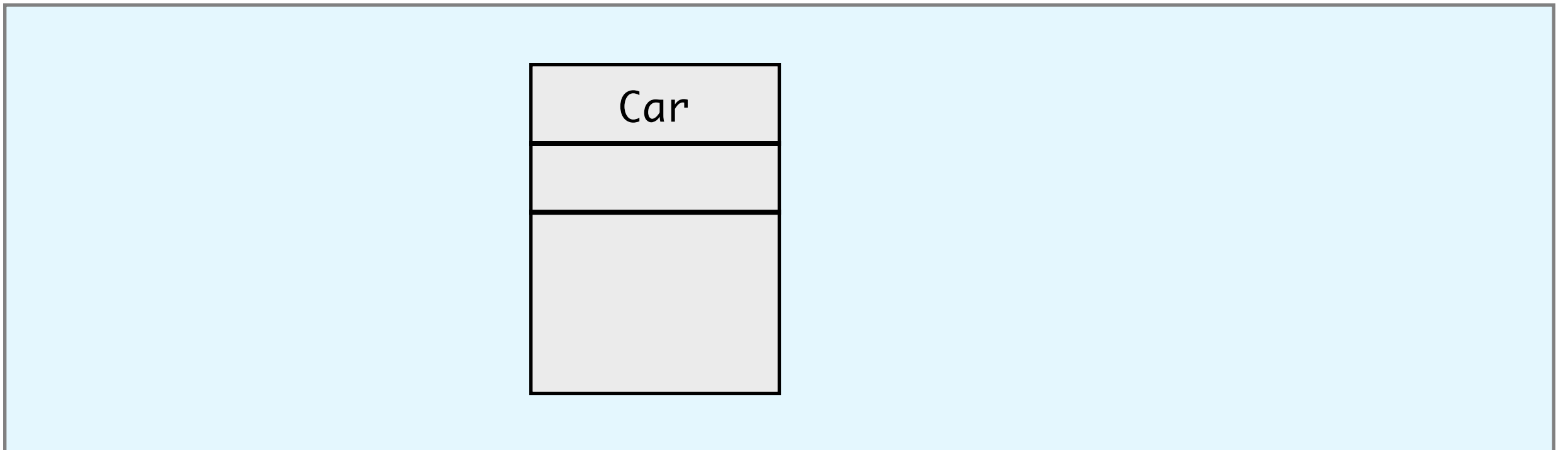
Stack



Heap

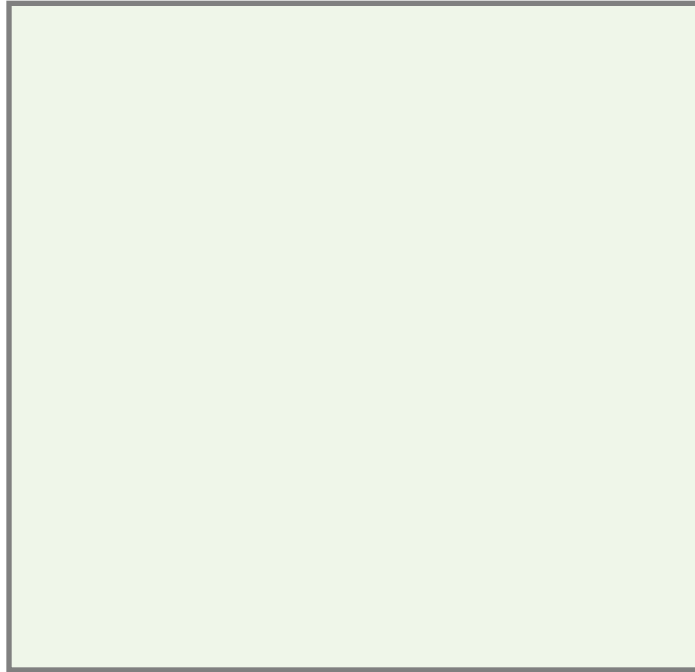


Code

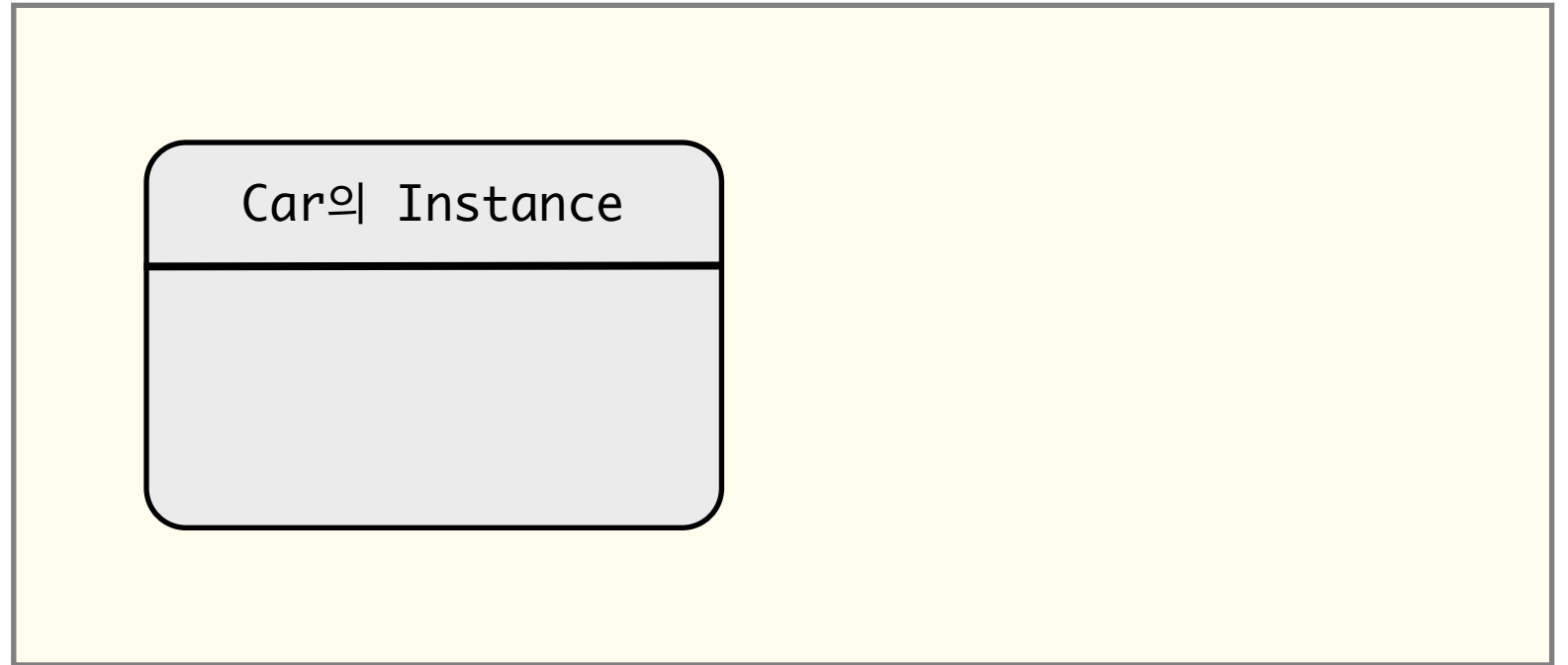


car1 = Car()

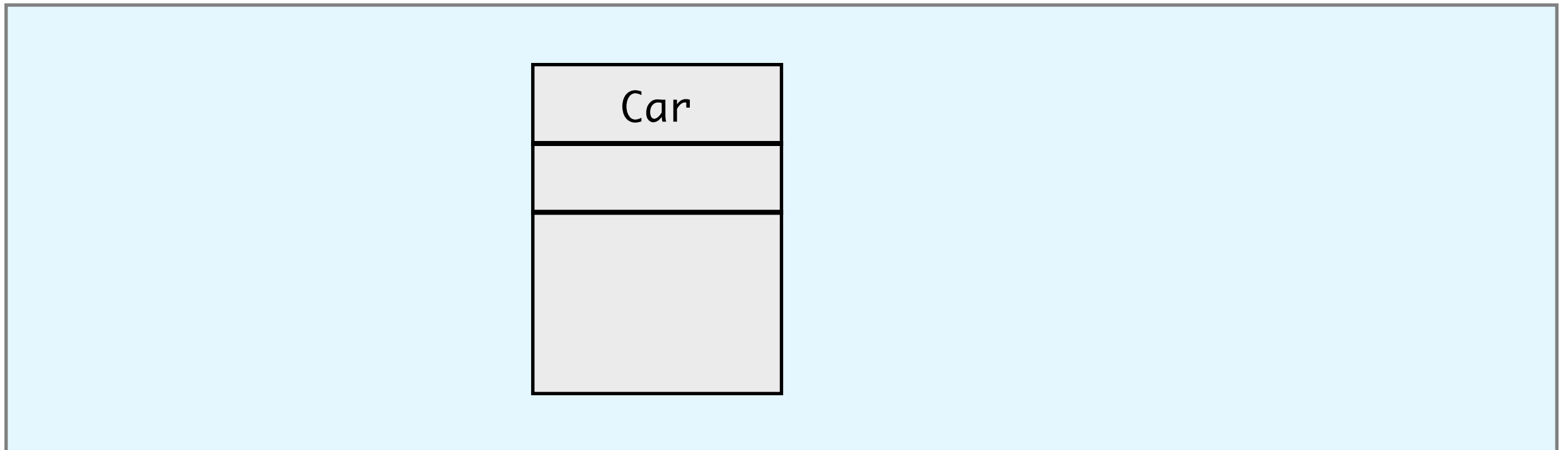
Stack



Heap

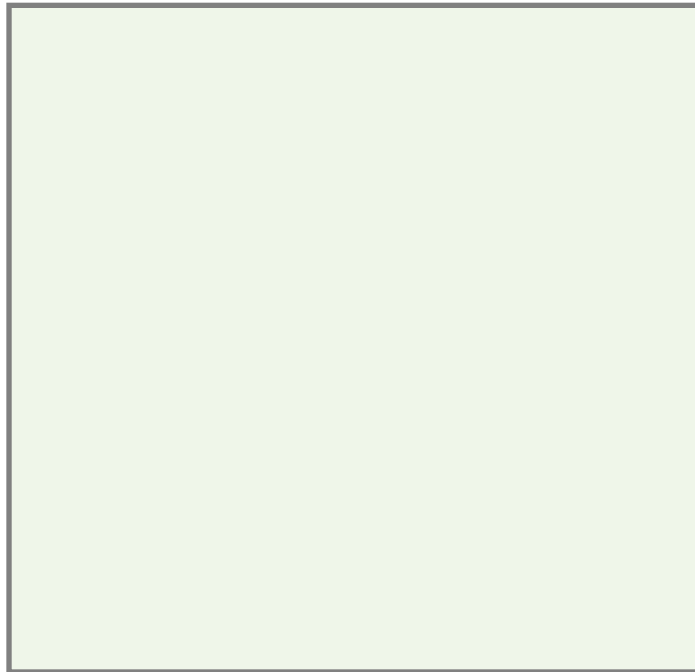


Code

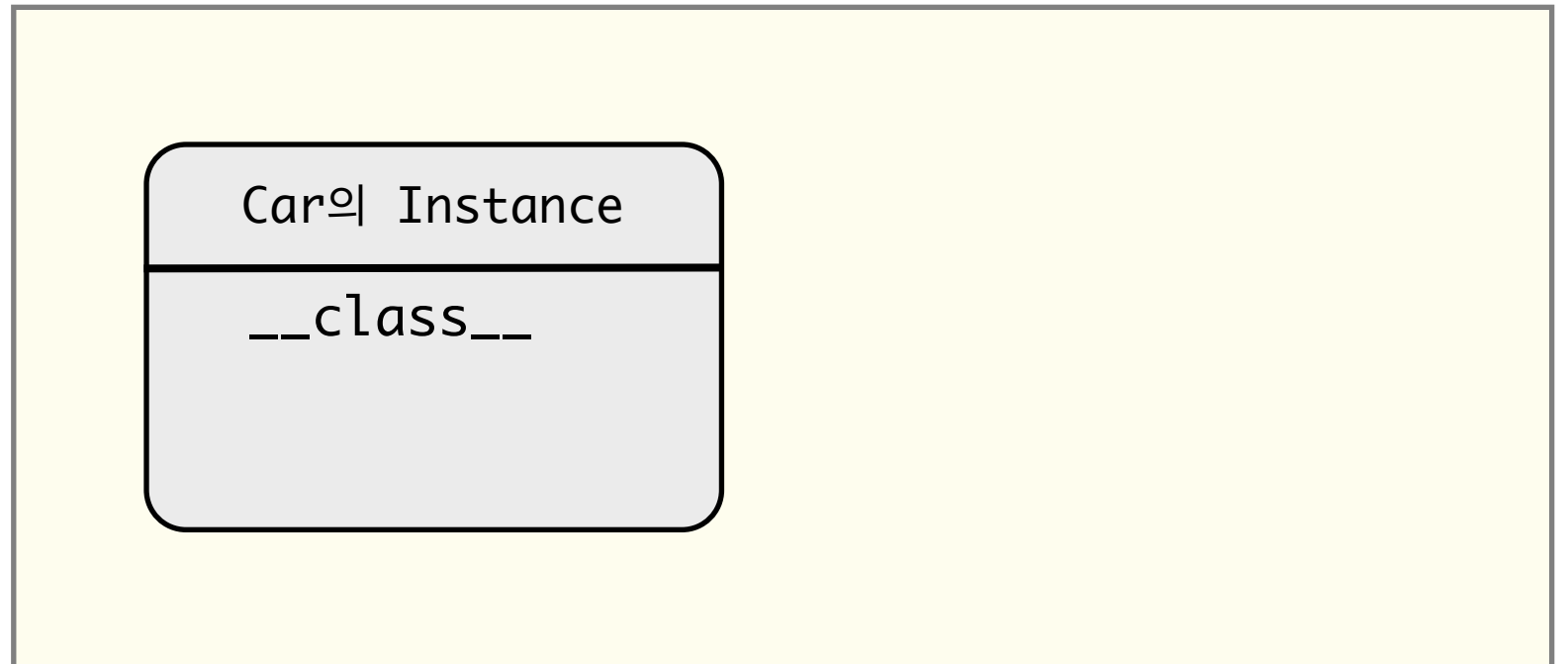


car1 = Car()

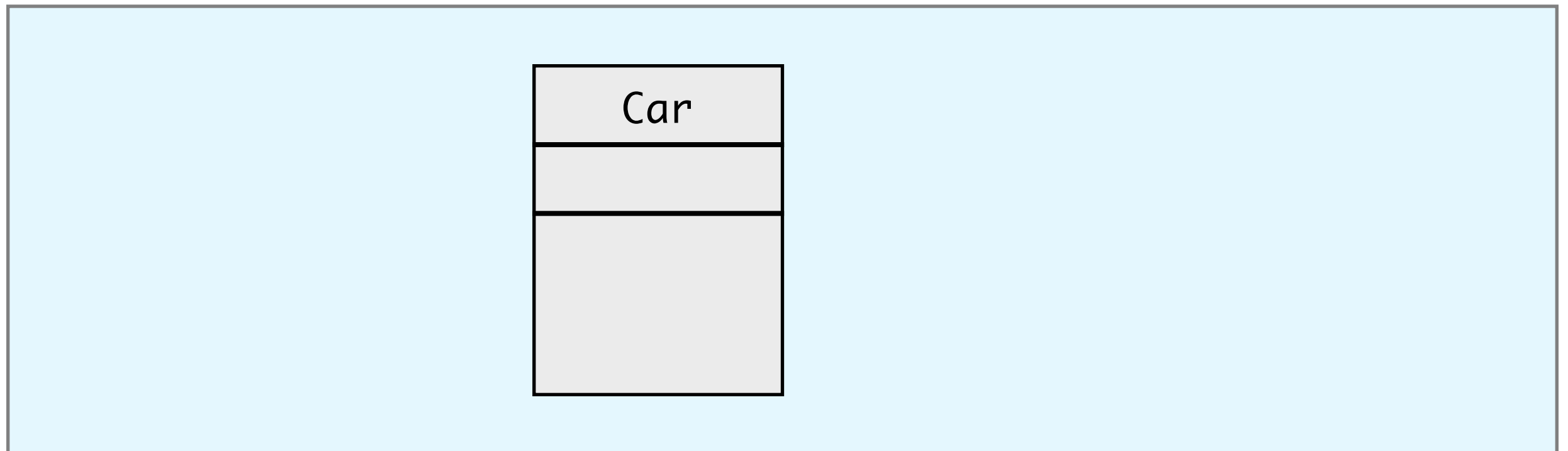
Stack



Heap

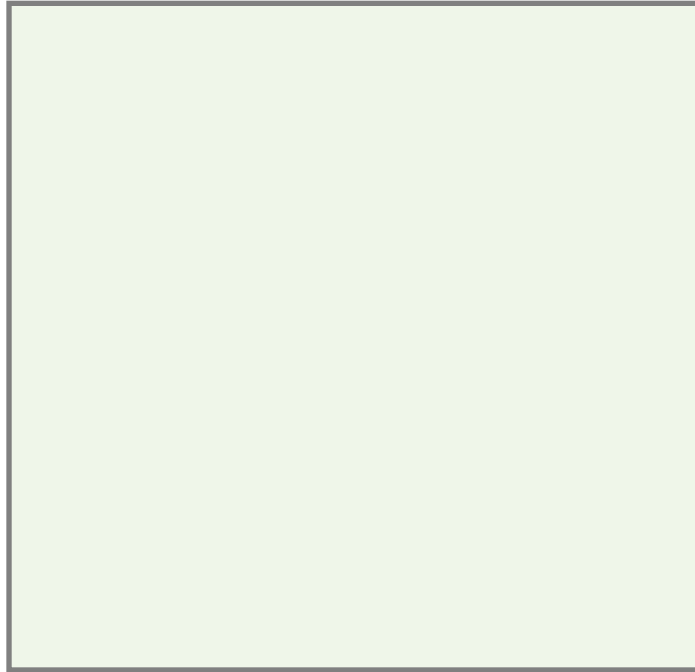


Code

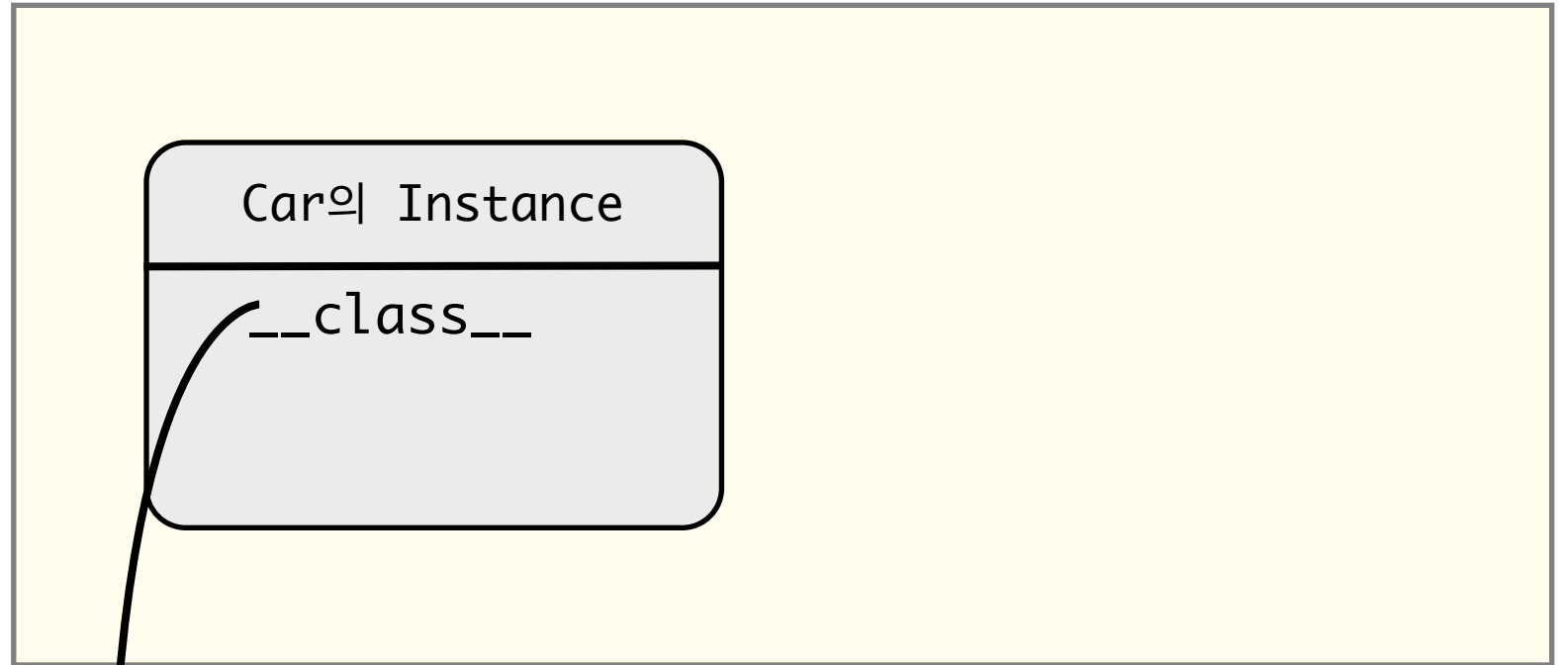


car1 = Car()

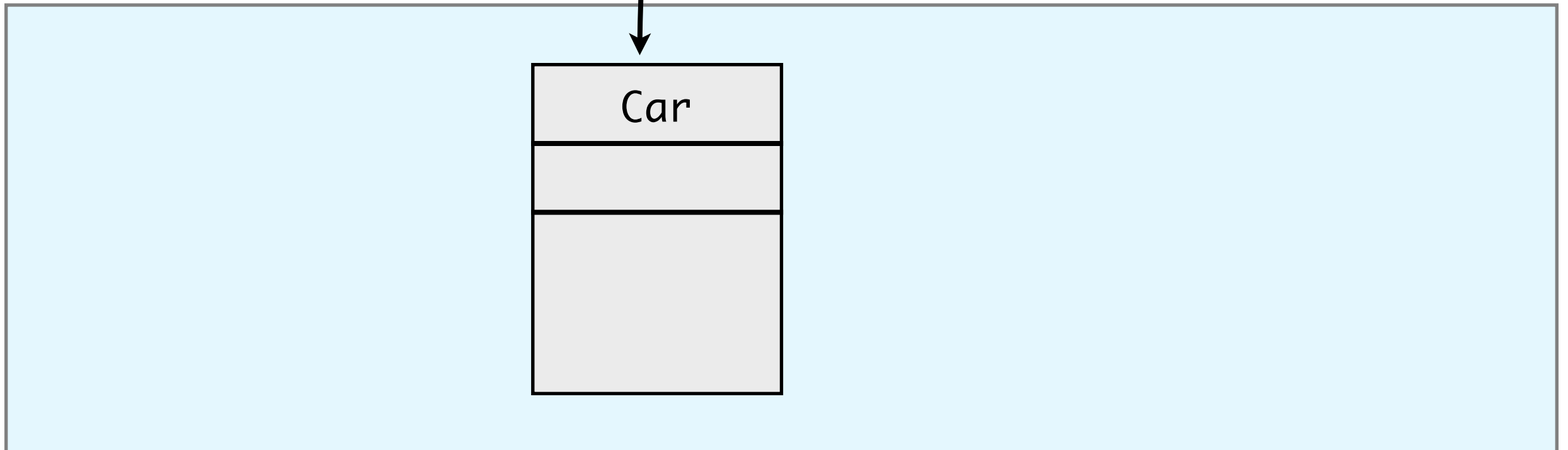
Stack



Heap

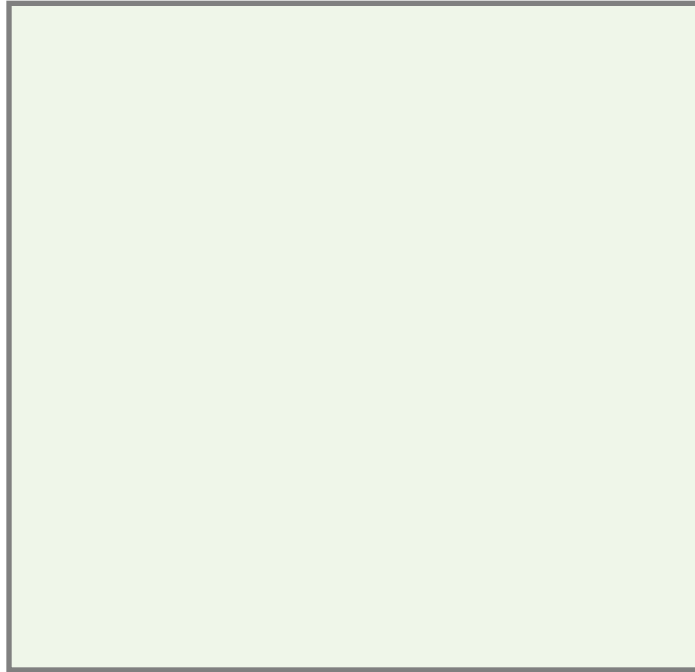


Code

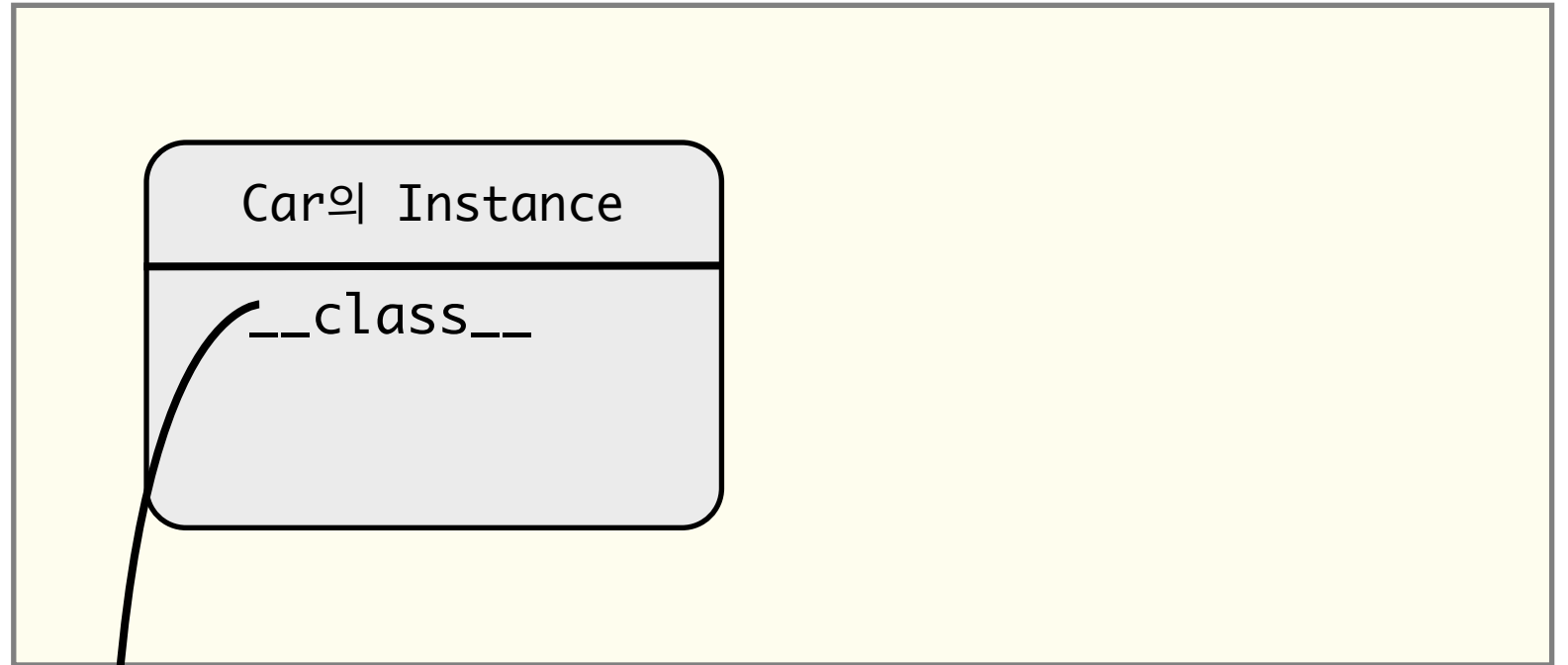


car1 = Car()

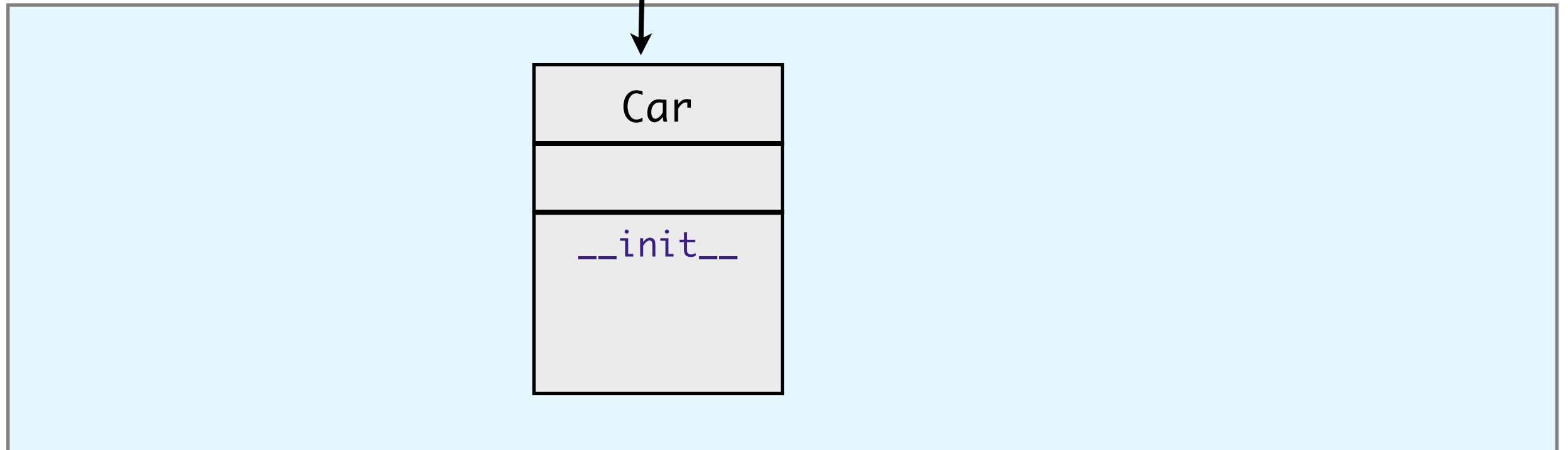
Stack



Heap

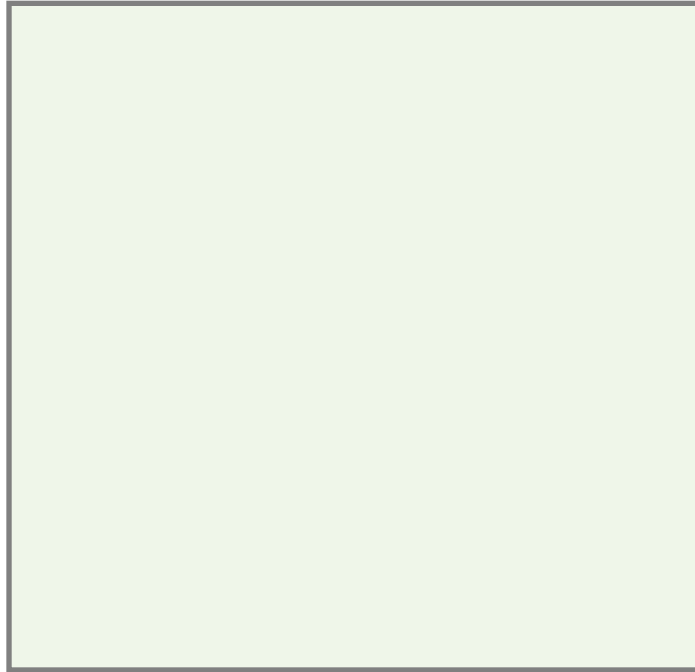


Code

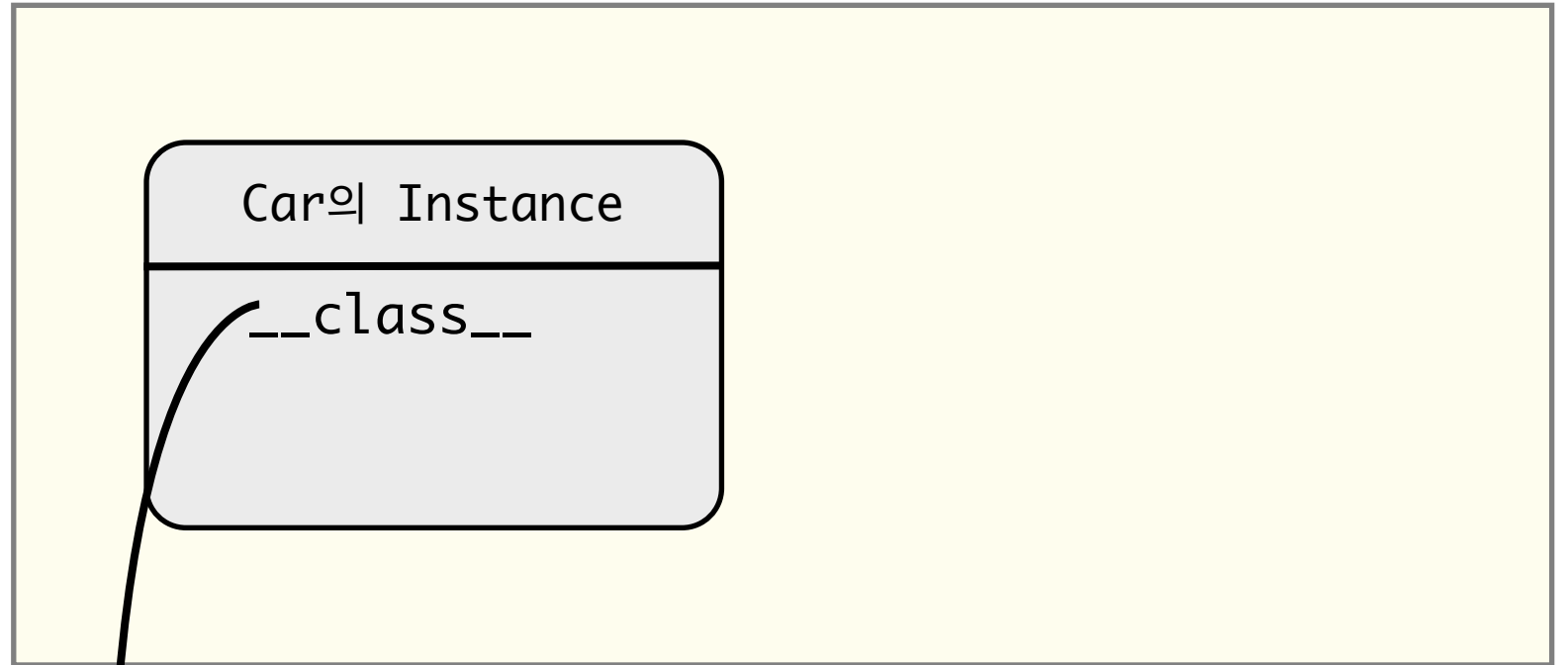


car1 = Car()

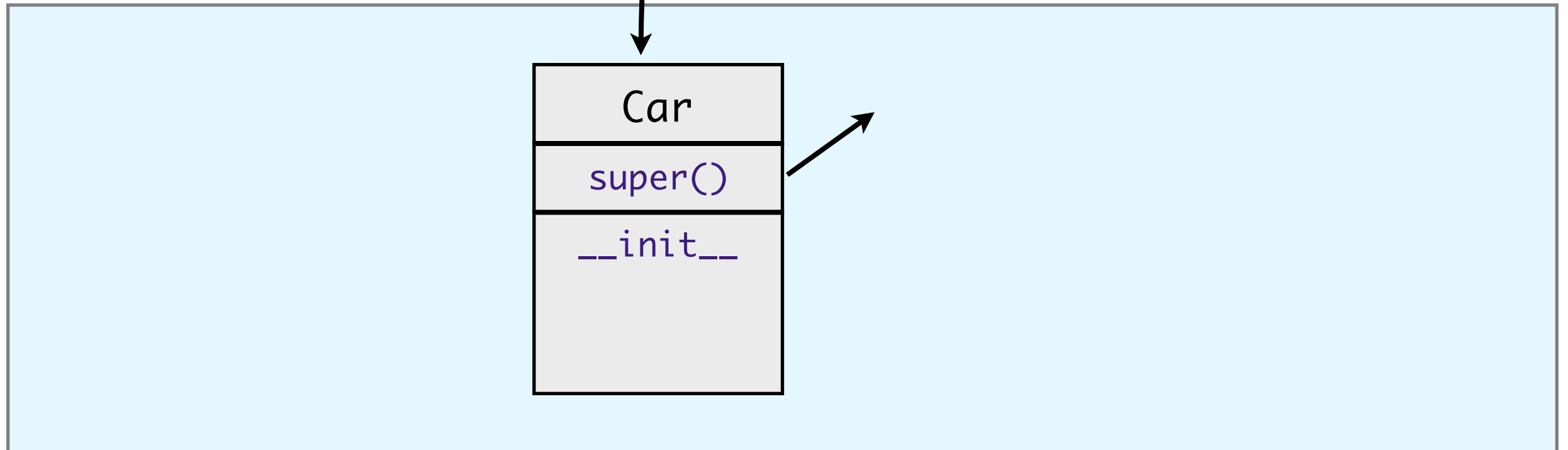
Stack



Heap

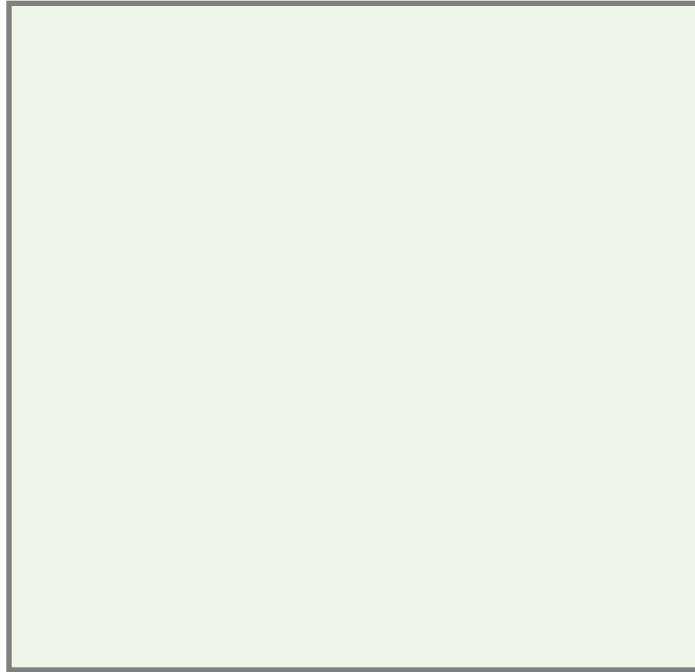


Code

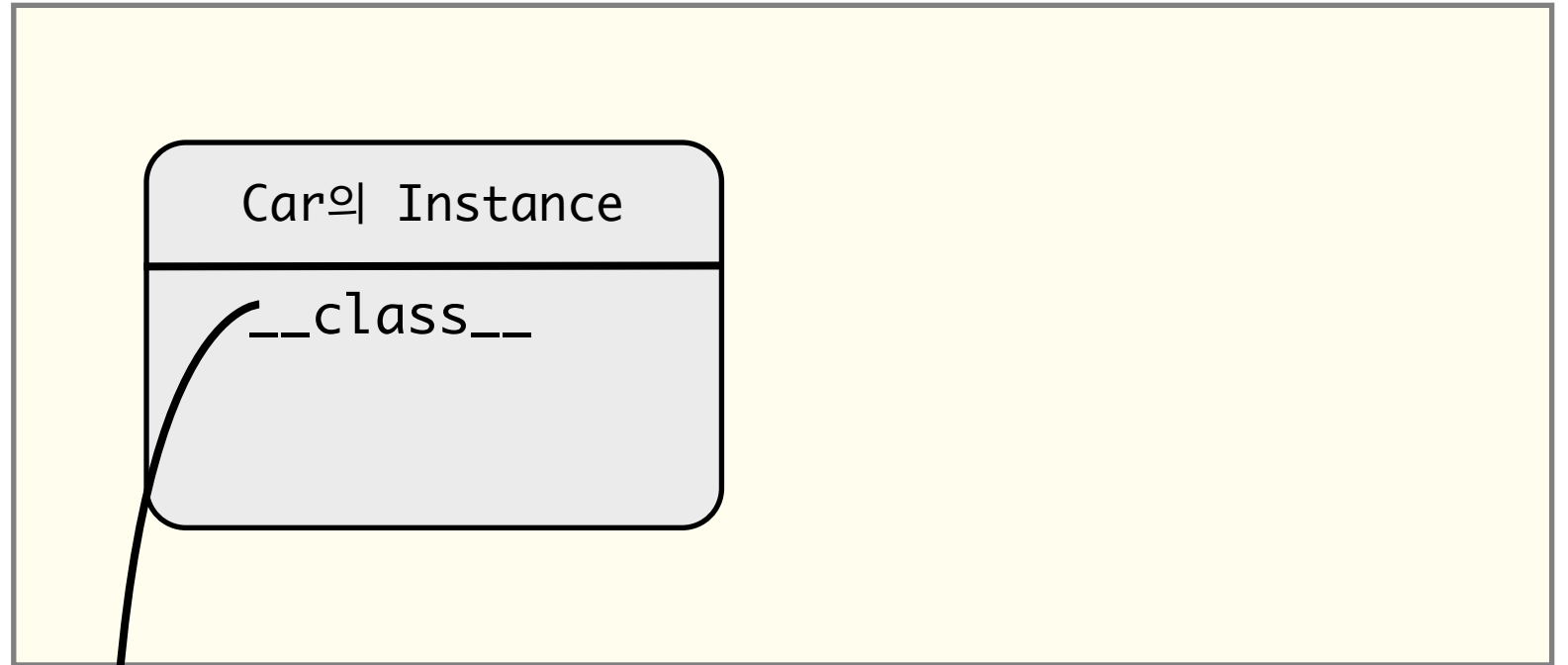


car1 = Car()

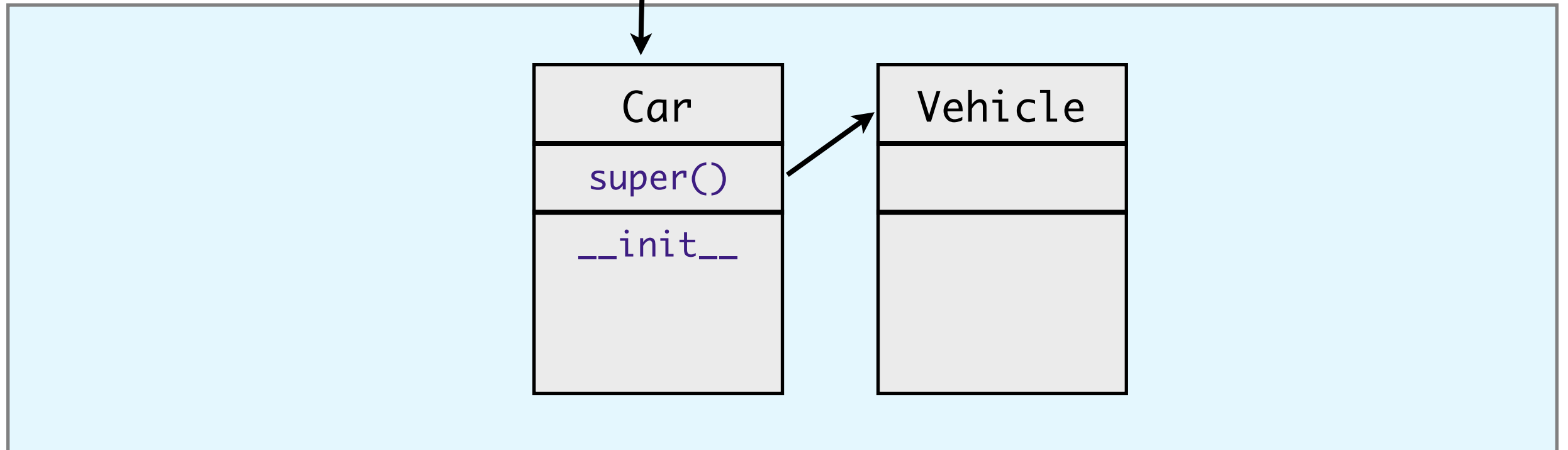
Stack



Heap

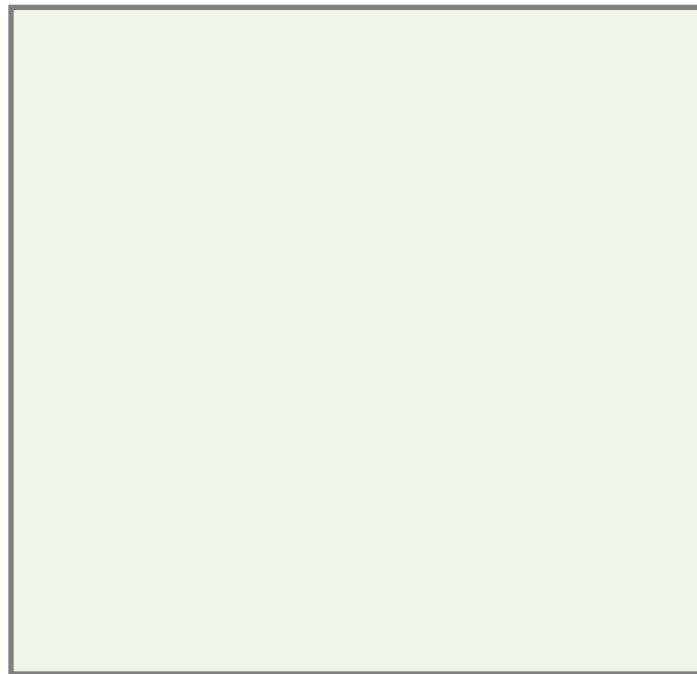


Code

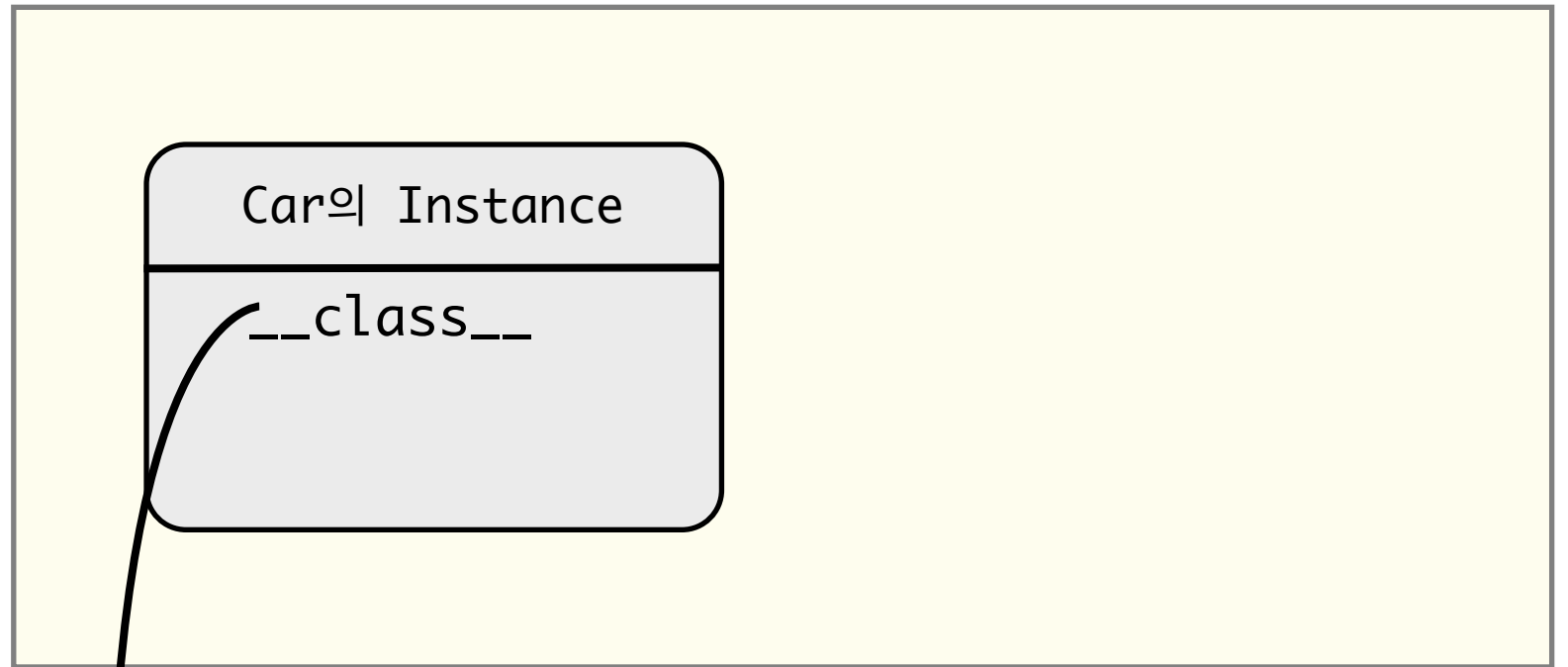


car1 = Car()

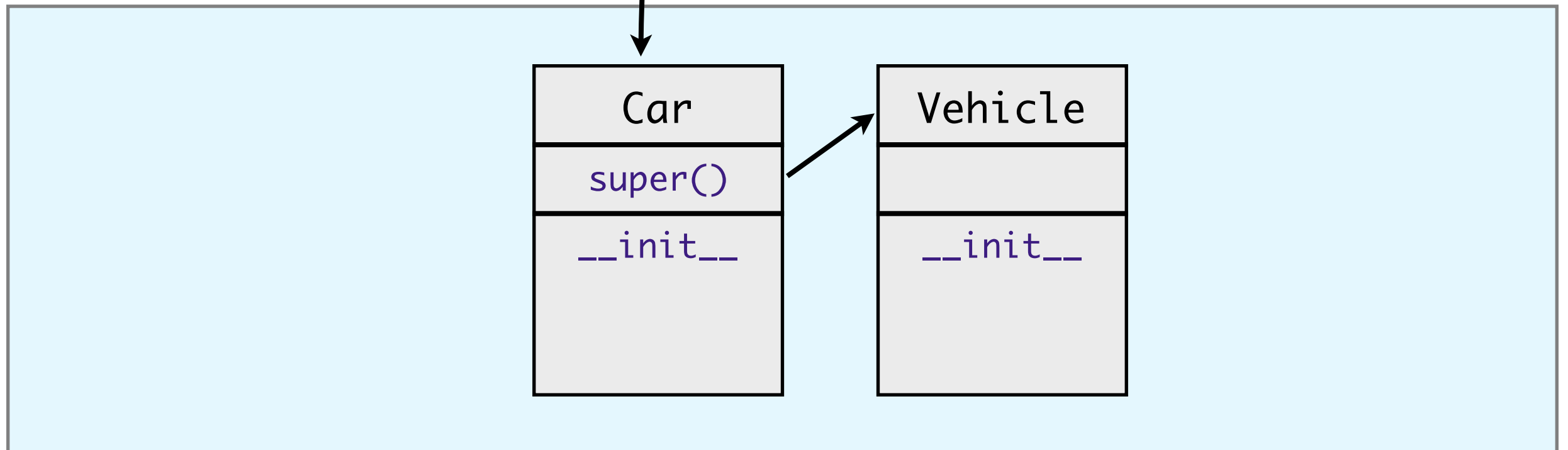
Stack



Heap

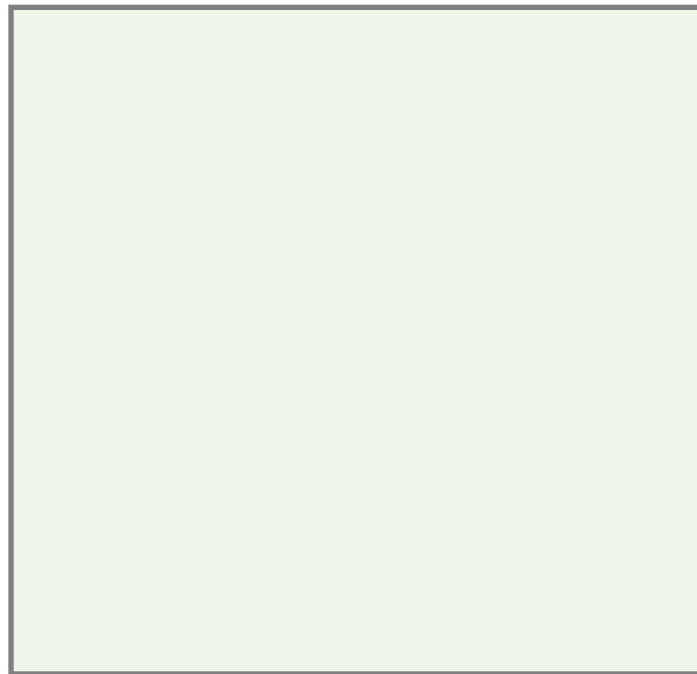


Code

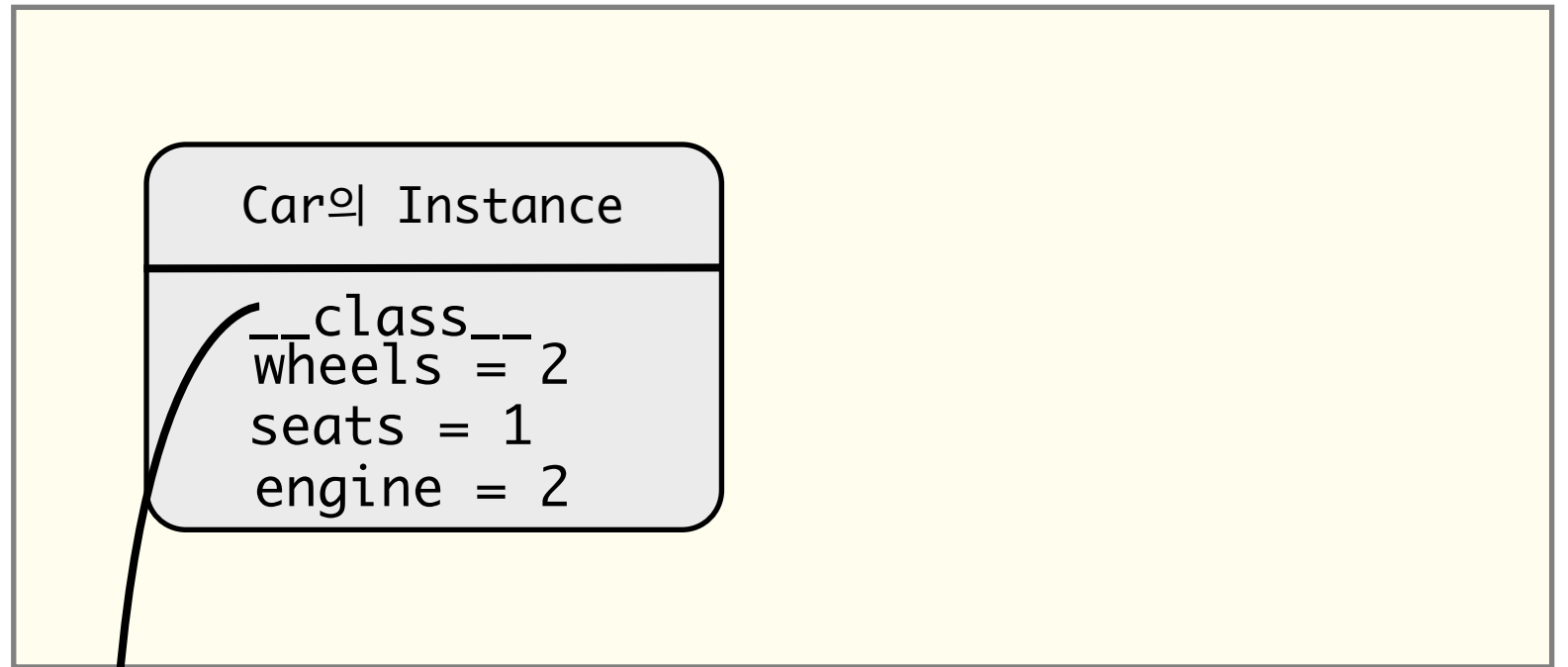


car1 = Car()

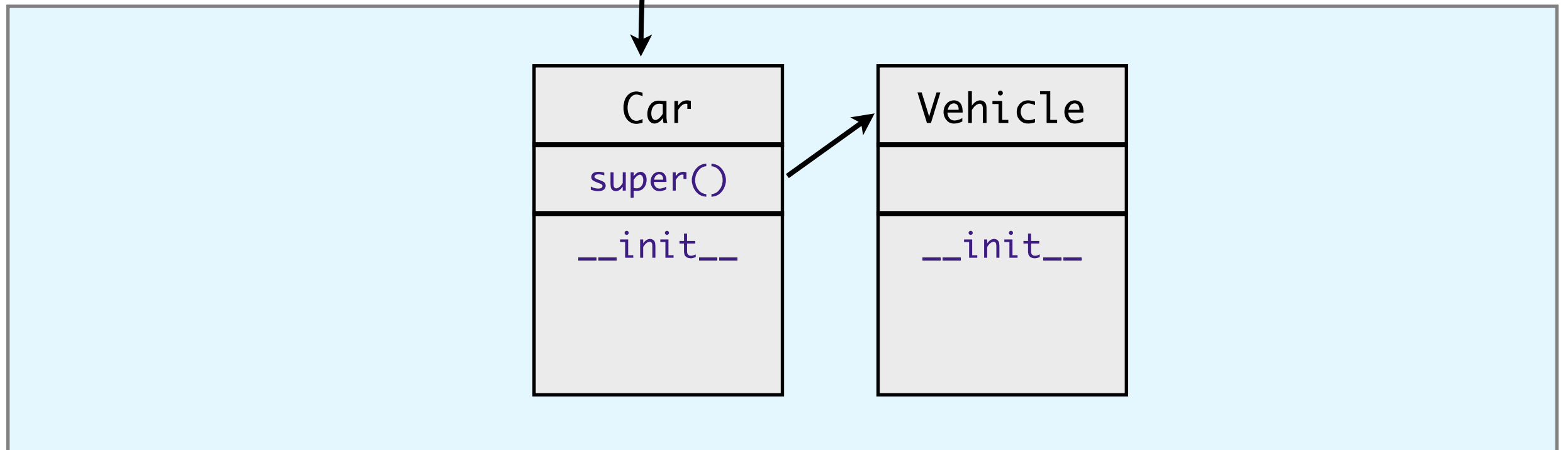
Stack



Heap

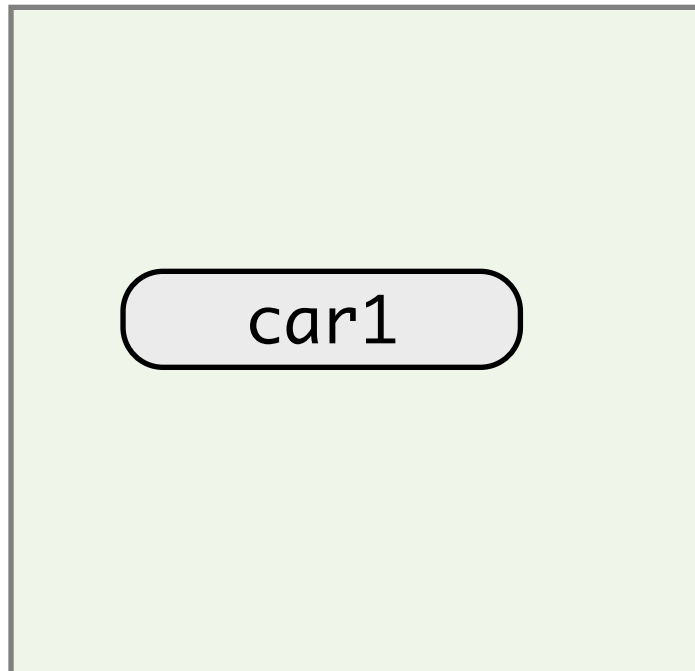


Code

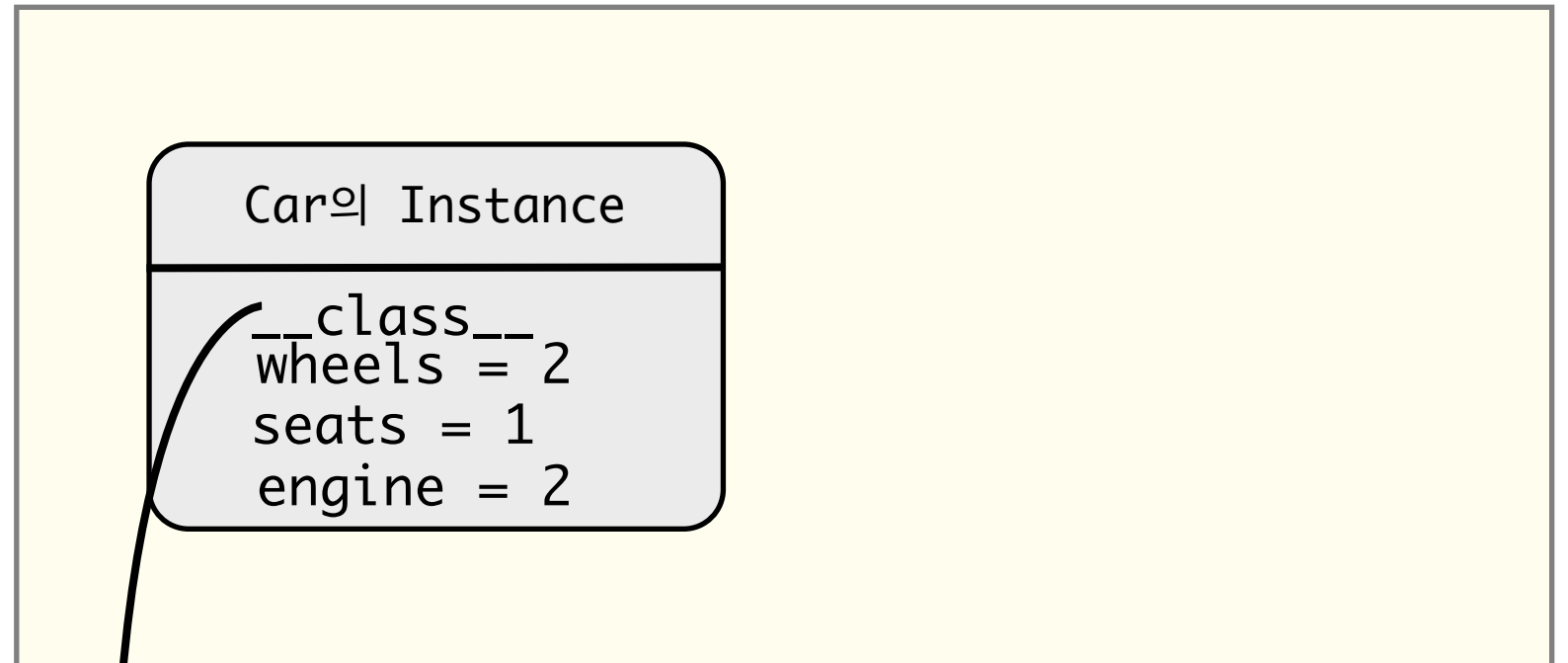


car1 = Car()

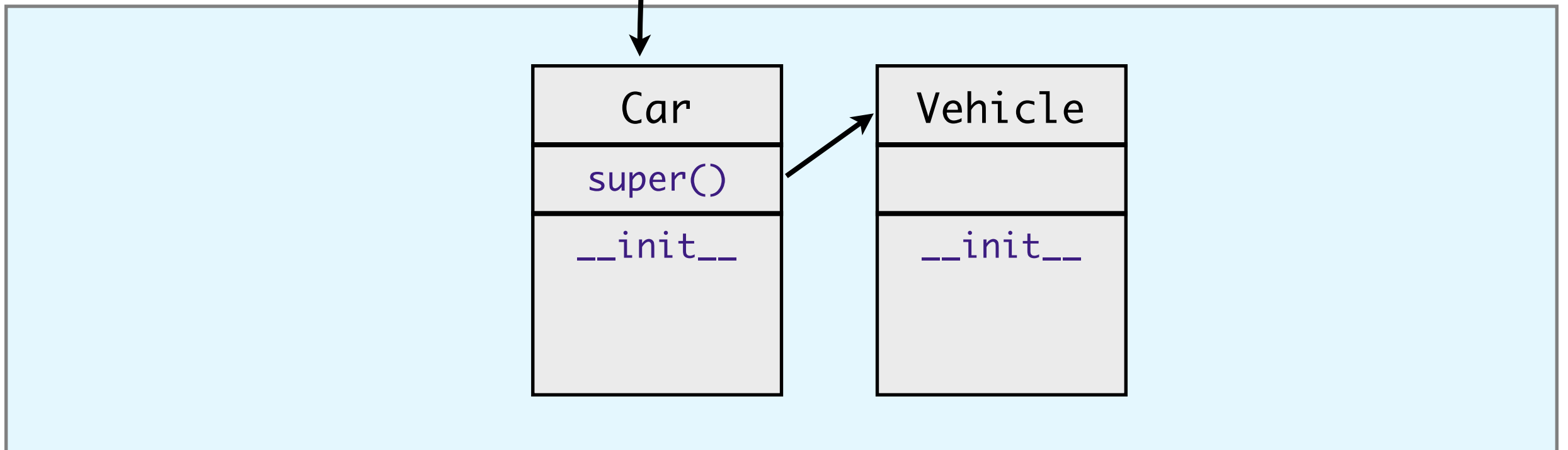
Stack



Heap

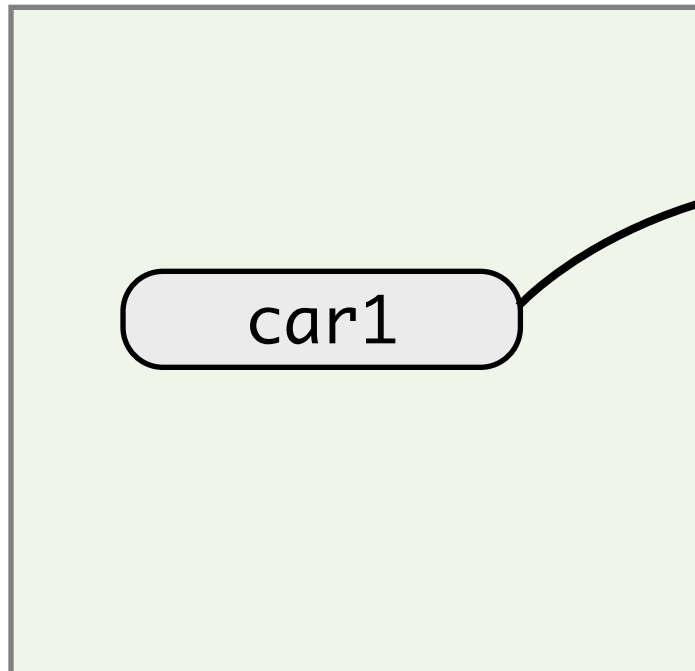


Code

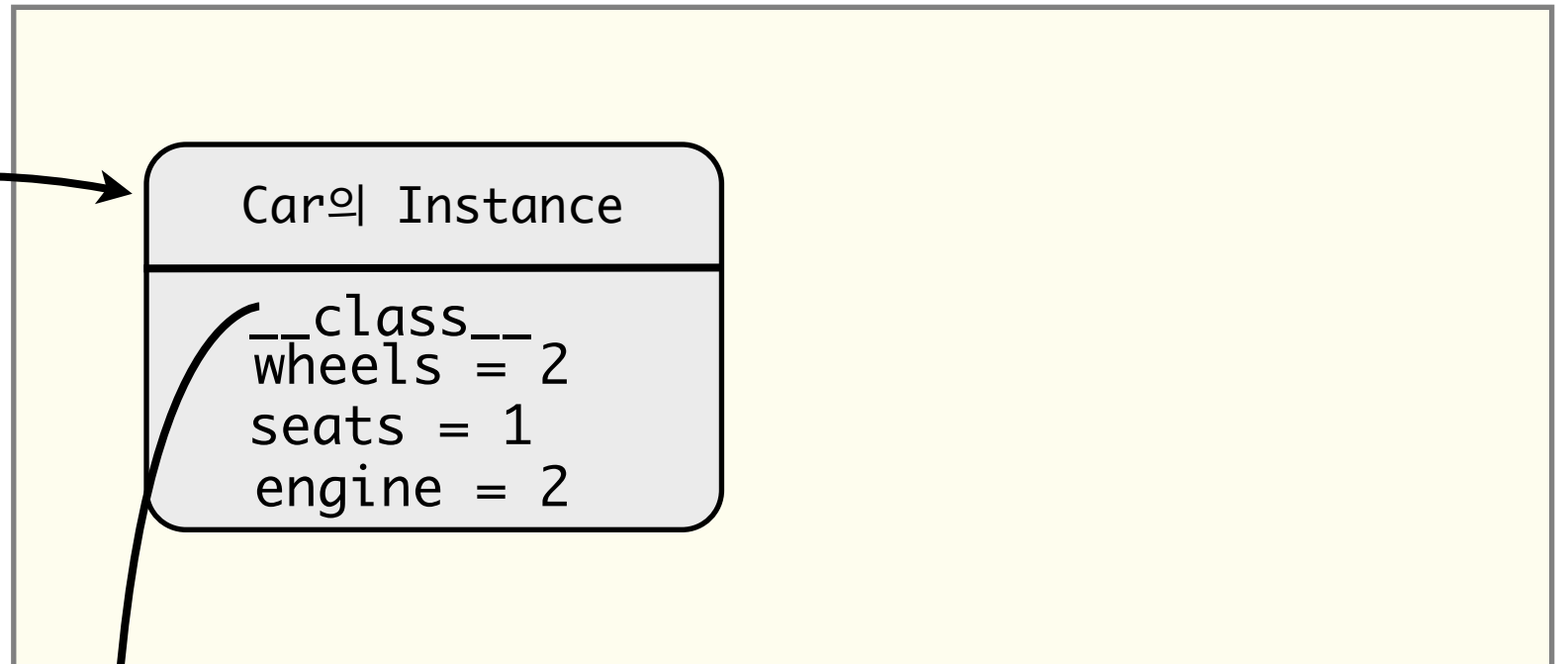


car1 = Car()

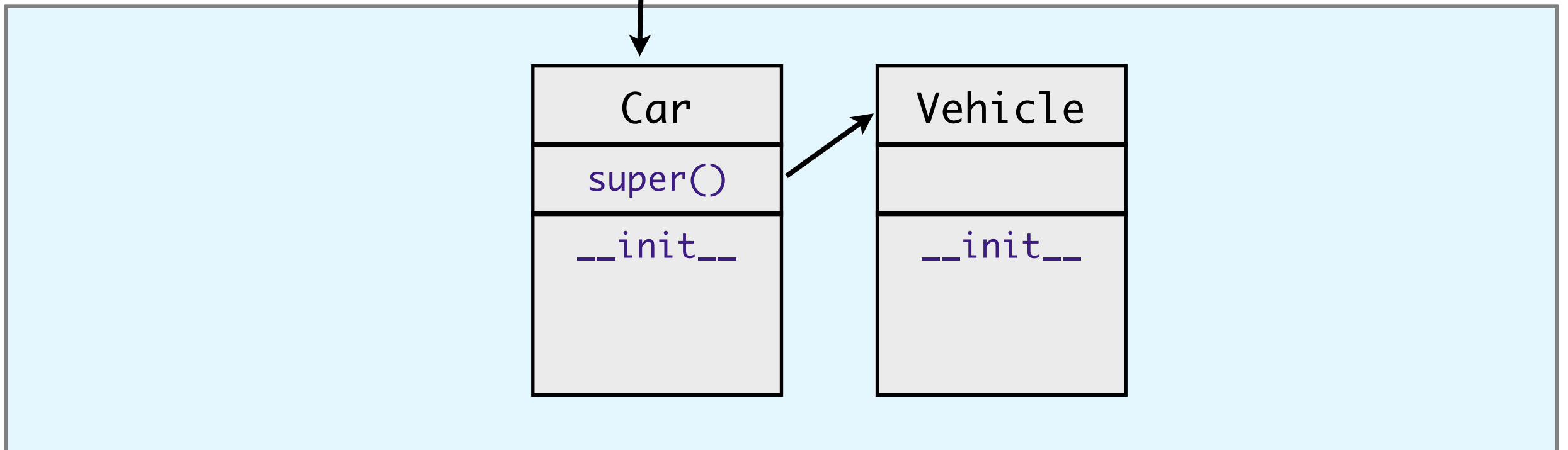
Stack



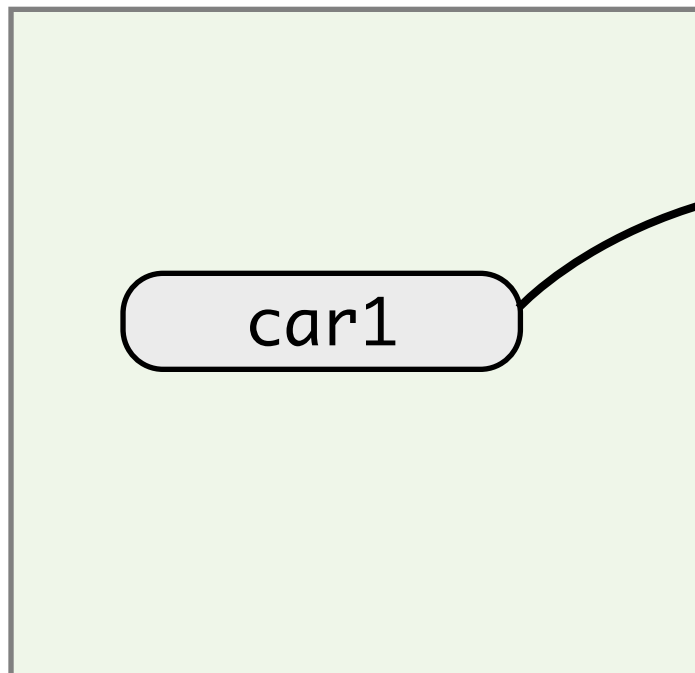
Heap



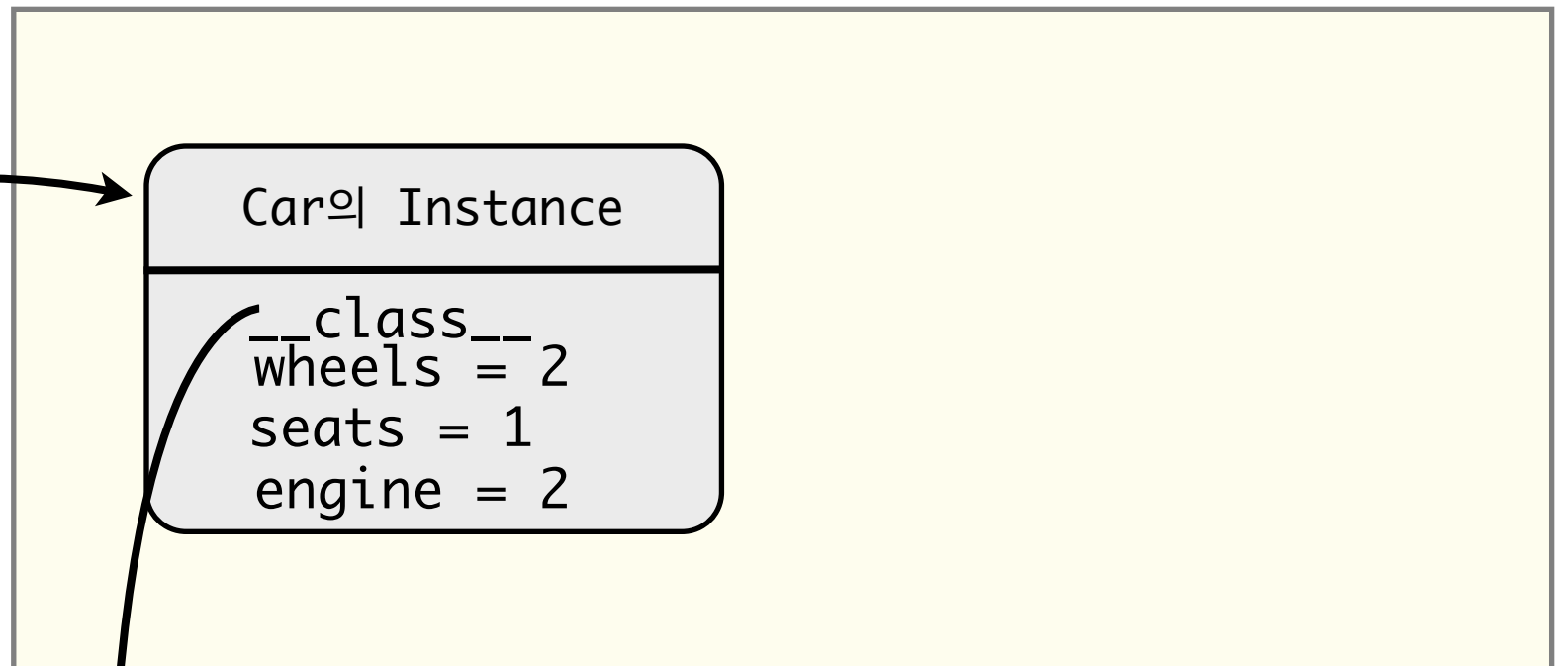
Code



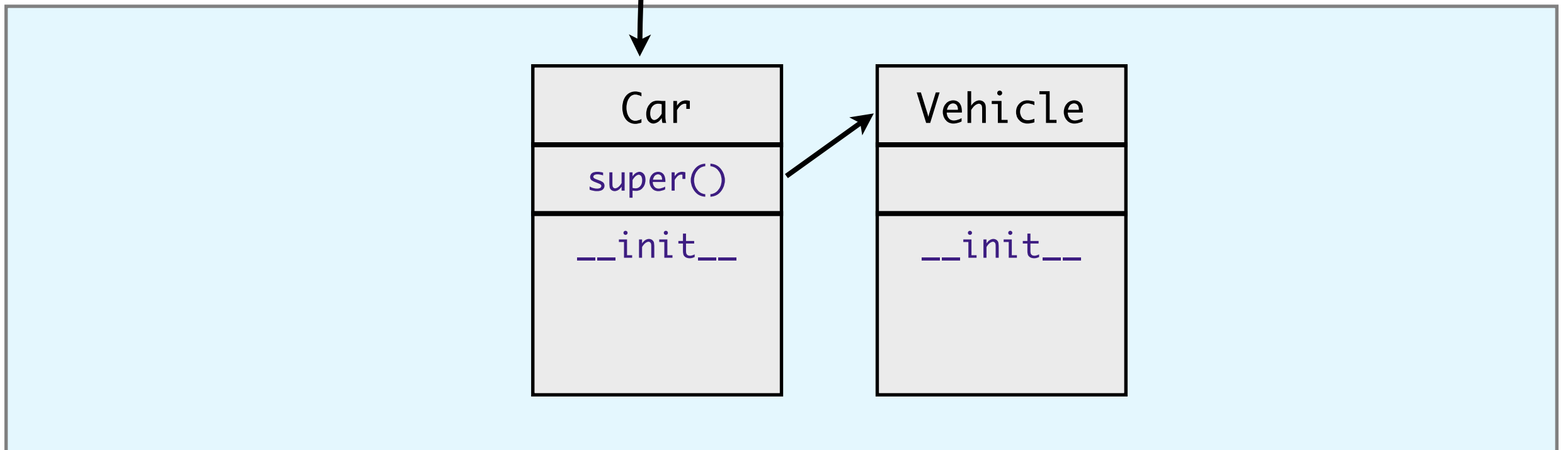
Stack



Heap

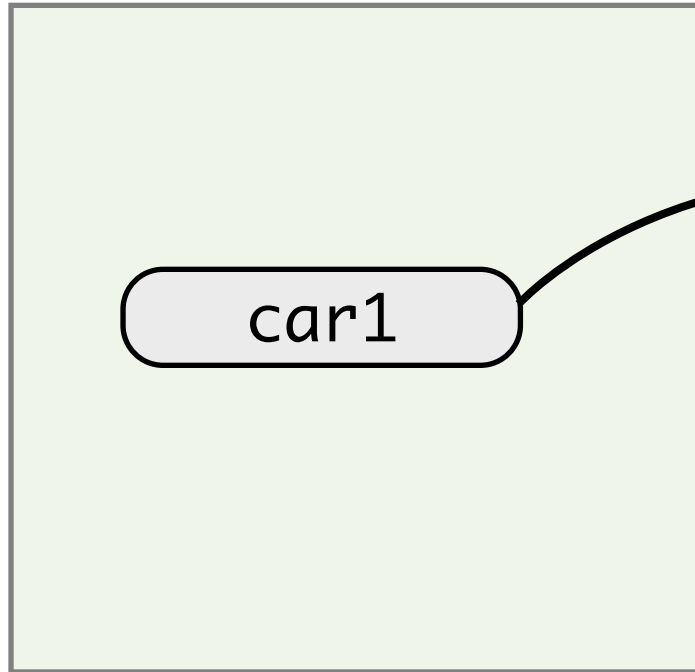


Code

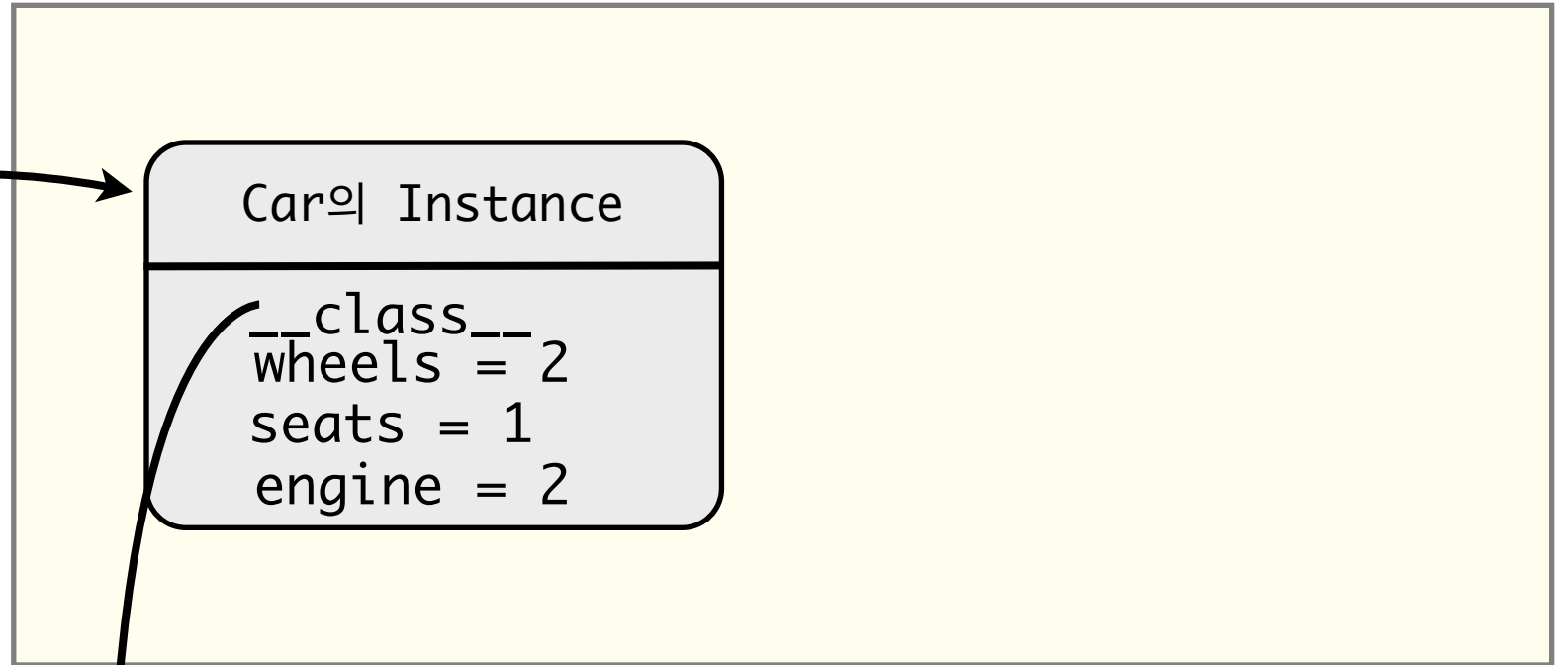


car1.drive()

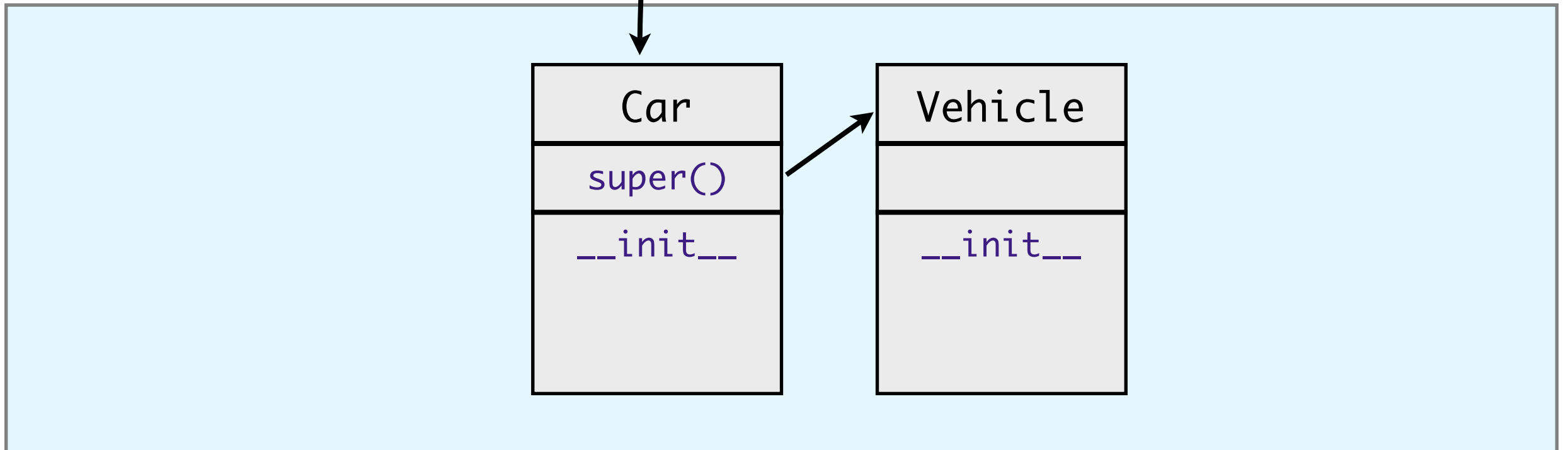
Stack



Heap

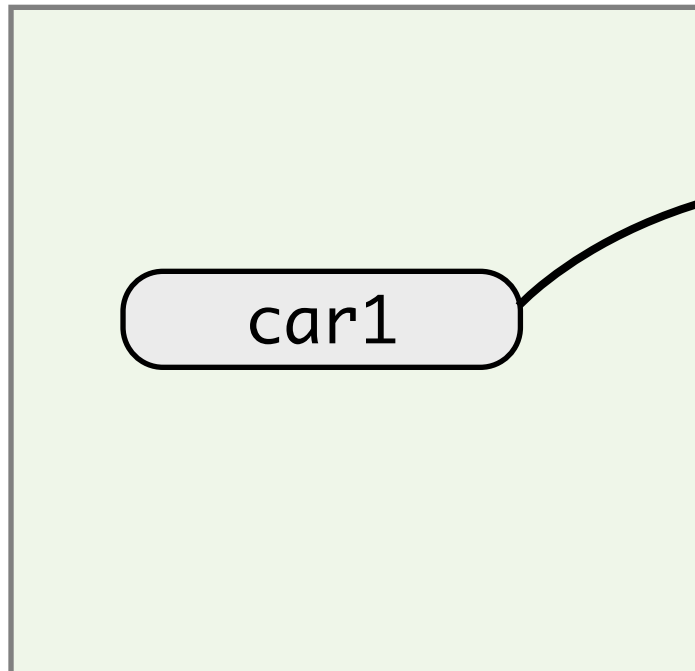


Code

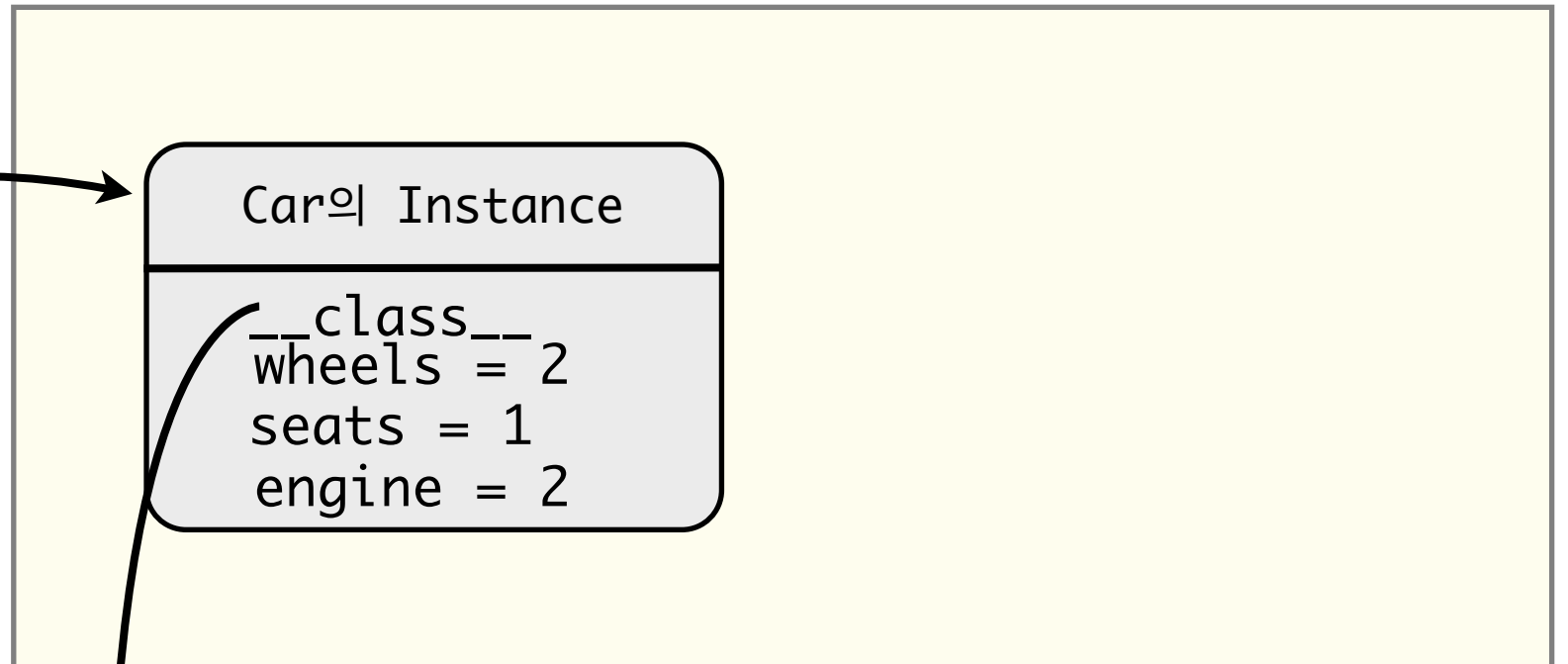


car1.drive()

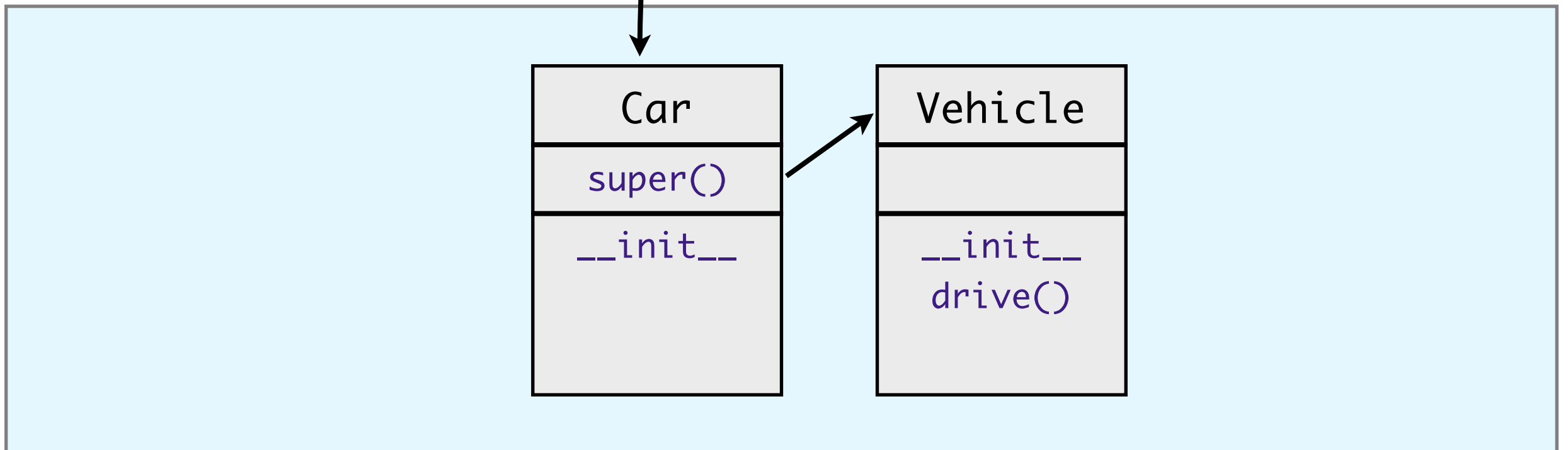
Stack



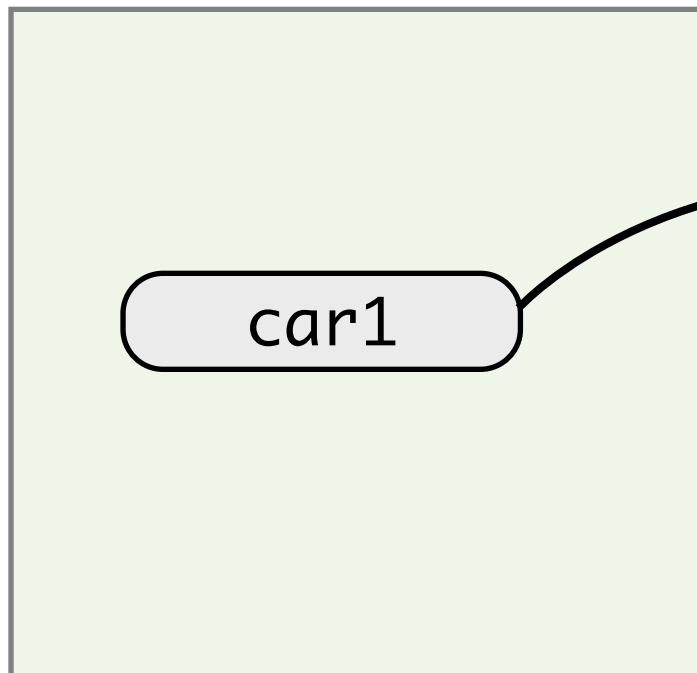
Heap



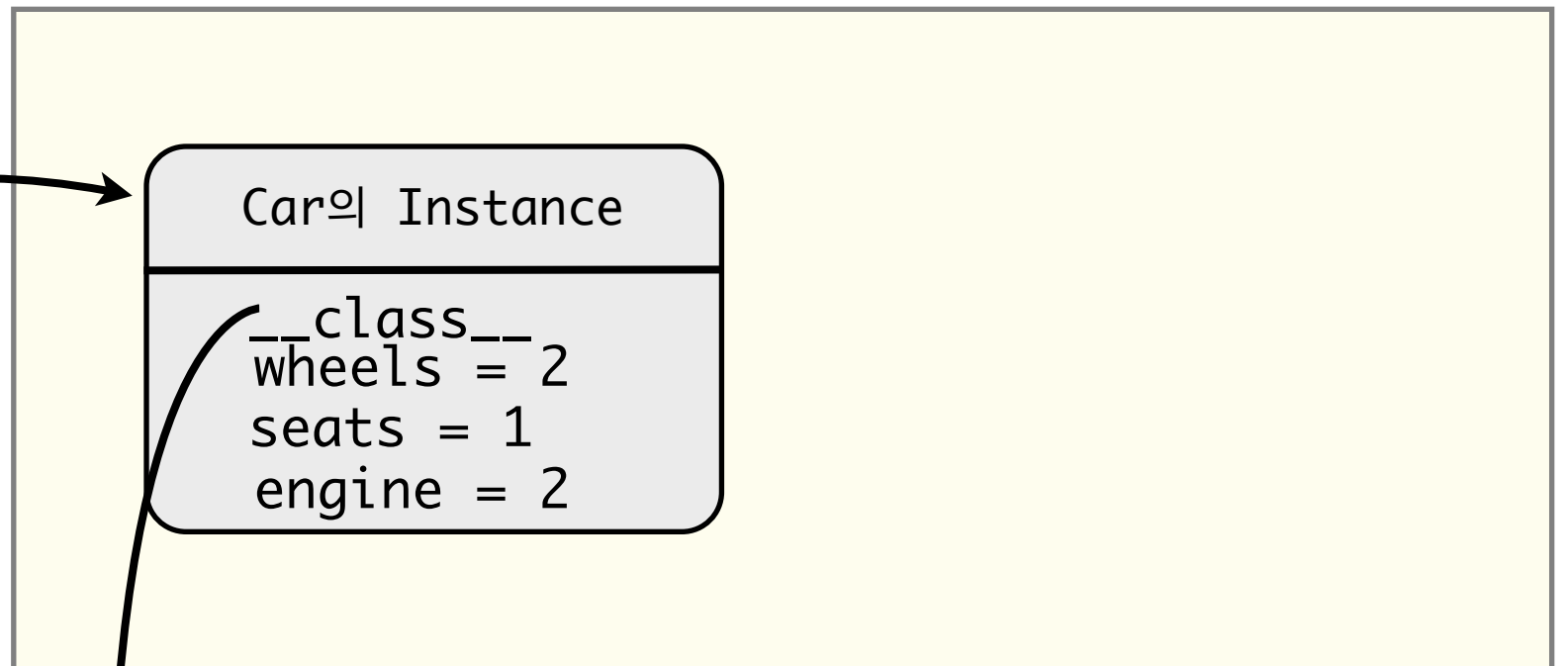
Code



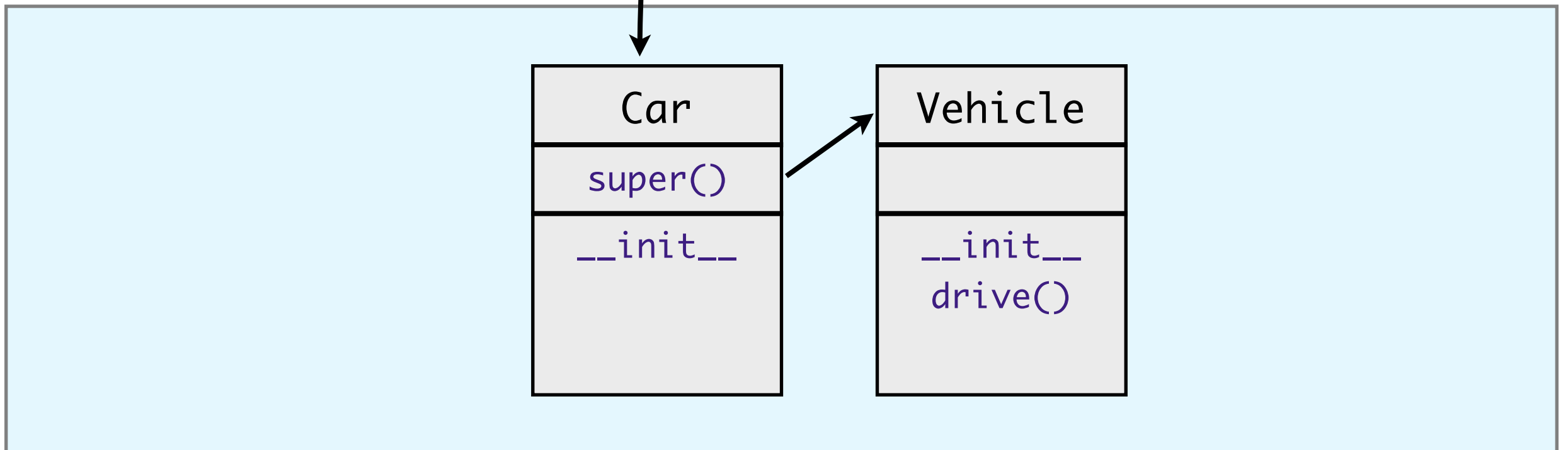
Stack



Heap



Code



상속

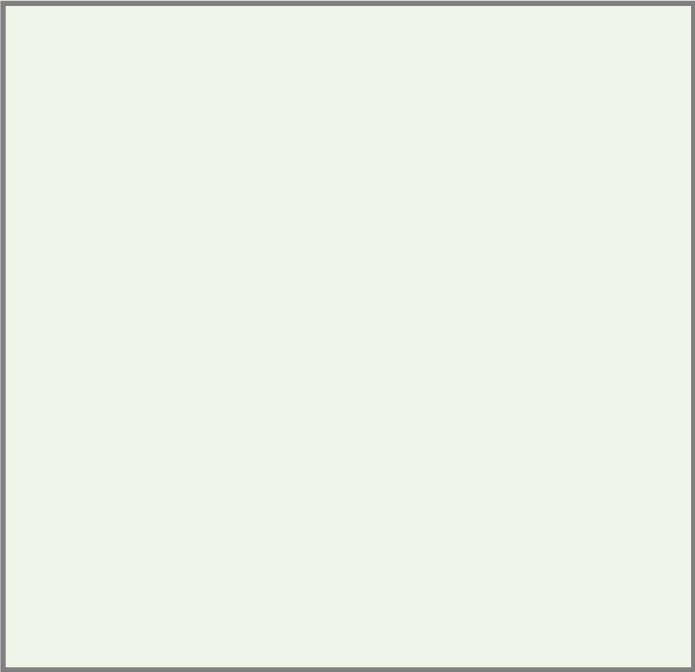
```
class Sedan(Car):  
    def load_passenger(self):  
        print('Load Passenger')
```

```
class Truck(Car):  
    def load_cargo(self):  
        print('Load Cargo')
```

```
sedan = Sedan()  
sedan.load_passenger()  
sedan.shift_gear()  
sedan.drive()
```

```
truck = Truck()  
truck.load_cargo()  
truck.shift_gear()
```

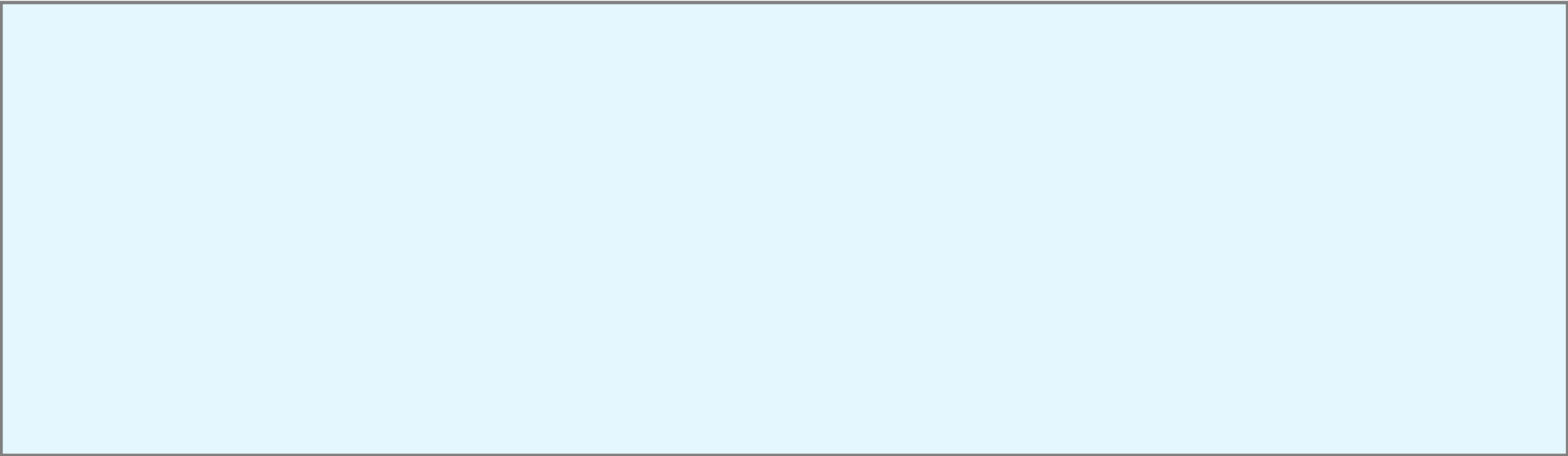
Stack



Heap

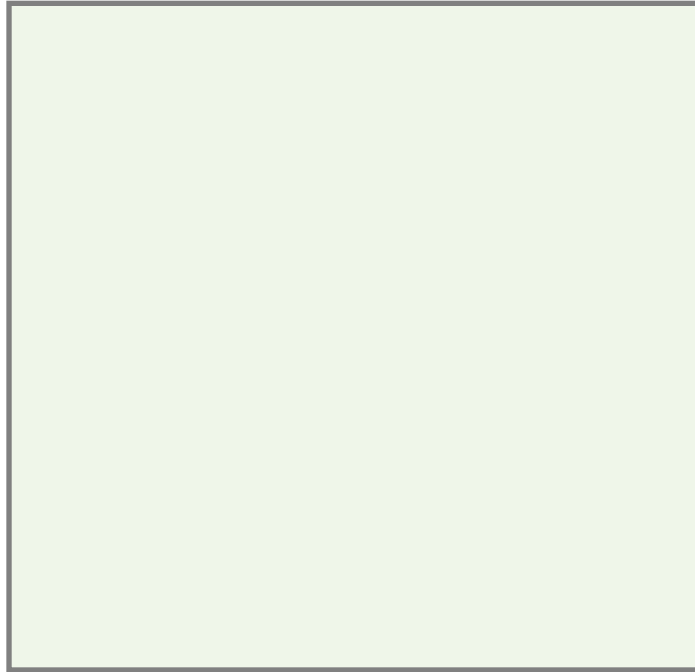


Code



sedan = Sedan()

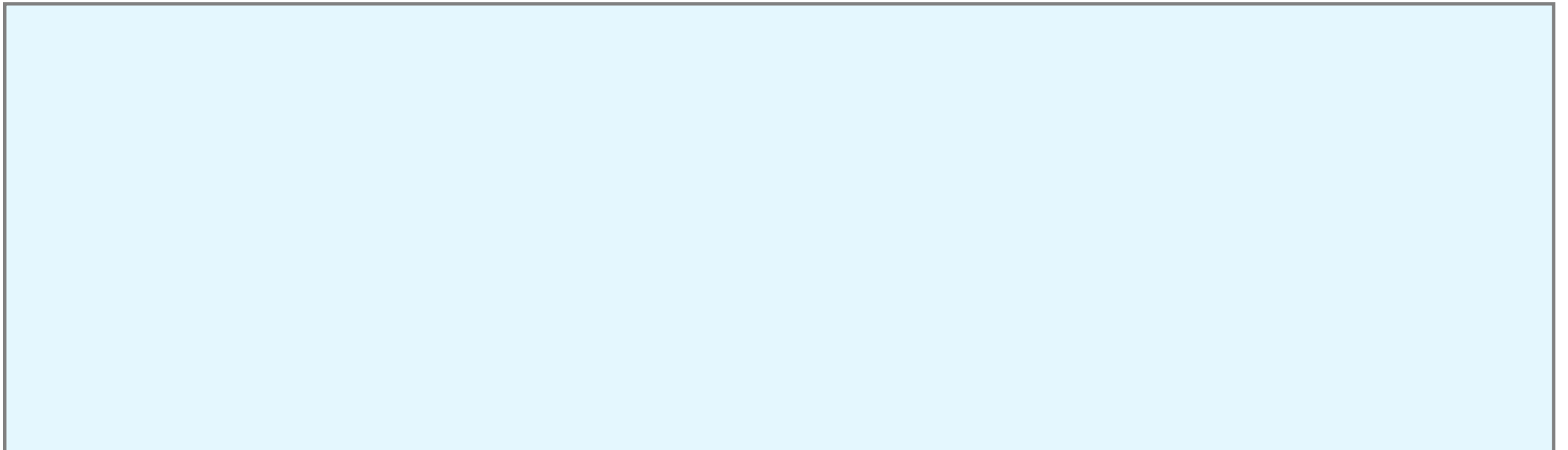
Stack



Heap

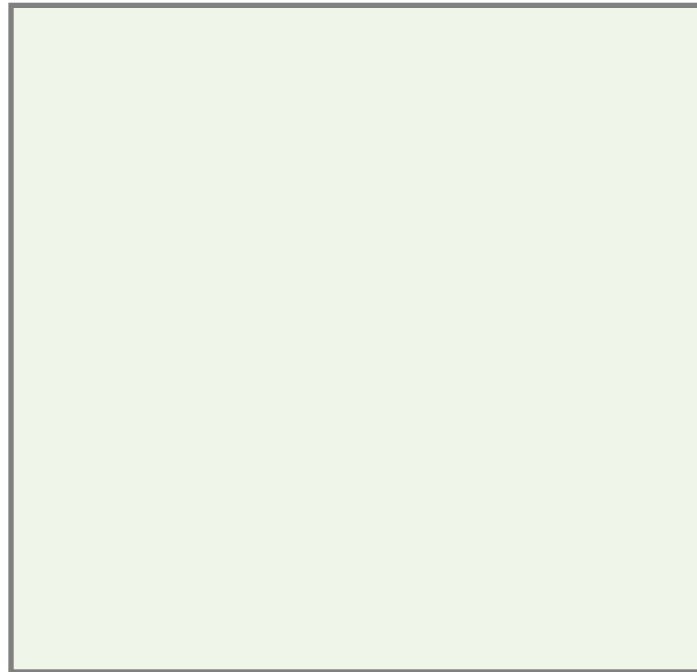


Code



sedan = Sedan()

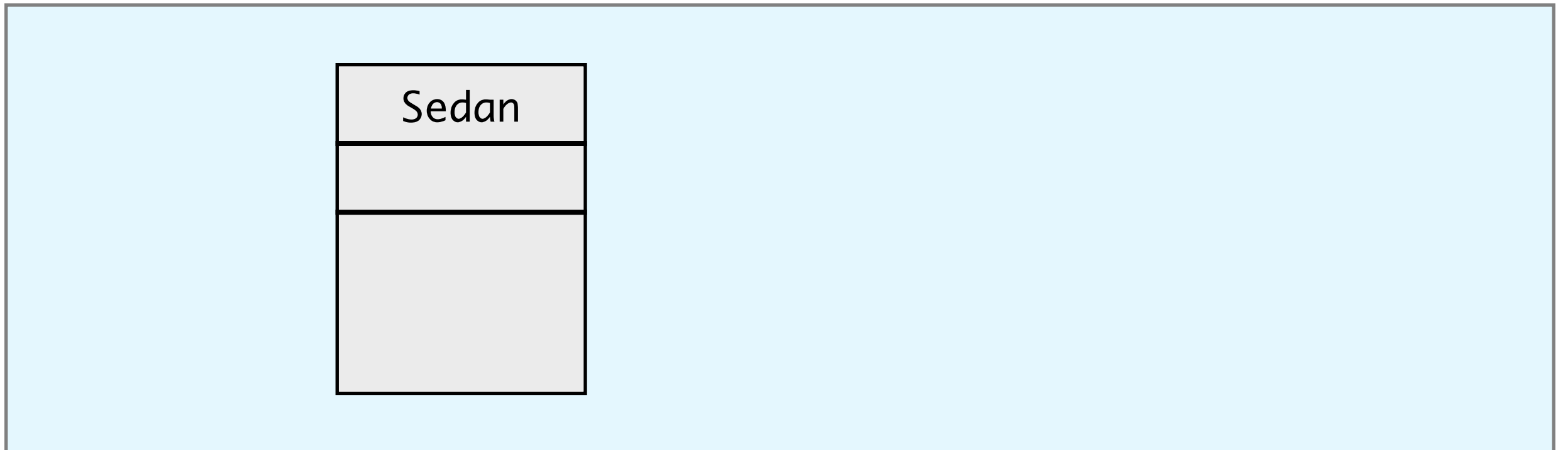
Stack



Heap

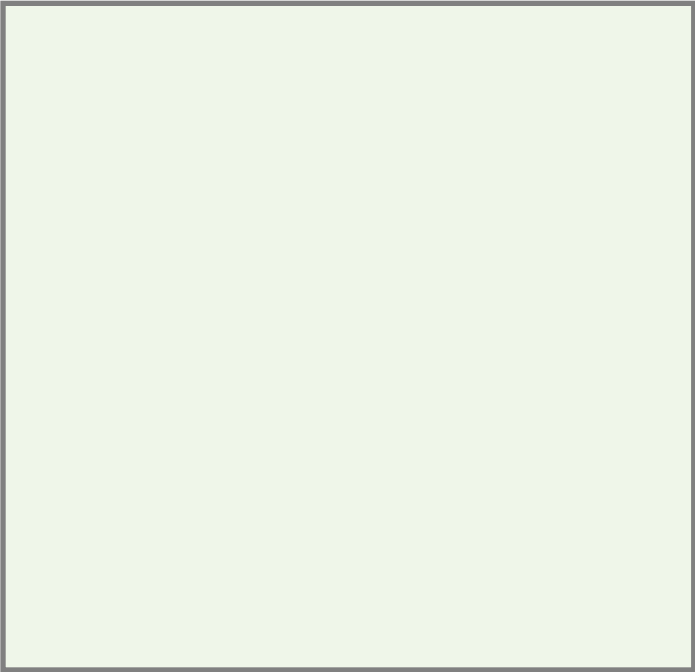


Code

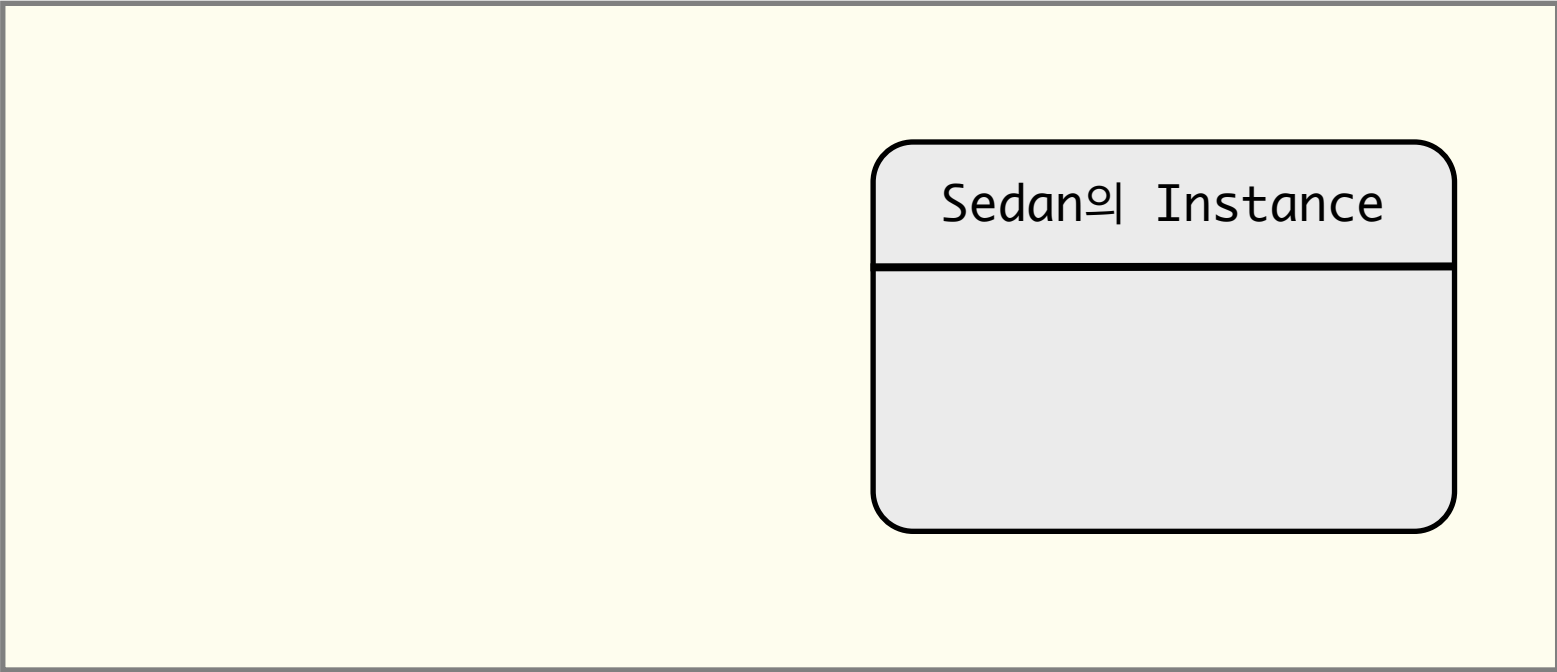


sedan = Sedan()

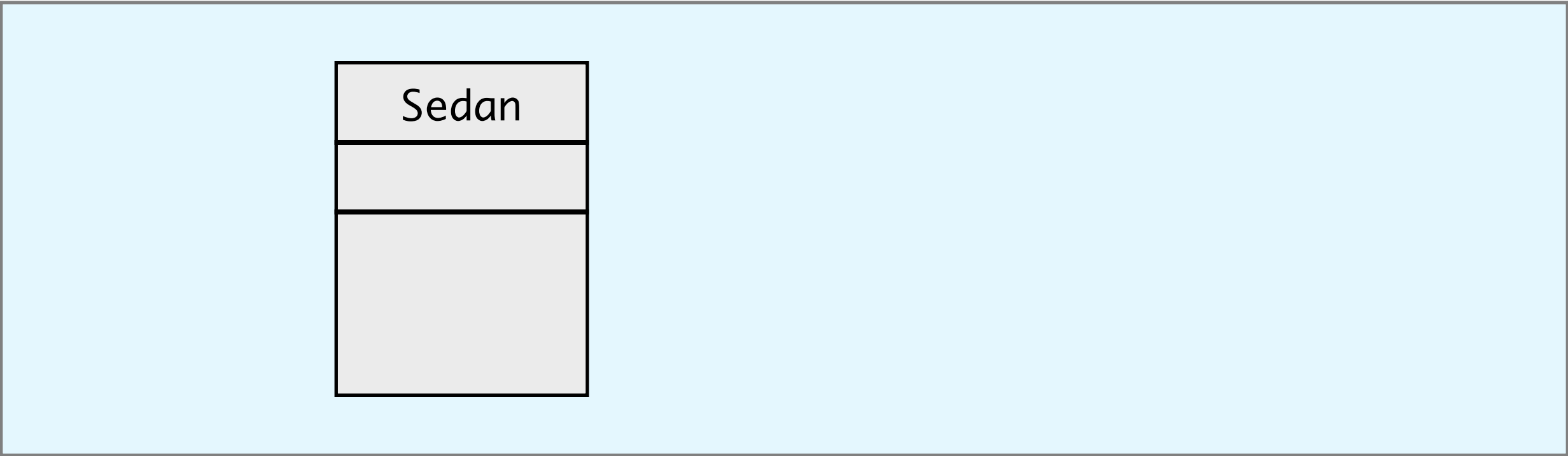
Stack



Heap

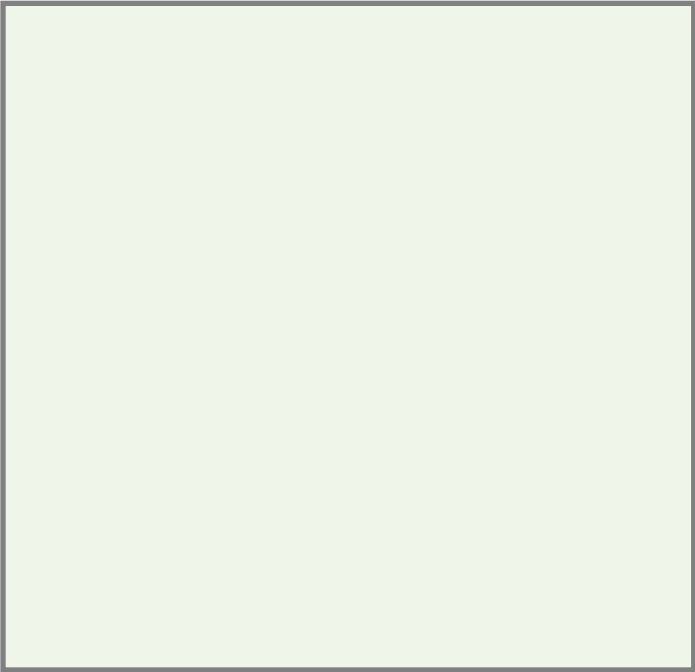


Code

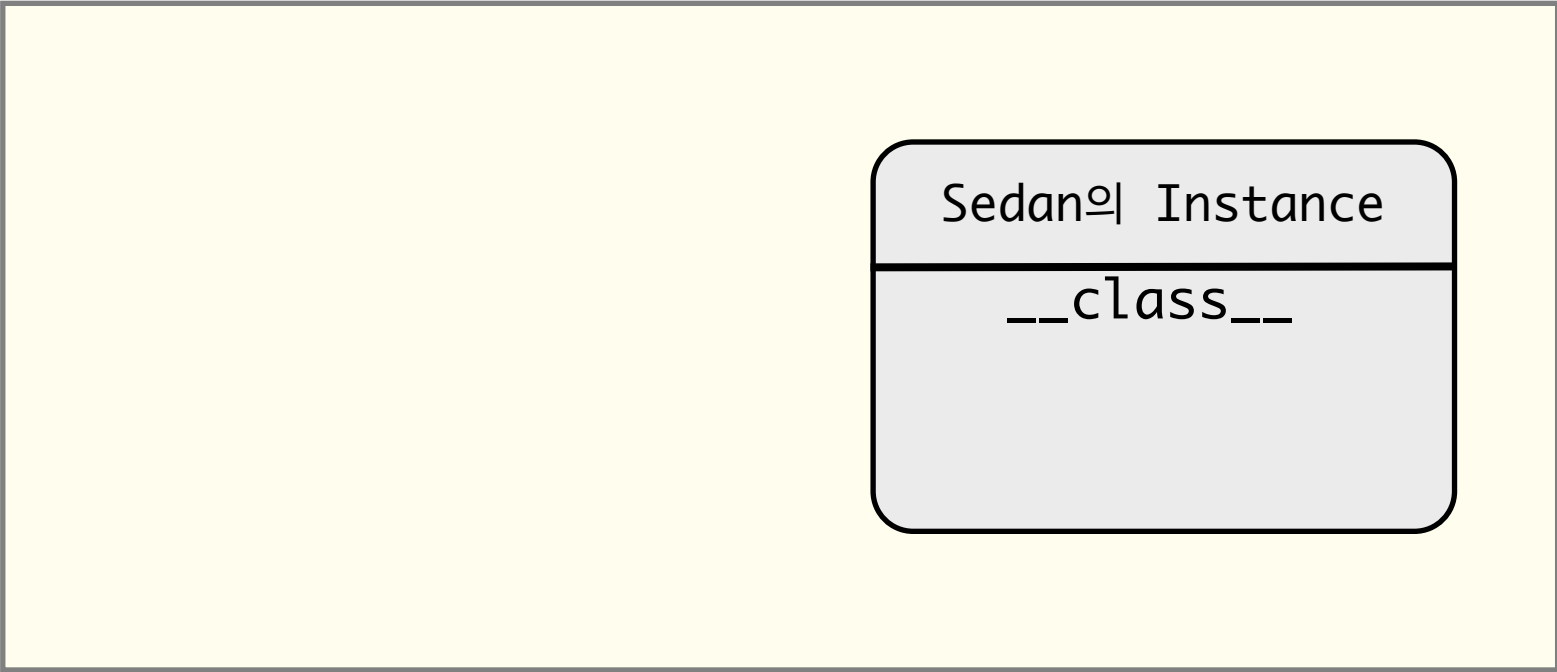


sedan = Sedan()

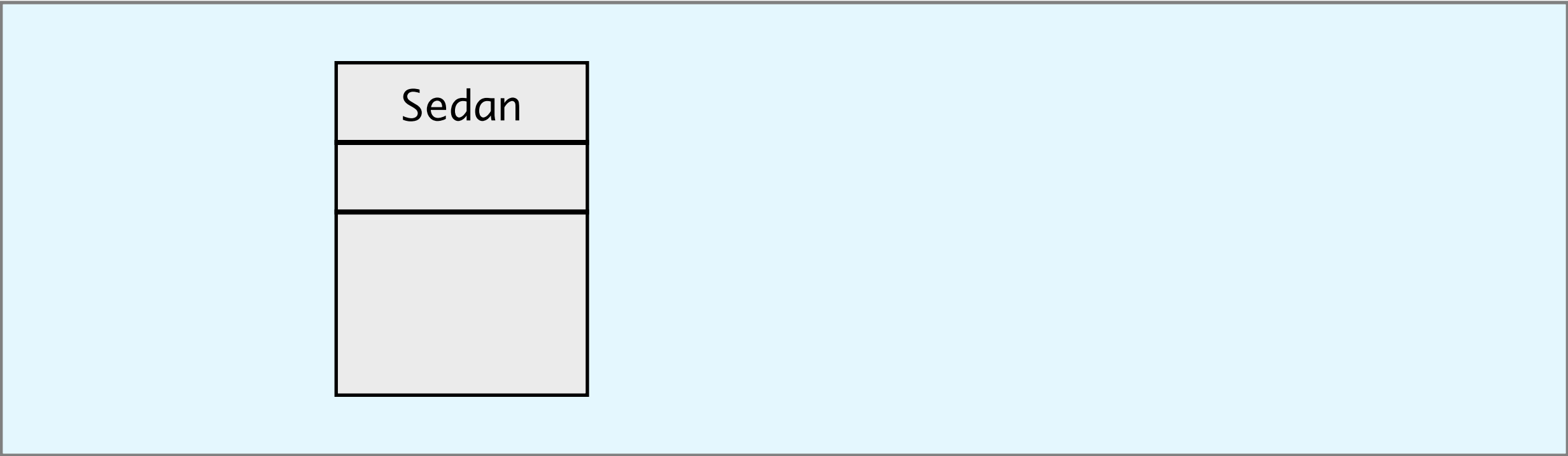
Stack



Heap

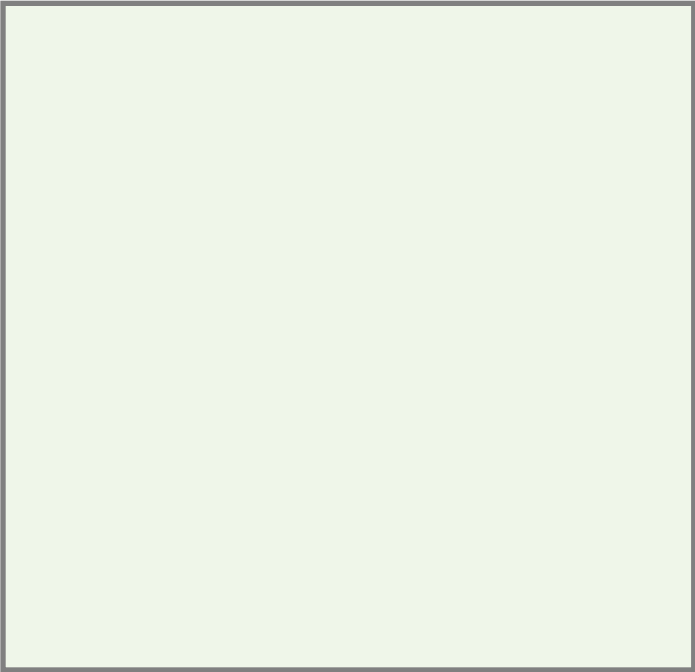


Code

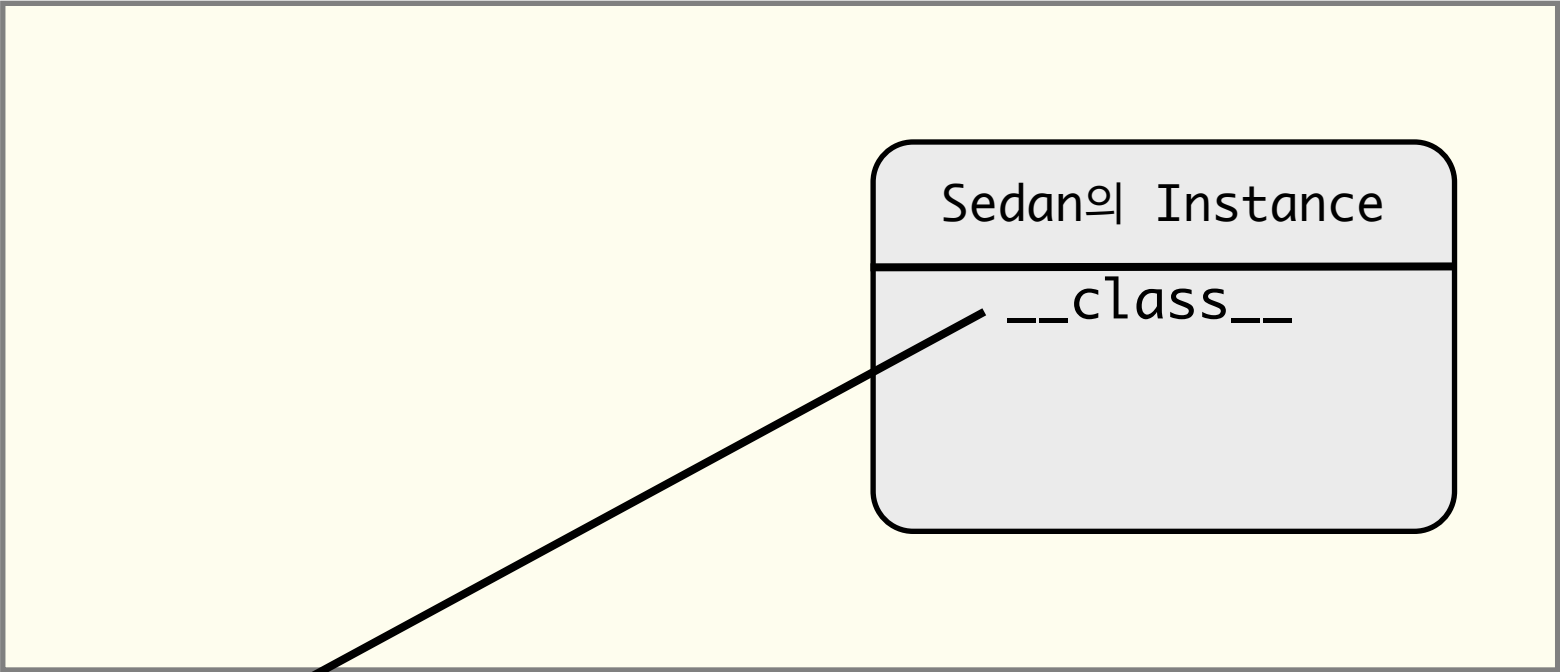


```
sedan = Sedan()
```

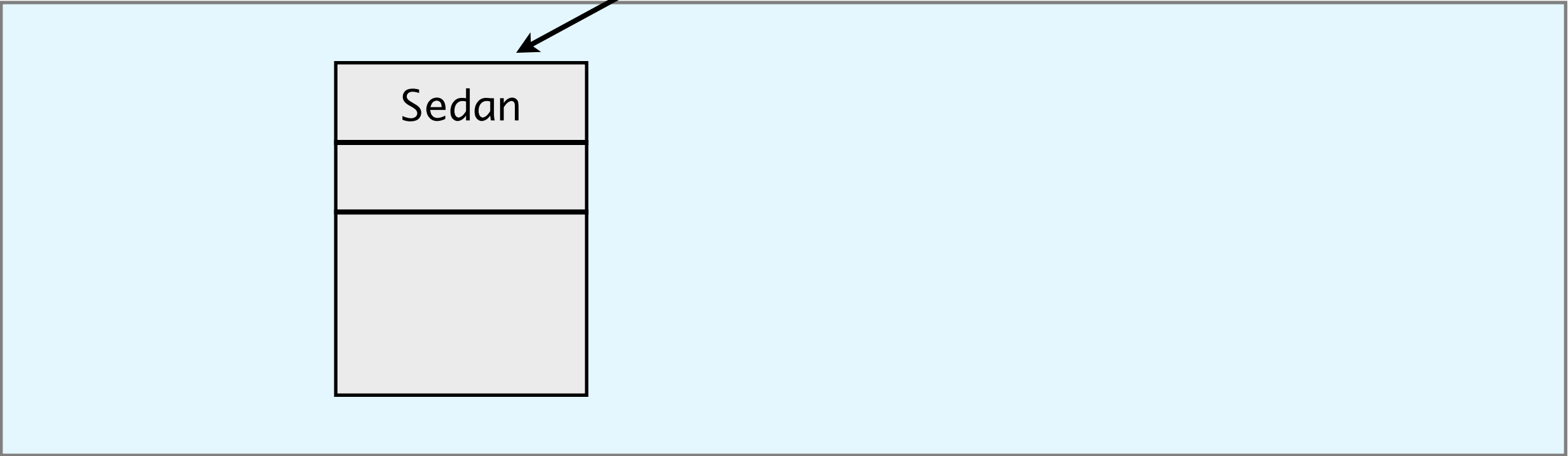
Stack



Heap

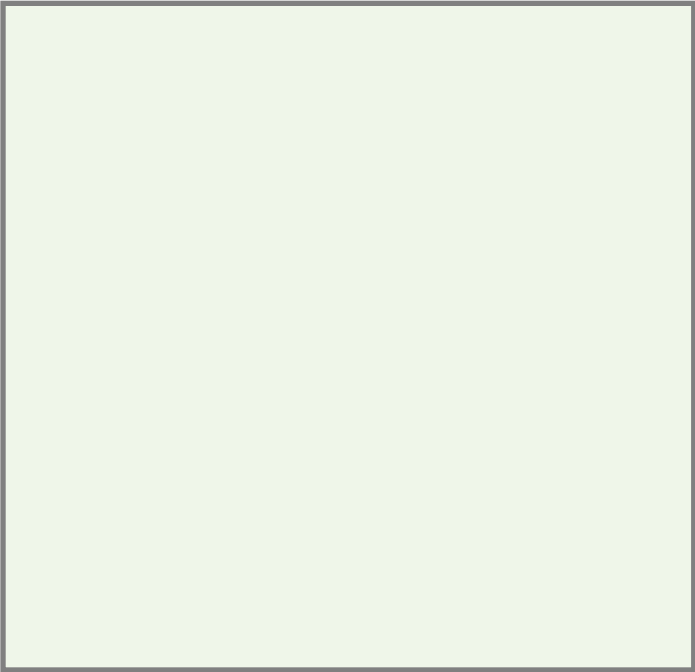


Code

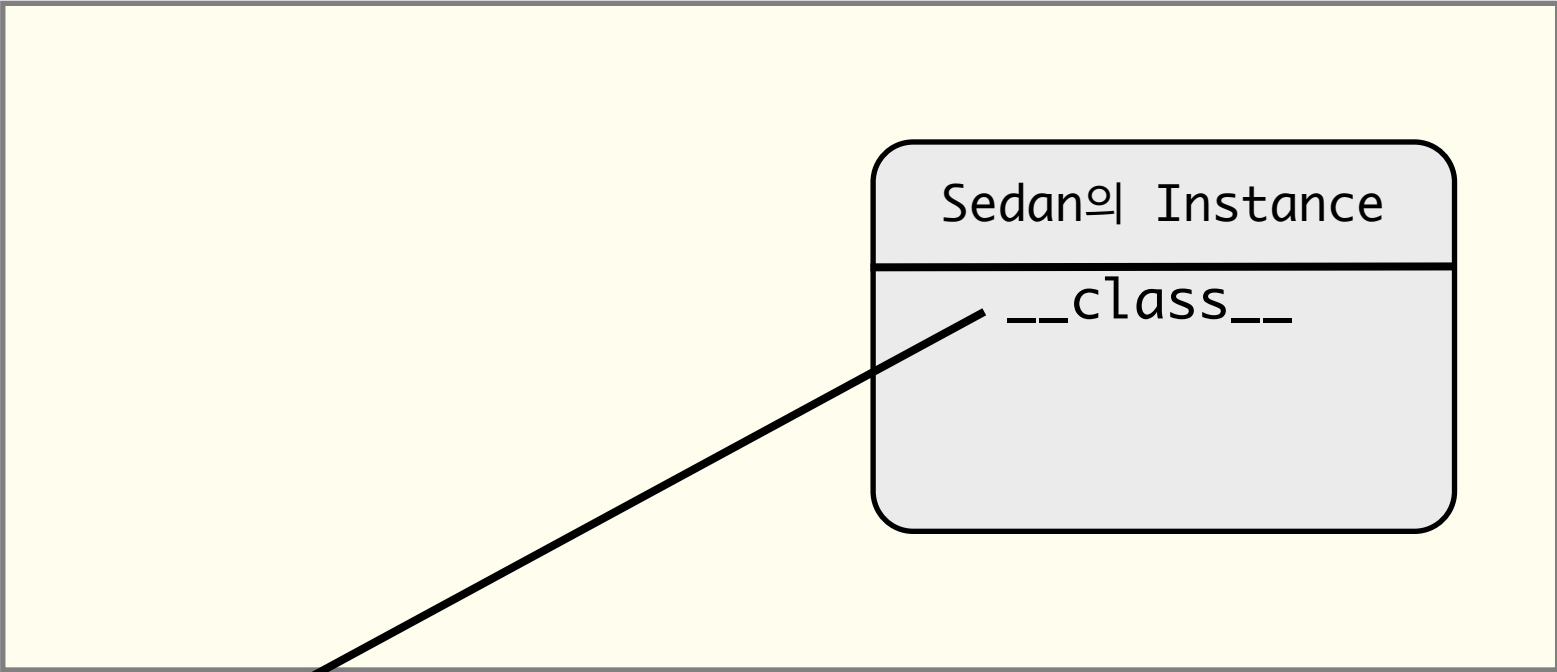


```
sedan = Sedan()
```

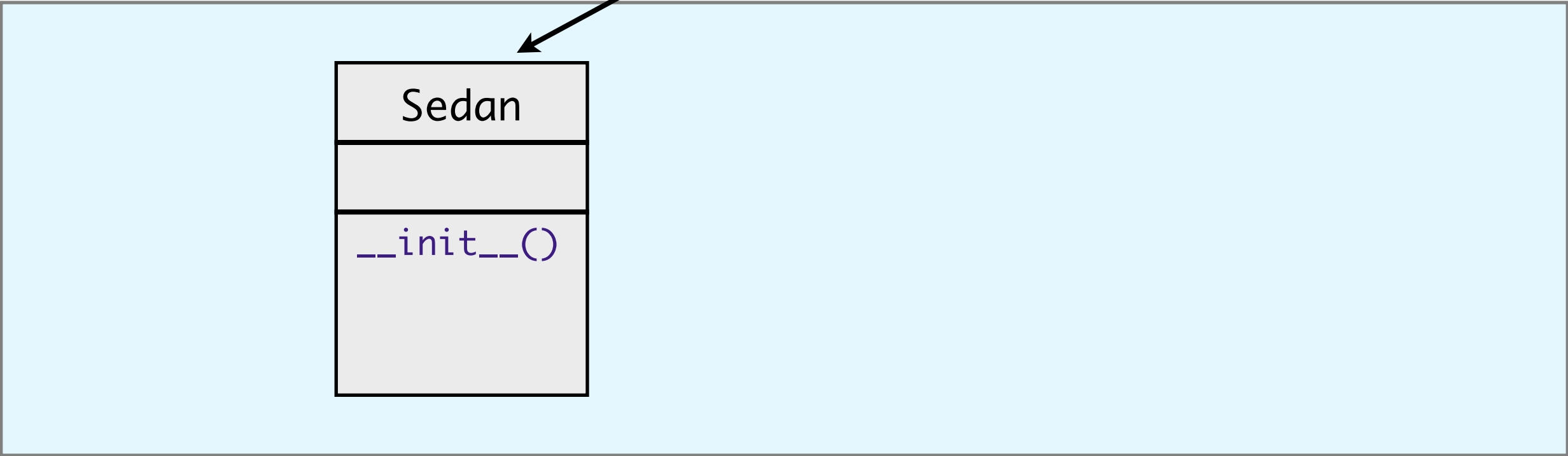
Stack



Heap

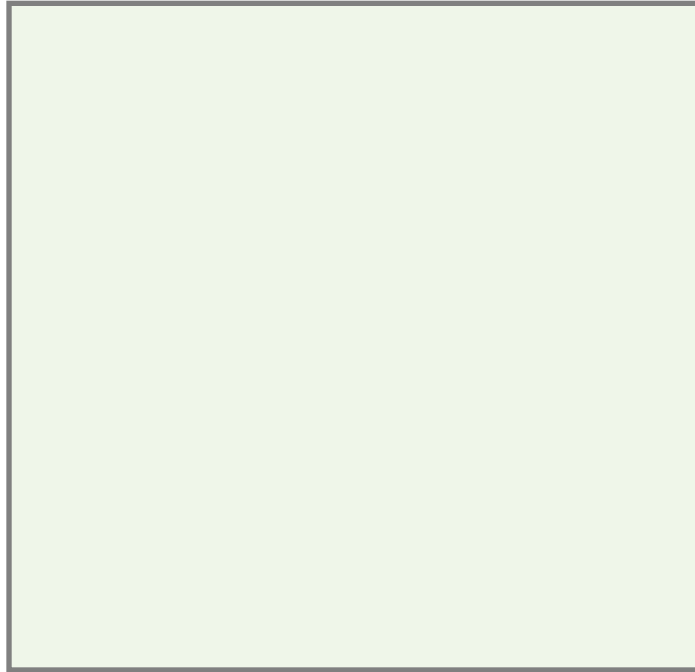


Code

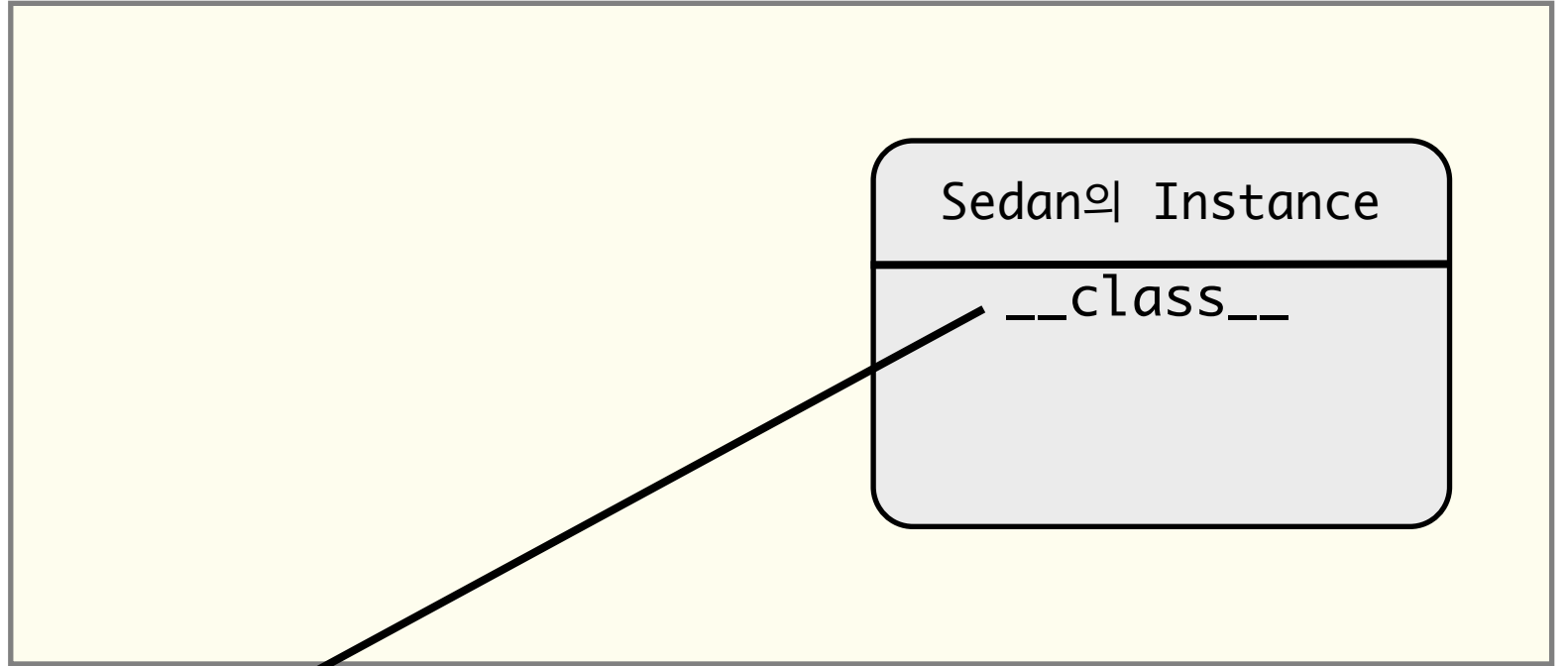


sedan = Sedan()

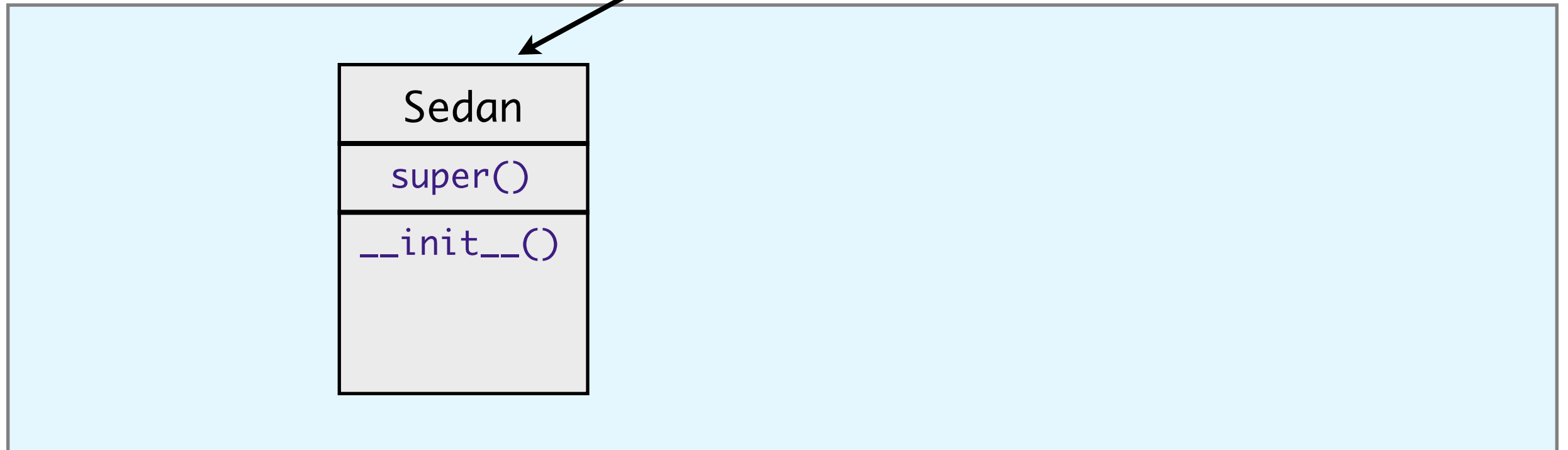
Stack



Heap

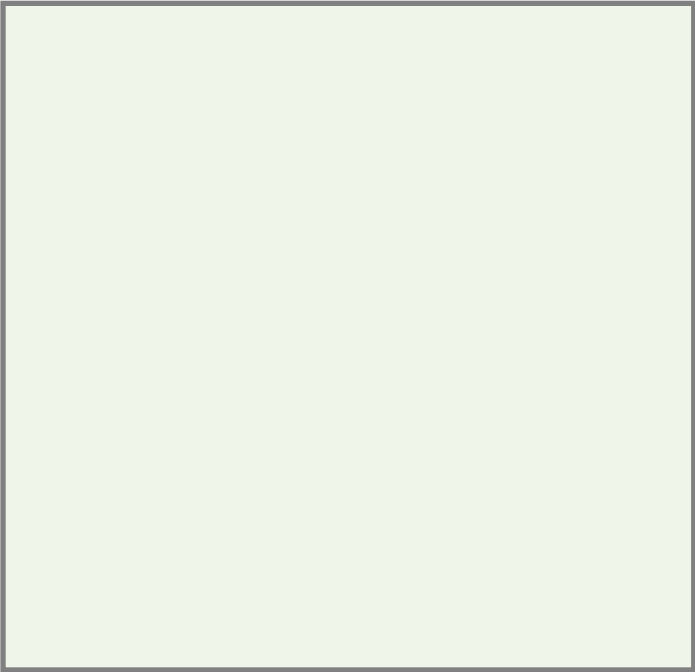


Code

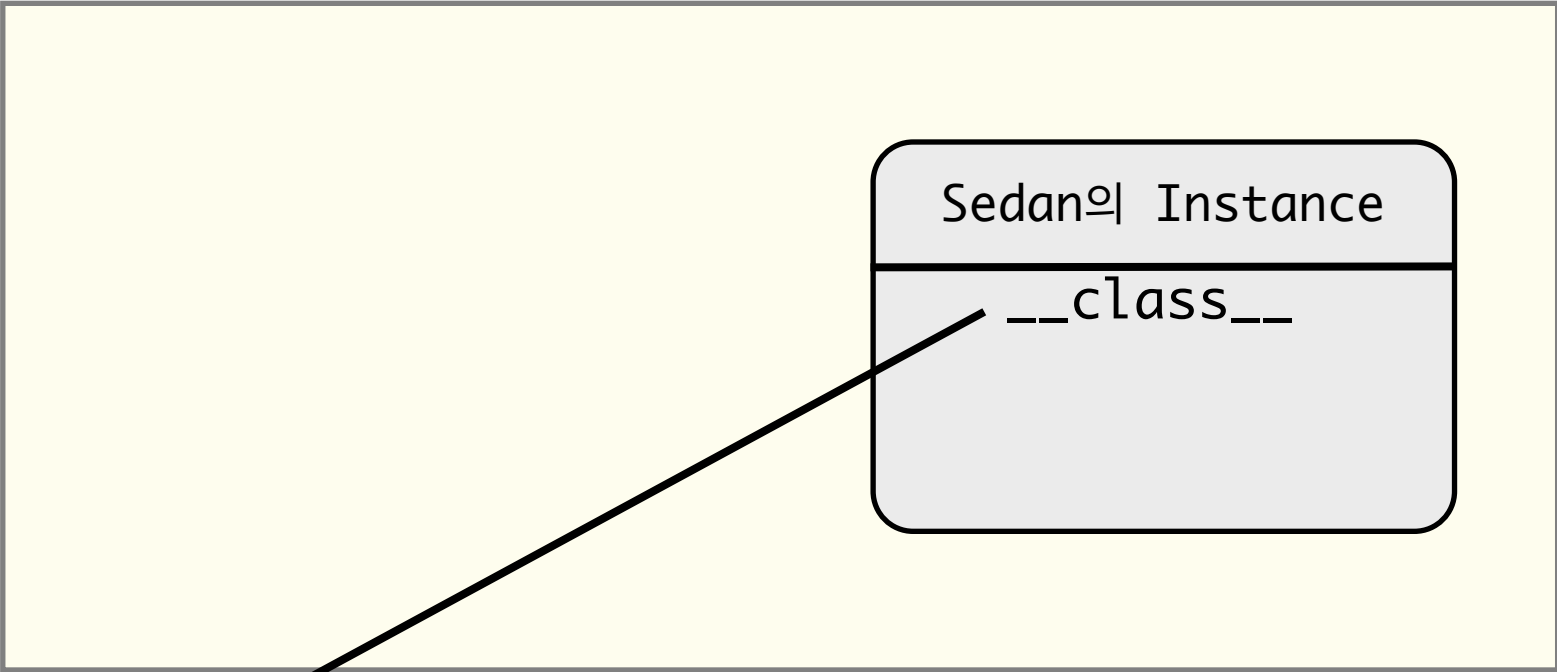


```
sedan = Sedan()
```

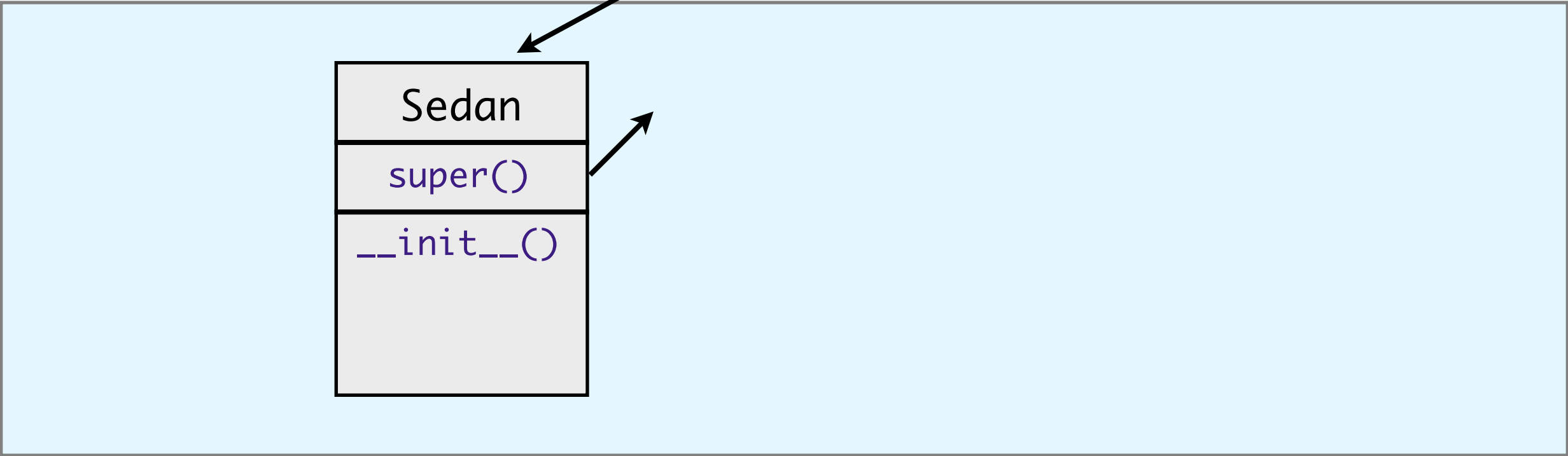
Stack



Heap

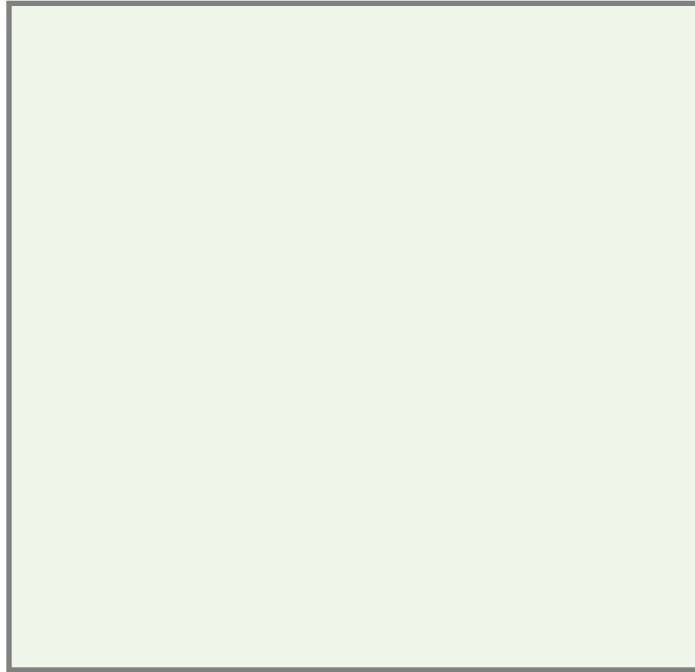


Code

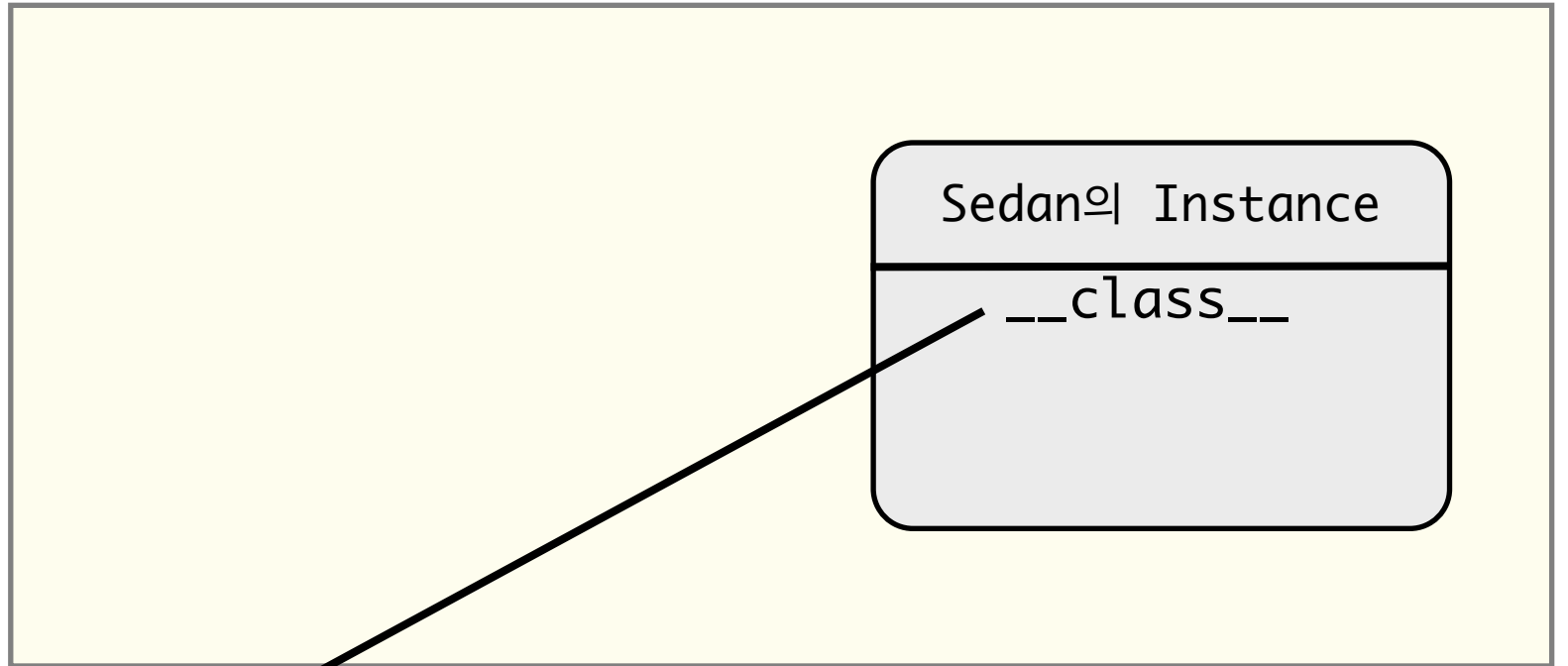


sedan = Sedan()

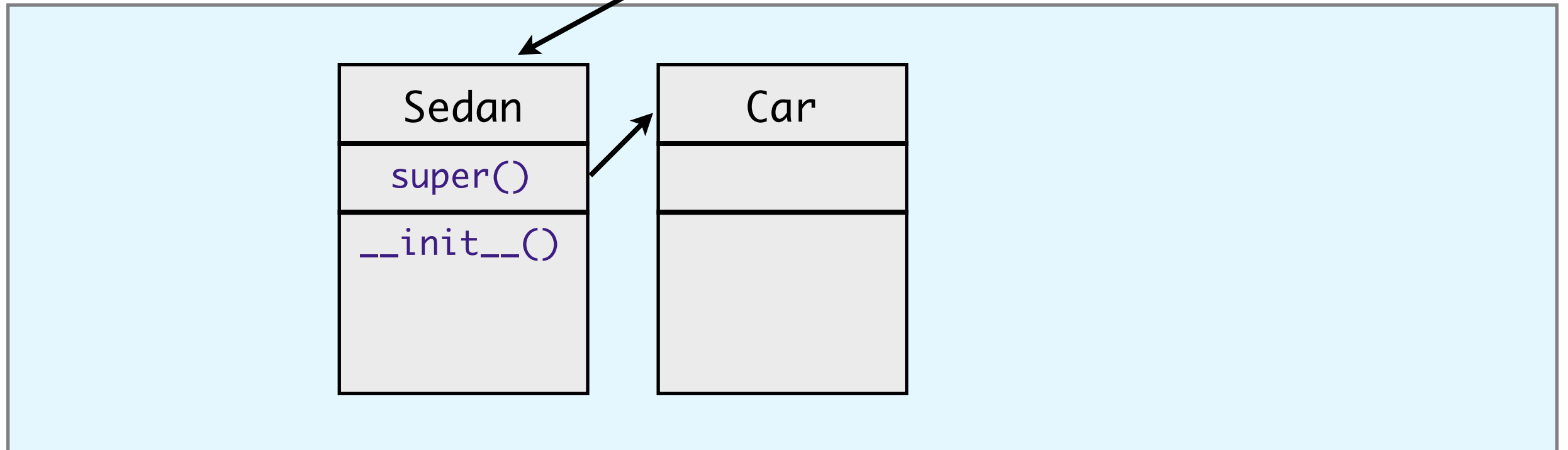
Stack



Heap

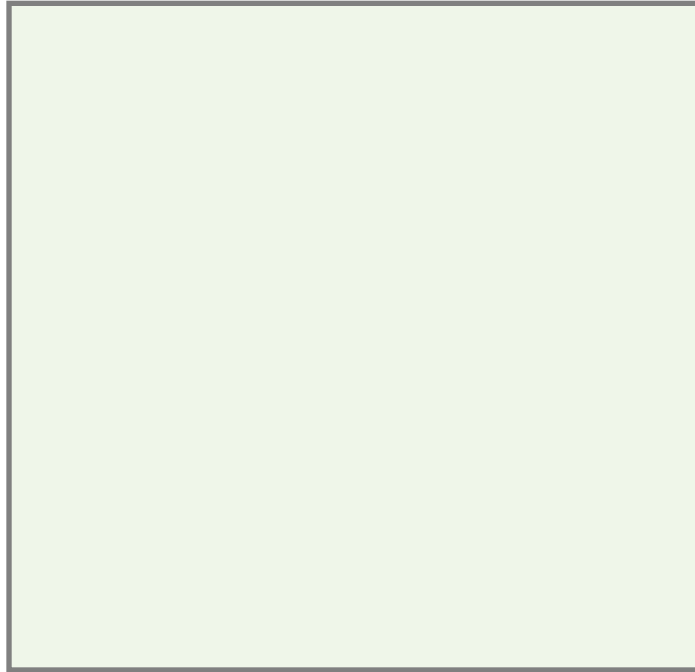


Code

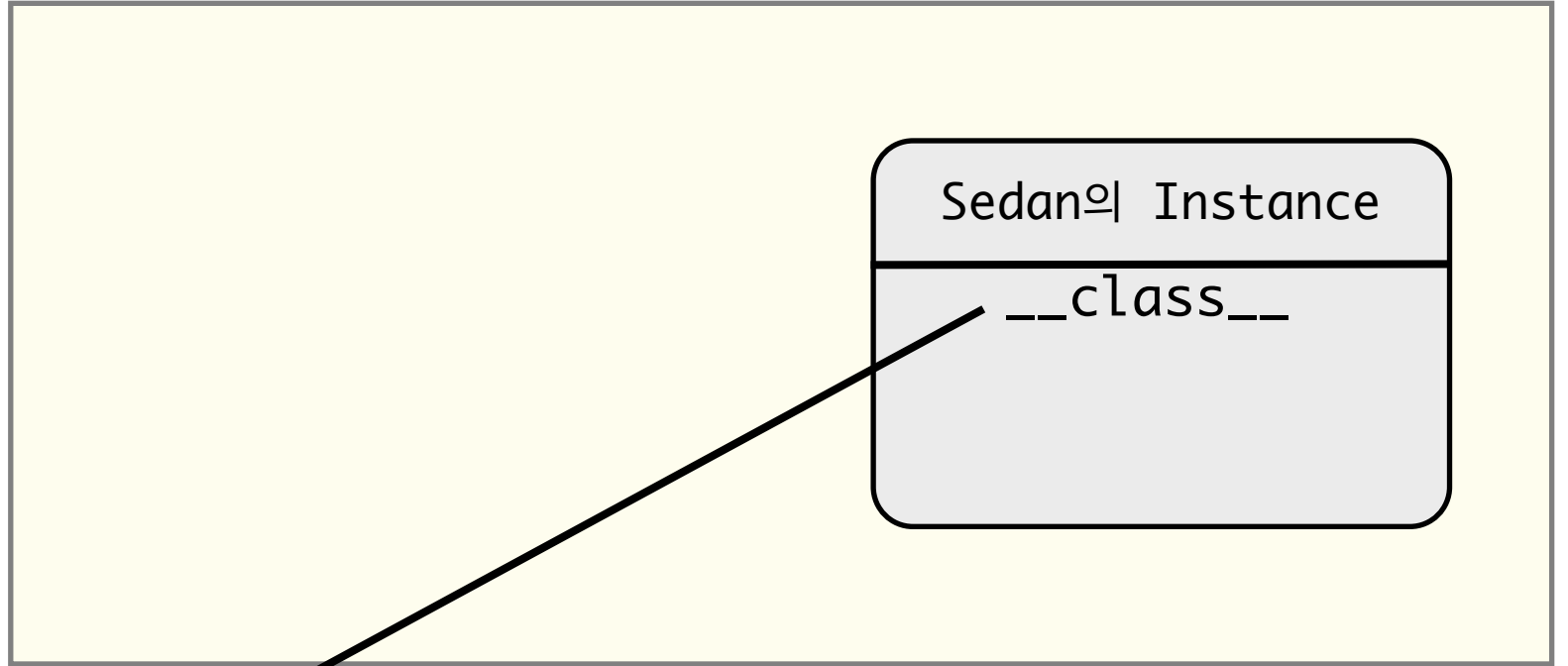


sedan = Sedan()

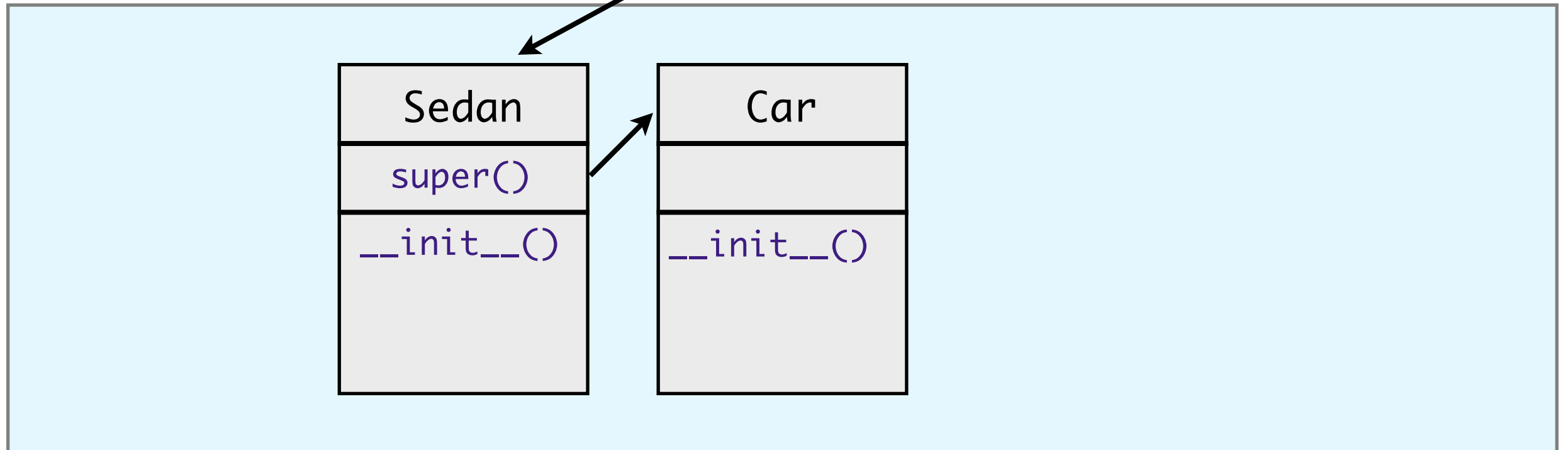
Stack



Heap

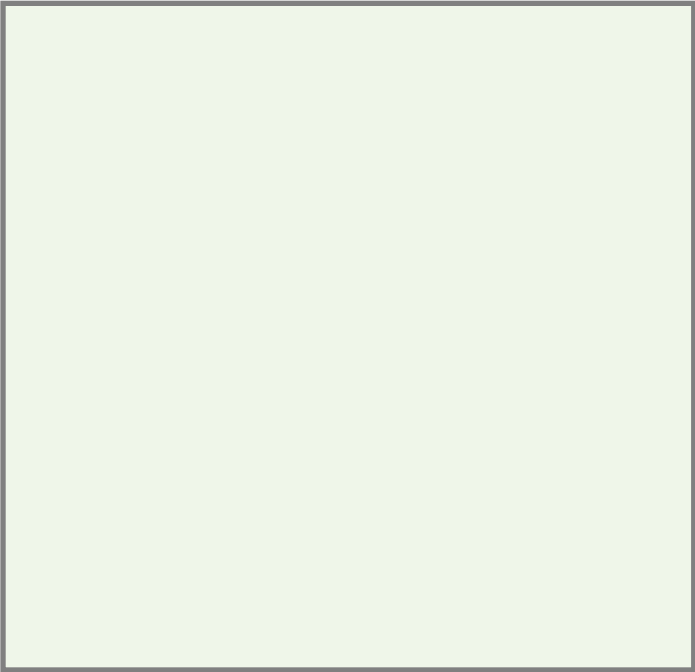


Code

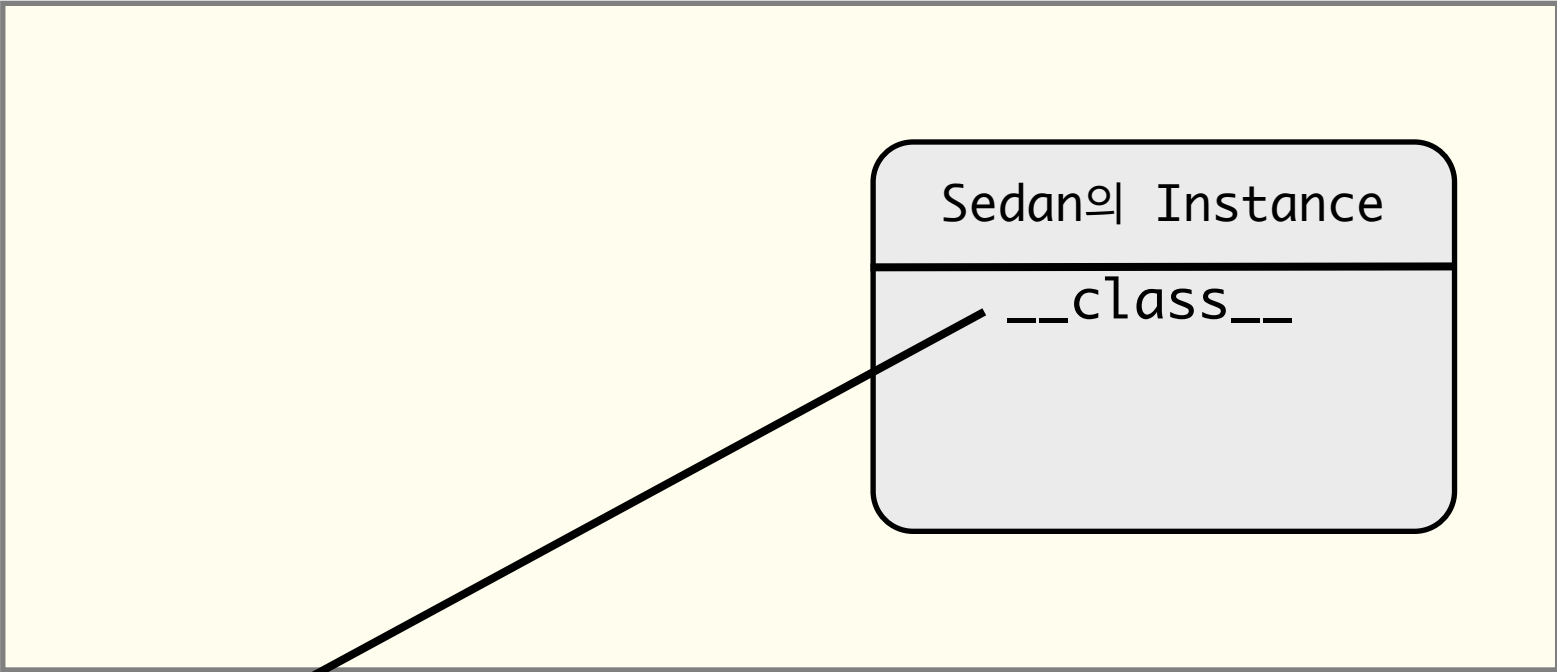


```
sedan = Sedan()
```

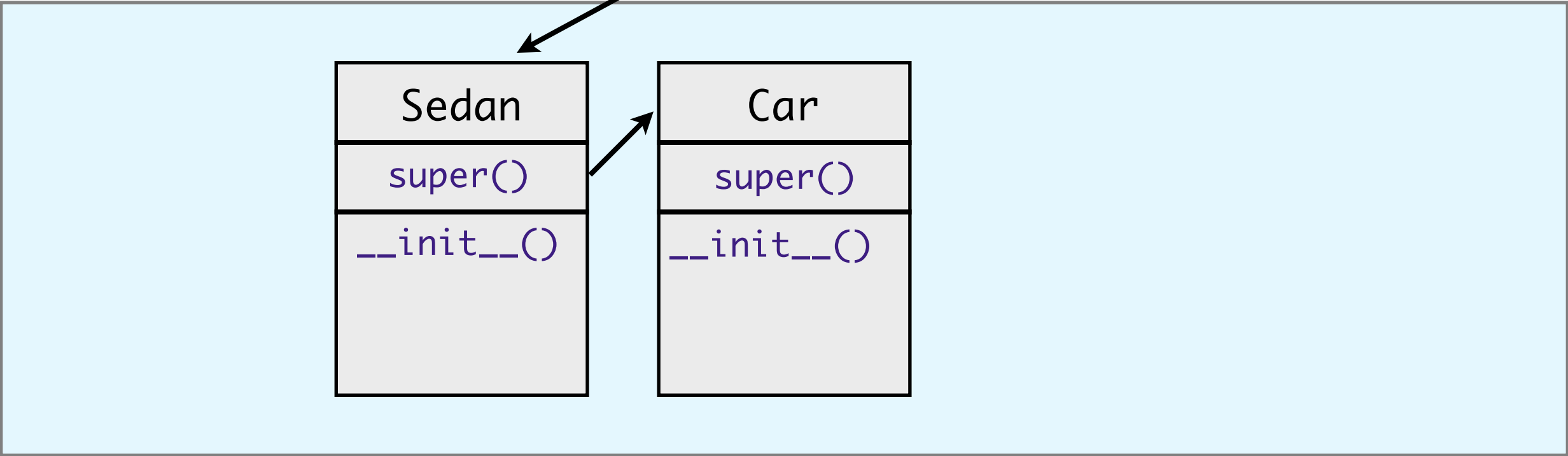
Stack



Heap

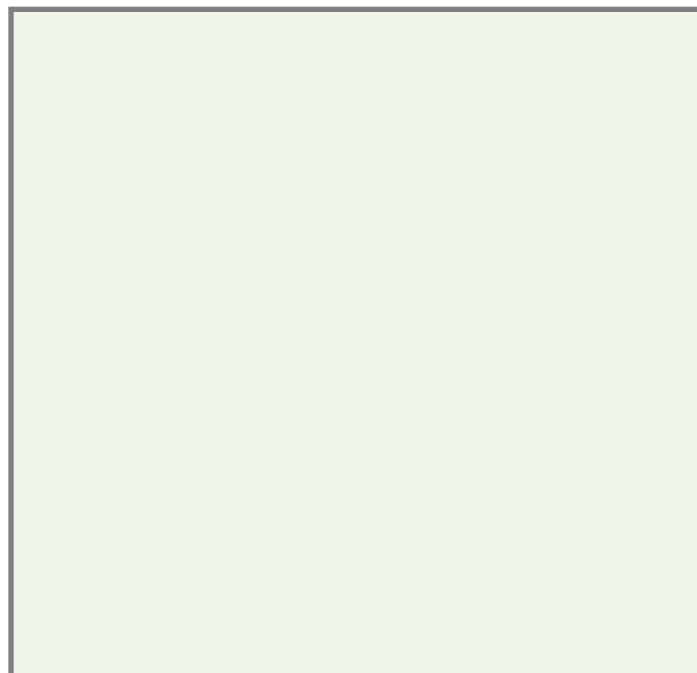


Code

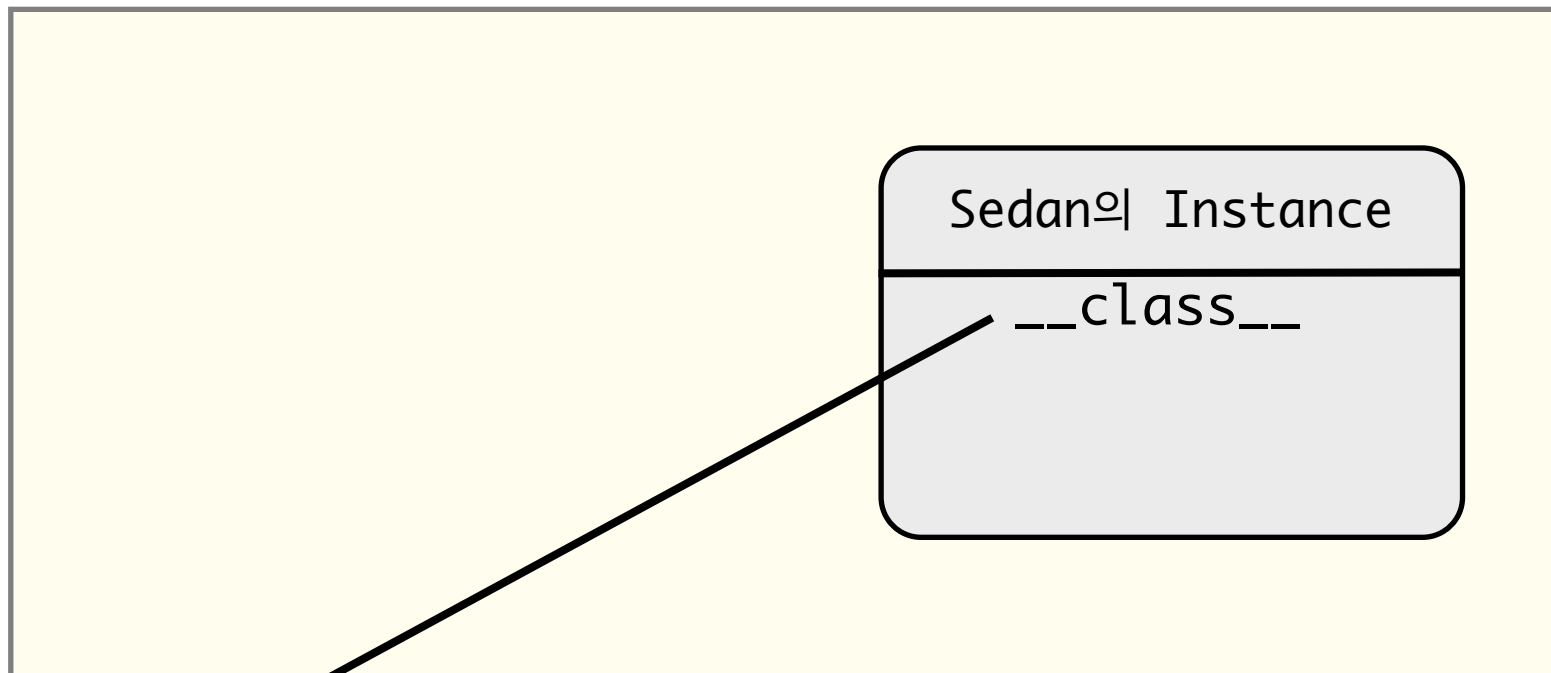


sedan = Sedan()

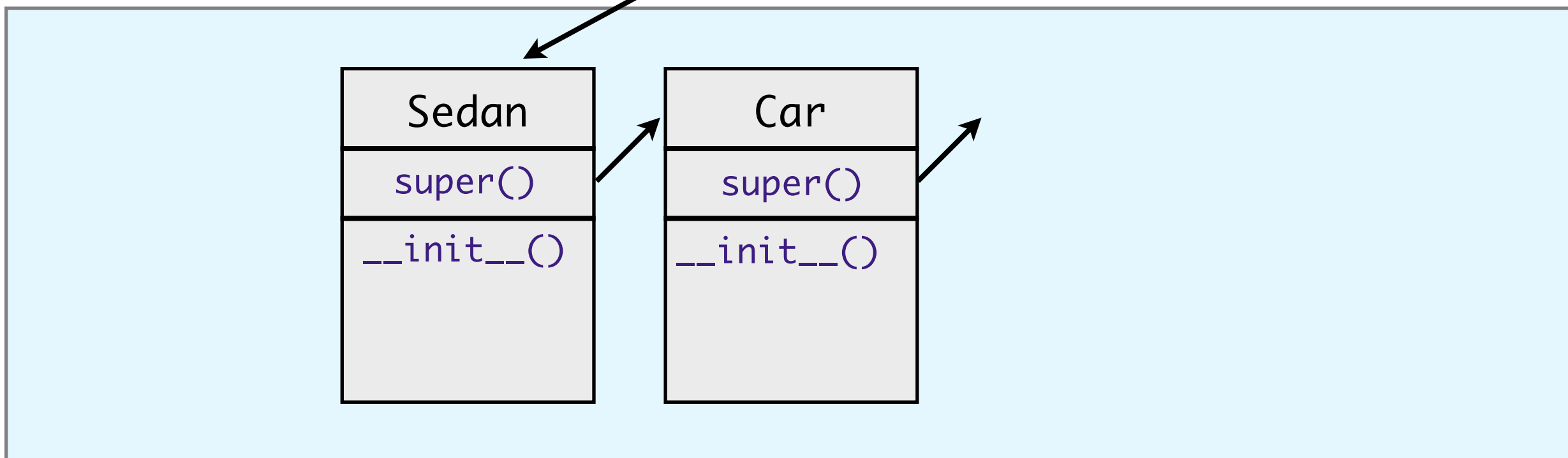
Stack



Heap

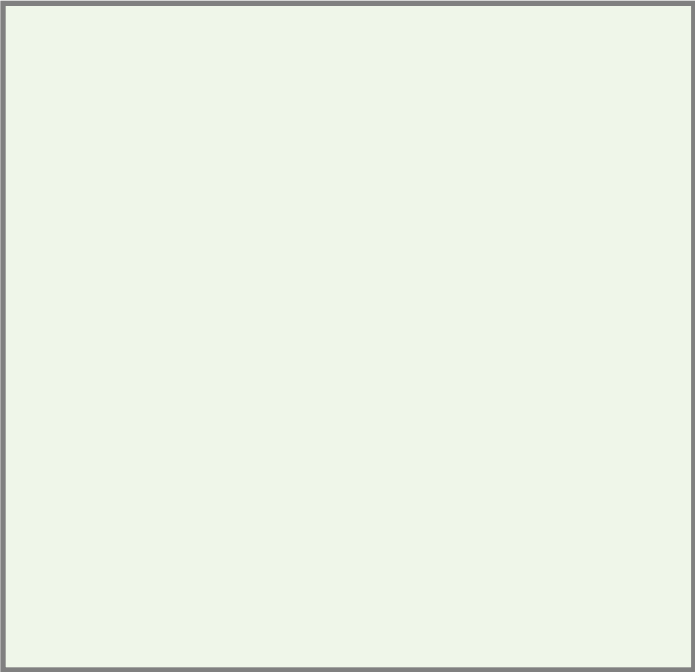


Code

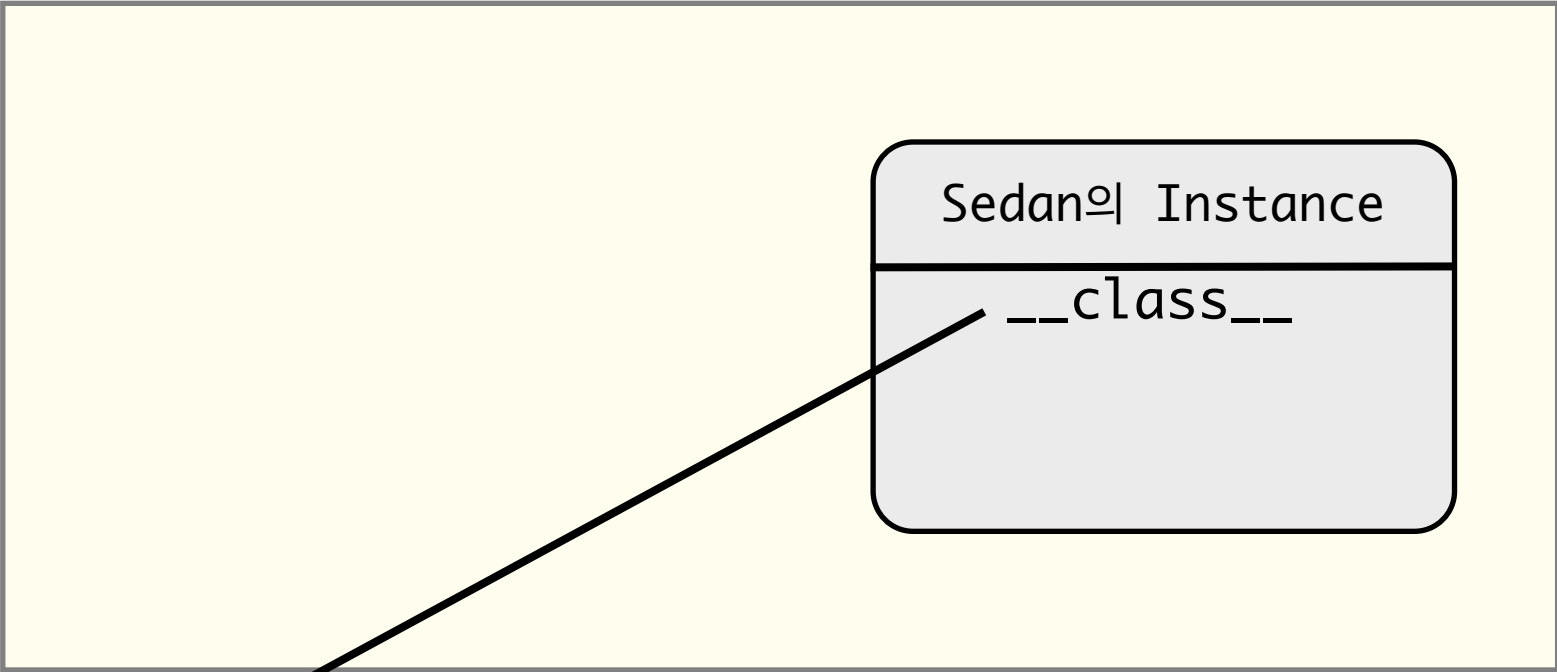


```
sedan = Sedan()
```

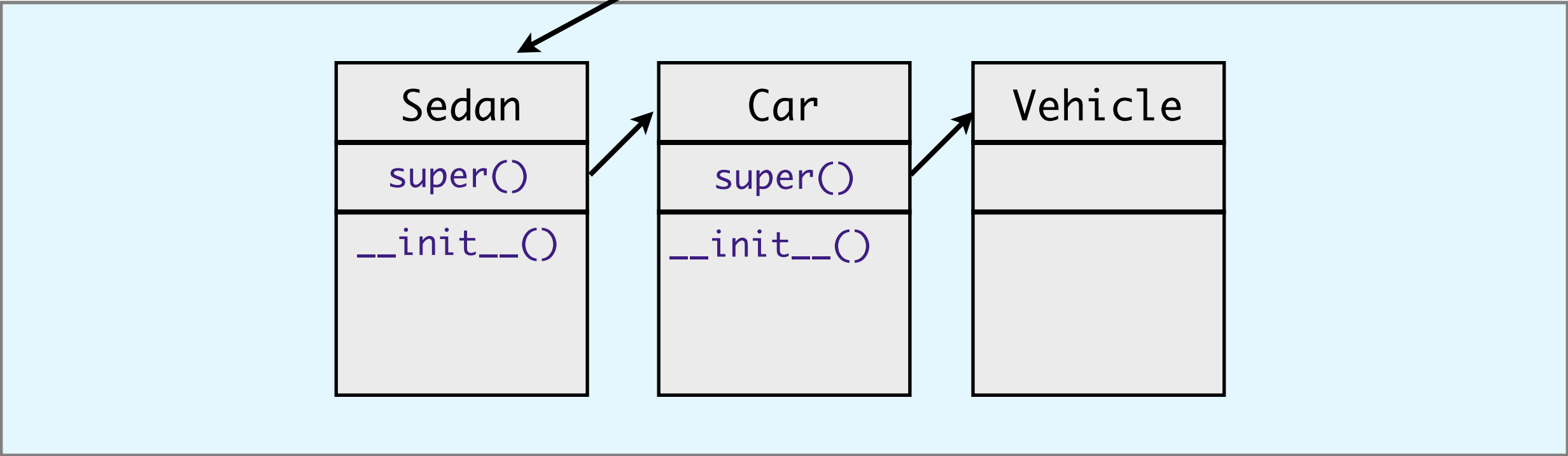
Stack



Heap

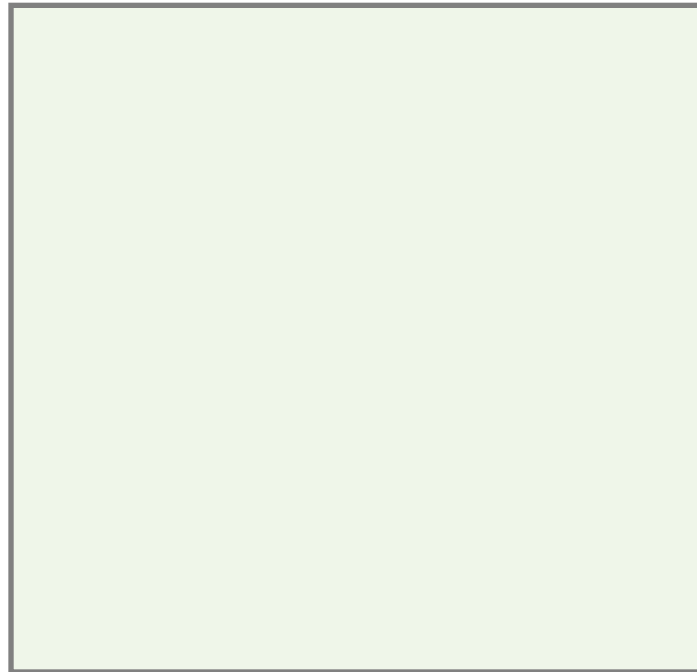


Code

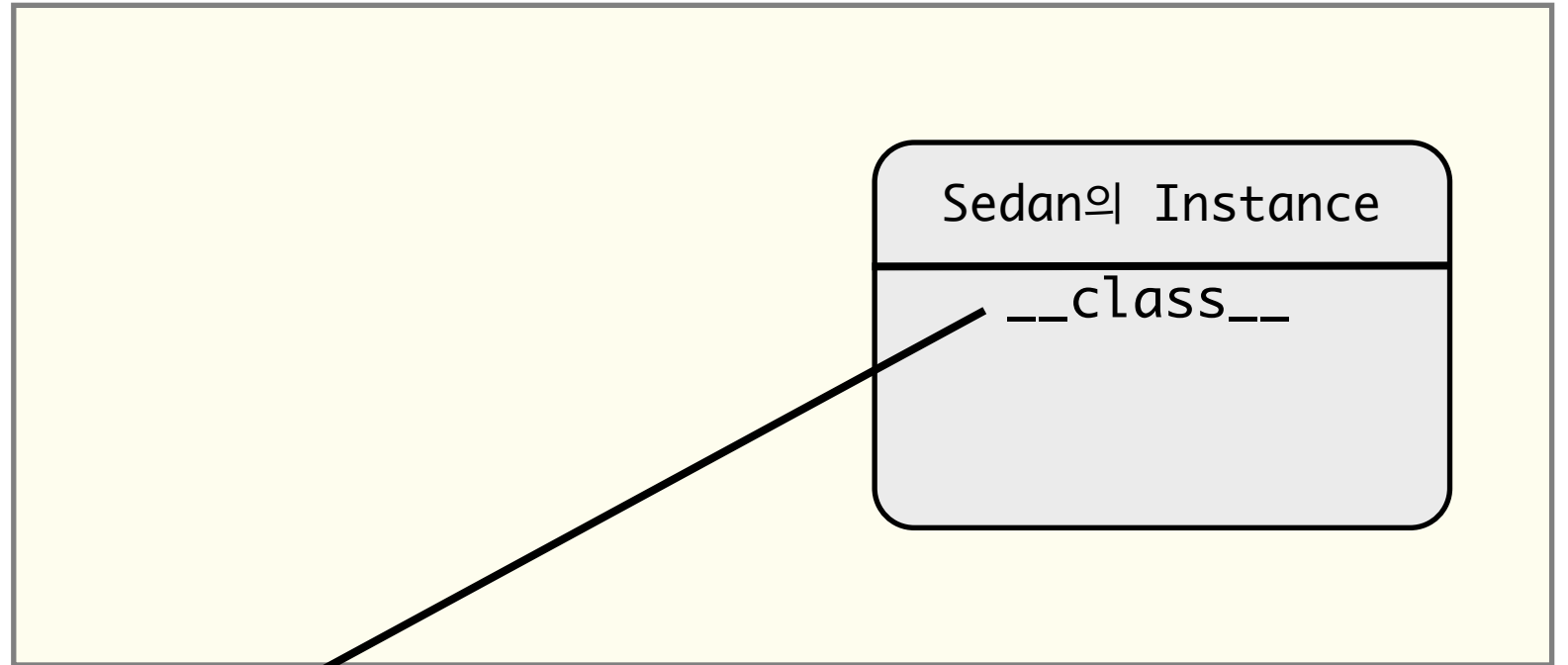


sedan = Sedan()

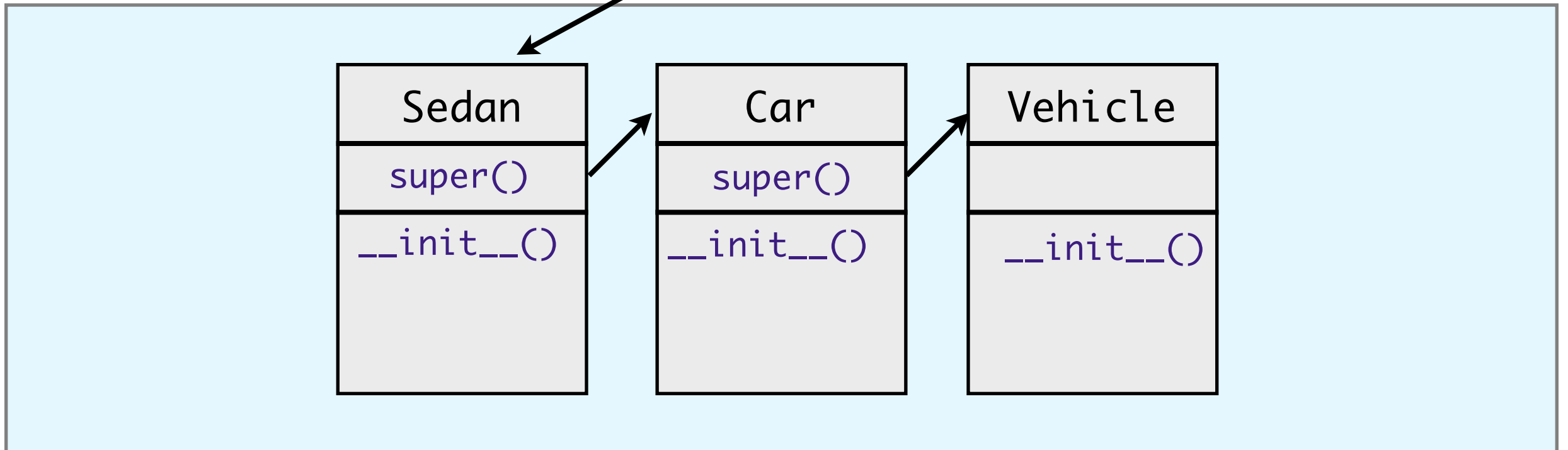
Stack



Heap

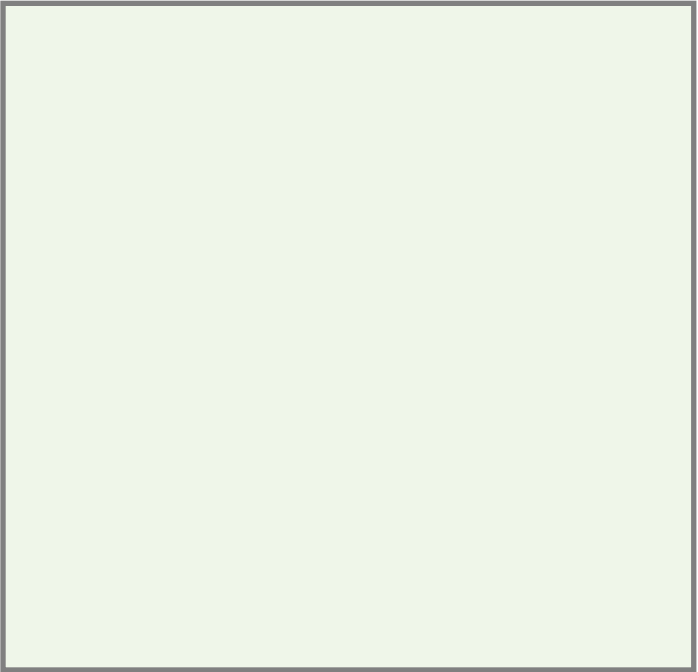


Code

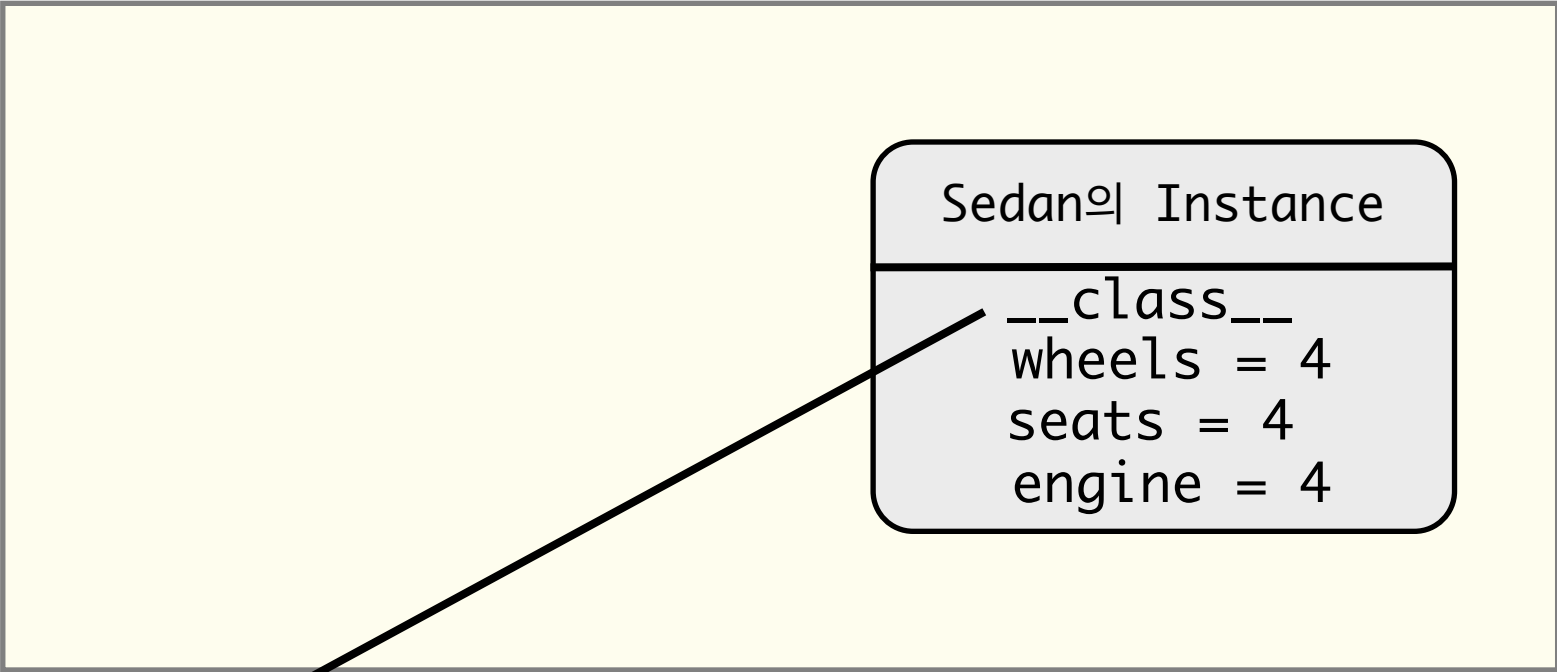


sedan = Sedan()

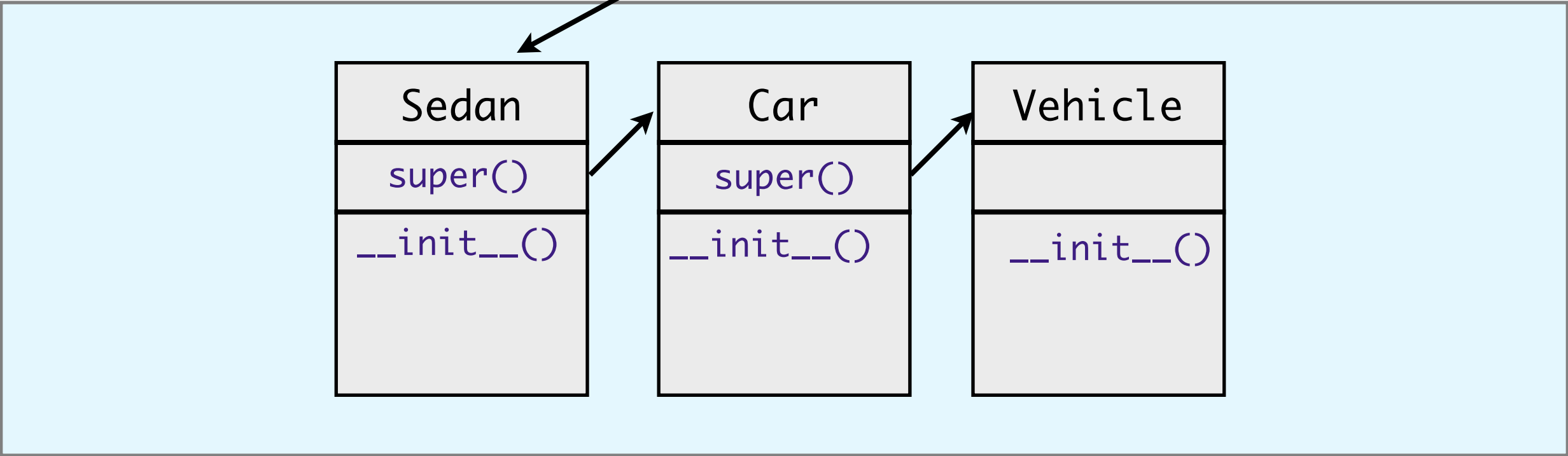
Stack



Heap

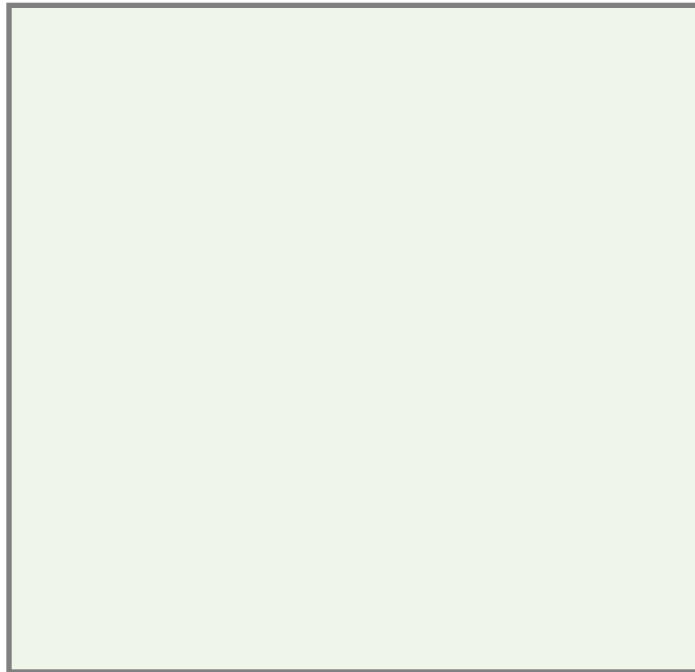


Code

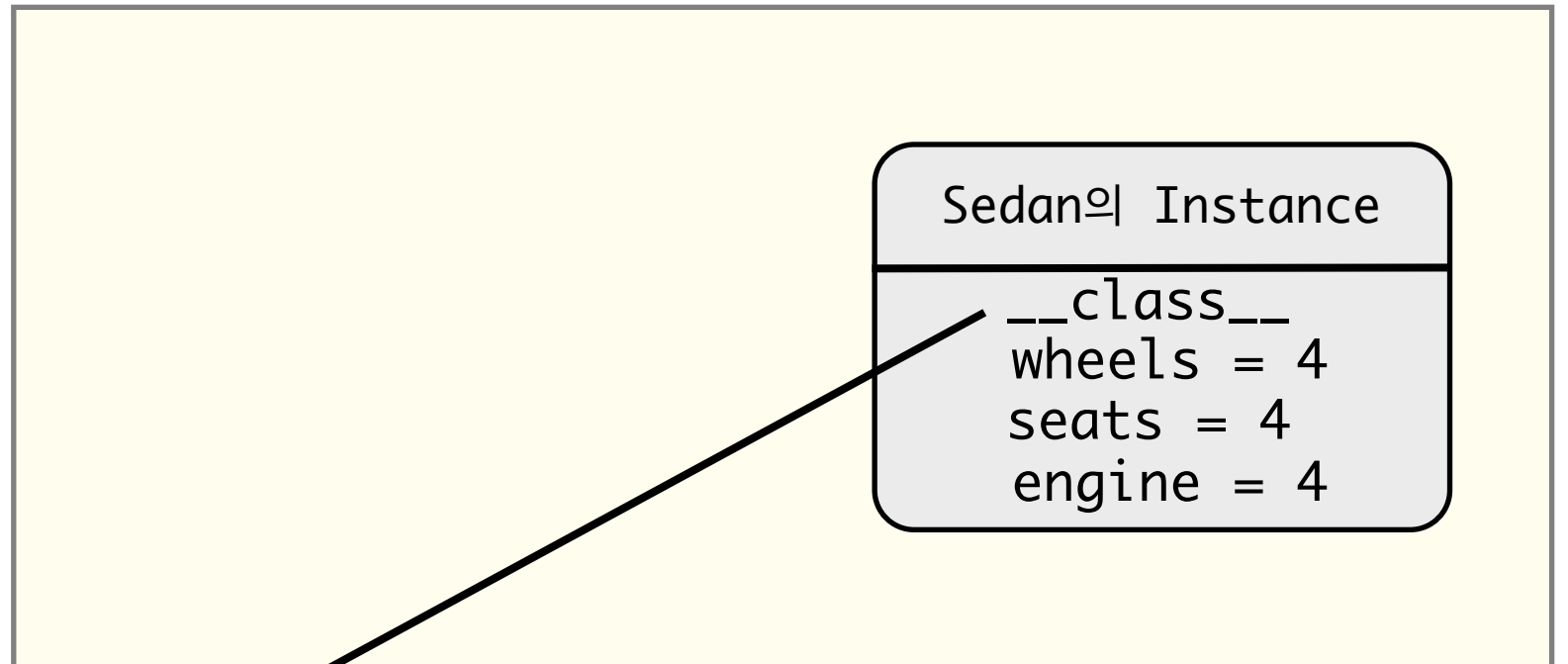


sedan = Sedan()

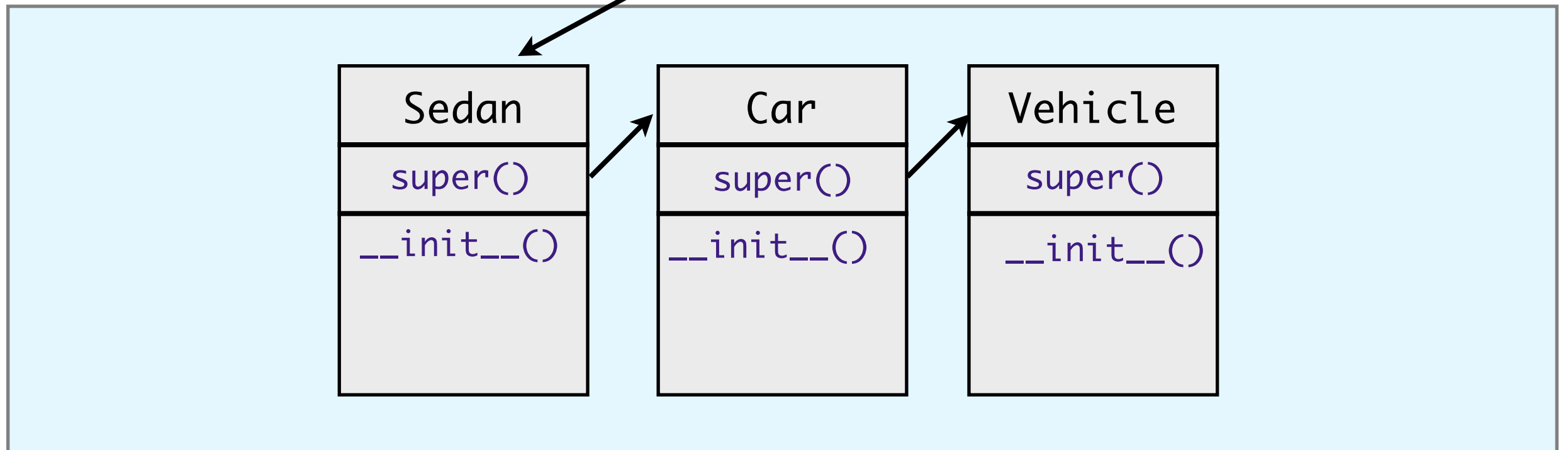
Stack



Heap

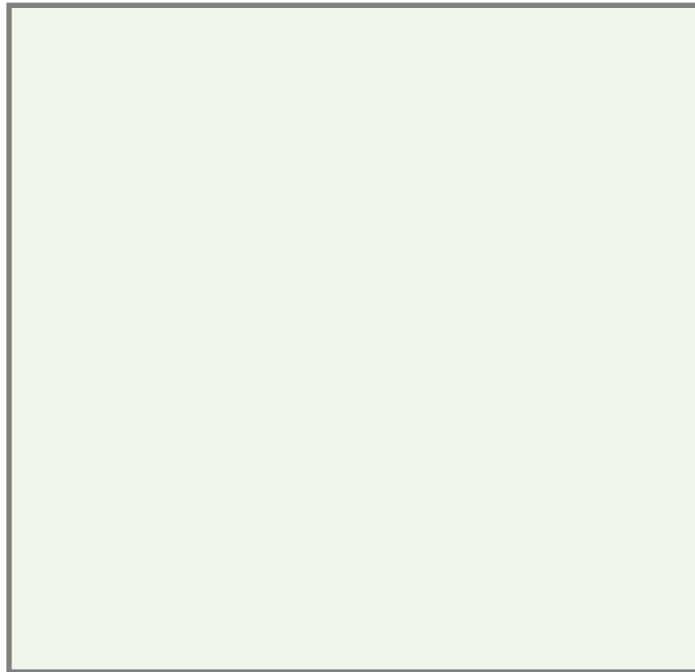


Code

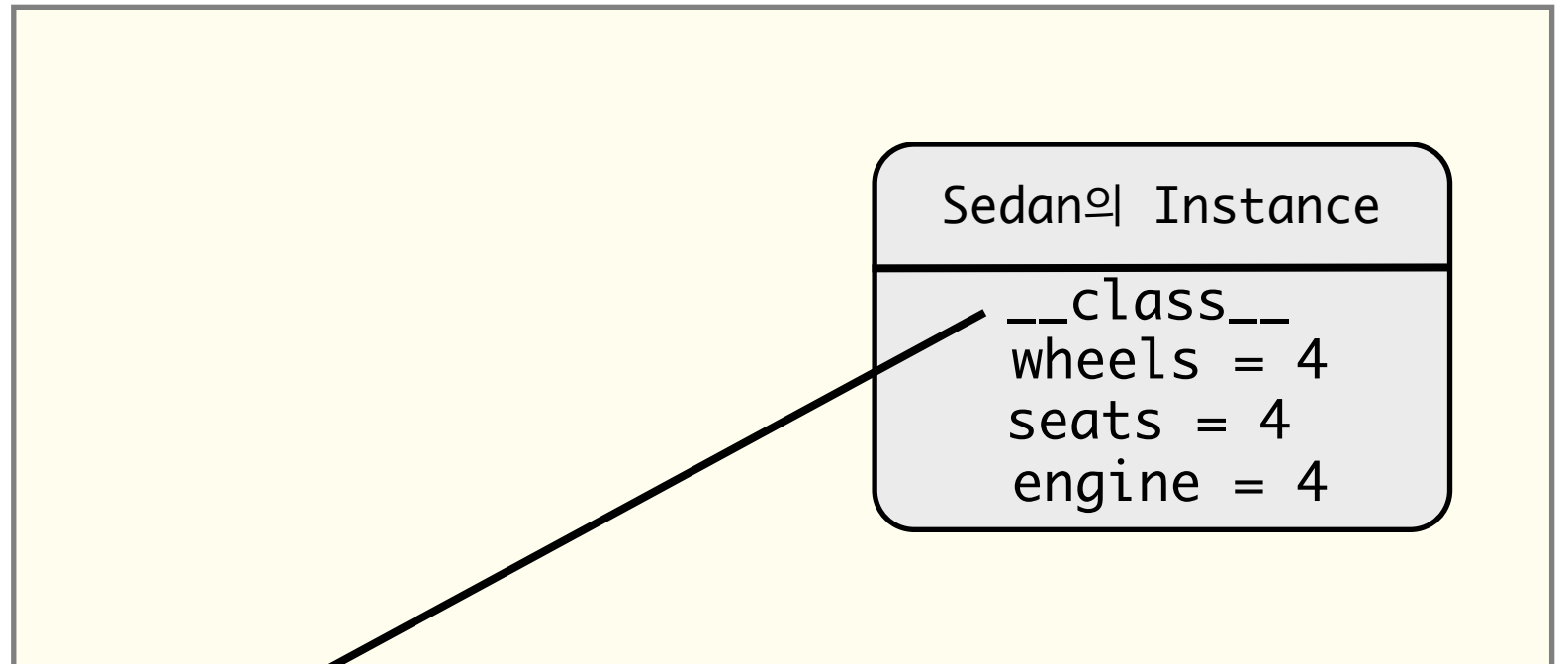


sedan = Sedan()

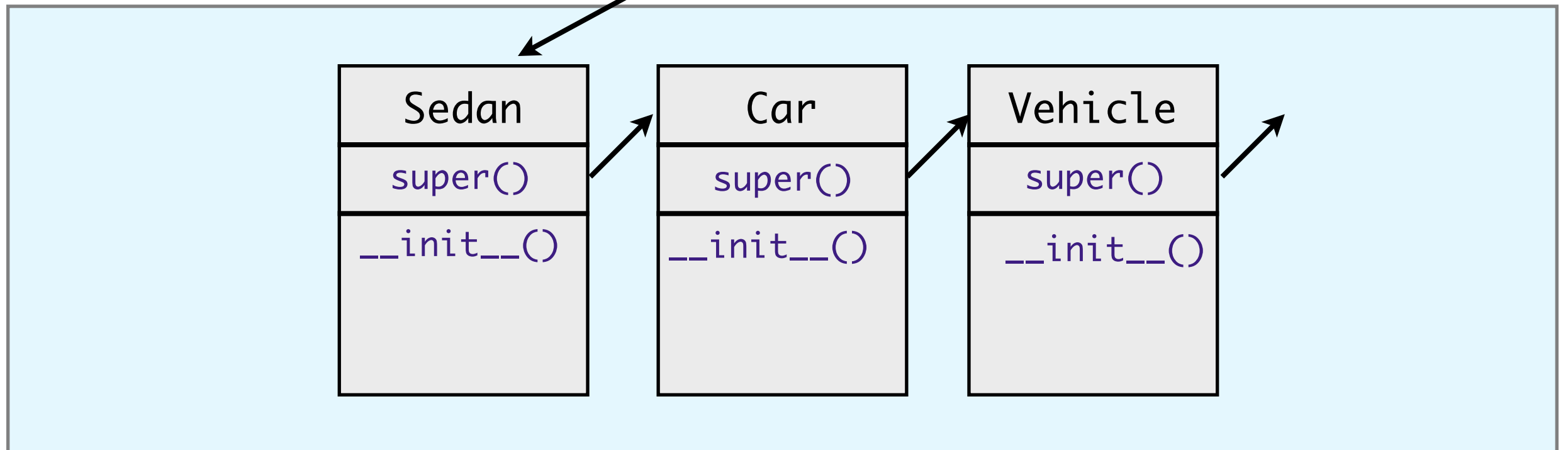
Stack



Heap

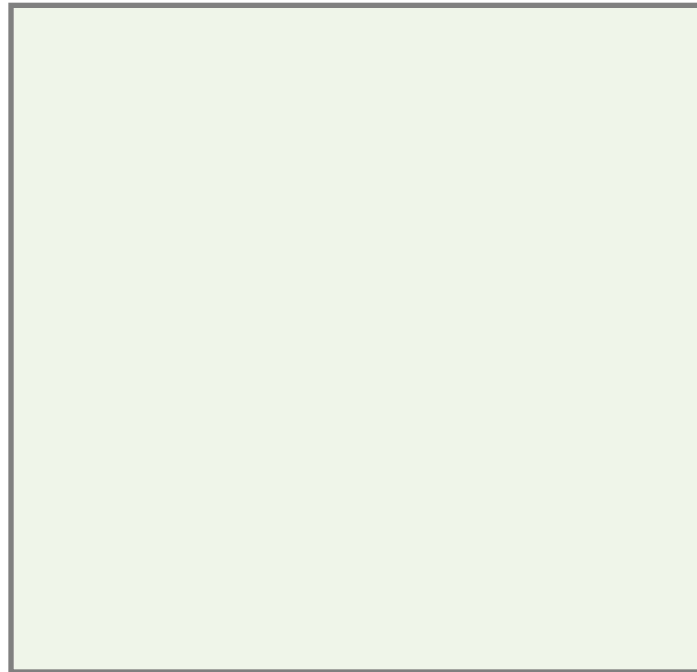


Code

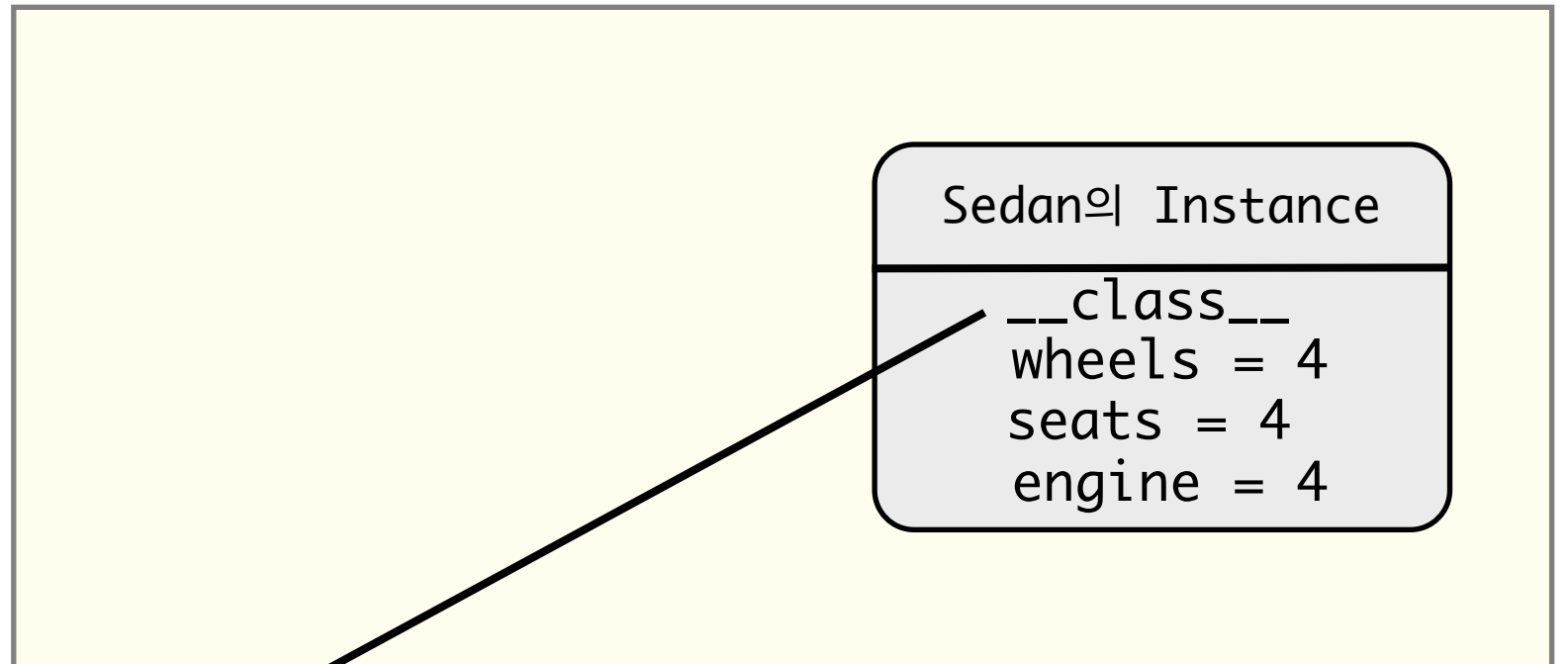


sedan = Sedan()

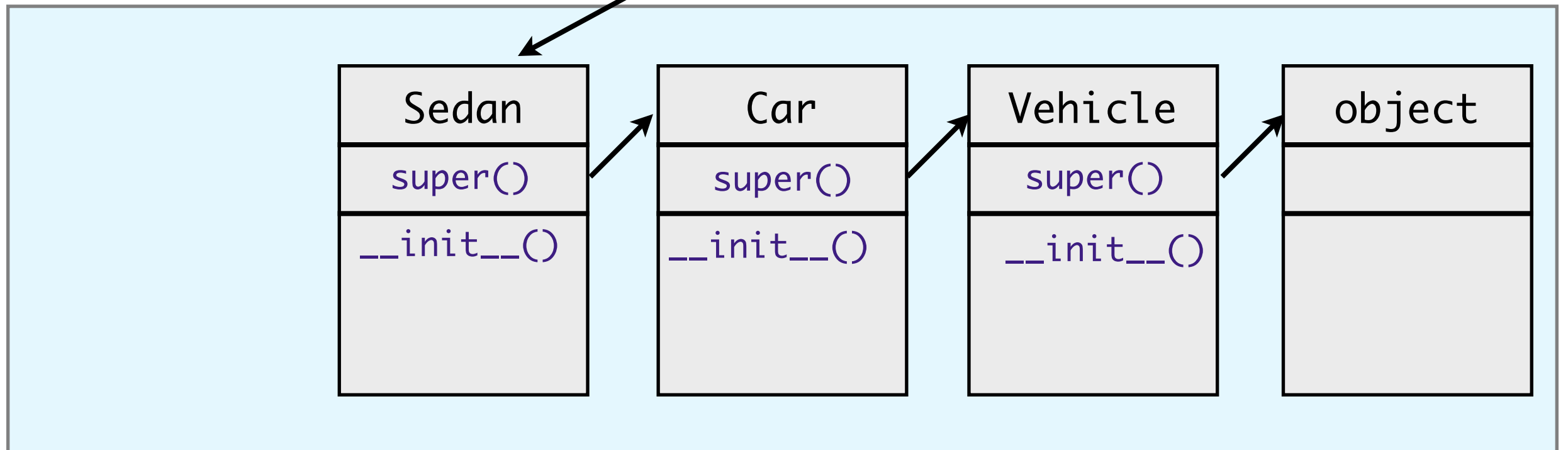
Stack



Heap

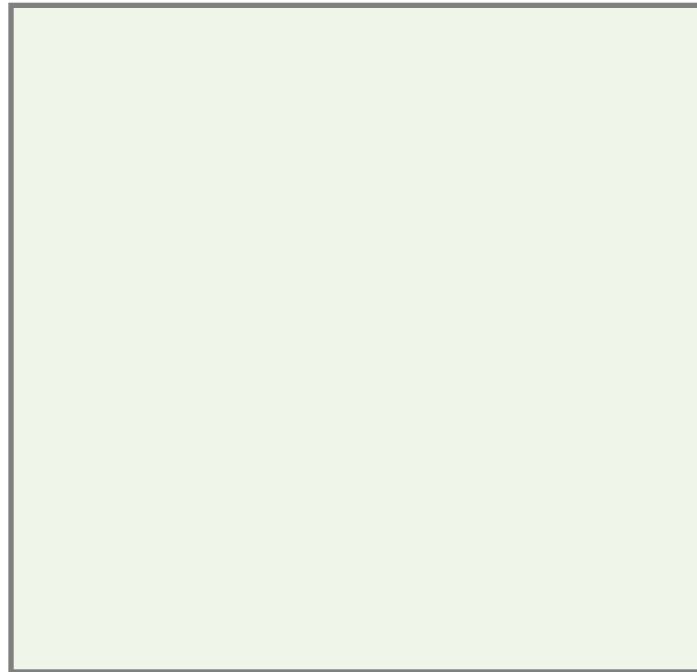


Code

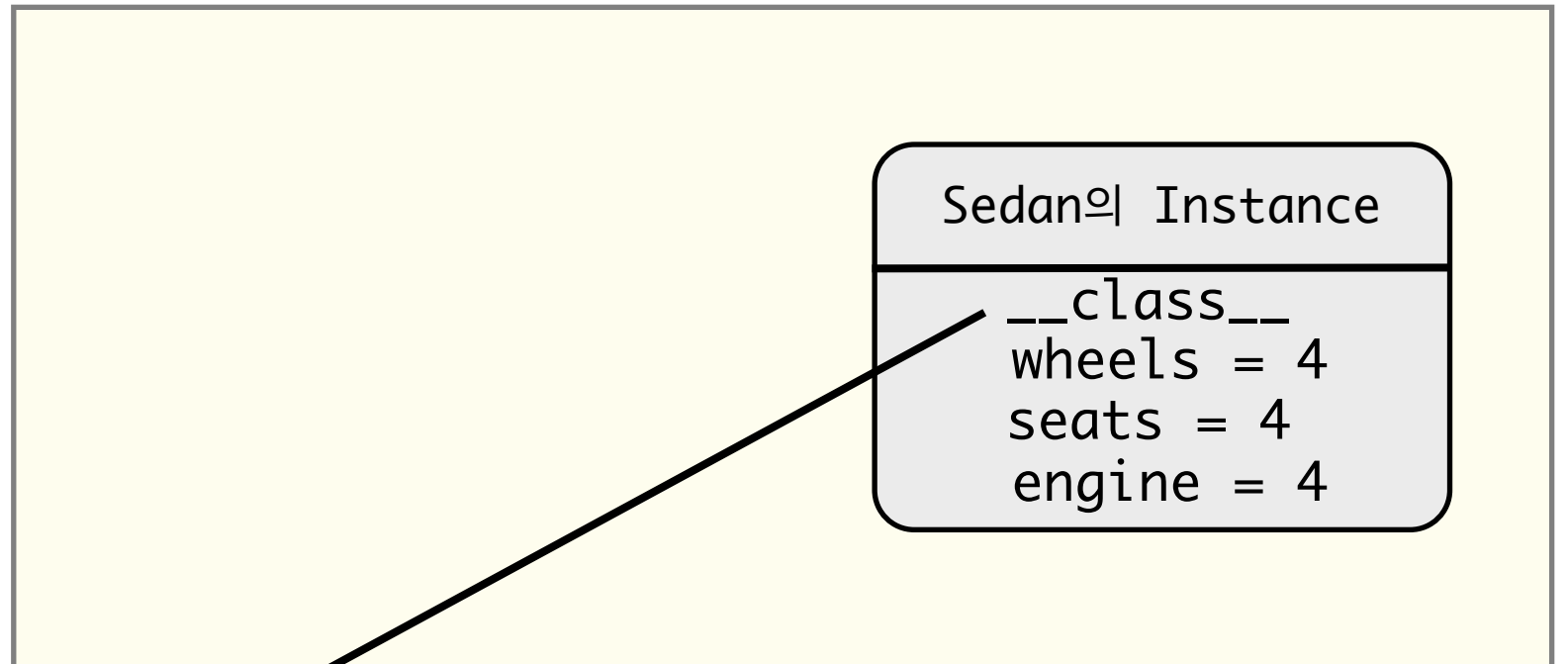


sedan = Sedan()

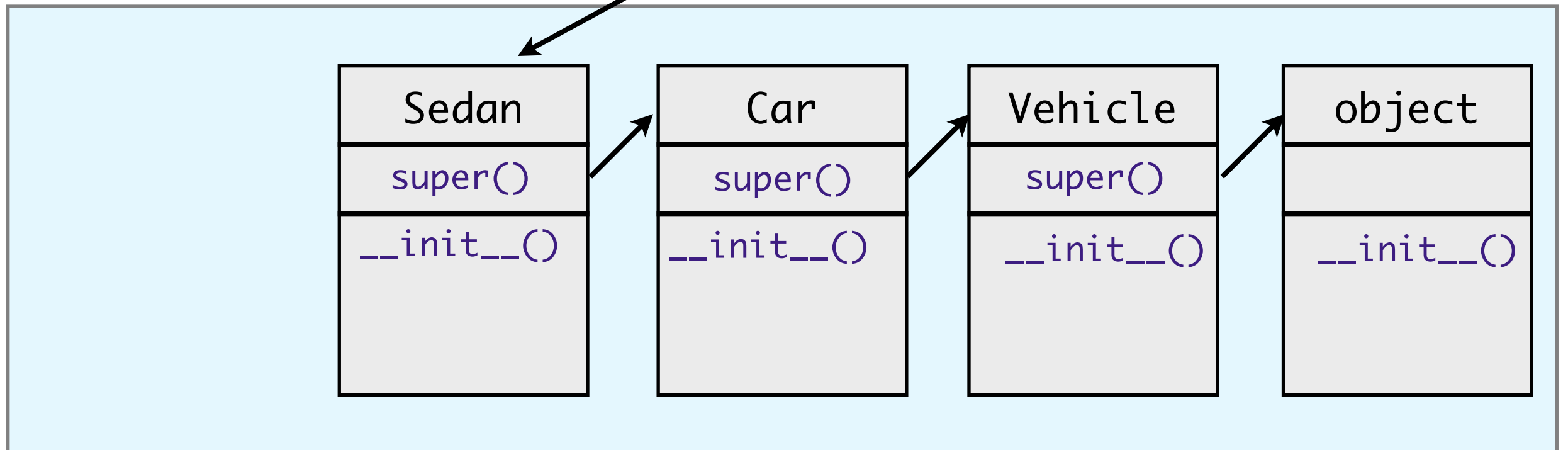
Stack



Heap



Code

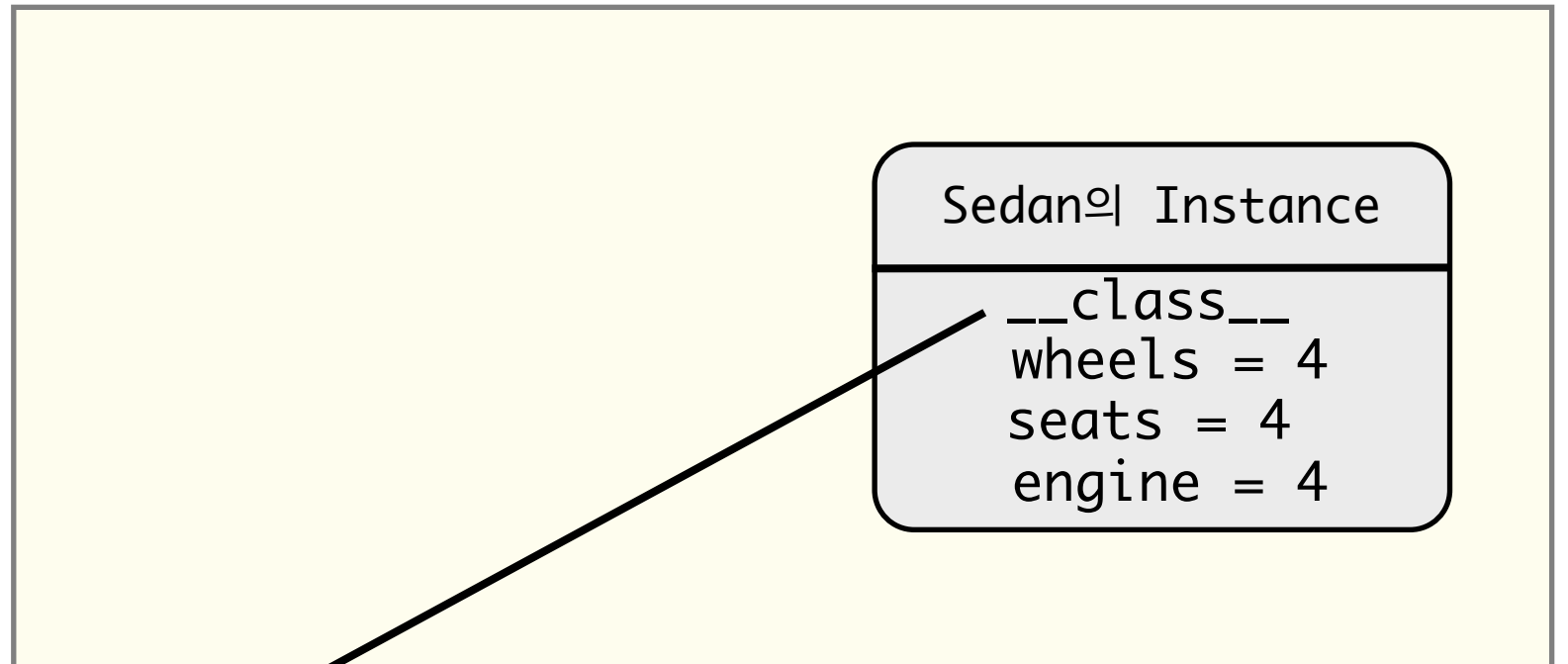


sedan = Sedan()

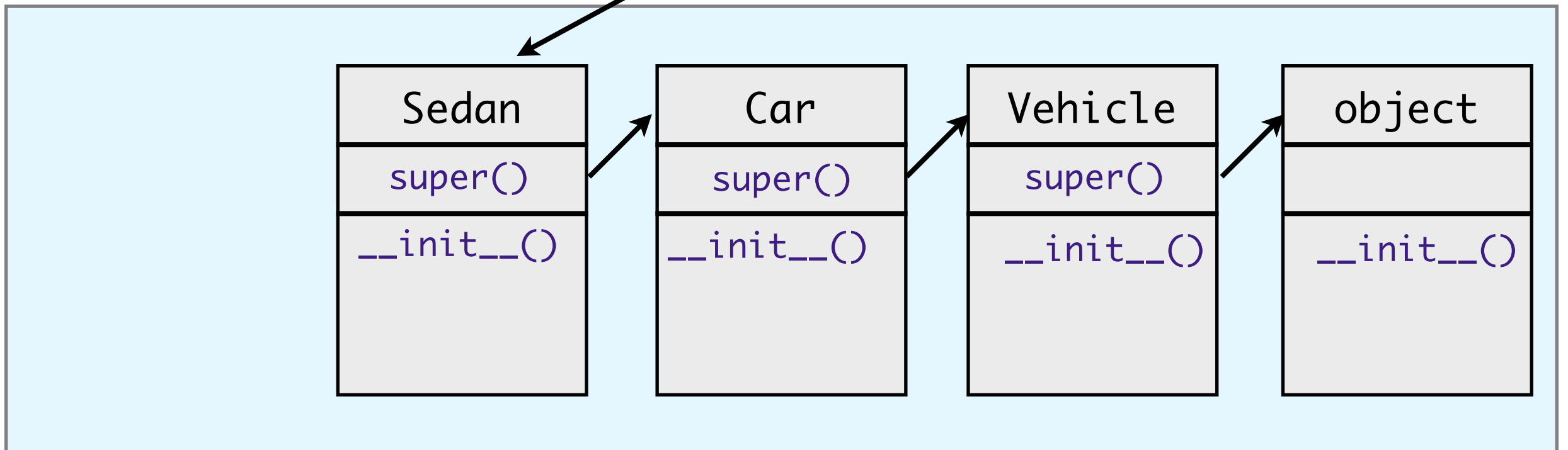
Stack



Heap



Code



sedan = Sedan()

Stack

Heap

sedan

Sedan의 Instance

__class__
wheels = 4
seats = 4
engine = 4

Code

Sedan

super()

__init__()

Car

super()

__init__()

Vehicle

super()

__init__()

object

__init__()

Stack

Heap

sedan

Sedan의 Instance

__class__
wheels = 4
seats = 4
engine = 4

Code

Sedan

super()

__init__()

Car

super()

__init__()

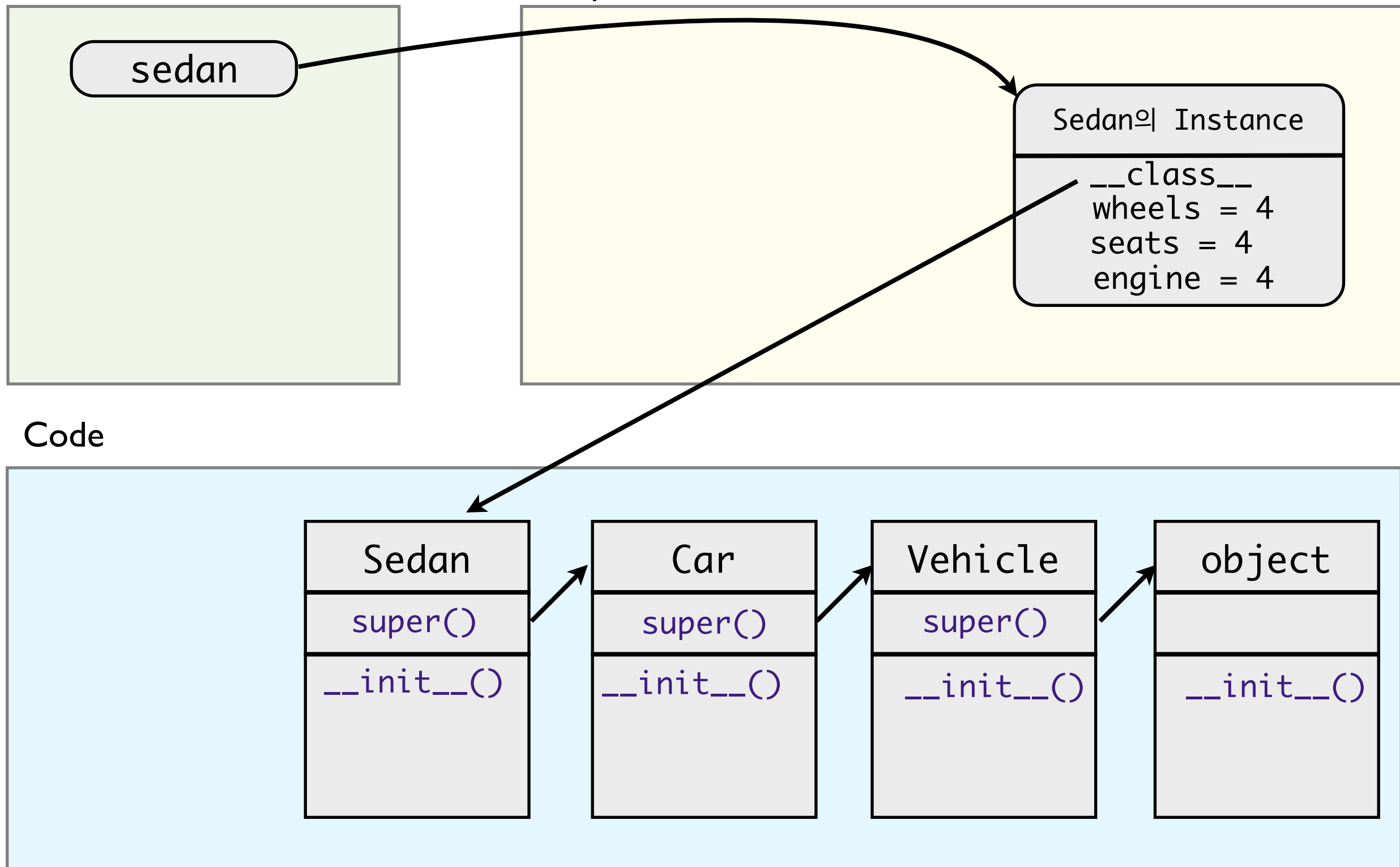
Vehicle

super()

__init__()

object

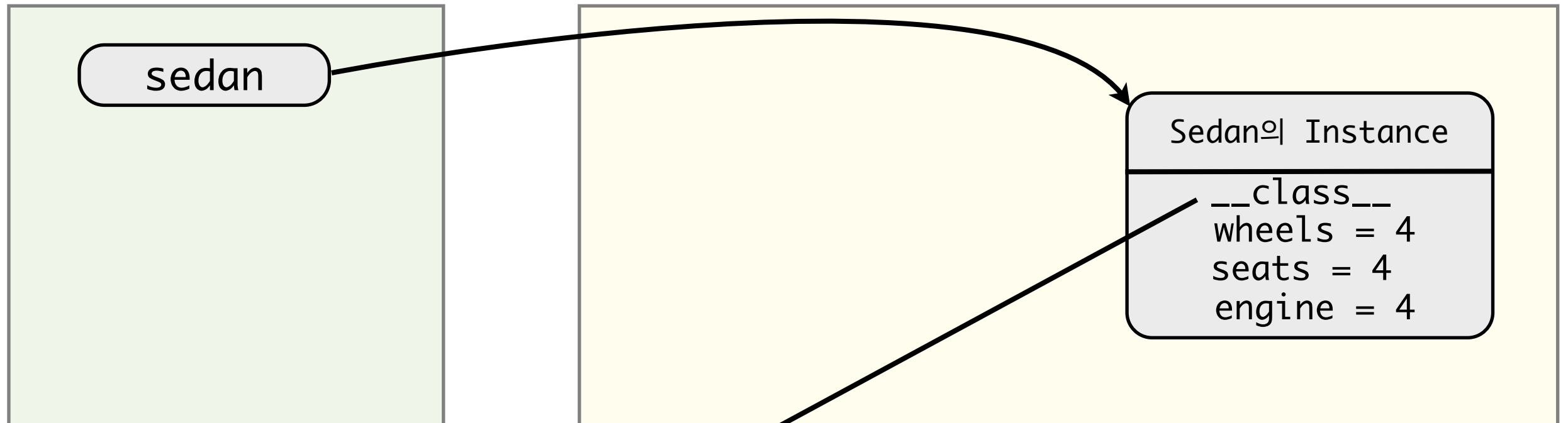
__init__()



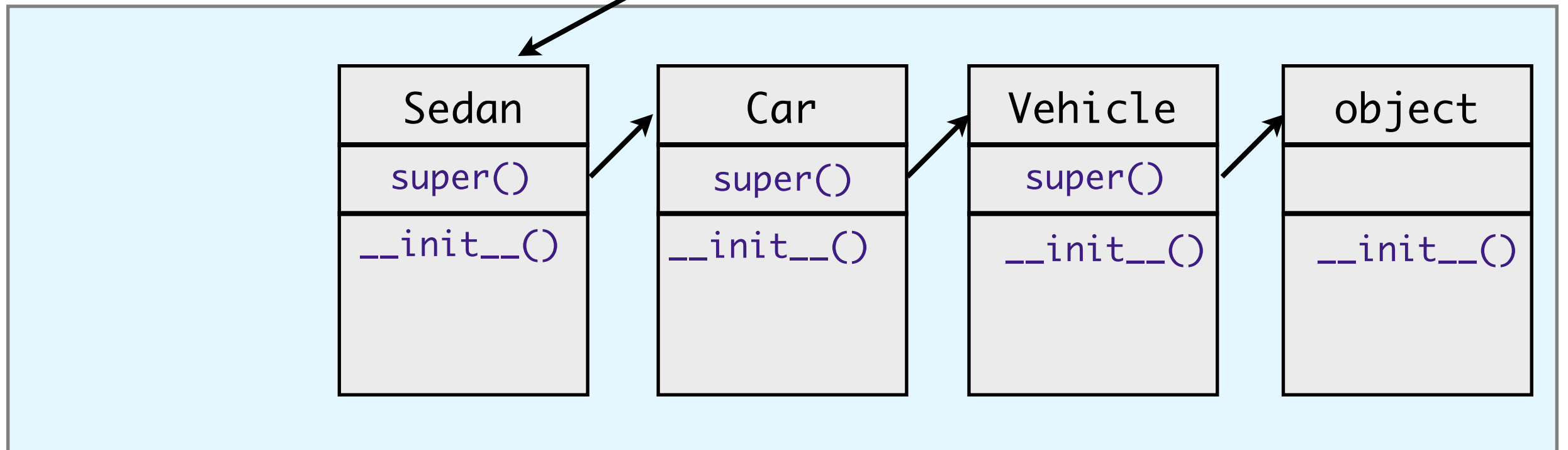
`sedan.load_passenger()`

Stack

Heap



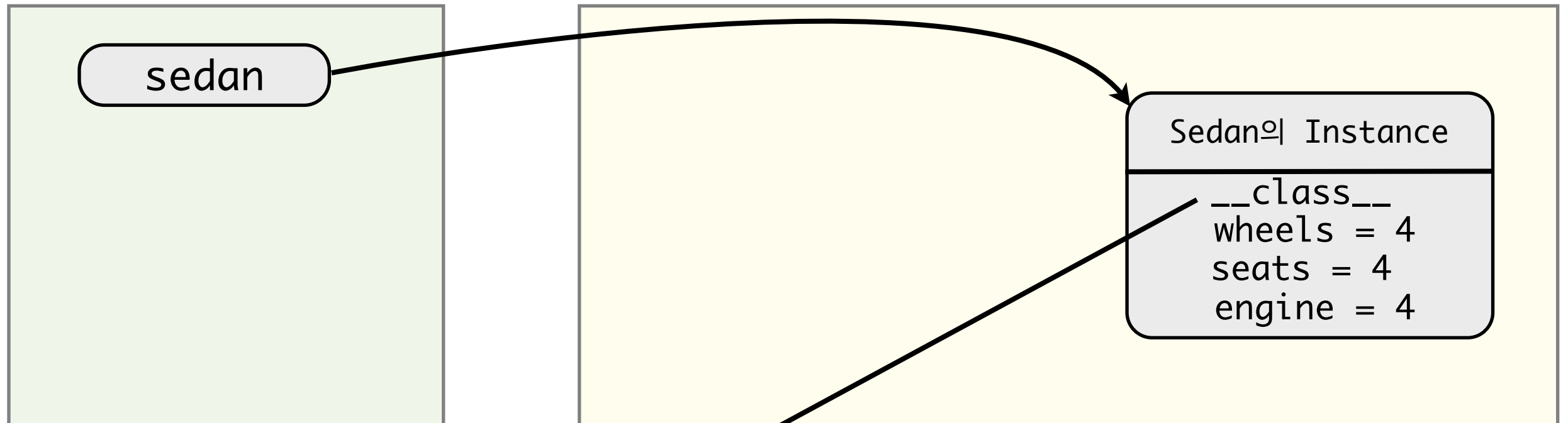
Code



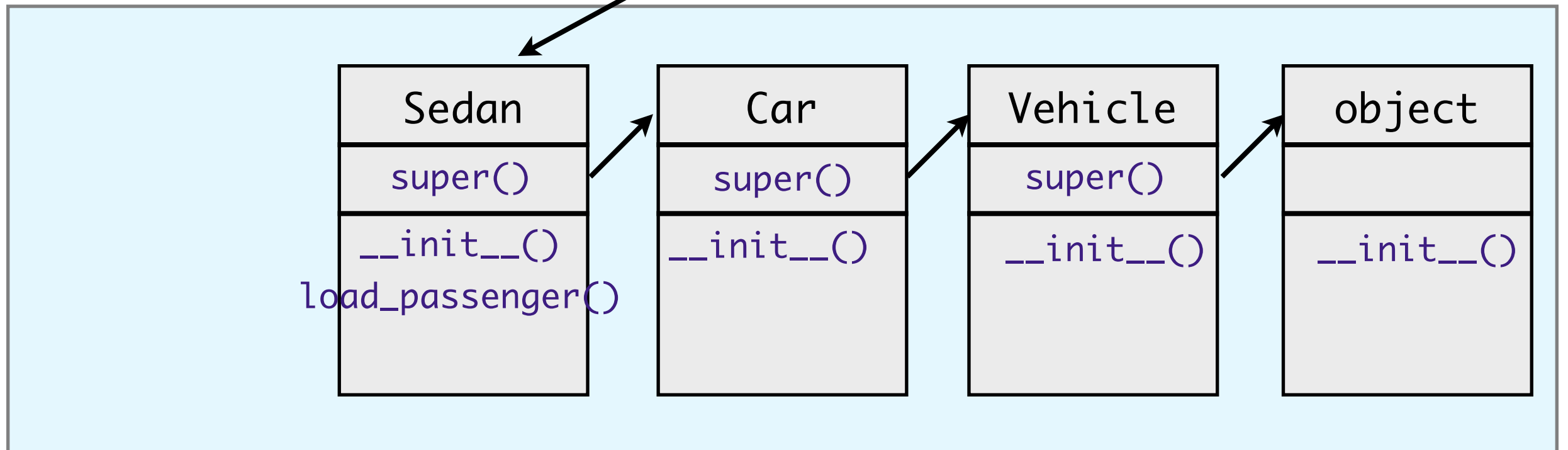
`sedan.load_passenger()`

Stack

Heap



Code



Stack

Heap

sedan

Sedan의 Instance

`__class__`
`wheels = 4`
`seats = 4`
`engine = 4`

Code

Sedan

`super()`

`__init__()`

`load_passenger()`

Car

`super()`

`__init__()`

Vehicle

`super()`

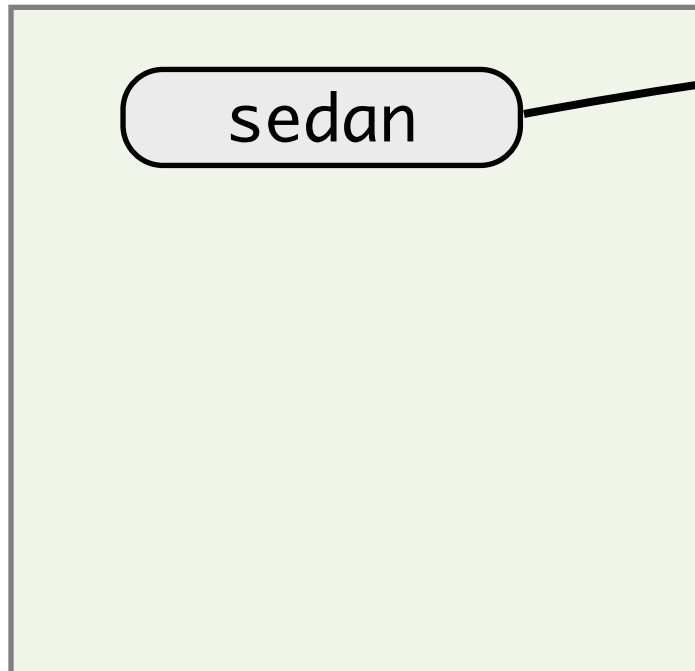
`__init__()`

object

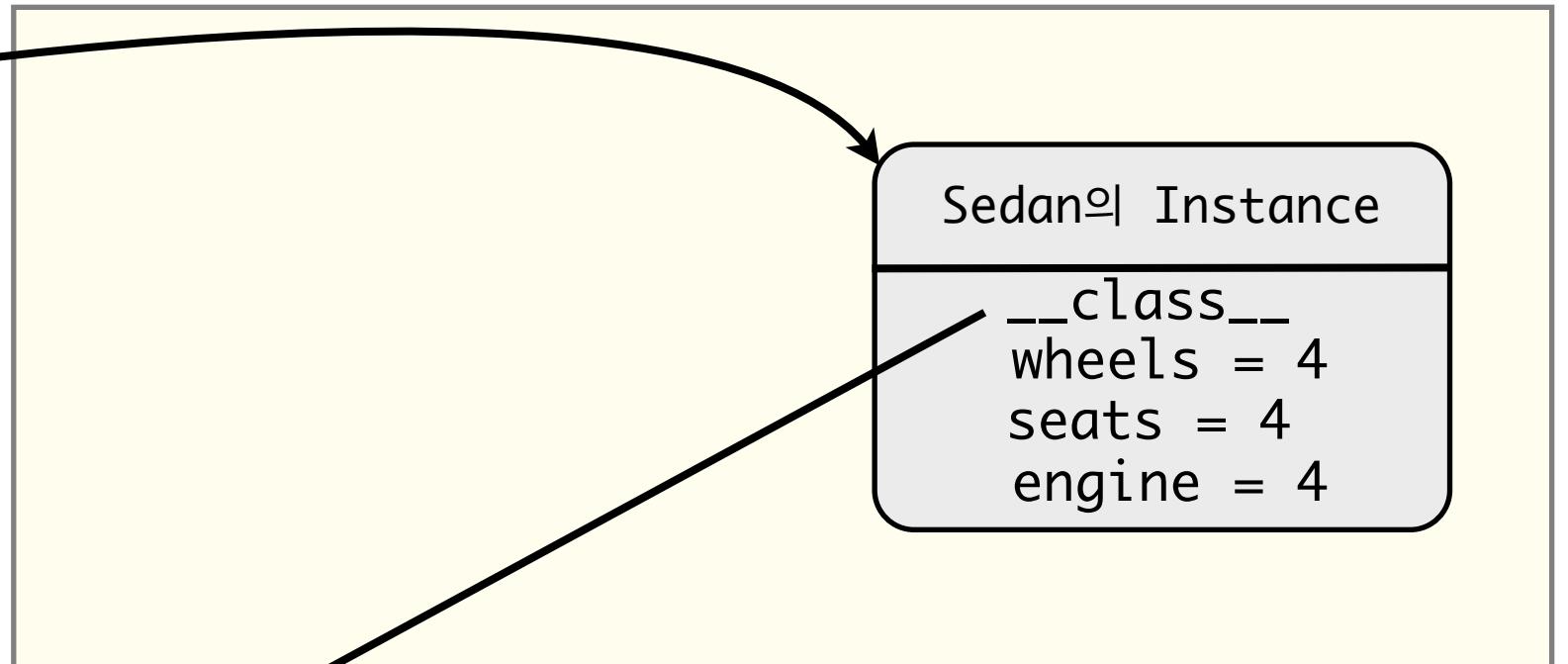
`__init__()`

`sedan.shift_gear()`

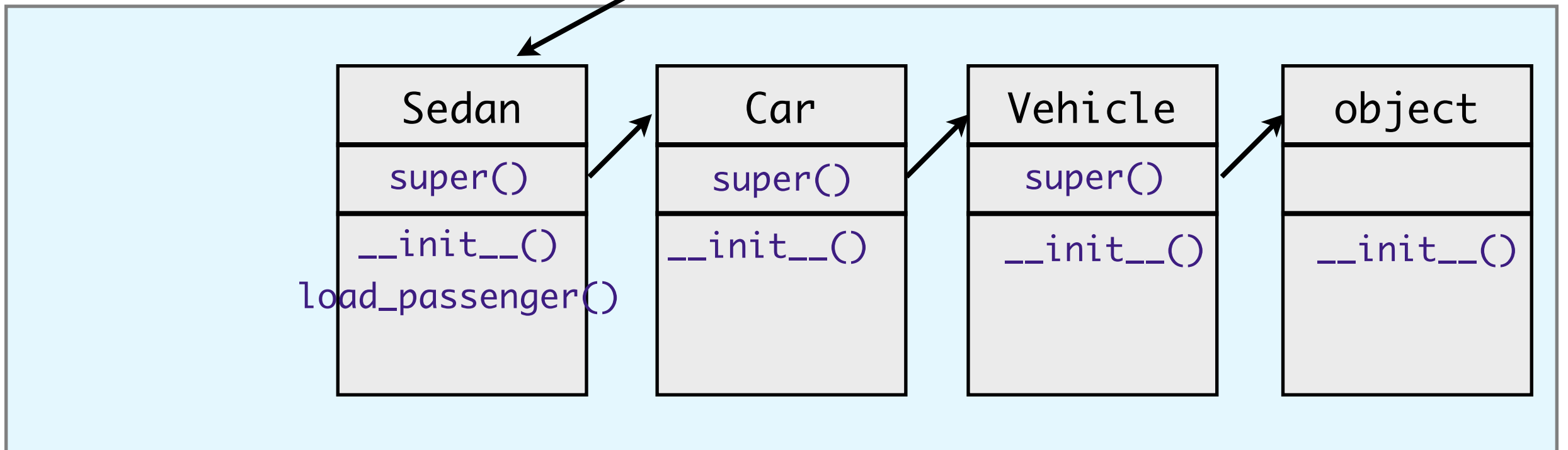
Stack



Heap



Code



sedan.shift_gear()

Stack

Heap

sedan

Sedan의 Instance

__class__
wheels = 4
seats = 4
engine = 4

Code

Sedan

super()

__init__()

load_passenger()

Car

super()

__init__()

shift_gear()

Vehicle

super()

__init__()

object

__init__()

Stack

Heap

sedan

Sedan의 Instance

__class__
wheels = 4
seats = 4
engine = 4

Code

Sedan

super()

__init__()

load_passenger()

Car

super()

__init__()

shift_gear()

Vehicle

super()

__init__()

object

__init__()

sedan.drive()

Stack

Heap

sedan

Sedan의 Instance

__class__
wheels = 4
seats = 4
engine = 4

Code

Sedan

super()

__init__()

load_passenger()

Car

super()

__init__()

shift_gear()

Vehicle

super()

__init__()

object

__init__()

sedan.drive()

Stack

Heap

sedan

Sedan의 Instance

__class__

wheels = 4

seats = 4

engine = 4

Code

Sedan

super()

__init__()

load_passenger()

Car

super()

__init__()

shift_gear()

Vehicle

super()

__init__()

drive()

object

__init__()

Stack

Heap

sedan

Sedan의 Instance

__class__
wheels = 4
seats = 4
engine = 4

Code

Sedan

super()

__init__()

load_passenger()

Car

super()

__init__()

shift_gear()

Vehicle

super()

__init__()

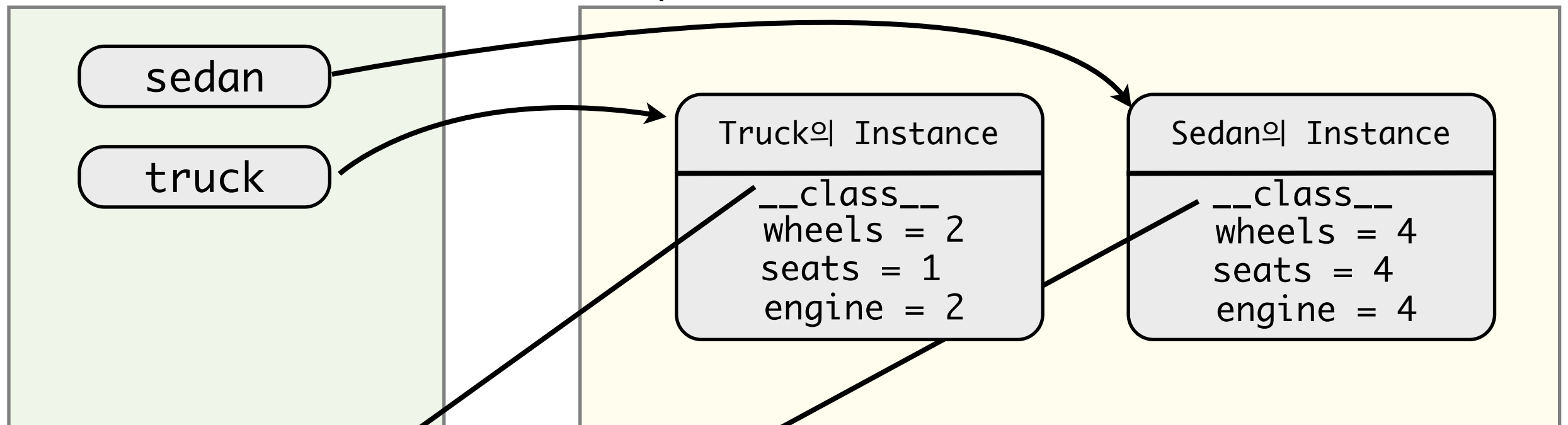
drive()

object

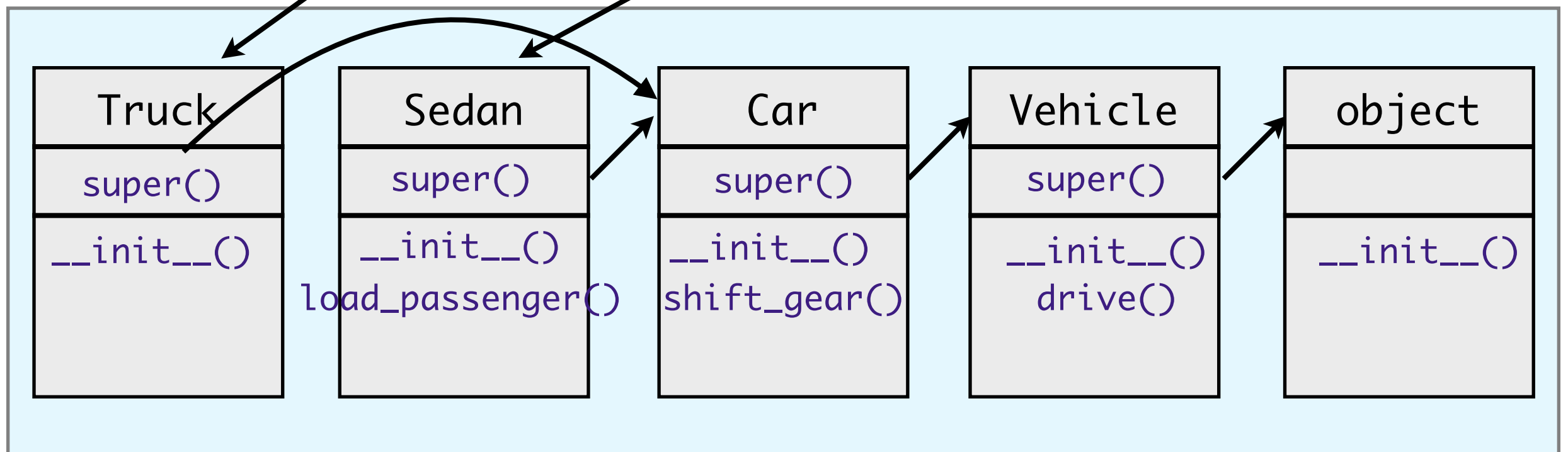
__init__()

Stack

Heap



Code



다형성

```
class Vehicle():
    def drive(self):
        print('Run')

class Sedan(Vehicle):
    def drive(self):
        print('Run Sedan')

class Truck(Vehicle):
    def drive(self):
        print('Run Truck')

class Boat():
    def drive(self):
        print('Run Boat')
```

```
def drive(vehicle):
    if isinstance(vehicle, Vehicle):
        vehicle.drive()

sedan = Sedan()
drive(sedan)

truck = Truck()
drive(truck)

boat = Boat()
drive(boat)
```

내장 함수

내장 함수

Built-in Functions

A

`abs()`
`aiter()`
`all()`
`any()`
`anext()`
`ascii()`

B

`bin()`
`bool()`
`breakpoint()`
`bytearray()`
`bytes()`

C

`callable()`
`chr()`
`classmethod()`
`compile()`
`complex()`

D

`delattr()`
`dict()`
`dir()`
`divmod()`

E

`enumerate()`
`eval()`
`exec()`

F

`filter()`
`float()`
`format()`
`frozenset()`

G

`getattr()`
`globals()`

H

`hasattr()`
`hash()`
`help()`
`hex()`

I

`id()`
`input()`
`int()`
`isinstance()`
`issubclass()`
`iter()`

L

`len()`
`list()`
`locals()`

M

`map()`
`max()`
`memoryview()`
`min()`

N

`next()`

O

`object()`
`oct()`
`open()`
`ord()`

P

`pow()`
`print()`
`property()`

R

`range()`
`repr()`
`reversed()`
`round()`

S

`set()`
`setattr()`
`slice()`
`sorted()`
`staticmethod()`
`str()`
`sum()`
`super()`

T

`tuple()`
`type()`

V

`vars()`

Z

`zip()`

`__import__()`

<https://docs.python.org/3/library/functions.html>

내장 함수

```
max_value = max(1, 30, 50)  
print(max_value)
```

```
min_value = min(1, 30, 50)  
print(min_value)
```

```
max_str = max('AAC', 'ABC', 'ACC')  
print(max_str)
```

```
min_str = min('AAC', 'ABC', 'ACC')  
print(min_str)
```

내장 함수

```
length = len("안녕하세요.")  
print(length)
```

```
result = eval("10+20+30+40")  
print(result)
```

```
result = eval("not 40>50")  
print(result)
```

```
list = [2, 5, 5, 3, 9, 1]  
result = sorted(list)  
print(result)
```

내장 함수

```
str1 = "안녕하세요"  
str_id = id(str1)  
print(str_id)
```

```
list_id = id([1,2,3,4,5])  
print(hex(list_id))  
print(oct(list_id))
```

```
print(type(3.14))  
print(type([1,2,3,4,5]))
```

```
print(abs(-5))
```

Python 기본 내장 모듈

Python Module Index

[_](#) | [a](#) | [b](#) | [c](#) | [d](#) | [e](#) | [f](#) | [g](#) | [h](#) | [i](#) | [j](#) | [k](#) | [l](#) | [m](#) | [n](#) | [o](#) | [p](#) | [q](#) | [r](#) | [s](#) | [t](#) | [u](#) | [v](#) | [w](#) | [x](#) | [z](#)

_	
__future__	<i>Future statement definitions</i>
__main__	<i>The environment where top-level code is run. Covers command-line interfaces, import-time behavior, and ``__name__ == '__main__'``.</i>
_thread	<i>Low-level threading API.</i>
a	
abc	<i>Abstract base classes according to :pep: `3119`.</i>
aifc	Deprecated: <i>Read and write audio files in AIFF or AIFC format.</i>
argparse	<i>Command-line option and argument parsing library.</i>
array	<i>Space efficient arrays of uniformly typed numeric values.</i>
ast	<i>Abstract Syntax Tree classes and manipulation.</i>
asynchat	Deprecated: <i>Support for asynchronous command/response protocols.</i>
asyncio	<i>Asynchronous I/O.</i>
asyncore	Deprecated: <i>A base class for developing asynchronous socket handling services.</i>
atexit	<i>Register and execute cleanup functions.</i>
audioop	Deprecated: <i>Manipulate raw audio data.</i>
b	
base64	<i>RFC 4648: Base16, Base32, Base64 Data Encodings; Base85 and Ascii85</i>
bdb	<i>Debugger framework.</i>
binascii	<i>Tools for converting between binary and various ASCII-encoded binary representations.</i>
bisect	<i>Array bisection algorithms for binary searching.</i>
builtins	<i>The module that provides the built-in namespace.</i>

<https://docs.python.org/3/py-modindex.html>

내장모듈 활용 - datetime

```
import datetime #import 키워드를 사용해서 모듈을 추가
```

```
today = datetime.date.today()  
print(today)
```

```
now = datetime.datetime.now()  
print(now)
```

```
print(now.year)  
print(now.month)  
print(now.day)
```

```
dir(datetime)
```

내장모듈 활용 - datetime

```
import datetime as dt # as를 사용해서 별칭을 지정
```

```
today = dt.date.today()  
print(today)
```

```
now = dt.datetime.now()  
print(now)
```

```
know = now + dt.timedelta(hours=9)  
print(know)
```

```
time_diff = know - now  
print(time_diff)
```

내장모듈 활용 - datetime

```
from datetime import datetime, date
```

```
xmas1 = datetime(2023, 12, 25, 0, 0, 0)  
print(xmas1)
```

```
xmas2 = date(2023, 12, 15)  
print(xmas2)
```

실습

datetime module을 사용해서 올해의 남은 일 수와 지금부터 100일 후의 날짜와 시간을 출력하시오.

내장모듈 활용 - time

```
import time
```

```
print(time.time())
```

```
local_time = time.localtime(time.time())
```

```
str_time = time.strftime('%Y-%m-%d %H:%M:%S', localtime)
```

```
print(str_time)
```

내장모듈 활용 - time

```
import time

print("시작")
time.sleep(5)
print("종료")

start_time = time.time()
time.sleep(5)
end_time = time.time()
elapsed_time = end_time - start_time
print(elapsed_time)
```

실습

time module과 for문을 사용해서 1~1000까지의 합을 구하는데 경과시간을 출력하시오.

실습

```
import time
start_time = time.time()
sum = 0
for i in range(1001):
    sum += i

end_time = time.time()
elapsed_time = end_time - start_time
print(elapsed_time)
```

내장모듈 활용 - math

```
import math
```

```
dir(math)
```

```
print(math.ceil(3.1))
```

```
print(math.floor(3.9))
```

```
print(math.fabs(-3.1))
```

```
print(math.pi)
```

```
print(math.sin(math.pi/2))
```

내장모듈 활용 - random

```
import random
```

```
print(random.random()) # 0~1미만의 실수 난수 발생
```

```
print(random.randrange(1, 7)) # 1~6까지의 정수 난수 발생
```

```
print(random.randint(1, 7)) # 1~7까지의 정수 난수 발생
```

```
list1 = [1,2,3,4,5,6,7,8,9,10]
```

```
random.shuffle(list1) # list의 아이템 순서를 바꿈
```

```
print(list1)
```

```
print(random.choice(list1)) # list의 아이템 중 하나를 임의로 선택
```

```
print(random.sample(list1, 5)) # list의 아이템 중 5개의 아이템을  
임의로 선택
```

실습

random 모듈을 이용해서 두 가지 방법으로 lotto 발생 함수를 만드시오.

1. while 루프와 randint 함수를 이용
2. shuffle 함수를 이용
3. sample 함수를 이용

실습

```
set1 = set()
while len(set1) < 6 :
    set1.add(random.randint(1, 45))

print(set1)
result = sorted(set1)
print(type(result))
print(result)
```

실습

```
list1 = list(range(1, 46))  
random.shuffle(list1)  
result = list1[0:6]  
result.sort()  
print(result)
```

```
list1 = list(range(1, 46))  
result = random.sample(list1, 6)  
result.sort()  
print(result)
```

내장모듈 활용 - os

```
import os
dir(os)

print(os.sep) # 경로구분자
cur_dir = os.getcwd() # 현재 작업경로
print(cur_dir)
test_dir = os.path.join(cur_dir, 'test') # 경로 생성

print(test_dir)

print(os.path.exists(test_dir)) #경로 유무
if not os.path.exists(test_dir):
    os.mkdir(test_dir) #디렉터리 생성
print(os.path.exists(test_dir))
```

file 처리

file 읽기

```
f=open("test.txt", 'r')  
text=f.read()  
print(text)  
f.close()
```

I/O Mode	Syntax	Behavior
Read	'r'	Opens the contents of a file for reading into the file interface, allowing for lines to be read-in successively.
Write	'w'	Creates a file with the specified name and allows for text to be written to the file; note that specifying a pre-existing filename will overwrite the existing file.
Append	'a'	Opens an existing file and allows for text to be written to it, starting at the conclusion of the original file contents.
Read and Write	'r+'	Opens a file such that its contents can be both read-in and written-to, thus offering great versatility.

Python's available file-access modes are summarized here.

file 읽기

```
with open('test.txt', 'r+') as f:  
    text = f.read()  
    print(text)
```

```
with open('test.txt', 'r+') as f:  
    lines = f.readlines()  
  
    for line in lines:  
        print(line, end=" ")
```

file 읽기

```
with open('test1.txt', 'r') as f:  
    print(f.readline(), end="")  
    print(f.readline(), end="")  
    print(f.readline(), end="")  
    print(f.readline(), end="")
```

```
with open('test1.txt', 'r', encoding='utf8') as f:  
    texts = f.readlines()  
    for text in texts:  
        if not text: break  
        print(text, end='')
```

file 쓰기

```
text = "안녕하세요.\n 파이썬입니다.\n 반갑습니다."  
with open('test1.txt', 'w') as f:  
    f.write(text)
```

```
texts = ["안녕하세요.", "파이썬입니다.", "반갑습니다."]  
with open('test1.txt', 'a') as f:  
    f.writelines(texts)
```

file 쓰기

```
texts = [ "안녕하세요.", "파이썬입니다.", "반갑습니다." ]
```

```
with open( 'test1.txt', 'a' ) as f:  
    for text in texts :  
        f.writelines(text)  
        f.writelines( '\n' )
```

예외처리

예외 발생

```
str1 = input('피제수를 입력하시오. ')
num1 = int(str1)

str2 = input('제수를 입력하시오. ')
num2 = int(str2)

result = num1 / num2
print('{} / {} = {}'.format(num1, num2, num1/num2))

#ZeroDivisionError: division by zero
```

try-except

```
str1 = input('피젯수를 입력하시오. ')
str2 = input('젯수를 입력하시오. ')

try:
    num1 = int(str1)
    num2 = int(str2)
    result = num1 / num2
    print('{} / {} = {}'.format(num1, num2, num1 / num2))
except:
    print('입력값을 확인하시오.')
```

try-except

```
str1 = input('피젯수를 입력하시오. ')
str2 = input('젯수를 입력하시오. ')

try:
    num1 = int(str1)
    num2 = int(str2)
    result = num1 / num2
    print('{} / {} = {}'.format(num1, num2, num1 / num2))
except Exception as e:
    print("excpetion:", e)
```

많이 사용하는 표준예외

Exception	Description
ZeroDivisionError	0으로 나눌때 발생
IndexError	list의 Index의 범위를 벗어날때 발생
ValueError	유효하지않은 값이 매개변수로 전달될때 발생
TypeError	유효하지않은 Type이 매개변수로 전달될때 발생
NameError	존재하지않은 변수를 사용할때 발생
FileNotFoundError	존재하지 않은 파일을 열때 발생

try-except

```
str1 = input('피젯수를 입력하시오. ')
str2 = input('젯수를 입력하시오. ')

try:
    num1 = int(str1)
    num2 = int(str2)
    result = num1 / num2
    print('{} / {} = {}'.format(num1, num2, num1 / num2))
except ValueError:
    print('숫자를 입력하시오. ')
except ZeroDivisionError:
    print('0으로 나눌수 없습니다. ')
except Exception as e:
    print('excpetion:', e)
```

try-except-else

```
str1 = input('젯수를 입력하시오. ')
str2 = input('피젯수를 입력하시오. ')

try:
    num1 = int(str1)
    num2 = int(str2)
    result = num1 / num2
except Exception as e:
    print("excpetion:", e)
else :
    print('{} / {} = {}'.format(num1, num2, num1 / num2))
```

try-except-else-finally

```
str1 = input('젯수를 입력하시오. ')
str2 = input('피젯수를 입력하시오. ')

try:
    num1 = int(str1)
    num2 = int(str2)
    result = num1 / num2
except Exception as e:
    print('excpetion:', e)
else :
    print('{} / {} = {}'.format(num1, num2, num1/num2))
finally:
    print('처리가 완료되었습니다.')
```

실습

test11.txt 파일을 읽기모드로 여는데 예외가 발생했을때 '파일을 여는중 예외가 발생했습니다.' 라고 출력하는 코드를 작성하시오.

람다(lambda)

일반 함수와 람다 함수

- 일반 함수를 이용하면 필요한 기능의 재사용이 가능하다
 - 함수를 정의한 후 필요할 때마다 호출하여 사용
- 가끔씩은 함수를 만들지 않고 함수화된 기능만을 불러 사용하고자 할 경우가 있다
 - 프로그램을 단순하게 만들 수 있다
 - 1회용으로 함수를 만들고 싶은데 `def xxx():` 과 같은 이름을 짓는 것이 번거롭게 느껴질 수 있다.
 - 코드의 가독성이 증대된다.

람다 함수

- 람다, 람다 표현식라고도 불리우는 람다 함수는 이름이 없는 함수
 - 익명함수(anonymous function)로 부르기도 함
 - 이름이 필요할 경우 할당하여 사용할 수 있다
- 주의 : 표현식 안에서 새 변수를 선언할 수 없다
- 반환값은 변수없이 식 한줄로 표현할 수 있어야 하며 복잡한 함수가 필요하면 def 로 함수를 정의하여야 한다.
 - return 문이 필요없다

람다함수 정의

```
def add(x, y):  
    return x + y  
  
f = lambda x, y: x + y  
  
print(add(1, 2))  
print(f(1, 2))
```

람다함수 정의

```
def add(x, y):  
    return x + y
```

```
f = lambda x, y: x + y
```

```
print(add(1, 2))
```

```
print(f(1,2))
```

```
f = lambda x: x if x % 3 == 0 else 0
```

```
print(f(3))
```

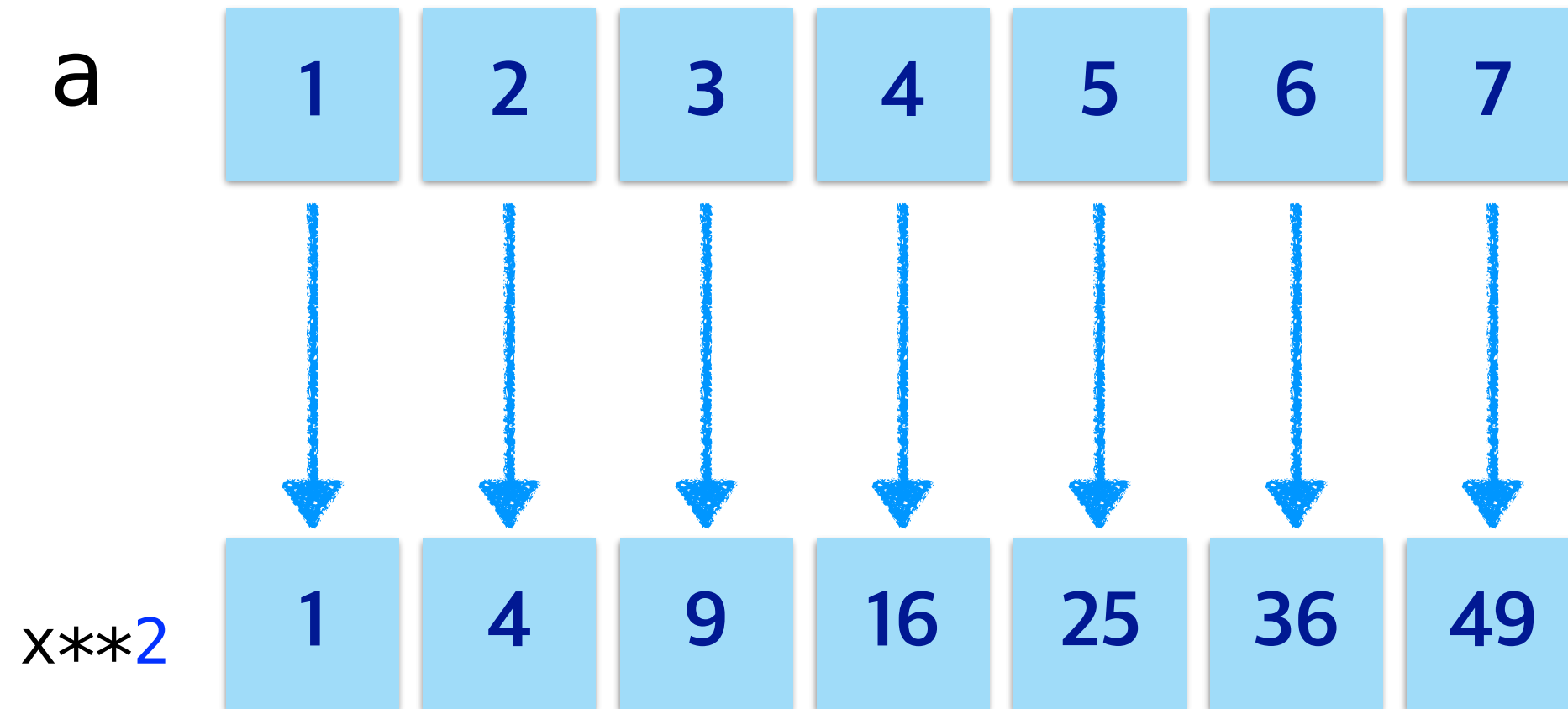
```
print(f(4))
```

맵(map) 함수와 람다

- 파이썬은 map() 이라는 내장함수를 제공하는데 열거가능한 자료형의 각 요소들에 대해서 매핑 함수를 적용한다

```
a = [1, 2, 3, 4, 5, 6, 7]
```

```
map(lambda x: x**2, a)
```



맵(map) 함수 사용

```
nums = [1, 2, 3, 4, 5]
def pow(x):
    return x ** 2
f = lambda x: x**2
print(list(map(pow, nums)))
print(list(map(f, nums)))
print(list(map(lambda x: x**2, nums)))
```

맵(map) 함수 사용

```
nums1 = [1, 2, 3, 4, 5]
```

```
nums2 = [6, 7, 8, 9, 10]
```

```
print(list(map(lambda x, y: x*y, nums1, nums2)))
```

실습

map과 lambda를 이용해서 1~10 list의 각 아이템에 5배를
가진 list를 만들고 출력하시오.

필터(filter) 함수와 람다

- 리스트에서 다룬 filter() 함수는 순환가능한 요소들을 함수에 넣어 그 리턴값이 참인 것만 묶어서 반환한다.

필터(filter) 함수 사용

```
ages = [18, 19, 39, 12, 43, 13, 21, 25]

def adult_func(age):
    if age >= 19:
        return True
    else:
        return False

for age in filter(adult_func, ages):
    print(age)
print()
for age in filter(lambda age: age >= 19, ages):
    print(age)
```

필터(filter) 함수 사용

```
ages = [18, 19, 39, 12, 43, 13, 21, 25]  
adult_ages = list(filter(lambda age: age >= 19, ages))  
print(adult_ages)
```

실습

1부터 10 사이의 정수를 가진 리스트에서 짝수만 나오게 필터링한 결과 리스트의 모든 원소에 대해 제곱을 수행해서 리스트로 반환하는 코드를 필터와 맵과 람다를 이용해서 작성하시요.

실습

```
nums = list(range(1, 10))  
  
result = list(map(lambda x: x**2,  
                  filter(lambda num: num % 2 == 0, nums)))  
print(result)
```

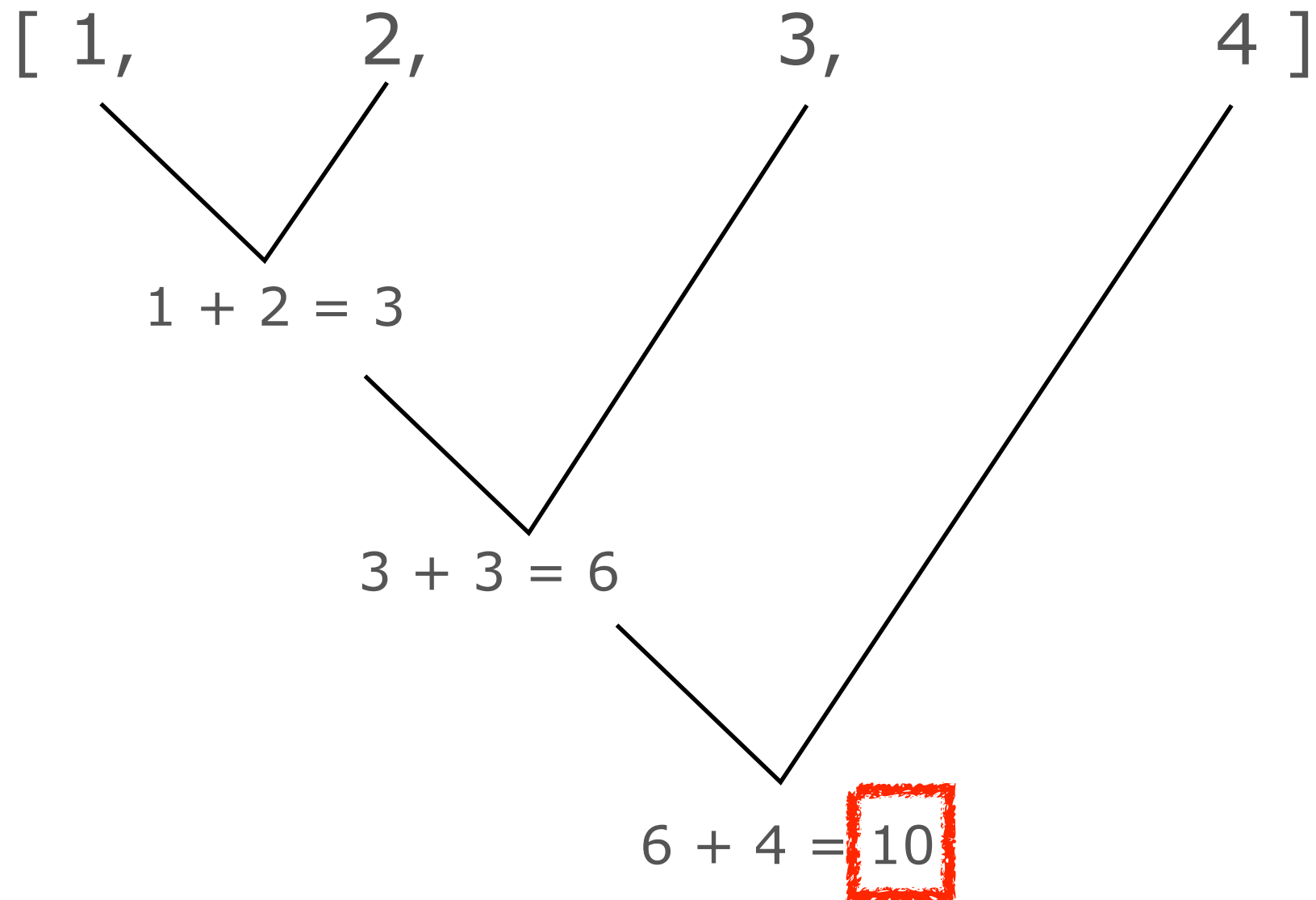
[4, 16, 36, 64]

reduce()

- 리듀스 함수는 functools 모듈에 포함되어 있는데 순환 가능 요소들을 같은 방법으로 연산하여 하나의 값을 얻는데 사용할 수 있다
- 문제는 이 결론을 얻는 것이 쉽게 이해하기 어렵다는 점이다

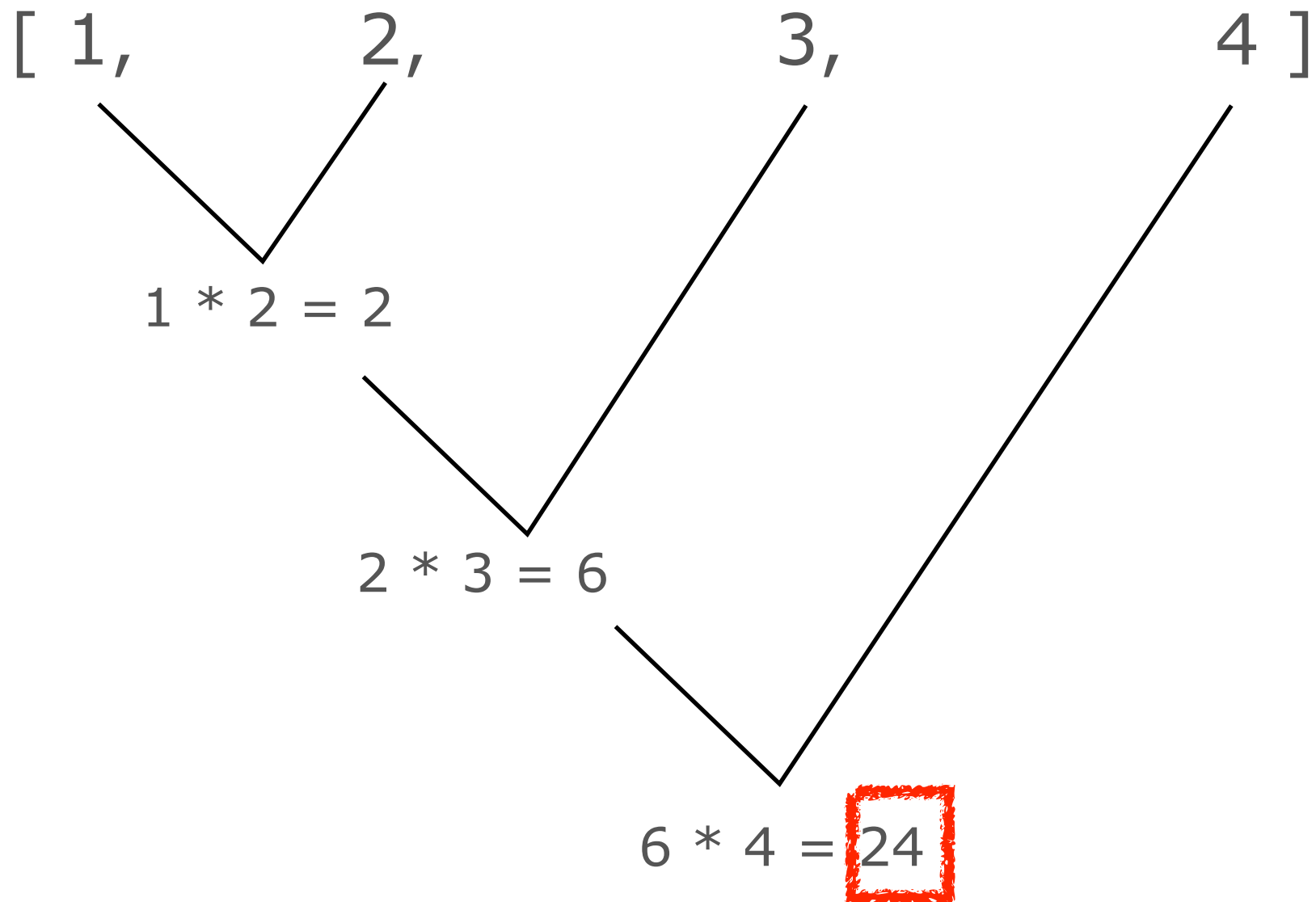
a = [1, 2, 3, 4]

n = reduce(**lambda** x, y: **x + y**, a)



a = [1, 2, 3, 4]

n = reduce(lambda x, y: x * y, a)



reduce 함수 사용

```
from functools import reduce
```

```
nums = [1, 2, 3, 4]
```

```
sum = reduce(lambda x, y: x+y, nums)
```

```
print(sum)
```

```
mul = reduce(lambda x, y: x*y, nums)
```

```
print(mul)
```

실습

1부터 10 사이의 정수를 가진 리스트에서 짝수만 나오게 필터링한 결과 리스트의 모든 원소의 곱을 구하는 코드를 lambda, filter, reduce 함수를 사용해서 작성하시요.

실습

```
from functools import reduce
```

```
nums = list(range(1, 10))
```

```
result = reduce(lambda x, y: x*y,
```

```
                filter(lambda num: num % 2 == 0, nums))
```

```
print(result)
```

축약표현을 이용한
list생성

list 축약표현

[{표현식} for {변수} in iterable]

```
list1 = list(range(1, 10))
```

```
list2 = list(map(lambda x: x**2, list1)) #map, lambda  
print(list2)
```

```
list3 = [x**2 for x in list1] #축약표현  
print(list3)
```

```
list4 = [x**2 for x in range(1, 10)] #축약표현  
print(list4)
```

list 축약표현

[{표현식} for {변수} in iterable if 조건문]
조건이 참인것만 필터링됨

```
list1 = list(range(1, 10))
```

```
# 조건문에 따라 5보다 큰 값만 필터링
```

```
list5 = [x**2 for x in list1 if x>5 ]
```

```
print(list5)
```

```
# 조건문에 따라 짝수 값만 필터링
```

```
list6 = [x**2 for x in range(1, 10) if x%2==0]
```

```
print(list6)
```

실습

축약표현으로 다음의 리스트를 만드시오.

1. 1~10까지 숫자의 제곱을 포함하는 리스트
2. 1~20까지의 짝수를 포함하는 리스트
3. 1~20까지의 숫자중 3의 배수를 포함하는 리스트
4. 1~20까지의 숫자중 5의 배수의 제곱을 포함하는 리스트